

Laslo Kraus

REŠENI ZADACI

IZ

PROGRAMSKOG JEZIKA

C++

AKADEMSKA MISAO

Beograd, 2011

Laslo Kraus

REŠENI ZADACI IZ PROGRAMSKOG JEZIKA C++
Treće dopunjeno izdanje

Recenzenti
Dr Igor Tartalja
Mr Đorđe Đurđević

Izdavač
AKADEMSKA MISAO
Bul. Kralja Aleksandra 73, Beograd

Lektor
Anđelka Kovačević

Dizajn naslovne strane
Zorica Marković, akademski slikar

Štampa
Planeta print, Beograd

Tiraž
300 primeraka

ISBN 978-86-7466-411-7

Predgovor

Ova zbirka zadataka je pomoćni udžbenik za učenje programiranja na jeziku C++. Zadaci prate gradivo autorove knjige *Programski jezik C++ sa rešenim zadacima*. Podrazumeva se, kao i u toj knjizi, da je čitalac savladao programiranje na jeziku C. Zbirka je namenjena za upotrebu u fakultetskoj nastavi, ali može da se koristi i za samostalno produbljivanje znanja iz programiranja.

Rešenja svih zadataka su potpuna u smislu da priloženi programi mogu da se izvršavaju na računaru. Pored samih tekstova programa priloženo je samo malo objašnjenja, prvenstveno u obliku slika i formula. Očekuje se da će izvođač nastave dati dodatna usmena objašnjenja slušaocima. Uz malo više napora zadaci mogu da se shvate i samostalno. Uz svaki program dat je i primer izvršavanja da bi se olakšalo razumevanje rada programa.

Kroz zadatke, pored elemenata samog jezika, prikazani su osnovni principi objektno orijentisanog programiranja: sakrivanje podataka, ponovno korišćenje koda, nasleđivanje i polimorfizam. Prikazani su i najčešće korišćeni postupci u programiranju: pretraživanje i uređivanje nizova, obrada znakovnih podataka, rad s bitovima, rad s dinamičkim strukturama podataka (kao što su liste i stabla) i rad s datotekama. Posebna pažnja posvećena je i inženjerskim aspektima programiranja: preglednosti, razumljivosti i efikasnosti.

Izvorni tekstovi svih programa iz ove zbirke mogu da se preuzmu preko Interneta sa adrese home.etf.rs/~kraus/knjige/. Svoja zapažanja čitaoci mogu da upute elektronskom poštom na adresu kraus@etf.rs.

Beograd, avgust 2011.

Laslo Kraus

NAPOMENA: Fotokopiranje ili umnožavanje na bilo koji način ili ponovno objavljivanje ove knjige - u celini ili u delovima - nije dozvoljeno bez prethodne izričite saglasnosti i pismenog odobrenja izdavača.

Sadržaj

Predgovor	3
Sadržaj	4
Preporučena literatura	6
0 Pripremni zadaci	7
Zadatak 0.1 Izostavljanje elemenata niza na osnovu binarne maske	8
Zadatak 0.2 Rekurzivno izračunavanje skalarnog proizvoda dva vektora	9
Zadatak 0.3 Presek dva skupa u dinamičkoj zoni memorije	10
Zadatak 0.4 Obrtanje redosleda elemenata jednostruko spregnute liste	11
1 Proširenja jezika C	13
Zadatak 1.1 Ispisivanje pozdrava	14
Zadatak 1.2 Izračunavanje zbira niza brojeva	15
Zadatak 1.3 Uređivanje dinamičkog niza brojeva	16
Zadatak 1.4 Obrada jednostruko spregnute liste brojeva	17
Zadatak 1.5 Uređivanje niza brojeva metodom podele	18
Zadatak 1.6 Izostavljanje suvišnih razmaka među rečima	20
Zadatak 1.7 Uređivanje imena gradova u dinamičkoj matrici	21
Zadatak 1.8 Određivanje polarnih koordinata tačke	22
Zadatak 1.9 Izračunavanje površine trougla	23
Zadatak 1.10 Paket funkcija za obradu redova brojeva neograničenog kapaciteta	24
Zadatak 1.11 Paketi funkcija za obradu tačaka, pravougaonika i nizova pravougaonika u ravni	26
2 Klase	29
Zadatak 2.1 Tačke u ravni	30
Zadatak 2.2 Uglovi	31
Zadatak 2.3 Redovi brojeva ograničenih kapaciteta	33
Zadatak 2.4 Uređeni skupovi brojeva	36
Zadatak 2.5 Trouglovi u ravni	39
Zadatak 2.6 Kvadri u dinamičkoj zoni memorije	41
Zadatak 2.7 Redovi u ravni koji ne smeju da se preklapaju	43
Zadatak 2.8 Kalendarski datumi	46
Zadatak 2.9 Liste brojeva	49
Zadatak 2.10 Uređena stabla brojeva	55
Zadatak 2.11 Nizovi materijalnih tačaka	61
Zadatak 2.12 Liste datuma	64
Zadatak 2.13 JMBG, osobe i imenici	67

3 Operatorske funkcije	71
Zadatak 3.1 Kompleksni brojevi	72
Zadatak 3.2 Vremenski intervali	74
Zadatak 3.3 Nizovi kompleksnih brojeva	76
Zadatak 3.4 Kvadri s automatski generisanim identifikacionim brojevima	78
Zadatak 3.5 Polinomi s realnim koeficijentima	80
Zadatak 3.6 Studenti koji ne smeju da se kopiraju	84
Zadatak 3.7 Redovi brojeva neograničenih kapaciteta	87
Zadatak 3.8 Tekstovi	90
Zadatak 3.9 Tekstovi s uštedom memorije	92
Zadatak 3.10 Karte i predstave	96
Zadatak 3.11 Zapisi artikala i inventari	99
Zadatak 3.12 Otpornici i redne veze otpornika	103
Zadatak 3.13 Tačke, trouglovi, skupovi trouglova u ravni	106
4 Izvedene klase	109
Zadatak 4.1 Valjci i kante	110
Zadatak 4.2 Osobe, daci i zaposleni	112
Zadatak 4.3 Neuređene i uređene liste celih brojeva	115
Zadatak 4.4 Predmeti, sfere i kvadri	118
Zadatak 4.5 Geometrijske figure, krugovi, kvadrati i trouglovi u ravni	121
Zadatak 4.6 Vektori, brzine i pokretni objekti i tačke u prostoru	126
Zadatak 4.7 Tačke, linije, duži, izlomljene linije i poligoni u ravni	129
Zadatak 4.8 Objekti, skupovi objekata, kompleksni brojevi i tekstovi	135
Zadatak 4.9 Geometrijska tela, sfere, valjci i redovi tela	140
Zadatak 4.10 Osobe, studenti i imenici	144
Zadatak 4.11 Osobe, vozila, teretna vozila i putnička vozila	148
Zadatak 4.12 Izrazi, konstante, promenljive, dodele vrednosti i aritmetičke operacije	153
Zadatak 4.13 Naredbe, proste naredbe, sekvence, selekcije i ciklusi	160
5 Izuzeci	167
Zadatak 5.1 Vektori realnih brojeva sa zadatim opsezima indeksa	168
Zadatak 5.2 Racionalni brojevi	171
Zadatak 5.3 Matrice racionalnih brojeva	174
Zadatak 5.4 Nizovi, funkcije i verižni razlomci	180
Zadatak 5.5 Podaci, skalarni podaci i nizovi	184
Zadatak 5.6 Funkcije i greške; izračunavanje određenog integrala	189
Zadatak 5.7 Predmeti, celi brojevi, zbirke i nizovi predmeta	195
Zadatak 5.8 Vektori, figure, tačke i mnogouglovi u prostoru	199
Zadatak 5.9 Proizvodi, sfere, kvadri, mašine i radnici	203
Zadatak 5.10 Radnici, prodavci, šefovi i firme	209
Zadatak 5.11 Vozila, lokomotive, putnički vagoni i vozovi	214
Zadatak 5.12 Električni potrošači, uređaji, grupe uređaja i izvori	220
Zadatak 5.13 Funkcije za koje mogu da se stvaraju izvodi, delegati, monomi, eksponencijalne funkcije i zbrovi funkcija	225

6	Generičke funkcije i klase	231
Zadatak 6.1	Generička funkcija za fuziju uređenih nizova	232
Zadatak 6.2	Generički stekovi zadatih kapaciteta	235
Zadatak 6.3	Generičke klase za upoređivanje podataka i uređivanje nizova podataka	237
Zadatak 6.4	Generički nizovi, boje, tačke, obojene figure, krugovi, pravougaonici, trouglovi, mnogouglovi i crteži u ravni	239
Zadatak 6.5	Generičke liste; datumi, osobe, ispiti, đaci i škole	247
Zadatak 6.6	Vozila, bicikli, kamioni, generički nizovi, etape, vožnje i trkački automobili	253
Zadatak 6.7	Tereti, sanduci, burad, generički nizovi, vozila, lokomotive, vagoni i vozovi	259
Zadatak 6.8	Simboli, fontovi, vektori, duži, tekstovi, generički nizovi i crteži	266
Zadatak 6.9	Akteri, časovnici, proizvodi i generička skladišta, proizvođači i potrošači	273
7	Standardna biblioteka	279
Zadatak 7.1	Stekovi i redovi tačaka neograničenih kapaciteta	280
Zadatak 7.2	Predmeti, tela, sfere, kvadri i sklopovi	284
Zadatak 7.3	Obrada sekvencijalne tekstualne datoteke	289
Zadatak 7.4	Obrada rečenica u tekstualnoj sekvencijalnoj datoteci	290
Zadatak 7.5	Obrada sekvencijalne binarne datoteke	291
Zadatak 7.6	Obrada relativne binarne datoteke	293
Zadatak 7.7	Klasa rečnika	295
Zadatak 7.8	Klasa relativnih binarnih datoteka i obrada liste u relativnoj binarnoj datoteci	299

0 Pripremni zadaci

Preporučena literatura

- [1] Laslo Kraus: **Programski jezik C++ sa rešenim zadacima**, *osmo prerađeno izdanje*, Akademska misao, Beograd, 2011.
- [2] **Working Paper for Draft Proposed International Standard for Information Systems – Programming Language C++**, Accredited Standard Committee X3, 1996.
- [3] Bjarne Stroustrup: **The C++ Programming Language**, *third edition*, Addison-Wesley Publishing Company, Reading, Massachusetts, 1997.
- [4] Laslo Kraus: **Programski jezik C sa rešenim zadacima**, *sedmo izdanje*, Akademska misao, Beograd, 2008.
- [5] Laslo Kraus: **Rešeni zadaci iz programskog jezika C**, *treće izdanje*, Akademska misao, Beograd, 2009.

Zadatak 0.1 Izostavljanje elemenata niza na osnovu binarne maske

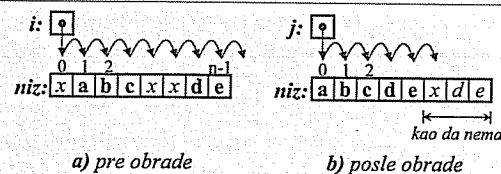
Napisati na jeziku C funkciju za izostavljanje svih elemenata numeričkog niza niz dužine n elemenata za koje na odgovarajućim mestima niza bitova maska stoji 0. Niz bitova se smešta u potreban broj bajtova od kojih svaki (sem možda poslednjeg) sadrži po 8 bitova.

Napisati na jeziku C program za ispitivanje prethodne funkcije.

Rešenje:

Izostavljanje određenih elemenata:

- Elementi se ispituju redom (indeks i). Ako i -ti element treba da ostane, premešta se na j -to mesto.
- Nova dužina niza je završna vrednost indeksa j .



```
/* reduk.c - Izostavljanje elemenata niza na osnovu maske. */
```

```
void redukcija(int niz[], const char maska[], int *n) {
    int i, j, k; char m;
    for (i=j=k=0; i<*n; i++) {
        if (i % 8 == 0) m = maska[k++];
        if (m & 1) niz[j++] = niz[i];
        m >>= 1;
    }
    *n = j;
}
```

```
#include <stdio.h>
```

```
void main() {
    while (1) {
        int n, i, niz[120];
        char maska[15];
        printf("n? "); scanf("%d", &n);
        if (n<=0 || n>120) break;
        printf("niz? ");
        for (i=0; i<n; scanf("%d", &niz[i++]));
        printf("maska? ");
        for (i=0; i<(n+7)/8; i++) {
            int m; scanf("%x", &m); maska[i] = m;
        }
        redukcija(niz, maska, &n);
        printf("niz=");
        for (i=0; i<n; printf(" %d", niz[i++]));
        putchar('\n');
    }
}
```

Izdvajanje desnog bita:

```

  7 6 5 4 3 2 1 0
m: x x x x x x x x } &
  0 0 0 0 0 0 0 1
  0 0 0 0 0 0 0 x

```

```

n? 11
niz? 1 2 3 4 5 6 7 8 9 0 1
maska? b5 4 1011010100000100
niz= 1 3 5 6 8 1
n? 8
niz? 1 2 3 4 5 6 7 8
maska? ff
niz= 1 2 3 4 5 6 7 8
n? 5
niz? 1 2 3 4 5
maska? 0
niz=
n? 0

```

Zadatak 0.2 Rekurzivno izračunavanje skalarnog proizvoda dva vektora

Napisati na jeziku C rekurzivnu funkciju za izračunavanje skalarnog proizvoda dva vektora.

Napisati na jeziku C program koji pročita dva vektora s glavnog ulaza, izračuna njihov skalarni proizvod, ispiše dobijeni rezultat na glavnom izlazu i ponavlja prethodne korake sve dok ne dobije signal za završetak rada programa. Program treba da je pogodan za skretanje glavnog ulaza na datoteku.

Rešenje:

```
/* skalpro.c - Skalarni proizvod dva vektora. */

double skal_pro(const double *a, const double *b, int n) {
    return n ? (*a) * (*b) + skal_pro(a+1, b+1, n-1) : 0;
}

#include <stdio.h>

void main() {
    double a[100], b[100]; int i, n;
    while (scanf("%i", &n) != EOF) {
        for (i=0; i<n; scanf("%lf", &a[i++]));
        for (i=0; i<n; scanf("%lf", &b[i++]));
        printf("g\n", skal_pro(a,b,n));
    }
}
```

```

% cc skalpro.c -o skalpro
% skalpro <skalpro.pod >skalpro.rez
% cat skalpro.pod
5
1 2 3 4 5
5 4 3 2 1
4
-1 1 -2 2
4 5 6 7
% cat skalpro.rez
35
3

```

Zadatak 0.3 Presek dva skupa u dinamičkoj zoni memorije

Skup realnih brojeva predstavlja se pomoću strukture od dva člana koji su broj elemenata skupa i pokazuje na niz u dinamičkoj zoni memorije koji sadrži same elemente skupa. Napisati na jeziku C funkciju za nalaženje preseka dva takva skupa.

Napisati na jeziku C program koji pročita dva skupa realnih brojeva, pronalazi njihov presek, ispisuje dobijeni rezultat i ponavlja prethodne korake sve dok za broj elemenata skupa ne pročita negativnu vrednost.

Rešenje:

```
/* presek.c - Presek dva skupa. */
#include <stdio.h>
#include <stdlib.h>

typedef struct { int vel; double *niz; } Skup;

Skup presek(Skup s1, Skup s2) {
    Skup s; int i, j, k, vel = (s1.vel < s2.vel ? s1.vel : s2.vel);
    double *niz = malloc(vel * sizeof(double));
    for (i=k=0; i<s1.vel; i++) {
        for (j=0; j<s2.vel && s2.niz[j]!=s1.niz[i]; j++);
        if (j < s2.vel) niz[k++] = s2.niz[j];
    }
    s.vel = k; s.niz = realloc(niz, k * sizeof(double));
    return s;
}

void main() {
    Skup s1, s2, s; int i;
    while (scanf("%d", &s1.vel), s1.vel >= 0) {
        s1.niz = malloc(s1.vel * sizeof(double));
        for (i=0; i<s1.vel; scanf("%lf", &s1.niz[i++]));
        scanf("%d", &s2.vel);
        s2.niz = malloc(s2.vel * sizeof(double));
        for (i=0; i<s2.vel; scanf("%lf", &s2.niz[i++]));
        s = presek(s1, s2);
        for (i=0; i<s.vel; printf("%lf ", s.niz[i++])); putchar('\n');
        free(s1.niz); free(s2.niz); free(s.niz);
    }
}
```

presek.pod

```
4 1 2 3 4
5 3 4 5 6 7
6 9 3 5 1 2 8
4 6 2 8 5
-1
```

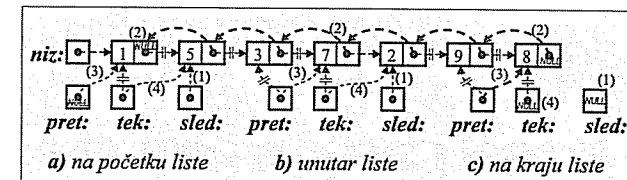
```
% cc presek.c -o presek
% presek <presek.pod
3.000000 4.000000
5.000000 2.000000 8.000000
```

Zadatak 0.4 Obrtanje redosleda elemenata jednostruko spregnute liste

Niz realnih brojeva predstavlja se u obliku jednostruko spregnute liste. Napisati na jeziku C funkciju za obrtanje redosleda elemenata jednog takvog niza (liste), tj. za zamenu prvog elementa s poslednjim, drugog s pretposlednjim itd.

Napisati na jeziku C program kojim se obrađuje proizvoljan broj nizova brojeva pomoću prethodne funkcije i ispisuju dobijeni rezultati na glavnom izlazu. Na raspolaganju stoje gotove funkcije za čitanje niza brojeva uz formiranje liste i za brisanje liste. Program treba da završi s radom kada se pročita prazan niz s glavnog ulaza.

Rešenje:



```
/* obrni.c - Obrtanje redosleda elemenata liste. */
#include <stdio.h>

typedef struct elem { float broj; struct elem *sled; } Elem;

Elem *obrni(Elem *niz) {
    Elem *tek = niz, *pret = NULL, *sled;
    while (tek) { sled = tek->sled; tek->sled = pret; pret = tek; tek = sled; }
    return pret;
}

Elem *citaaj_niz(void);

void brisi_niz(Elem *);

void main() {
    Elem *niz, *tek;
    while ((niz = citaaj_niz()) != NULL) {
        niz = obrni(niz);
        for (tek=niz; tek; tek=tek->sled) printf("%g", tek->broj);
        putchar('\n');
        brisi_niz(niz);
    }
}
```

1 Proširenja jezika C

Zadatak 1.1 Ispisivanje pozdrava

Napisati na jeziku C++ program koji ispisuje tekst na glavnom izlazu računara.

Rešenje:

```
// pozdrav.C - Ispisivanje pozdrava.

#include <iostream>
using namespace std;

int main() {
    cout << "Pozdrav svima!" << endl;
    return 0;
}
```

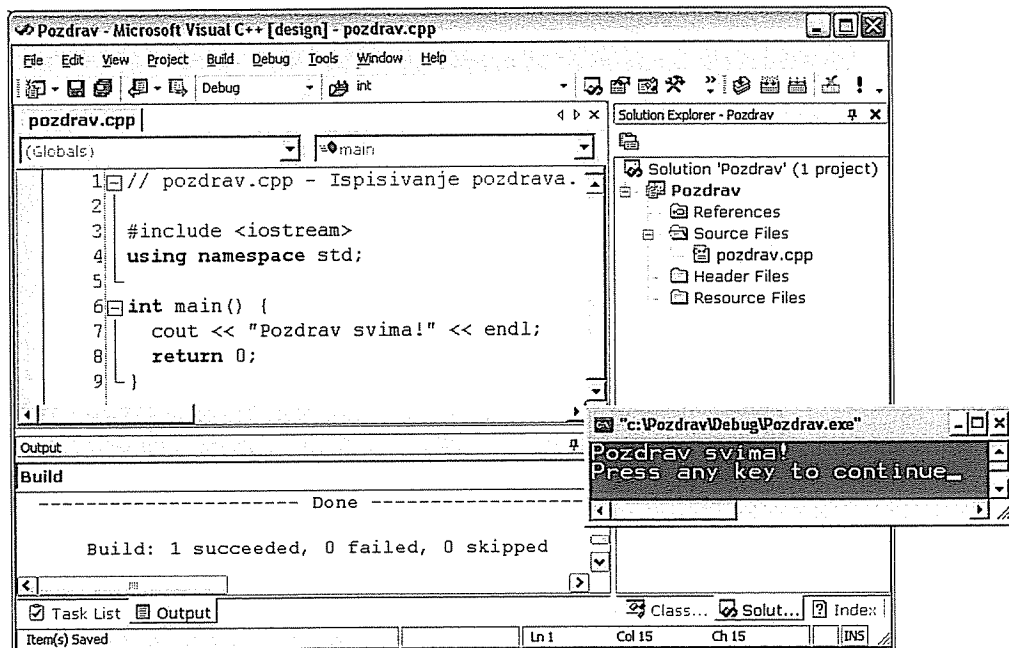
Obrada programa:

1) pod UNIX-om (Linux na PC-u)

```
% vi imeprog.C
% CC imeprog.C -o imeprog
% CC imeprog.C -lm -o imeprog
% imeprog
% imeprog <podaci >rezult
```

```
% vi pozdrav.C
% CC pozdrav.C -o pozdrav
% pozdrav
Pozdrav svima!
```

2) u programskom okruženju Visual Studio .Net:



Zadatak 1.2 Izračunavanje zbira niza brojeva

Napisati na jeziku C++ program za izračunavanje zbira elemenata niza celih brojeva. Program treba da obrađuje više kompleta podataka.

Rešenje:

```
// zbir.C - Zbir niza celih brojeva.

#include <iostream>
using namespace std;

int main() {

    const int DUZ = 100;

    while (true) {
        cout << "\nDuzina niza? ";
        int n; cin >> n;
        if (n <= 0 || n > DUZ) break;
        int a[DUZ]; cout << "Elementi niza? ";
        for (int i=0; i<n; cin >> a[i++]);
        int s = 0;
        for (int i=0; i<n; s+=a[i++]);
        cout << "Zbir elemenata: " << s << endl;
    }
    return 0;
}
```

```
% CC zbir.C -o zbir
% zbir
```

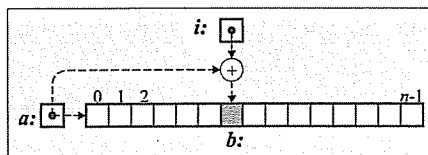
```
Duzina niza? 8
Elementi niza? 1 2 3 4 5 6 7 8
Zbir elemenata: 36

Duzina niza? 0
```

Zadatak 1.3 Uređivanje dinamičkog niza brojeva

Napisati na jeziku C++ program za uređivanje dinamičkog niza celih brojeva. Program treba da obrađuje više kompleta podataka.

Rešenje:



```
// uredil.C - Uređivanje dinamičkog niza celih brojeva.
```

```
#include <iostream>
using namespace std;

int main() {
    while (true) {
        int n; cout << "Duzina niza? "; cin >> n;
        if (n <= 0) break;
        int* a = new int [n];
        cout << "Pocetni niz? ";
        for (int i=0; i<n; cin >> a[i++]);
        for (int i=0; i<n-1; i++) {
            int& b = a[i]; // b je u stvari a[i].
            for (int j=i+1; j<n; j++)
                if (a[j]<b) { int c=b; b=a[j]; a[j]=c; }
        }
        cout << "Uredjeni niz: ";
        for (int i=0; i<n; cout << a[i++] << ' ');
        cout << endl;
        delete [] a;
    }
    return 0;
}
```

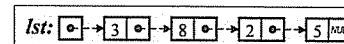
```
% CC uredil.C -o uredil
% uredil
Duzina niza? 10
Pocetni niz? 2 0 3 8 7 1 9 4 6 5
Uredjeni niz: 0 1 2 3 4 5 6 7 8 9
Duzina niza? 0
```

Zadatak 1.4 Obrada jednostruko spregnute liste brojeva

Napisati na jeziku C++ funkcije za čitanje, ispisivanje i uništavanje jednostruko spregnute liste celih brojeva.

Napisati na jeziku C++ glavnu funkciju za ispitivanje prethodnih funkcija.

Rešenje:



```
// listal.C - Obrada jednostruko spregnute liste.
```

```
#include <iostream>
using namespace std;

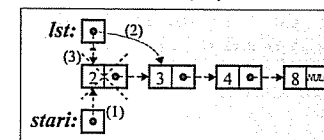
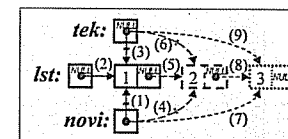
struct Elem { int broj; Elem* sled; };

Elem* citaj() { // Stvaranje liste čitajući.
    Elem *lst=0, *tek=0;
    while (true) {
        int broj; cin >> broj;
        if (broj == 9999) break;
        Elem* novi = new Elem;
        novi->broj = broj; novi->sled = 0;
        tek = (lst==0 ? lst : tek->sled) = novi;
    }
    return lst;
}

void pisi(Elem* lst) { // Ispisivanje liste.
    while (lst) { cout << lst->broj << ' '; lst = lst->sled; }
}

void brisi(Elem* lst) { // Uništavanje liste.
    while (lst) { Elem* stari = lst; lst = lst->sled; delete stari; }
}

int main() { // Ispitivanje funkcija.
    while (true) {
        Elem* lista;
        cout << "\nLista? ";
        if ((lista = citaj())==0) break;
        cout << "Procitano: "; pisi(lista); cout << endl;
        brisi(lista);
    }
    return 0;
}
```



```
% CC listal.C -o listal
% listal
Lista? 1 2 3 4 5 6 7 8 9 9999
Procitano: 1 2 3 4 5 6 7 8 9
Lista? 9999
```



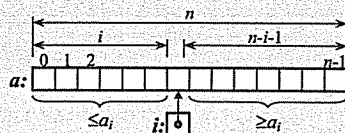
Zadatak 1.5 Uređivanje niza brojeva metodom podele

Napisati na jeziku C++ funkciju za uređivanje niza celih brojeva metodom podele (*quick sort*).

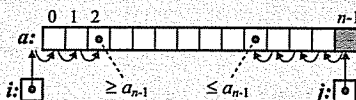
Napisati na jeziku C++ program za ispitivanje prethodne funkcije.

Rešenje:

Niz se podeli oko elementa i tako da levo svi brojevi budu $\leq a_i$, a desno svi budu $\geq a_i$. Levi i desni deo niza uređuju se nezavisno.



Podela se pravi premeštanjem elemenata većih od a_{n-1} s početka niza prema kraju, a elemenata manjih od a_{n-1} skraja prema početku. Na kraju, a_{n-1} se stavi negde u sredinu niza.



```
// podela.C - Uređivanje niza brojeva metodom podele (quick sort).
```

```
inline void zameni(int& x, int& y)
{ int z = x; x = y; y = z; }
```

```
static int podeli(int a[], int n) {
    int& b = a[n-1], i = -1, j = n-1;
    while (i < j) {
        do i++; while (a[i] < b);
        do j--; while (j >= 0 && a[j] > b);
        zameni(a[i], (i < j) ? a[j] : b);
    }
    return i;
}
```

```
void uredi(int a[], int n) {
    if (n > 1) {
        int i = podeli(a, n);
        uredi(a, i);
        uredi(a+i+1, n-i-1);
    }
}
```

```
// podelat.C - Ispitivanje funkcije za uređivanje niza metodom podele.
```

```
#include <stdlib.h>
#include <iostream>
using namespace std;

void uredi(int[], int);

int main() {
    while (true) {
        int n; cout << "\n\nDuzina niza? "; cin >> n;
        if (n <= 0) break;
        int* a = new int [n];
        cout << "\nPocetni niz:\n\n";
        for (int i=0; i<n; i++)
            cout << (a[i] = (int)(rand() / (RAND_MAX + 1.) * 10))
                << ((i%30==29 || i==n-1) ? '\n' : ' ');
        uredi(a, n);
        cout << "\nUredjeni niz:\n\n";
        for (int i=0; i<n; i++)
            cout << a[i] << ((i%30==29 || i==n-1) ? '\n' : ' ');
        delete [] a;
    }
    return 0;
}
```

```
% CC podelat.C podela.C -o podelat
% podelat
```

Duzina niza? 97

Pocetni niz:

```
0 3 0 3 2 5 1 7 9 2 4 1 6 5 0 1 8 6 7 8 9 2 4 9 8 9 8 4 6 6
5 5 7 1 4 1 6 4 4 5 3 8 2 1 4 6 1 5 6 6 0 7 2 4 3 3 5 6 2 1
8 2 3 4 2 4 0 4 9 0 4 0 0 2 9 3 4 1 1 2 2 7 8 9 7 4 5 0 1 4
4 5 2 1 7 6 8
```

Uredjeni niz:

```
0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
2 2 2 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5
5 5 5 5 5 6 6 6 6 6 6 6 6 6 7 7 7 7 7 7 8 8 8 8 8 8 8
9 9 9 9 9 9 9
```

Duzina niza? 0

Zadatak 1.6 Izostavljanje suvišnih razmaka među rečima

Napisati na jeziku C++ program kojim se tekst s glavnog ulaza računara prepisuje na glavni izlaz računara uz razdvajanje reči sa po jednim znakom razmaka. Na ulazu reči su razdvojene proizvoljnim brojem znakova razmaka i/ili tabulacije. Tekst se završava signalom za kraj datoteke. Raspodela reči u redove treba da se očuva.

Rešenje:

```
// razmak.C - Izostavljanje suvišnih razmaka među rečima.
```

```
#include <iostream>
using namespace std;

int main() {
    int znak; bool ima = true;

    while ((znak = cin.get()) != EOF) // while (cin.get(znak))
        if (znak != ' ' && znak != '\t')
            { cout.put(znak); ima = znak == '\n'; }
        else if (!ima) { cout.put(' '); ima = true; }
    return 0;
}
```

razmak.pod

```
Ova linija sadrži suvisne znakove razmaka.
U ovoj liniji nema suvisnih znakova razmaka.
===
```

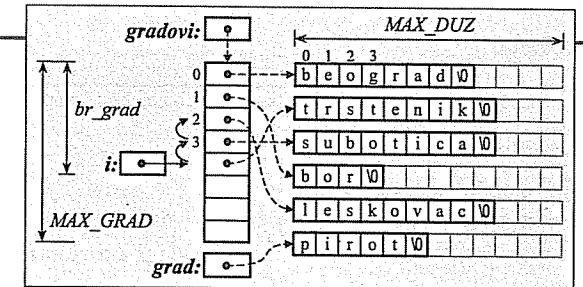
```
% CC razmak.C -o razmak
% razmak <razmak.pod
Ova linija sadrži suvisne znakove razmaka.
U ovoj liniji nema suvisnih znakova razmaka.
===
```

Zadatak 1.7 Uređivanje imena gradova u dinamičkoj matrici

Napisati program na jeziku C++ za čitanje niza imena gradova uz njihovo uređivanje po abecednom redu i ispisivanje rezultata. Iz svakog reda treba da se čita po jedno ime sve dok se ne pročita prazan red. Niz imena smestiti u dinamičku zonu memorije.

Rešenje:

```
// gradovi.C - Uređivanje
// imena gradova smeštenih
// u dinamičku matricu.
#include <string.h>
#include <iostream>
using namespace std;
const int MAX_GRAD = 100;
const int MAX_DUZ = 30;
int main() {
    // Čitanje i obrada pojedinačnih imena:
    cout << "\nNeuredjeni niz imena gradova:\n\n";
    char** gradovi = new char* [MAX_GRAD];
    int br_grad=0;
    do {
        char* grad = new char [MAX_DUZ];
        cin.getline(grad, MAX_DUZ); // Čitanje sledećeg imena.
        int duz = strlen(grad);
        // Kraj ako je dužina imena nula:
        if (!duz) { delete [] grad; break; }
        char* pom = new char [duz+1]; // Skraćivanje niza.
        strcpy(pom, grad);
        delete [] grad; grad = pom;
        cout << grad << endl; // Ispisivanje imena.
        // Uvrštavanje novog imena
        // u uređeni niz starih imena:
        int i;
        for (i=br_grad-1; i>=0; i--)
            if (strcmp(gradovi[i], grad) > 0)
                gradovi[i+1] = gradovi[i];
            else break;
        gradovi[i+1] = grad;
        // Nastavak rada ako ima još
        // slobodnih vrsta u matrici:
    } while (++br_grad < MAX_GRAD);
    // Skraćivanje niza pokazivača:
    char** pom = new char* [br_grad];
    for (int i=0; i<br_grad; i++)
        pom[i] = gradovi[i];
    delete [] gradovi; gradovi = pom;
    // Ispisivanje uređenog niza imena:
    cout << "\nUredjeni niz imena gradova:\n\n";
    for (int i=0; i<br_grad; i++) cout<<gradovi[i++]<<endl;
    // Uništavanje matrice.
    for (int i=0; i<br_grad; delete gradovi[i++]);
    delete [] gradovi;
    return 0;
}
```



gradovi.pod

```
beograd
trstenik
subotica
bor
leskovac
pirot
```

% gradovi <gradovi.pod

Neuredjeni niz imena gradova:

```
beograd
trstenik
subotica
bor
leskovac
pirot
```

Uredjeni niz imena gradova:

```
beograd
bor
leskovac
pirot
subotica
trstenik
```

Zadatak 1.8 Određivanje polarnih koordinata tačke

Napisati na jeziku C++ funkcije za pretvaranje pravouglih koordinata u ravni u polarne koordinate korišćenjem pokazivača i upućivača.

Napisati na jeziku C++ glavnu funkciju za ispitivanje prethodnih funkcija.

Rešenje:

```
// polar.C - Određivanje polarnih koordinata tačke.

#include <math.h>

// Pomoću pokazivača.
void polar(double x, double y, double* r, double* fi) {
    *r = sqrt(x*x + y*y);
    *fi = (x==0 && y==0) ? 0 : atan2(y, x);
}

// Pomoću upućivača.
void polar(double x, double y, double& r, double& fi) {
    r = sqrt(x*x + y*y);
    fi = (x==0 && y==0) ? 0 : atan2(y, x);
}

#include <iostream>
using namespace std;

int main() {
    while (true) {
        double x, y; cout << "\nx,y? "; cin >> x >> y;
        if (x == 1e38) break;
        double r, fi;
        polar(x, y, &r, &fi);
        cout << "r,fi: " << r << ' ' << fi << endl;
        polar(x, y, r, fi);
        cout << "r,fi: " << r << ' ' << fi << endl;
    }
    return 0;
}
```

```
% polar.C -o polar
% polar

x,y? 1 1
r,fi: 1.41421 0.785398
r,fi: 1.41421 0.785398

x,y? -3 4
r,fi: 5 2.2143
r,fi: 5 2.2143

x,y? 1e38 0
```

Zadatak 1.9 Izračunavanje površine trougla

Napisati na jeziku C++ funkcije za izračunavanje površine opšteg, jednakokrakog i jednakostraničnog trougla koji je zadat dužinama stranica.

Napisati na jeziku C++ glavnu funkciju za ispitivanje prethodnih funkcija.

Rešenje:

```
// trougaol.C - Izračunavanje površine trougla.

#include <math.h>
#include <iostream>
using namespace std;

// Opšti trougao.
double P(double a, double b, double c) {
    if (a>0 && b>0 && c>0 && a+b>c && b+c>a && c+a>b) {
        double s = (a + b + c) / 2;
        return sqrt(s * (s-a) * (s-b) * (s-c));
    } else return -1;
}

// Jednakokraki trugao.
inline double P(double a, double b) { return P(a, b, b); }

// Jednakostranični trougao.
inline double P(double a) { return P(a, a, a); }

// Glavna funkcija.
int main() {
    while (true) {
        cout << "Vrsta (1, 2, 3, 0)? "; int k; cin >> k;
        if (k<1 || k>3) break;
        double a, b, c, s;
        switch (k) {
            case 1: cout << "a? "; cin >> a;
                    s = P(a); break;
            case 2: cout << "a,b? "; cin >> a >> b;
                    s = P(a, b); break;
            case 3: cout << "a,b,c? "; cin >> a >> b >> c;
                    s = P(a, b, c); break;
        }
        if (s > 0) cout << "P= " << s << endl;
        else cout << "Greska!\n";
    }
    return 0;
}
```

```
% CC trougaol.C -o trougaol
% trougaol
Vrsta (1, 2, 3, 0)? 1
a? 2
P= 1.73205
Vrsta (1, 2, 3, 0)? 2
a,b? 1 2
P= 0.968246
Vrsta (1, 2, 3, 0)? 3
```

```
a,b,c? 3 4 5
P= 6
Vrsta (1, 2, 3, 0)? 3
a,b,c? 1 2 4
Greska!
Vrsta (1, 2, 3, 0)? 2
a,b? 2 1
Greska!
Vrsta (1, 2, 3, 0)? 0
```


Zadatak 1.10 Paket funkcija za obradu redova brojeva neograničenog kapaciteta

Red celih brojeva neograničenog kapaciteta predstavlja se pomoću strukture koja sadrži pokazivače na početak i na kraj reda. Napisati na jeziku C++ paket funkcija za obradu redova celih brojeva.

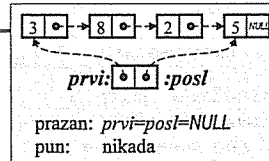
Napisati na jeziku C++ program za ispitivanje prethodnih funkcija.

Rešenje:

```
// redl.h - Deklaracija paketa za obradu
//          redova celih brojeva.
```

```
struct Elem { int broj; Elem* sled; };
struct Red { Elem *prvi, *posl; };
```

```
inline Red pravi()                // Stvaranje praznog reda.
{ Red r; r.prvi = r.posl = 0; return r; }
inline bool prazan(const Red& r)   // Da li je prazan?
{ return r.prvi == 0; }
void dodaj(Red& r, int b);         // Dodavanje broja na kraj reda.
int uzmi(Red& r);                 // Uzimanje broja s početka reda.
int duz(const Red& r);            // Određivanje dužine reda.
void pisi(const Red& r);          // Ispisivanje reda.
void brisi(Red& r);              // Pražnjenje reda.
```



```
// redl.C - Definicije paketa za obradu redova celih brojeva.
```

```
#include "redl.h"
#include <stdlib.h>
#include <iostream>
using namespace std;
```

```
void dodaj(Red& r, int b) {          // Dodavanje broja na kraj reda.
    Elem* novi = new Elem;
    novi->broj = b; novi->sled = 0;
    if (!r.prvi) r.prvi = novi; else r.posl->sled = novi;
    r.posl = novi;
}
```

```
int uzmi(Red& r) {                  // Uzimanje broja s početka reda.
    if (!r.prvi) exit(1);
    int b = r.prvi->broj;
    Elem* stari = r.prvi;
    r.prvi = r.prvi->sled;
    if (!r.prvi) r.posl = 0;
    delete stari;
    return b;
}
```

```
int duz(const Red& r) {             // Određivanje dužine reda.
    int d = 0;
    for (Elem* tek=r.prvi; tek; tek=tek->sled) d++;
    return d;
}
```

```
void pisi(const Red& r) {           // Ispisivanje reda.
    for (Elem* tek=r.prvi; tek; tek=tek->sled)
        cout << tek->broj << ' ';
}
```

```
void brisi(Red& r) {                // Pražnjenje reda.
    while (r.prvi) {
        Elem* stari = r.prvi; r.prvi=r.prvi->sled; delete stari;
    }
    r.posl = 0;
}
```

```
// redlt.C - Ispitivanje paketa za obradu redova celih brojeva.
```

```
#include "redl.h"
#include <iostream>
using namespace std;
```

```
int main() {
    Red r = pravi();
    for (bool dalje=true; dalje; ) {
        cout << "\n1 Dodaj broj      4 Pisi red\n"
                "2 Uzmi broj        5 Brisi red\n"
                "3 Uzmi duzinu       0 Zavrshi\n\n"
                "Vas izbor? ";
        int izb; cin >> izb;
        switch (izb) {
            case 1: int b; cout << "Broj? "; cin >> b; dodaj(r, b); break;
            case 2: if (!prazan(r)) cout << "Broj= " << uzmi(r) << endl;
                    else cout << "Red je prazan!\n"; break;
            case 3: cout << "Duzina= " << duz(r) << endl; break;
            case 4: cout << "Red= "; pisi(r); cout << endl; break;
            case 5: brisi(r); break;
            case 0: dalje = false; break;
            default: cout << "Nedozvoljen izbor!\n"; break;
        }
    }
    return 0;
}
```

```
% CC redlt.C redl.C -o redlt
% redlt

1. Dodaj broj      4. Pisi red
2. Uzmi broj      5. Brisi red
3. Uzmi duzinu    0. Zavrshi
```

```
Vas izbor? 1
Broj? 1
```

```
Vas izbor? 1
Broj? 2
```

```
Vas izbor? 1
Broj? 3
```

```
...
```

```
Vas izbor? 4
Red= 1 2 3
```

```
Vas izbor? 2
Broj= 1
```

```
Vas izbor? 2
Broj= 2
```

```
Vas izbor? 2
Broj= 3
```

```
Vas izbor? 2
Red je prazan!
```

```
Vas izbor? 0
```

Zadatak 1.11 Paketi funkcija za obradu tačaka, pravougaonika i nizova pravougaonika u ravni

Tačka u ravni predstavlja se strukturom koja sadrži koordinate tačke. Napisati na jeziku C++ paket funkcija za obradu tačaka koji sadrži funkcije za sastavljanje tačke od njenih koordinata (podrazumevano (0,0)), čitanje tačke s glavnog ulaza i ispisivanje tačke na glavnom izlazu.

Pravougaonik u ravni s ivicama paralelnim koordinatnim osama predstavlja se strukturom koja sadrži dve tačke koje čine naspramna temena pravougaonika. Napisati na jeziku C++ paket funkcija za obradu pravougaonika koji sadrži funkcije za sastavljanje pravougaonika na osnovu dva temena (podrazumevano (0,0) i (1,1)), čitanje pravougaonika s glavnog ulaza, ispisivanje pravougaonika na glavnom izlazu i izračunavanje površine pravougaonika.

Dinamički niz pravougaonika u ravni predstavlja se strukturom koja sadrži dužinu niza i pokazivač na elemente niza. Napisati na jeziku C++ paket funkcija za obradu nizova pravougaonika koji sadrži funkcije za sastavljanje praznog niza, čitanje niza s glavnog ulaza, ispisivanje niza na glavnom izlazu, uništavanje niza, kopiranje niza i uređivanje niza po površinama pravougaonika u nizu.

Napisati na jeziku C++ program za ispitivanje prethodnih paketa funkcija.

Rešenje:

```
// tacka1.h - Deklaracije paketa za obradu tačaka u ravni.
#include <iostream>
using namespace std;

namespace Geometr {
    struct Tacka { double x, y; };

    inline void pravi(Tacka& T, double x=0, double y=0) { T.x = x; T.y = y; }

    inline void citaj(Tacka& T) { cin >> T.x >> T.y; }

    inline void pisi(const Tacka& T)
    { cout << '(' << T.x << ', ' << T.y << ')'; }

    const Tacka ORG = {0, 0}, JED = {1, 1};
}
```

```
// pravoug1.h - Deklaracije paketa za obradu pravougaonika u ravni.
#include "tacka1.h"
#include <iostream>
#include <cmath>
using namespace std;

namespace Geometr {
    struct Pravoug { Tacka A, C; };

    inline void pravi(Pravoug& p, Tacka A=ORG, Tacka C=JED)
    { p.A = A; p.C = C; }

    inline void pravi(Pravoug& p, double xA, double yA, double xC, double yC)
    { pravi(p.A, xA, yA); pravi(p.C, xC, yC); }

    inline void citaj(Pravoug& p) { citaj(p.A); citaj(p.C); }

    inline void pisi(const Pravoug& p)
    { cout << '['; pisi(p.A); cout << ', ' << pisi(p.C); cout << ']' << endl; }

    inline double P(const Pravoug& p)
    { return fabs(p.A.x-p.C.x) * fabs(p.A.y-p.C.y); }
}
```

```
// nizpravoug.h - Deklaracije paketa za obradu
// dinamičkih nizova pravougaonika u ravni.

#include "pravoug1.h"
#include <iostream>
using namespace std;

namespace Geometr {
    struct NizPrav { int n; Pravoug* a; };

    inline void pravi(NizPrav& niz) { niz.n = 0; niz.a = 0; }

    void citaj(NizPrav& niz);

    void pisi(const NizPrav& niz);

    void brisi(NizPrav& niz);

    void kopiraj(NizPrav& niz1, const NizPrav& niz2);

    void uredi(NizPrav& niz);
}
```

```
// nizpravoug.C - Definicije paketa za obradu
// dinamičkih nizova pravougaonika u ravni.

#include "nizpravoug.h"

void Geometr::citaj(NizPrav& niz) {
    cout << "Dužina niza? "; cin >> niz.n;
    if (niz.n > 0) {
        niz.a = new Pravoug [niz.n];
        for (int i=0; i<niz.n; i++) {
            cout << "Pravoug[" << i << "] (xA,yA,xC,yC)? ";
            citaj(niz.a[i]);
        }
    } else { niz.n = 0; niz.a = 0; }
}

void Geometr::pisi(const NizPrav& niz) {
    for (int i=0; i<niz.n; i++) {
        cout << "Pravoug[" << i << "] = "; pisi(niz.a[i]);
        cout << ' ' << P(niz.a[i]) << endl;
    }
}

void Geometr::brisi(NizPrav& niz) { delete [] niz.a; niz.a = 0; niz.n = 0; }

void Geometr::kopiraj(NizPrav& niz1, const NizPrav& niz2) {
    brisi(niz1);
    if (niz2.n > 0) {
        niz1.a = new Pravoug [niz1.n = niz2.n];
        for (int i=0; i<niz1.n; i++) niz1.a[i] = niz2.a[i];
    }
}

void Geometr::uredi(NizPrav& niz) {
    for (int i=0; i<niz.n-1; i++)
        for (int j=i+1; j<niz.n; j++)
            if (P(niz.a[j]) < P(niz.a[i]))
                { Pravoug p = niz.a[i]; niz.a[i] = niz.a[j]; niz.a[j] = p; }
}
```

```
// pravouglt.C - Ispitivanje paketa za obradu pravougaonika u ravni.
```

```
#include "nizpravoug.h"
#include <iostream>
using namespace std;
using namespace Geometr;

int main() {
    while (true) {
        NizPrav niz;
        cout << endl; citaj(niz);
        if (niz.a == 0) break;
        uredi(niz);
        cout << endl; pisi(niz);
        brisi(niz);
    }
    return 0;
}
```

```
% CC pravouglt.C nizpravoug.C -o pravouglt
% pravouglt
```

```
Duzina niza? 4
Pravoug[0] (xA,yA,xC,yC)? 1 1 2 4
Pravoug[1] (xA,yA,xC,yC)? 0 1 -2 2
Pravoug[2] (xA,yA,xC,yC)? 0 0 2 -2
Pravoug[3] (xA,yA,xC,yC)? -1 -1 -2 -2
```

```
Pravoug[0] = [(-1,-1),(-2,-2)] 1
Pravoug[1] = [(0,1),(-2,2)] 2
Pravoug[2] = [(1,1),(2,4)] 3
Pravoug[3] = [(0,0),(2,-2)] 4
```

```
Duzina niza? 0
```

2 Klase

Zadatak 2.1 Tačke u ravni

Napisati na jeziku C++ klasu tačka u ravni. Predvideti: postavljanje i dohvaćanje koordinata, izračunavanje rastojanja do zadate tačke, čitanje tačke i pisanje tačke.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:

```
// tacka2.h - Definicija klase tačka u ravni.
class Tacka {
    double x, y;           // Koordinate.
public:
    void postavi(double a, double b) // Postavljanje
    { x = a; y = b; }           // koordinata.
    double aps() const { return x; } // Apscisa.
    double ord() const { return y; } // Ordinata.
    double rastojanje(Tacka) const; // Rastojanje do tačke.
    void citaj();              // Čitanje tačke.
    void pisi() const;         // Pisanje tačke.
};
```

Tacka
- x, y
+ postavi()
+ aps()
+ ord()
+ rastojanje()
+ citaj()
+ pisi()

// Definicije ugrađenih metoda izvan klase.

```
#include <iostream>
using namespace std;

inline void Tacka::citaj() // Čitanje tačke.
{ cin >> x >> y; }

inline void Tacka::pisi() const // Pisanje tačke.
{ cout << '(' << x << ', ' << y << ')'; }
```

// tacka2.C - Definicije metoda klase tačka u ravni.

```
#include "tacka2.h"
#include <cmath>
using namespace std;

double Tacka::rastojanje(Tacka t) const // Rastojanje do tačke.
{ return sqrt(pow(x-t.x,2) + pow(y-t.y,2)); }
```

// tacka2t.C - Ispitivanje klase tačka u ravni.

```
#include "tacka2.h"
#include <iostream>
using namespace std;

int main() {
    cout << "t1? "; double x, y; cin >> x >> y;
    Tacka t1; t1.postavi(x, y);
    cout << "t2? "; Tacka t2; t2.citaj();
    cout << "t1=(" << t1.aps() << ', ' << t1.ord()
    << ")", t2=""; t2.pisi(); cout << endl;
    cout << "Rastojanje=" << t1.rastojanje(t2) << endl;
    return 0;
}
```

```
% CC tacka2t.C tacka2.C -o tacka2t
% tacka2t
t1? 1 1
t2? 4 5
t1=(1,1), t2=(4,5)
Rastojanje=5
```

Zadatak 2.2 Uglovi

Napisati na jeziku C++ klasu uglova. Predvideti:

- stvaranje ugla na osnovu zadatog broja radijana (podrazumevano 0),
- stvaranje ugla na osnovu zadatog broja stepeni, minuta i sekundi,
- dohvaćanje veličine ugla u radijanima,
- dohvaćanje delova ugla (celobrojnih stepeni, minuta i sekundi) odvojeno i odjednom,
- dodavanje vrednosti drugog ugla tekućem uglu,
- množenje ugla realnim brojem, i
- čitanje ugla s glavnog ulaza i ispisivanje ugla na glavnom izlazu u radijanima i u stepenima.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:

// ugao.h - Definicija klase uglova.

```
#include <iostream>
using namespace std;

const double FAKTOR = 3.14159265358979323 / 180;

class Ugao {
    double ugao;           // Ugao u radijanima.
public:
    // Konstruktori:
    Ugao(double u=0)        // - podrazumevani i
    { ugao = u; }           // konverzije,
    Ugao(int stp, int min=0, int sek=0) // - na osnovu stepeni.
    { ugao = ((sek/60.+min)/60+stp) * FAKTOR; }

    double rad() const { return ugao; } // Radijani.
    int stp() const      // Stepeni.
    { return ugao / FAKTOR; }

    int min() const      // Minuti.
    { return int(ugao / FAKTOR * 60) % 60; }

    int sek() const      // Sekunde.
    { return int(ugao / FAKTOR * 3600) % 60; }

    void razlozi(int& st, int& mi, int& se) const { // Sva tri dela
    st = stp(); mi = min(); se = sek(); // odjednom.
    }

    Ugao& dodaj(Ugao u) // Dodavanje ugla.
    { ugao += u.ugao; return *this; }

    Ugao& pomnozi(double a) // Množenje realnim brojem.
    { ugao *= a; return *this; }

    void citaj() { cin >> ugao; } // Čitanje u radijanima.
    void citajStep() { // Čitanje u stepenima.
    int stp, min, sek; cin >> stp >> min >> sek;
    *this = Ugao(stp, min, sek);
    }

    void pisi() const { cout << ugao; } // Pisanje u radijanima.
    void pisiStep() const { // Pisanje u stepenima.
    cout << '(' << stp() << ':' << min() << ':' << sek() << ')';
    }
};
```

```
// ugaot.C - Ispitivanje klase uglova.
```

```
#include "ugao.h"
#include <iostream>
using namespace std;

int main() {
    Ugao u1, u2;
    cout << "Prvi ugao [rad]? "; u1.citaj();
    cout << "Drugi ugao [rad]? "; u2.citaj();
    Ugao sr = Ugao(u1).dodaj(u2).pomnozi(0.5);
    cout << "Srednja vrednost= "; sr.pisi();      cout << ' ';
    sr.pisiStep(); cout << endl;

    return 0;
}
```

```
% CC ugaot.C ugao.C -o ugaot
% ugaot
Prvi ugao [rad]? 1
Drugi ugao [rad]? 3
Srednja vrednost= 2 (114:35:29)
```

Zadatak 2.3 Redovi brojeva ograničenih kapaciteta

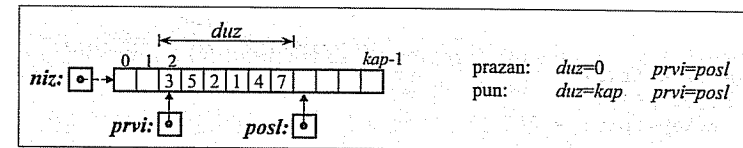
Red je linearna struktura podataka kod kojeg se podaci stavljaju na jednom kraju a uzimaju na drugom kraju. Može da bude ograničenog i neograničenog kapaciteta. Napisati na jeziku C++ klasu redova ograničenih kapaciteta za cele brojeve. Predvideti:

- stvaranje praznog reda sa zadatim kapacitetom i reda kao kopiju drugog reda,
- uništavanje reda,
- dodavanje jednog podatka i uzimanje jednog podatka,
- ispitivanje da li je red pun i da li je prazan,
- pisanje sadržaja reda na glavnom izlazu, *i*
- pražnjenje reda.

Napisati na jeziku C++ interaktivan program za ispitivanje prethodne klase.

Rešenje:

Red: ⇐ 3 5 2 1 4 7 ⇐
 prvi posl



```
// red2.h - Definicija klase redova ograničenih kapaciteta.
```

```
class Red {
    int *niz, kap, duz, prvi, posl;
public:
    explicit Red(int k=10);            // Stvaranje praznog reda (nije konverzija).
    Red(const Red& rd);                // Stvaranje reda od kopije drugog.
    ~Red() { delete [] niz; }            // Uništavanje reda.
    void stavi(int b);                  // Stavljanje broja u red.
    int uzmi();                         // Uzimanje broja iz reda.
    bool prazan() const { return duz == 0; } // Da li je red prazan?
    bool pun() const { return duz == kap; } // Da li je red pun?
    void pisi() const;                  // Pisanje sadržaja reda.
    void prazni() { duz = prvi = posl = 0; } // Pražnjenje reda.
};
```

```
// red2.C - Definicije metoda klase redova ograničenih kapaciteta.
```

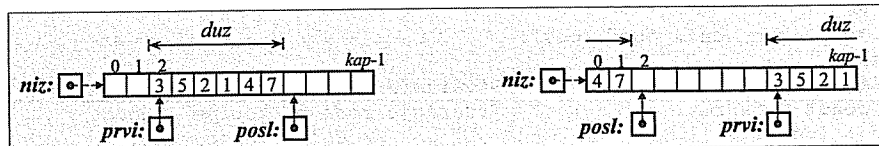
```
#include "red2.h"
#include <iostream>
#include <cstdlib>
using namespace std;

Red::Red(int k) { // Stvaranje praznog reda.
    niz = new int [kap = k];
    duz = prvi = posl = 0;
}

Red::Red(const Red& rd) { // Stvaranje reda od kopije drugog.
    niz = new int [kap = rd.kap];
    for (int i=0; i<kap; i++) niz[i] = rd.niz[i];
    duz = rd.duz; prvi = rd.prvi; posl = rd.posl;
}

void Red::stavi(int b) { // Stavljanje broja u red.
    if (duz == kap) exit(1);
    niz[posl++] = b;
    if (posl == kap) posl = 0;
    duz++;
}

int Red::uzmi() { // Uzimanje broja iz reda.
    if (duz == 0) exit(2);
    int b = niz[prvi++];
    if (prvi == kap) prvi = 0;
    duz--;
    return b;
}
```



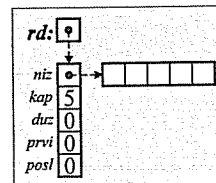
```
void Red::pisi() const { // Pisanje sadržaja reda.
    for (int i=0; i<duz; cout<<niz[(prvi+i)%kap]<<' ');
}
```

```
// red2t.C - Ispitivanje klase redova ograničenih kapaciteta.
```

```
#include "red2.h"
#include <iostream>
using namespace std;

int main() {
    Red* rd = new Red(5); bool kraj = false;
    while (!kraj) {
        cout << "\n1. Stvaranje reda\n"
                "2. Stavljanje podatka u red\n"
                "3. Uzimanje podatka iz reda\n"
                "4. Ispisivanje sadržaja reda\n"
                "5. Praznjenje reda\n"
                "0. Zavrsetak rada\n\n"
                "Vas izbor? ";

```



```
int izbor; cin >> izbor;
switch (izbor) {
case 1: // Stvaranje novog reda:
    cout << "Kapacitet? "; int k; cin >> k;
    if (k > 0) { delete rd; rd = new Red(k); }
    else cout << "*** Nedozvoljen kapacitet! ***\n";
    break;
case 2: // Stavljanje podatka u red:
    if (!rd->pun()) {
        cout << "Broj? "; int b; cin >> b; rd->stavi(b);
    } else cout << "*** Red je pun! ***\n";
    break;
case 3: // Uzimanje podatka iz reda:
    if (!rd->prazan())
        cout << "Broj= " << rd->uzmi() << endl;
    else cout << "*** Red je prazan! ***\n";
    break;
case 4: // Ispisivanje sadržaja reda:
    cout << "Red= "; rd->pisi(); cout << endl;
    break;
case 5: // Praznjenje reda:
    rd->prazni(); break;
case 0: // Zavrsetak rada:
    kraj = true; break;
default: // Pogrešan izbor:
    cout << "*** Nedozvoljen izbor! ***\n"; break;
}
return 0;
}
```

```
% CC red2t.C red2.C -o red2t
% red2t
```

1. Stvaranje reda
2. Stavljanje podatka u red
3. Uzimanje podatka iz reda
4. Ispisivanje sadržaja reda
5. Praznjenje reda
0. Zavrsetak rada

```
Vas izbor? 1
Kapacitet? 3
```

```
Vas izbor? 2
Broj? 1
```

```
Vas izbor? 2
Broj? 2
```

```
Vas izbor? 2
Broj? 3
```

```
Vas izbor? 2
Broj? 3
```

```
Vas izbor? 2
*** Red je pun! ***
```

```
Vas izbor? 4
Red= 1 2 3
```

```
Vas izbor? 3
Broj= 1
```

```
Vas izbor? 3
Broj= 2
```

```
Vas izbor? 3
Broj= 3
```

```
Vas izbor? 3
*** Red je prazan! ***
```

```
Vas izbor? 55
*** Nedozvoljen izbor! ***
```

```
Vas izbor? 0
```

Zadatak 2.4 Uređeni skupovi brojeva

Napisati na jeziku C++ klasu uređenih skupova realnih brojeva. Predvideti:

- stvaranje praznog skupa, skupa koji sadrži jedan broj i kao kopiju drugog skupa,
- uništavanje skupa,
- nalaženje unije, preseka i razlike dva skupa,
- čitanje skupa s glavnog ulaza,
- ispisivanje skupa na glavnom izlazu, i
- dohvaćanje broja elemenata skupa.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:

```
// skup1.h - Definicija klase uređenih skupova.

class Skup {
    int vel; double* niz;           // Veličina i elementi skupa.
    void kopiraj(const Skup&);      // Kopiranje skupa.
    void brisi() { delete [] niz; niz = 0; vel = 0; } // Pražnjenje skupa.
public:
    Skup() { niz = 0; vel = 0; }    // Stvaranje praznog skupa.
    Skup(double a) {                 // Konverzija broja u skup.
        niz = new double [vel = 1];
        niz[0] = a;
    }
    Skup(const Skup& s) { kopiraj(s); } // Inicijalizacija skupom.
    ~Skup() { brisi(); }             // Uništavanje skupa.
    void unija (const Skup&, const Skup&); // Unija dva skupa.
    void presek (const Skup&, const Skup&); // Presek dva skupa.
    void razlika(const Skup&, const Skup&); // Razlika dva skupa.
    void pisi () const;              // Pisanje skupa.
    void citaj();                   // Čitanje skupa.
    int velicina() const { return vel; } // Veličina skupa.
};
```

```
// skup1.C - Definicije metoda klase uređenih skupova.

#include "skup1.h"
#include <iostream>
using namespace std;

void Skup::kopiraj(const Skup& s) { // Kopiranje skupa.
    niz = new double [vel = s.vel];
    for (int i=0; i<vel; i++) niz[i] = s.niz[i];
}

void Skup::unija (const Skup& s1, const Skup& s2) { // Unija dva skupa.
    Skup s;
    s.niz = new double [s1.vel+s2.vel]; s.vel = 0;
    for (int i=0, j=0; i<s1.vel || j<s2.vel; )
        s.niz[s.vel++] = i==s1.vel ? s2.niz[j++] :
                           j==s2.vel ? s1.niz[i++] :
                           s1.niz[i]<s2.niz[j] ? s1.niz[i++] :
                           s1.niz[i]>s2.niz[j] ? s2.niz[j++] :
                           (j++, s1.niz[i++]);
    brisi(); kopiraj(s);
}

void Skup::presek(const Skup& s1, const Skup& s2) { // Presek dva skupa.
    Skup s;
    s.niz = new double [s1.vel<s2.vel ? s1.vel : s2.vel]; s.vel = 0;
    for (int i=0, j=0; i<s1.vel && j<s2.vel; )
        if (s1.niz[i] < s2.niz[j]) i++;
        else if (s1.niz[i] > s2.niz[j]) j++;
        else s.niz[s.vel++] = s1.niz[i++], j++;
    brisi(); kopiraj(s);
}

void Skup::razlika(const Skup& s1, const Skup& s2) { // Razlika dva skupa.
    Skup s;
    s.niz = new double [s1.vel]; s.vel = 0;
    for (int i=0, j=0; i<s1.vel; )
        if (j == s2.vel) s.niz[s.vel++] = s1.niz[i++];
        else if (s1.niz[i] < s2.niz[j]) s.niz[s.vel++] = s1.niz[i++];
        else if (s1.niz[i] > s2.niz[j]) j++;
        else i++, j++;
    brisi(); kopiraj(s);
}

void Skup::pisi() const { // Pisanje skupa.
    cout << '{';
    for (int i=0; i<vel; i++) { cout << niz[i]; if (i < vel-1) cout << ','; }
    cout << '>';
}

void Skup::citaj() { // Čitanje skupa.
    brisi();
    int vel; cin >> vel;
    for (int i=0; i<vel; i++) { double broj; cin>>broj; unija(*this, broj); }
}
```

```
// skuplt.C - Ispitivanje klase uredenih skupova.
```

```
#include "skupl.h"
#include <iostream>
using namespace std;

int main() {
    char jos;
    do {
        Skup s1; cout << "niz1? "; s1.citaj();
        Skup s2; cout << "niz2? "; s2.citaj();
        cout << "s1  ="; s1.pisi(); cout << endl;
        cout << "s2  ="; s2.pisi(); cout << endl;
        Skup s; s.unija (s1, s2); cout << "s1+s2="; s.pisi(); cout << endl;
        s.presek (s1, s2); cout << "s1*s2="; s.pisi(); cout << endl;
        s.razlika(s1, s2); cout << "s1-s2="; s.pisi(); cout << endl;
        cout << "\nJos? "; cin >> jos;
    } while (jos=='d' || jos=='D');
    return 0;
}
```

```
% CC skupl.C skuplt.C -o skuplt
```

```
% skuplt
niz1? 4   1 2 3 4
niz2? 5   3 4 5 6 7
s1  = {1,2,3,4}
s2  = {3,4,5,6,7}
s1+s2={1,2,3,4,5,6,7}
s1*s2={3,4}
s1-s2={1,2}

Jos? d
niz1? 6   9 3 5 1 7 3
niz2? 4   6 2 8 8
s1  = {1,3,5,7,9}
s2  = {2,6,8}
s1+s2={1,2,3,5,6,7,8,9}
s1*s2={}
s1-s2={1,3,5,7,9}

Jos? n
```

Zadatak 2.5 Trouglovi u ravni

Napisati na jeziku C++ klasu trouglova. Predvideti: ispitivanje da li tri duži mogu biti stranice trougla, stvaranje trougla, dohvatanje dužina stranica, izračunavanje obima i površine, čitanje trougla s glavnog ulaza i ispisivanje trougla na glavnom izlazu.

Napisati na jeziku C++ program koji pročita dinamički niz trouglova, uredi niz po neopadajućem redosledu površina trouglova i ispiše dobijeni rezultat.

Rešenje:

```
// trougao2.h - Definicija klase trouglova.
```

```
#include <cstdlib>
using namespace std;

class Trougao {
    double a, b, c; // Stranice trougla.
public:
    static bool moze(double a, double b, double c) { // Da li su stranice
        return a>0 && b>0 && c>0 && // prihvatljive?
            a+b>c && b+c>a && c+a>b;
    }
    Trougao(double aa=1, double bb=1, double cc=1) { // Postavljanje
        if (!moze(aa, bb, cc)) exit(1); // koordinata.
        a = aa; b = bb; c = cc;
    }
    double dohvA() const { return a; } // Dohvatanje stranica.
    double dohvB() const { return b; }
    double dohvC() const { return c; }
    double O() const { return a + b + c; } // Obim trougla.
    double P() const; // Površina trougla.
    bool citaj(); // Čitanje trougla.
    void pisi() const; // Pisanje trougla.
};
```

```
// trougao2.C - Definicije metoda klase trouglova.
```

```
#include "trougao2.h"
#include <iostream>
#include <cmath>
using namespace std;

double Trougao::P() const { // Površina trougla.
    double s = O() / 2;
    return sqrt(s * (s-a) * (s-b) * (s-c));
}

bool Trougao::citaj() { // Čitanje trougla.
    double aa, bb, cc;
    cin >> aa >> bb >> cc;
    if (!moze(aa, bb, cc)) return false;
    a = aa; b = bb; c = cc;
    return true;
}

void Trougao::pisi() const { // Pisanje trougla.
    cout << "Troug(" << a << ', ' << b << ', ' << c << ')';
}
```



```
// trougao2t.C - Ispitivanje klase trouglova.

#include "trougao2.h"
#include <iostream>
using namespace std;

int main() {
    cout << "Broj trouglova? "; int n; cin >> n;
    Trougao* niz = new Trougao [n];
    for (int i=0; i<n; ) {
        cout << i+1 << ". trougao? ";
        double a, b, c; cin >> a >> b >> c;
        if (Trougao::moze(a,b,c)) niz[i++] = Trougao(a, b, c);
        else cout << "*** Neprihvatljive stranice! ***\n";
    }
    for (int i=0; i<n-1; i++)
        for (int j=i+1; j<n; j++)
            if (niz[j].P() < niz[i].P())
                { Trougao pom = niz[i]; niz[i] = niz[j]; niz[j] = pom; }
    cout << "\nUredjeni niz trouglova:\n";
    for (int i=0; i<n; i++) {
        cout << i+1 << ": "; niz[i].pisi();
        cout << " P=" << niz[i].P() << endl;
    }
    delete [] niz;
    return 0;
}
```

```
% CC trougao2.C trougao2t.C -o trougao2t
% trougao2t
Broj trouglova? 5
1. trougao? 1 1 1
2. trougao? 3 4 5
3. trougao? 1 2 2
4. trougao? 1 3 5
*** Neprihvatljive stranice! ***
4. trougao? 2 3 4
5. trougao? 5 6 7

Uredjeni niz trouglova:
1: Troug(1,1,1) P=0.433013
2: Troug(1,2,2) P=0.968246
3: Troug(2,3,4) P=2.90474
4: Troug(3,4,5) P=6
5: Troug(5,6,7) P=14.6969
```

Zadatak 2.6 Kvadri u dinamičkoj zoni memorije

Napisati na jeziku C++ klasu kvadara čiji svi primeri moraju biti u dinamičkoj zoni memorije. Ukupna zapremina svih postojećih kvadara nijednog trenutka ne sme da pređe određenu najveću vrednost. Predvideti:

- postavljanje i dohvatanje vrednosti dozvoljene ukupne zapremine svih kvadara,
- dohvatanje trenutne ukupne zapremine svih kvadara,
- stvaranje kvadra zadatih dužina ivica,
- čitanje kvadra s glavnog ulaza,
- dohvatanje dužina ivica kvadra,
- izračunavanje zapremine kvadra, i
- ispisivanje kvadra na glavnom izlazu.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:

```
// kvadar1.h - Definicija klase kvadara.

#include <iostream>
using namespace std;

class Kvarar {
    static double Vmax, Vuk; // Dozvoljena i trenutna ukupna zapremina.
    double a, b, c; // Dužine ivica kvadra.

    Kvarar(double aa, double bb, double cc) // Konstruktor samo za interne
        { a = aa; b = bb; c = cc; } // potrebe.

    Kvarar(const Kvarar&) {} // Ne sme da se kopira.

public:
    static bool postaviVmax(double maxV) { // Postavljanje maksimalne
        if (maxV < Vuk) return false; // zapremine.
        Vmax = maxV;
        return true;
    }

    static double dohvVmax() // Dohvatanje maksimalne
        { return Vmax; } // zapremine.

    static double dohvVuk() // Dohvatanje trenutne
        { return Vuk; } // ukupne zapremine.

    static Kvarar* pravi(double a, double b, double c) { // Stvaranje
        double V = a * b * c; // dinamičkog kvadra.
        if (a<=0 || b<=0 || c<=0 || Vuk+V>Vmax) return 0;
        Vuk += V; return new Kvarar(a, b, c);
    }

    static Kvarar* citaj() { // Čitanje kvadra.
        double a, b, c; cin >> a >> b >> c;
        return pravi(a, b, c);
    }

    ~Kvarar() { Vuk -= a * b * c; } // Uništavanje kvadra.
    double dohvA() const { return a; } // Dohvatanje dužina ivica.
    double dohvB() const { return b; }
    double dohvC() const { return c; }

    double V() const { return a * b * c; } // Zapremina kvadra.

    void pisi() const // Pisanje kvadra.
        { cout << "kvadar[" << a << ',' << b << ',' << c << ']' << endl; }
};
```

```
// kvadar1.C - Definicije statičkih polja klase kvadara.
```

```
#include "kvadar1.h"
```

```
double Kvadar::Vmax = 0, Kvadar::Vuk = 0;
```

```
// kvadar1t.C - Ispitivanje klase kvadara.
```

```
#include "kvadar1.h"
```

```
#include <iostream>
```

```
#include <cstdlib>
```

```
using namespace std;
```

```
int main(int, char* varg[]) {
    Kvadar::postaviVmax(atoi(varg[1]));
```

```
    struct Elem {                // Element liste kvadara.
```

```
        Kvadar* kvad; Elem* sled;
```

```
        Elem(Kvadar* kv) { kvad = kv; sled = 0; }
```

```
        ~Elem() { delete kvad; }
```

```
};
```

```
for (char jos='d'; jos=='d' || jos=='D'; cout<<"\nJos? ", cin>>jos) {
    Elem *prvi = 0, *posl = 0;
```

```
    while (true) {                // Čitanje kvadara i stvaranje liste.
```

```
        cout << "a,b,c? ";
```

```
        if (Kvadar* kv = Kvadar::citaj())
```

```
            posl = (!prvi ? prvi : posl->sled) = new Elem(kv);
```

```
        else break;
```

```
    }
```

```
    cout << "Zapremine: ";        // Zapremine pročitanih kvadara.
```

```
    for (Elem* tek=prvi; tek; tek=tek->sled) cout << ' ' << tek->kvad->V();
```

```
    cout << "\nUkupno: " << Kvadar::dohvVuk() << endl;
```

```
                                // Uništavanje liste kvadara.
```

```
    while (prvi) { Elem* stari = prvi; prvi = prvi->sled; delete stari; }
```

```
    cout << "Ukupno posle brisanja: " << Kvadar::dohvVuk() << endl;
```

```
    }
    return 0;
}
```

```
% CC kvadar1.C kvadar1t.C -o kvadar1t
```

```
% kvadar1t 100
```

```
a,b,c? 1 2 3
```

```
a,b,c? 2 3 4
```

```
a,b,c? 3 4 5
```

```
a,b,c? 4 5 6
```

```
Zapremine: 6 24 60
```

```
Ukupno: 90
```

```
Ukupno posle brisanja: 0
```

```
Jos? d
```

```
a,b,c? 1 1 1
```

```
a,b,c? 2 2 2
```

```
a,b,c? 3 3 3
```

```
a,b,c? 0 0 0
```

```
Zapremine: 1 8 27
```

```
Ukupno: 36
```

```
Ukupno posle brisanja: 0
```

```
Jos? n
```

Zadatak 2.7 Krugovi u ravni koji ne smeju da se preklapaju

Napisati na jeziku C++ klasu krugova u ravni koji ne smeju da se preklapaju. Predvideti:

- proveravanje da li krug zadatog poluprečnika i koordinata centra sme da postoji,
- stvaranje kruga zadatog poluprečnika i koordinata centra,
- uništavanje kruga,
- izračunavanje rastojanja najbližih tačaka dva kruga,
- premeštanje kruga na drugo mesto u ravni i pomeranje kruga za određeni pomak,
- ispisivanje kruga na glavnom izlazu, i
- ispisivanje svih krugova na glavnom izlazu.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:

```
// krug1.h - Definicija klase krugova
// u ravni.
```

```
#include "tacka2.h" → Zadatak 2.1
```

```
#include <cmath>
```

```
#include <iostream>
```

```
using namespace std;
```

```
class Krug {
```

```
    Tacka c; double r;
```

```
    Krug *pret, *sled;
```

```
    static Krug* prvi;
```

```
    Krug() { r = -1; }
```

```
    Krug(const Krug&) {}
```

```
public:
```

```
    static bool moze(double r, double x, double y); // Može li postojati?
```

```
    Krug(double rr, double x, double y);           // Stvaranje kruga.
```

```
    ~Krug();                                         // Uništavanje kruga.
```

```
    friend double rastojanje(const Krug& k1, const Krug& k2);
```

```
                                                    // Rastojanje dva kruga.
```

```
    bool pomeri(double dx, double dy);             // Premeštanje kruga.
```

```
    void pisi() const;                             // Pisanje kruga.
```

```
    static void pisiSve();                          // Pisanje svih krugova.
```

```
};
```

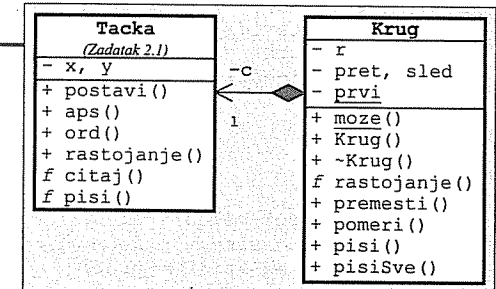
```
// Definicije funkcija za neposredno ugrađivanje u kod.
```

```
inline double rastojanje(const Krug& k1, const Krug& k2)
```

```
{ return k1.c.rastojanje(k2.c) - k1.r - k2.r; } // Rastojanje dva kruga.
```

```
inline void Krug::pisi() const
```

```
{ cout << "K[" << r << ', ' << c.pisi(); cout << ']' << endl; }
```



```
// krug1.C - Definicije metoda i statičkih polja klase krugova.
```

```
#include "krug1.h"
```

```
Krug* Krug::prvi = 0; // Početak zajedničke liste.
```

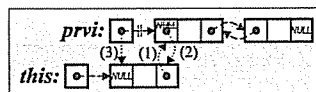
```
bool Krug::moze(double r, double x, double y){ // Može li postojati?
    Krug k; k.r = r; k.c.postavi(x, y);
    Krug* tek = prvi;
    while (tek && rastojanje(k,*tek)>=0) tek=tek->sled;
    k.r = -1;
    return tek == 0;
}
```

```
Krug::Krug(double rr, double x, double y) { // Stvaranje kruga.
```

```

if (!moze(rr, x, y)) exit(1);
r = rr; c.postavi(x, y);
sled = prvi; pret = 0;
if (prvi) prvi->pret = this;
prvi = this;

```

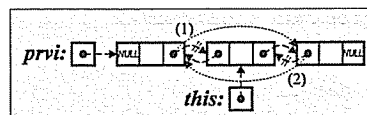


```
Krug::~~Krug() { // Uništavanje kruga (izbacivanje iz liste).
```

```

    if (r > 0) {
        if (pret) pret->sled = sled;
        else prvi = sled;
        if (sled) sled->pret = pret;
    }
}

```



```
bool Krug::premesti(double x, double y) { // Premeštanje kruga.
```

```

    if (!moze(r, x, y)) return false;
    c.postavi(x, y);
    return true;
}

```

```
bool Krug::pomeri(double dx, double dy) { // Pomeranje kruga.
```

```

    if (!moze(r, c.aps()+dx, c.ord()+dy)) return false;
    c.postavi(c.aps()+dx, c.ord()+dy);
    return true;
}

```

```
void Krug::pisiSve() { // Pisanje svih krugova.
```

```
cout << "\nSvi krugovi u memoriji:\n";
for (Krug* tek=prvi; tek; tek=tek->sled)
{ tek->pisi(); cout << endl; }
```

```
// kruglt.C - Ispitivanje klase krugova.
```

```
#include "krug1.h"
```

```
#include <iostream>
using namespace std;
```

```
int main() {
    char jos;
    do {
        Krug* krugovi[100]; int n = 0;
        while (true) {
            cout << "Poluprecnik? "; double r; cin >> r;
            if (r <= 0) break;
            cout << "Koordinate centra? "; double x, y; cin >> x >> y;
            if (Krug::moze(r, x, y)) {
                krugovi[n++] = new Krug(r, x, y);
            } else cout << "*** Ne moze da se smesti! ***\n\a";
        }
        Krug::pisiSve();
        for (int i=0; i<n; delete krugovi[i++]);
        cout << "\nJos? "; cin >> jos;
    } while (jos!='N' && jos!='n');
    return 0;
}
```

```
% CC krug1t.C krug1.C tacka2.C -o krug1t
```

```
% kruglt
Poluprecnik? 1
Koordinate centra? 1 1
Poluprecnik? 2
Koordinate centra? 3 3
*** Ne moze da se smesti! ***
Poluprecnik? 2
Koordinate centra? 4 4
Poluprecnik? 3
Koordinate centra? -1 5
Poluprecnik? 0
```

Svi krugovi u memoriji:
 $K[3, (-1, 5)]$
 $K[2, (4, 4)]$
 $K[1, (1, 1)]$

```

Jos? d
Poluprecnik? 1
Koordinate centra? 1 1
Poluprecnik? 1
Koordinate centra? 3 3
Poluprecnik? 2
Koordinate centra? 4 4
*** Ne moze da se smesti! ***
Poluprecnik? 0

```

Svi krugovi u memoriji:
 $K[1, (3, 3)]$
 $K[1, (1, 1)]$

JOS? n

Zadatak 2.8 Kalendarski datumi

Napisati na jeziku C++ klasu kalendarskih datuma. Predvideti:

- ispitivanje da li je data godina prestupna,
- ispitivanje da li tri cela broja predstavljaju ispravan datum,
- stvaranje datuma,
- dohvaćanje delova datuma,
- čitanje novog datuma s glavnog ulaza,
- pisanje datuma na glavnom izlazu,
- određivanje rednog broja dana u godini počev od 1.1.1. i u toku nedelje u datumu,
- dohvaćanje broja dana u mesecu u datumu,
- dodavanje datumu i oduzimanje od datuma jednog dana i zadatog broja dana,
- određivanje broja dana između dva datuma, *i*
- dohvaćanje imena meseca i imena dana datuma.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:

```
// datum1.h - Definicija klase kalendarskih datuma.

class Datum {
    static const short duz[][12], // Brojevi dana po mesecima.
                        prot[][12]; // Protekli dani od početka godine.
    static const char *imeD[], // Imena dana.
                    *imeM[]; // Imena meseci.
    short dan, mes, god; // Delovi datuma.

public:
    static bool prestupna(short g); // Da li je prestupna godina?
    bool prestupna() const { return prestupna(god); }
    static bool moze(short d, short m, short g); // Da li je ispravan?
    Datum(short d, short m, short g); // Stvaranje datuma.
    short dohvDan() const { return dan; } // Dohvaćanje delova datuma.
    short dohvMes() const { return mes; }
    short dohvGod() const { return god; }
    static Datum citaj(); // Čitanje datuma.
    void pisi() const; // Pisanje datuma.
    int danUGod() const; // Redni broj dana u godini.
    long ukDan() const; // Redni broj dana od 1.1.1.
    int danUNed() const; // Redni broj dana u nedelji.
    int duzMes() const; // Broj dana u mesecu.
    void sutra(); // Seledeći datum.
    void juce(); // Prethodni datum.
    void dodaj(unsigned k); // Dodavanje celog broja.
    void oduzmi(unsigned k); // Oduzimanje celog broja.
    friend long razlika(Datum dat1, Datum dat2); // Razlika dva datuma.
    const char* imeDan() const; // Ime dana.
    const char* imeMes() const; // Ime meseca.
};

// Definicije metoda koje se ugrađuju neposredno u kod.

inline bool Datum::prestupna(short g)
{ return g%4==0 && g%100!=0 || g%400==0; }

inline bool Datum::moze(short d, short m, short g)
{ return g>0 && m>0 && m<=12 && d>0 && d<=duz[prestupna(g)][m-1]; }
```

```
#include <cstdlib>
using namespace std;

inline Datum::Datum(short d, short m, short g) {
    if (!moze(d, m, g)) exit(1);
    dan = d; mes = m; god = g;
}

inline int Datum::danUGod() const
{ return prot[prestupna()][mes-1] + dan; }

inline int Datum::danUNed() const { return (ukDan() + 6) % 7 + 1; }

inline int Datum::duzMes() const { return duz[prestupna()][mes-1]; }

inline long razlika(Datum dat1, Datum dat2)
{ return dat1.ukDan() - dat2.ukDan(); }

inline const char* Datum::imeMes() const { return imeM[mes-1]; }

inline const char* Datum::imeDan() const { return imeD[danUNed()-1]; }
```

```
// datum1.C - Definicije statičkih polja i metoda
// klase kalendarskih datuma.

#include "datum1.h"
#include <iostream>
using namespace std;

const short Datum::duz[][12] = // Brojevi dana po mesecima.
    {{31,28,31,30,31,30,31,31,30,31,30,31},
     {31,29,31,30,31,30,31,31,30,31,30,31}},
    Datum::prot[][12] = // Brojevi dana od početka godine.
    {{0,31,59,90,120,151,181,212,243,273,304,334},
     {0,31,60,91,121,152,182,213,244,274,305,335}};

const char *Datum::imeD[] = {"ponedeljak", "utorak", "sreda", "cetvrtak",
                             "petak", "subota", "nedelja"},
    *Datum::imeM[] = {"januar", "februar", "mart", "april", "maj",
                     "jun", "jul", "avgust", "septembar",
                     "oktobar", "novembar", "decembar"};

Datum Datum::citaj() {
    short d, m, g;
    while (true) {
        cin >> d >> m >> g;
        if (moze(d, m, g)) break;
        printf("\n*** Neispravan datum, unesite ponovo: ");
    }
    return Datum(d, m, g);
}

void Datum::pisi() const {
    cout << dan/10 << dan%10 << ". " << imeM[mes-1] << " " << god << ".";
}

long Datum::ukDan() const {
    short g = god - 1;
    return g*365L + g/4 - g/100 + g/400 + danUGod();
}
```

```

void Datum::sutra() {
    if (dan < duzMes()) dan++;
    else {
        dan = 1;
        if (mes < 12) mes++;
        else { mes = 1; god++; }
    }
}

void Datum::juce() {
    if (dan > 1) dan--;
    else {
        if (mes > 1) mes--;
        else { mes = 12; god--; }
        dan = duzMes();
    }
}

void Datum::dodaj (unsigned k) { for (unsigned i=0; i<k; i++) sutra(); }

void Datum::oduzmi(unsigned k) { for (unsigned i=0; i<k; i++) juce(); }

```

// datumlt.C - Ispitivanje klase kalendarskih datuma.

```

#include "datuml.h"
#include <iostream>
using namespace std;

int main() {
    cout << "Danasnji datum? "; Datum dat1 = Datum::citaj();
    cout << "Datum rođenja? "; Datum dat2 = Datum::citaj();
    dat1.pisi();
    cout << " je " << dat1.imeDan() << ", "
         << dat1.danUGod() << ". dan u godini.\n"
         << "Rodili ste se u " << dat2.imeDan() << " i imate "
         << razlika(dat1, dat2) << " dana.\n";
    return 0;
}

```

% CC datuml.C datumlt.C -o datumlt

```

% datumlt
Danasnji datum? 22 5 2011
Datum rođenja? 12 5 1986
22. maj 2011. je nedelja, 142. dan u godini.
Rodili ste se u ponedeljak i imate 9141 dana.

```

Zadatak 2.9 Liste brojeva

Napisati na jeziku C++ klasu listi celih brojeva. Predvideti:

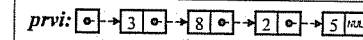
- stvaranje prazne liste, liste od jednog broja i liste kao kopiju druge liste,
- uništavanje liste,
- određivanje broja elemenata liste,
- ispisivanje liste na glavnom izlazu,
- dodavanje broja na početak liste,
- dodavanje broja na kraj liste,
- čitanje liste s glavnog ulaza dodajući brojeve na početak liste,
- čitanje liste s glavnog ulaza dodajući brojeve na kraj liste,
- umetanje broja u uređenu listu,
- brisanje svih elemenata liste, i
- izostavljanje iz liste svakog pojavljivanja datog broja.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Ostvariti prethodnu klasu i pomoću rekurzivnih metoda.

Rešenje:

1) Iterativno (ciklusima)

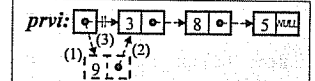


// lista2.h - Definicija klase listi celih brojeva (iterativno).

```

class Lista {
    struct Elem {                // ELEMENT LISTE:
        int broj;                // - sadržaj,
        Elem* sled;              // - pokazivač na sledeći element,
        Elem(int b, Elem* s=0)   // - konstruktor.
        { broj=b; sled = s; }
    };
    Elem* prvi;                  // Pokazivač na početak liste.
public:
    Lista() { prvi = 0; }        // Konstruktor:
    Lista(int b)                 // - podrazumevani,
    { prvi = new Elem(b); }      // - konverzije,
    Lista(const Lista& lst);      // - kopije.
    ~Lista() { brisi(); }        // Destruktor.
    int duz() const;             // Broj elemenata liste.
    void pisi() const;           // Pisanje liste.
    void naPocetak(int b) {      // Dodavanje na
        prvi = new Elem(b, prvi); // početak.
    }
    void naKraj(int b);          // Dodavanje na kraj.
    void citaj1(int n);          // Čitanje liste stavljajući brojeve na početak.
    void citaj2(int n);          // Čitanje liste stavljajući brojeve na kraj.
    void umetni(int b);          // Umetanje u uređenu listu.
    void brisi();                // Brisanje svih elemenata liste.
    void izostavi(int b);        // Izostavljanje svakog pojavljivanja.
};

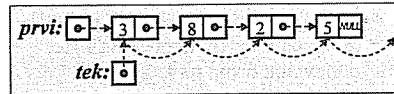
```



// lista2.C - Definicije metoda listi celih brojeva (iterativno).

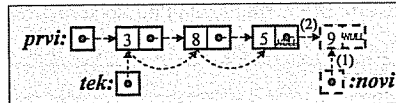
```
#include "lista2.h"
#include <iostream>
using namespace std;
```

```
int Lista::duz() const {           // Broj elemenata liste.
    int n = 0;
    for (Elem* tek=prvi; tek; tek=tek->sled)
        n++;
    return n;
}
```

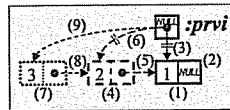


```
void Lista::pisi() const {         // Pisanje liste.
    for (Elem* tek=prvi; tek; tek=tek->sled)
        cout << tek->broj << ' ';
}
```

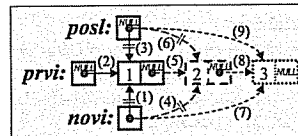
```
void Lista::naKraj(int b) {       // Dodavanje na kraj.
    Elem* novi = new Elem(b);
    if (!prvi) prvi = novi;
    else {
        Elem* tek = prvi;
        while (tek->sled) tek = tek->sled;
        tek->sled = novi;
    }
}
```



```
void Lista::citaaj1(int n) { // Čitanje stavljajući brojeve na početak.
    brisi();
    for (int i=0; i<n; i++) {
        int b; cin >> b;
        prvi = new Elem(b, prvi);
    }
}
```

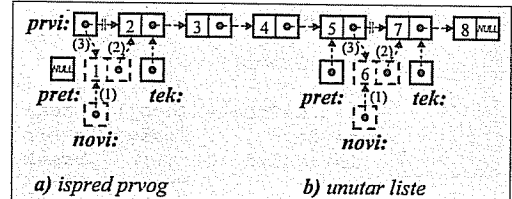


```
void Lista::citaaj2(int n) { // Čitanje stavljajući brojeve na kraj.
    brisi(); Elem* posl = 0;
    for (int i=0; i<n; i++) {
        int b; cin >> b;
        Elem* novi = new Elem(b);
        if (!prvi) prvi = novi; else posl->sled = novi;
        posl = novi;
    }
}
```

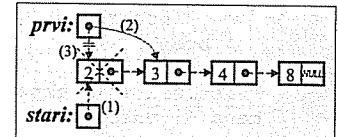


```
Lista::Lista(const Lista& lst) { // Konstruktor kopije.
    prvi = 0;
    for (Elem* tek=lst.prvi, *posl=0; tek; tek=tek->sled) {
        Elem* novi = new Elem(tek->broj);
        if (!prvi) prvi = novi; else posl->sled = novi;
        posl = novi;
    }
}
```

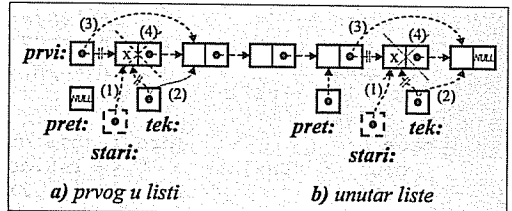
```
void Lista::umetni(int b) {       // Umetanje u uredenu listu.
    Elem *tek=prvi, *pret=0;
    while (tek && tek->broj < b)
        { pret = tek; tek = tek->sled; }
    Elem* novi = new Elem(b, tek);
    if (!pret) prvi = novi;
    else pret->sled = novi;
}
```



```
void Lista::brisi() {            // Brisanje svih elemenata liste.
    while (prvi) {
        Elem* stari = prvi;
        prvi = prvi->sled;
        delete stari;
    }
}
```



```
void Lista::izostavi(int b) {    // Izostavljanje svakog pojavljivanja.
    Elem *tek = prvi, *pret = 0;
    while (tek)
        if (tek->broj != b) {
            pret = tek; tek = tek->sled;
        } else {
            Elem* stari = tek;
            tek = tek->sled;
            if (!pret) prvi = tek;
            else pret->sled = tek;
            delete stari;
        }
}
```



// lista2t.C - Ispitivanje klase listi celih brojeva.

```
#include "lista2.h"
#include <iostream>
using namespace std;
```

```
int main() {
    Lista lst; bool kraj = false;
    while (!kraj) {
        cout << "1. Dodavanje broja na pocetak liste\n"
                "2. Dodavanje broja na kraj liste\n"
                "3. Umetanje broja u uredjenu listu\n"
                "4. Izostavljanje broja iz liste\n"
                "5. Brisanje svih elemenata liste\n"
                "6. Citanje uz obrtanje redosleda brojeva\n"
                "7. Citanje uz cuvanje redosleda brojeva\n"
                "8. Odredjivanje duzine liste\n"
                "9. Ispisivanje liste\n"
                "0. Zavrsetak rada\n\n";
        int izbor; cin >> izbor;
    }
}
```

```

switch (izbor) {
case 1: case 2: case 3: case 4:
    cout << "Broj? "; int broj; cin >> broj;
    switch (izbor) {
        case 1: // Dodavanje broja na pocetak liste:
            lst.naPocetak(broj); break;
        case 2: // Dodavanje broja na kraj liste:
            lst.naKraj(broj); break;
        case 3: // Umetanje broja u uredenu listu:
            lst.umetni(broj); break;
        case 4: // Izostavljanje broja iz liste:
            lst.izostavi(broj); break;
    } break;
case 5: // Brisanje svih elemenata liste:
    lst.brisi(); break;
case 6: case 7: // Citanje liste:
    cout << "Duzina? "; int n; cin >> n;
    cout << "Elementi? ";
    switch(izbor) {
        case 6: // - uz obrtanje redosleda brojeva:
            lst.citaj1(n); break;
        case 7: // - uz cuvanje redosleda brojeva:
            lst.citaj2(n); break;
    } break;
case 8: // Odredjivanje duzine liste:
    cout << "Duzina= " << lst.duz() << endl; break;
case 9: // Ispisivanje liste:
    cout << "Lista= "; lst.pisi(); cout << endl; break;
case 0: // Zavrsetak rada:
    kraj = true; break;
default: // Pogresan izbor:
    cout << "*** Neozvoljen izbor! ***\a\n"; break;
}
return 0;
}

```

```

% CC lista2.C lista2t.C -o lista2t
% lista2t

```

```

1. Dodavanje broja na pocetak liste
2. Dodavanje broja na kraj liste
3. Umetanje broja u uredjenu listu
4. Izostavljanje broja iz liste
5. Brisanje svih elemenata liste
6. Citanje uz obrtanje redosleda brojeva
7. Citanje uz cuvanje redosleda brojeva
8. Odredjivanje duzine liste
9. Ispisivanje liste
0. Zavrsetak rada

```

```

Vas izbor? 1
Broj? 1
...
Vas izbor? 1
Broj? 2
...
Vas izbor? 1

```

```

Broj? 3
...
Vas izbor? 2
Broj? 4
...
Vas izbor? 2
Broj? 5
...
Vas izbor? 2
Broj? 6
...
Vas izbor? 8
Duzina= 6
...
Vas izbor? 9
Lista= 3 2 1 4 5 6
...
Vas izbor? 5
...
Vas izbor? 3
Broj? 5

```

```

...
Vas izbor? 3
Broj? 2
...
Vas izbor? 9
Lista= 1 2 2 4 5 5 6
...
Vas izbor? 4
Broj? 5
...
Vas izbor? 9
Lista= 1 2 2 4 6
...
Vas izbor? 6
Duzina? 5
Elementi? 1 2 3 4 5
...
Vas izbor? 3
Broj? 5
...
Vas izbor? 9
Lista= 5 4 3 2 1
...
Vas izbor? 7
Duzina? 5
Elementi? 1 2 3 4 5
...
Vas izbor? 9
Lista= 1 2 3 4 5
...
Vas izbor? 11
*** Neozvoljen izbor! ***
...
Vas izbor? 0

```

2) Rekurzivno

```

// lista3.h - Definicija klase listi
// celih brojeva (rekurzivno).

```

```

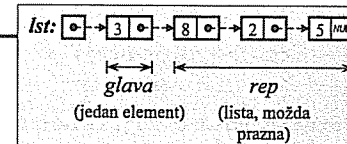
#include <iostream>
using namespace std;

```

```

class Lista {
    struct Elem { // Element liste:
        int broj; // - sadržaj,
        Elem* sled; // - pokazivač na sledeći,
        Elem(int b, Elem* s=0) // - konstruktor.
        { broj=b; sled = s; }
    };
    Elem* prvi; // Pokazivač na pocetak liste.
public: // Konstruktori:
    Lista() { prvi = 0; } // - podrazumevani,
    Lista(int b) // - konverzije,
    { prvi = new Elem(b); }
    Lista(const Lista& lst) // - kopije.
    { prvi = 0; kopiraj(lst.prvi); }
    ~Lista() { brisi(prvi); } // Destruktor.
    int duz() const // Broj elemenata liste.
    { return duz(prvi); }
    void pisi() const // Ispisivanje liste.
    { pisi(prvi); }
    void naPocetak(int b) // Dodavanje na pocetak.
    { prvi = new Elem(b, prvi); }
    void naKraj(int b) // Dodavanje na kraj.
    { naKraj(prvi, b); }
    void citaj1(int n) // Citanje liste stavljajući brojeve na pocetak.
    { brisi(); citaj1(prvi, n); }
    void citaj2(int n) // Citanje liste stavljajući brojeve na kraj.
    { brisi(); citaj2(prvi, n); }
    void umetni(int b) // Umetanje u uredenu listu.
    { umetni(prvi, b); }
    void brisi() // Brisanje svih elemenata liste.
    { brisi(prvi); }
    void izostavi(int b) // Izostavljanje svakog pojavljivanja.
    { izostavi(prvi, b); }
}

```



```

private:
    static Elem* kopiraj(Elem* lst) { // POMOĆNE REKURZIVNE METODE:
        // Kopiranje liste.
        return lst ? new Elem(lst->broj, kopiraj(lst->sled)) : 0;
    }

    static int duz(Elem* lst) // Broj elemenata liste.
    { return lst ? 1+duz(lst->sled) : 0; }

    static void pisi(Elem* lst) { // Ispisivanje liste.
        if (lst)
            cout << lst->broj << ' '; pisi(lst->sled);
    }

    static void naKraj(Elem*& lst, int b) { // Dodavanje na kraj.
        if (!lst) lst = new Elem(b);
        else naKraj(lst->sled, b);
    }

    static void citaj1(Elem*& lst, int n) { // Čitanje stavljajući na poč.
        if (n) {
            lst = new Elem(0);
            citaj1(lst->sled, n-1);
            cin >> lst->broj;
        }
    }

    static void citaj2(Elem*& lst, int n) { // Čitanje stavljajući na kraj.
        if (n) {
            int b; cin >> b; lst = new Elem(b);
            citaj2(lst->sled, n-1);
        }
    }

    static void umetni(Elem*& lst, int b) { // Umetanje u uređenu listu.
        if (!lst || lst->broj >= b)
            lst = new Elem(b, lst);
        else
            umetni(lst->sled, b);
    }

    static void brisi(Elem*& lst) { // Brisanje svih elemenata.
        if (lst) { brisi(lst->sled);
            delete lst; lst = 0; }
    }

    static void izostavi(Elem*& lst, int b) { // Izost. svakog pojavlj.
        if (lst) { izostavi(lst->sled, b);
            if (lst->broj == b) {
                Elem* stari = lst;
                lst = lst->sled; delete stari;
            }
        }
    }
};

```

Nova glava + kopija repa.

1 + dužina repa.

Piši glavu + piši rep.

Ako je lista prazna, napravi jedan element – inače, dodaj na kraj repa.

Čitaj rep + čitaj glavu.

Čitaj glavu + čitaj rep.

Ako je lista prazna ili je broj ispred glave, stavi listu iza broja – inače, umetni broj u rep.

Briši rep + briši glavu.

Izostavi broj iz repa. Posle briši glavu ako je jednak broju.

// lista3t.C – Ispitivanje klase listi celih brojeva.

#include "lista3.h"

... Nastavak se doslovce poklapa sa lista2t.C ...

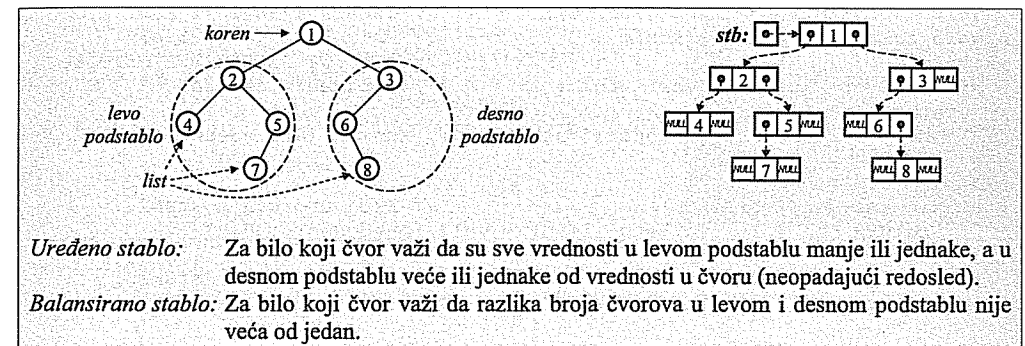
Zadatak 2.10 Uređena stabla brojeva

Napisati na jeziku C++ klasu uređenih binarnih stabala celih brojeva. Predvideti:

- stvaranje praznog stabla, stabla od jednog broja i kao kopiju drugog stabla,
- uništavanje stabla,
- ispitivanje da li je stablo prazno i određivanje broja čvorova u stablu,
- izračunavanje zbira brojeva u čvorovima stabla,
- ispisivanje sadržaja stabla na glavnom izlazu po prefiksnom (koren – levo podstablo – desno podstablo), infiksnom (levo podstablo – koren – desno podstablo) i postfiksnom (levo podstablo – desno podstablo – koren) redosledu,
- grafički prikaz sadržaja stabla na glavnom izlazu,
- određivanje broja pojavljivanja date vrednosti u stablu,
- određivanje vrednosti najmanjeg i najvećeg podatka u stablu,
- pronalaženje čvora u stablu koji sadrži zadati broj,
- dodavanje novog broja u stablo,
- čitanje stabla sa zadatim brojem čvorova s glavnog ulaza,
- brisanje svih čvorova stabla,
- izostavljanje datog broja iz stabla, i
- balansiranje stabla.

Napisati na jeziku C++ interaktivni program za ispitivanje prethodne klase.

Rešenje:



// stablo.h – Definicija klase uređenih binarnih stabala.

```

#include <iostream>
#include <iomanip>
#include <cstdlib>
using namespace std;

```

```

class Stablo {
    struct Cvor {
        int broj;
        Cvor *levo, *desno;
        Cvor(int b, Cvor* l=0, Cvor* d=0)
            { broj = b; levo = l; desno = d; }
    };
    Cvor* stb;

```

```

// ČVOR STABLA:
// - sadržaj čvora,
// - levo i desno podstablo,
// - konstruktor.

// Pokazivač na koren stabla.

```



```

public:
    Stablo() { stb = 0; } // Prazno stablo.
    Stablo(int b) { stb = new Cvor(b); } // Konverzija.
    Stablo(const Stablo& stb) { stb = kopiraj(stb); } // Kopija.
    ~Stablo() { brisi(stb); } // Destruktor.
    bool prazno() const { return stb == 0; } // Da li je stablo prazno?
    int vel() const { return vel(stb); } // Broj čvorova u stablu.
    int zbir() const { return zbir(stb); } // Zbir brojeva u stablu.
    void pisiKLD() const { pisiKLD(stb); } // Prefiksno pisanje.
    void pisiLKD() const { pisiLKD(stb); } // Infiksno pisanje.
    void pisiLDK() const { pisiLDK(stb); } // Postfiksno pisanje.
    void crtaj() const { crtaj(stb, 0); } // Grafički prikaz stabla.
    int pojav(int b) const // Broj pojavljivanja.
    { return pojav(stb, b); }
    int min() const { // Najmanji u stablu.
        if (!stb) exit(1); // (ne sme biti prazno)
        return min(stb);
    }
    int max() const { // Najveći u stablu.
        if (!stb) exit(1); // (ne sme biti prazno)
        return max(stb);
    }
    Cvor* nadji(int b) const // Traženje u stablu.
    { return nadji(stb, b); }
    void dodaj(int b) { dodaj(stb, b); } // Dodavanje u stablo.
    void citaj(int n); // Čitanje stabla.
    void brisi() { brisi(stb); } // Brisanje celog stabla.
    void izost(int b) { izost(stb, b); } // Izostavljanje iz stabla.
    void balans() { balans(stb); } // Balansiranje stabla.
private:
    static Cvor* kopiraj(Cvor* stb) { // POMOĆNE REKURZIVNE METODE:
        return stb ? // Kopiranje stabla.
            new Cvor(stb->broj, kopiraj(stb->levo), kopiraj(stb->desno)) : 0;
    }

    static int vel(Cvor* stb) // Broj čvorova u stablu.
    { return stb ? 1 + vel(stb->levo) + vel(stb->desno) : 0; }

    static int zbir(Cvor* stb) // Zbir brojeva u stablu.
    { return stb ? stb->broj + zbir(stb->levo) + zbir(stb->desno) : 0; }

    static void pisiKLD(Cvor* stb) { // Prefiksno ispisivanje.
        if (stb) {
            cout << stb->broj << ' '; pisiKLD(stb->levo); pisiKLD(stb->desno);
        }
    }

    static void pisiLKD(Cvor* stb) { // Infiksno ispisivanje.
        if (stb) {
            pisiLKD(stb->levo); cout << stb->broj << ' '; pisiLKD(stb->desno);
        }
    }

    static void pisiLDK(Cvor* stb) { // Postfiksno ispisivanje.
        if (stb) {
            pisiLDK(stb->levo); pisiLDK(stb->desno); cout << stb->broj << ' ';
        }
    }

```

```

static void crtaj(Cvor* stb, int nivo) { // Grafički prikaz stabla.
    if (stb) {
        crtaj(stb->desno, nivo+1);
        cout << setw(4*nivo) << " " << stb->broj << endl;
        crtaj(stb->levo, nivo+1);
    }
}

static int pojav(Cvor* stb, int b) { // Broj pojavljivanja
    return stb ? (stb->broj==b)+pojav(stb->levo,b) // u stablu
        +pojav(stb->desno,b) : 0;
}

static int min(Cvor* stb) // Najmanji u stablu.
{ return stb->levo ? min(stb->levo) : stb->broj; }

static int max(Cvor* stb) // Najveći u stablu.
{ return stb->desno ? max(stb->desno) : stb->broj; }

static Cvor* nadji(Cvor* stb, int b) { // Traženje u stablu.
    if (!stb) return 0;
    if (stb->broj == b) return stb;
    if (stb->broj > b) return nadji(stb->levo, b);
    return nadji(stb->desno, b);
}

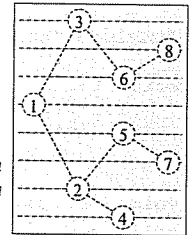
static void dodaj(Cvor*& stb, int b) { // Dodavanje u stablo.
    if (!stb) stb = new Cvor(b);
    else if (stb->broj > b) dodaj(stb->levo, b);
    else if (stb->broj < b) dodaj(stb->desno, b);
    else if (rand()/(RAND_MAX+1.) < 0.5)
        dodaj(stb->levo, b);
    else
        dodaj(stb->desno, b);
}

static void brisi(Cvor*& stb) { // Brisanje celog stabla.
    if (stb) {
        brisi(stb->levo); brisi(stb->desno); delete stb; stb = 0;
    }
}

static void izost(Cvor*& stb, int b) { // Izostavljanje iz stabla.
    if (stb) {
        if (stb->broj > b) izost(stb->levo, b);
        else if (stb->broj < b) izost(stb->desno, b);
        else if (stb->levo) {
            int m = max(stb->levo);
            stb->broj = m; izost(stb->levo, m);
        } else if (stb->desno) {
            int m = min(stb->desno);
            stb->broj = m; izost(stb->desno, m);
        } else {
            delete stb; stb = 0;
        }
    }
}

static void balans(Cvor*& stb); // Balansiranje stabla.
};

```



Ako broj nije u korenu, dovoljno je pretražiti samo levo ili samo desno podstablo.

Ako je stablo prazno, novi čvor postaje jedini u stablu. Inače, dodaje se u levo ili desno podstablo, zavisno o odnosu prema sadržaju korena. U slučaju jednakosti, levo ili desno podstablo se bira slučajno. – Novi čvor biće list u stablu.

Ako broj nije u listu, sadržaj čvora mora da se zameni nečim. U obzir dolazi najveći broj u levom podstablu ili najmanji broj u desnom podstablu.

// stablo.C - Deficije metoda klase uredenih binarnih stabala.

```
#include "stablo.h"
```

```
void Stablo::citaj(int n) { // Čitanje stabla.
    brisi(stb);
    for (int i=0; i<n; i++) { int b; cin >> b; dodaj(b); }
}
```

```
void Stablo::balans(Cvor*& stb) { // Balansiranje stabla.
    if (stb) {
        int k = vel(stb->levo) - vel(stb->desno);
        for (; k>1; k-=2) {
            dodaj(stb->desno, stb->broj);
            stb->broj = max(stb->levo);
            izost(stb->levo, stb->broj);
        }
        for (; k<-1; k+=2) {
            dodaj(stb->levo, stb->broj);
            stb->broj = min(stb->desno);
            izost(stb->desno, stb->broj);
        }
        balans(stb->levo);
        balans(stb->desno);
    }
}
```

Sve dok je levo podstablo preveliko, sadržaj korena se ubacuje u desno podstablo, a najveći broj iz levog podstabla premešta se u koren. Slično se postupa i ako je desno podstablo preveliko. Na kraju, potrebno je odvojeno balansirati levo i desno podstablo.

// stablot.C - Ispitivanje klase uredenih binarnih stabala.

```
#include "stablo.h"
#include <iostream>
using namespace std;
```

```
// Primena operacije na stablo za svaki pročitani broj:
void radi(Stablo& stb, void (Stablo::*pf)(int)) {
    int b; cout << "Brojevi? ";
    do { cin >> b; (stb.*pf)(b); } while (cin.get() != '\n');
    // do kraja reda
}
```

```
int main() {
    Stablo stb; bool kraj = false;
    while (!kraj) {
        cout << endl <<
            "a) Dodavanje brojeva      Pisanje stabla:\n"
            "b) Izostavljanje brojeva    1. koren-levo-desno\n"
            "c) Citanje stabla                2. levo-koren-desno (uredjeno)\n"
            "d) Najmanji element              3. levo-desno-koren\n"
            "e) Najveci element                4. crtanje\n"
            "f) Pretrazivanje                  i) Velicina stabla\n"
            "g) Balansiranje                    j) Zbir elemenata\n"
            "h) Praznjenje stabla              k) Broj pojavljivanja\n"
            "                                z) Zavrsetak rada\n"
            "Vas izbor? ";
        char izbor; cin >> izbor;
```

```
switch (izbor) {
    int broj;
    case 'a': case 'A': // Dodavanje brojeva u stablo:
        radi(stb, &Stablo::dodaj); break;
    case 'b': case 'B': // Izostavljanje brojeva iz stabla:
        radi(stb, &Stablo::izost); break;
    case 'c': case 'C': // Čitanje stabla:
        cout << "Duzina? "; int n; cin >> n;
        cout << "Brojevi? "; stb.citaj(n);
        break;
    case 'd': case 'D': case 'e': case 'E':
        if (!stb.prazno()) switch (izbor) {
            case 'd': case 'D': // Najmanji element stabla:
                cout << "min=" << stb.min(); break;
            case 'e': case 'E': // Najveci element stabla:
                cout << "max=" << stb.max(); break;
        } else cout << "*** Stablo je parzno! ***\n";
        break;
    case 'f': case 'F': // Pretrazivanje stabla:
        cout << "Broj? "; cin >> broj;
        cout << "Broj se" << (stb.nadji(broj) ? "" : " NE")
            << " nalazi u stablu.\n";
        break;
    case 'g': case 'G': // Balansiranje stabla:
        stb.balans(); break;
    case 'h': case 'H': // Praznjenje stabla:
        stb.brisi(); break;
    case 'i': case 'I': // Veličina stabla:
        cout << "Vel=" << stb.vel() << endl; break;
    case 'j': case 'J': // Zbir elemenata stabla:
        cout << "Zbir=" << stb.zbir() << endl; break;
    case 'k': case 'K': // Broj pojavljivanja datog broja:
        cout << "Broj? "; cin >> broj;
        cout << "Broj se pojavljuje " << stb.pojav(broj) << " puta.\n";
        break;
    case '1': // Pisanje stabla koren-levo-desno:
        cout << "Stablo=" << stb.pisiKLD(); cout << endl; break;
    case '2': // Pisanje stabla levo-koren-desno:
        cout << "Stablo=" << stb.pisiLKD(); cout << endl; break;
    case '3': // Pisanje stabla levo-desno-koren:
        cout << "Stablo=" << stb.pisiLDK(); cout << endl; break;
    case '4': // Crtanje stabla:
        stb.crtaj(); break;
    case 'z': case 'Z': // Zavrsetak rada:
        kraj = true; break;
    default: // Pogrešan izbor:
        cout << "*** Nedoizvoljen izbor! ***\n"; break;
}
return 0;
}
```

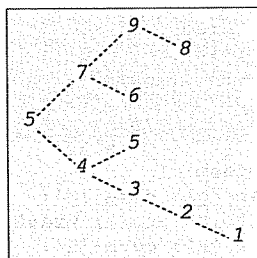
```
% CC stablo.C stablot.C -o stablot
% stablot
```

```
a) Dodavanje brojeva      Pisanje stabla:
b) Izostavljanje brojeva  1. koren-levo-desno
c) Citanje stabla         2. levo-koren-desno (uredjeno)
d) Najmanji element       3. levo-desno-koren
e) Najveći element        4. crtanje
f) Pretrazivanje          i) Velicina stabla
g) Balansiranje           j) Zbir elemenata
h) Praznjenje stabla      k) Broj pojavljivanja
                          z) Zavrsetak rada
```

```
Vas izbor? a
Brojevi? 5 7 4 3 9 8 2 6 5 1
```

```
Vas izbor? 4
```

```
9
 8
7 6
5 5
4 3
 2
 1
```



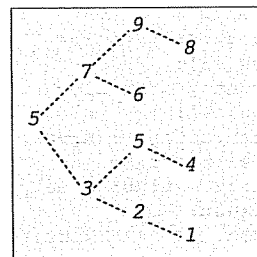
```
Vas izbor? 1
Stablo= 5 4 3 2 1 5 7 6 9 8
```

```
Vas izbor? 2
Stablo= 1 2 3 4 5 5 6 7 8 9
```

```
Vas izbor? 3
Stablo= 1 2 3 5 4 6 8 9 7 5
```

```
Vas izbor? g
```

```
Vas izbor? 4
9
 8
7 6
5 5
 4
 3
 2
 1
```



```
Vas izbor? 1
Stablo= 5 3 2 1 5 4 7 6 9 8
```

```
Vas izbor? 2
Stablo= 1 2 3 4 5 5 6 7 8 9
```

```
Vas izbor? 3
Stablo= 1 2 4 5 3 6 8 9 7 5
```

```
Vas izbor? z
```

Zadatak 2.11 Nizovi materijalnih tačaka

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima i destruktorom koji su potrebni za bezbedno korišćenje klase):

- Materijalna tačka** u prostoru zadaje se pomoću realne mase (podrazumevano 1) i tri realne koordinate (podrazumevano (0,0,0)). Može da se odredi rastojanje (r) do druge tačke, da se izračuna privlačna sila između tačke i zadate druge tačke ($F = \gamma \cdot m_1 \cdot m_2 / r^2$, $\gamma = 6,67 \cdot 10^{-11}$) i da se tačka ispiše na glavnom izlazu.
- Niz materijalnih tačaka** stvara se prazan, zadatog početnog kapaciteta (podrazumevano 5), posle čega se tačke dodaju pojedinačno na kraj niza. Ako se niz prepuni, kapacitet mu se poveća za 5. Može da se dohvati broj tačaka u nizu, da se dohvati tačka u nizu koja najviše privlači zadatu tačku i da se niz ispiše na glavnom izlazu.

Napisati na jeziku C++ **program** koji čitajući materijalne tačke s glavnog ulaza napravi niz materijalnih tačaka (čitanje se završava unosom negativne mase), ispiše na glavnom izlazu dobijeni niz kao i tačku koja najviše privlači tačku jedinične mase u koordinatnom početku i ponavlja prethodne korake sve dok ne pročita prazan niz (niz dužine 0).

Rešenje:

```
// mattacka.h - Definicija klase materijalnih tačaka.
```

```
#include <iostream>
#include <cmath>
using namespace std;

class MatTacka {
    double m, x, y, z; // Masa i koordinate tačke.
public:
    explicit MatTacka(double mm=1, double xx=0, // Stvaranje tačke.
                      double yy=0, double zz=0)
    { m = mm; x = xx; y = yy; z = zz; }
    double r(const MatTacka& T) const // Rastojanje do tačke.
    { return sqrt(pow(x-T.x,2) + pow(y-T.y,2) + pow(z-T.z,2)); }
    double F(const MatTacka& T) const // Privlačna sila.
    { return 6.67e-11 * m * T.m / pow(r(T),2); }
    void pisi() const // Pisanje tačke.
    { cout << '[' << m << ", (" << x << ", " << y << ", " << z << ") ]"; }
};
```

```
// nizmatc.h - Definicija klase nizova materijalnih tačaka.
```

```
#include "mattacka.h"

class NizMTac {
    MatTacka* niz; // Niz tačaka.
    int kap, duz; // Kapacitet i dužina niza.
public:
    explicit NizMTac(int k=5) { // Stvaranje praznog niza.
        niz = new MatTacka [kap = k];
        duz = 0;
    }
    NizMTac(const NizMTac& n); // Konstruktor kopije.
    ~NizMTac() { delete [] niz; } // Destruktor.
    int vel() const { return duz; } // Veličina niza.
    NizMTac& dodaj(const MatTacka& T); // Dodavanje tačke.
```

```
MatTacka maxF(const MatTacka& T) const; // Najjača tačka.
void pisi() const; // Pisanje niza.
};
```

// nizmtac.C - Definicije metoda klase nizova materijalnih tačaka.

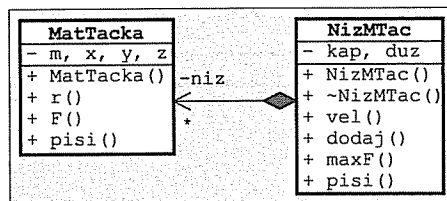
```
#include "nizmtac.h"
#include <iostream>
using namespace std;
```

```
NizMTac::NizMTac(const NizMTac& n) { // Konstruktor kopije.
    niz = new MatTacka [kap = n.kap];
    duz = n.duz;
    for (int i=0; i<duz; i++) niz[i] = n.niz[i];
}
```

```
NizMTac& NizMTac::dodaj(const MatTacka& T) { // Dodavanje tačke.
    if (duz == kap) {
        MatTacka* pom = new MatTacka [kap += 5];
        for (int i=0; i<duz; i++) pom[i] = niz[i];
        delete [] niz; niz = pom;
    }
    niz[duz++] = T;
    return *this;
}
```

```
MatTacka NizMTac::maxF(const MatTacka& T) const { // Najjača tačka.
    double max = T.F(niz[0]);
    int ind = 0;
    for (int i=1; i<duz; i++) {
        double F = T.F(niz[i]);
        if (F > max) { max = F; ind = i; }
    }
    return niz[ind];
}
```

```
void NizMTac::pisi() const { // Pisanje niza.
    for (int i=0; i<duz; i++) { niz[i].pisi(); cout << endl; }
}
```



// nizmtact.C - Ispitivanje kalse nizova materijalnih tačaka.

```
#include "nizmtac.h"
#include <iostream>
using namespace std;
```

```
int main() {
    while (true) {
        NizMTac niz;
        while (true) {
            cout << "m,x,y,z? "; double m, x, y, z; cin >> m >> x >> y >> z;
            if (m < 0) break;
            niz.dodaj(MatTacka(m,x,y,z));
        }
        if (niz.vel() == 0) break;
        cout << "Niz tacaka:\n"; niz.pisi();
        cout << "Najvise privlaci: ";
        niz.maxF(MatTacka()).pisi(); cout << endl;
    }
    return 0;
}
```

% CC nizmtac.C nizmtact.C -o nizmtact

```
% nizmtact
m,x,y,z? 5 3 4 5
m,x,y,z? 1 1 1 1
m,x,y,z? 8 2 9 4
m,x,y,z? -1 0 0 0
Niz tacaka:
[5, (3,4,5)]
[1, (1,1,1)]
[8, (2,9,4)]
Najvise privlaci: [1, (1,1,1)]
m,x,y,z? -1 0 0 0
```

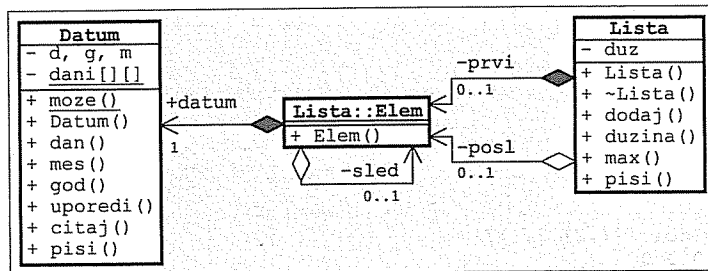
Zadatak 2.12 Liste datuma

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima i destruktorom koji su potrebni za bezbedno korišćenje klasa):

- Datum** se zadaje pomoću broja dana, meseca i godine. Može da se proveri da li tri cela broja predstavljaju ispravan datum, da se stvara datum na osnovu tri cela broja (podrazumevano 1.7.2011. – pogrešan datum prekida program), da se dohvataju delovi datuma, da se datum uporedi s drugim datumom (rezultat je <0, =0 ili >0, zavisno od toga da li je tekući datum pre, jednak ili posle zadatog datuma), da se datum pročita s glavnog ulaza (povratna vrednost je indikator uspeha) i da se datum ispiše na glavnom izlazu.
- Lista** datuma se stvara prazna, posle čega se datumi dodaju pojedinačno na kraj liste. Može da se odredi dužina liste, da se dohvati pokazivač na najkasniji datum u listi i da se lista ispiše na glavnom izlazu.

Napisati na jeziku C++ **program** koji čitajući datume s glavnog ulaza napravi listu datuma (čitanje se završava prvim neispravnim datumom), ispiše na glavnom izlazu dobijenu listu kao i najkasniji datum i ponavlja prethodne korake sve dok ne pročita praznu listu.

Rešenje:



```
// datum2.h - Definicija klase kalendarskih datuma.
```

```
#include <iostream>
#include <cstdlib>
using namespace std;

class Datum {
    int d, m, g; // Dan, mesec i godina.
    static int dani[2][12]; // Brojevi dana po mesecima.
public:
    static bool moze(int d, int m, int g) // Da li je ispravan?
    { return g>0 && m>0 && m<=12 && d>0 && d<=dani[g%4][m-1]; }
    explicit Datum(int dd=1, int mm=7, int gg=2011) { // Stvaranje datuma.
        if (!moze(dd, mm, gg)) exit(1);
        d = dd; m = mm; g = gg;
    }
    int dan() const { return d; } // Dohvatanje delova datuma.
    int mes() const { return m; }
    int god() const { return g; }
    int uporedi(const Datum& dat) const { // Upoređivanje datuma.
        if (g != dat.g) return g - dat.g;
        if (m != dat.m) return m - dat.m;
        return d - dat.d;
    }
}
```

```
bool citaj() { // Čitanje datuma.
    int d, m, g; cin >> d >> m >> g;
    if (!moze(d, m, g)) return false;
    *this = Datum(d, m, g); return true;
}

void pisi() const // Pisanje datuma.
{ cout << d << '.' << m << '.' << g << '.'; }
};
```

```
// datum2.C - Definicija statičkog polja uz klasu datuma.
```

```
#include "datum2.h"

int Datum::dani[2][12] = {
    {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31},
    {31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}
};
```

```
// listadat.h - Definicija klase listi datuma.
```

```
#include "datum2.h"

class Lista {
    struct Elem { // ELEMENT LISTE:
        Datum dat; // - sadržani datum,
        Elem* sled; // - pokazivač na sledeći element,
        Elem const(const Datum& d) // - konstruktor.
        { dat = d; sled = 0; }
    };
    Elem* prvi, *posl; // Pokazivač na početak i kraj liste.
    int duz; // Dužina liste.
public:
    Lista() { prvi = posl = 0; duz = 0; } // - podrazumevani,
    Lista(const Lista& lst); // - kopije.
    ~Lista(); // Destruktor.
    Lista& dodaj(const Datum& dat) { // Dodavanje datuma.
        posl = (!prvi ? prvi : posl->sled) = new Elem(dat);
        duz++;
        return *this;
    }
    int duzina() const { return duz; } // Dužina liste.
    const Datum* max() const; // Najkasniji (svežiji) datum.
    void pisi() const; // Pisanje liste.
};
```

```
// listadat.C - Definicije metoda klase listi datuma.
```

```
#include "listadat.h"
#include <iostream>
using namespace std;

Lista::Lista(const Lista& lst) { // Konstruktor kopije.
    prvi = posl = 0; duz = 0;
    for (Elem* tek=lst.prvi; tek; tek=tek->sled) dodaj(tek->dat);
}

Lista::~Lista() { // Destruktor.
    while (prvi) { Elem* stari = prvi; prvi = prvi->sled; delete stari; }
}
```

```
const Datum* Lista::max() const { // Najkasniji (svežiji) datum.
    if (!prvi) return 0;
    Datum* m = &prvi->dat;
    for (Elem* tek=prvi->sled; tek; tek=tek->sled)
        if (m->uporedi(tek->dat) < 0) m = &tek->dat;
    return m;
}

void Lista::pisi() const { // Pisanje liste.
    for (Elem* tek=prvi; tek; tek=tek->sled)
        { tek->dat.pisi(); cout << ' '; }
}
```

// listadatt.C - Ispitivanje klase listi datuma.

```
#include "listadat.h"
#include <iostream>
using namespace std;

int main() {
    while (true) {
        Lista lst;
        while (true) {
            cout << "Datum (d,m,g)? "; int d, m, g; cin >> d >> m >> g;
            if (!Datum::moze(d, m, g)) break;
            lst.dodaj(Datum(d, m, g));
        }
        if (lst.duzina() == 0) break;
        cout << "Lista= "; lst.pisi(); cout << endl;
        cout << "Najkasnije= "; lst.max()->pisi(); cout << endl;
    }
    return 0;
}
```

```
% CC datum2.C listadat.C listadatt.C -o listadatt
% listadatt
Datum (d,m,g)? 1 7 2011
Datum (d,m,g)? 12 5 1986
Datum (d,m,g)? 31 8 2011
Datum (d,m,g)? 3 12 1999
Datum (d,m,g)? 31 2 2001
Lista= 1.7.2011. 12.5.1986. 31.8.2011. 3.12.1999.
Najkasnije= 31.8.2011.
Datum (d,m,g)? 0 0 0
```

Zadatak 2.13 JMBG, osobe i imenici

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima i destruktorom koji su potrebni za bezbedno korišćenje klase):

- Jedinstveni matični broj građana (**JMBG**) sadrži nisku fiksne dužine od 13 cifara, koja se zadaje pri stvaranju i može da se dohvati. Može da se ispita da li je zadati JMBG veći od drugog zadatog JMBG i da se JMBG ispiše na glavnom izlazu računara.
- **Osoba** ima ime i JMBG koji se zadaju pri stvaranju i mogu da se dohvate. Ime je niska proizvoljne dužine. Osoba može da se ispiše na glavnom izlazu računara u obliku *ime (jmbg)*.
- **Imenik** može da sadrži proizvoljan broj osoba po rastućem redosledu njihovih JMBG. Stvara se prazan posle čega se osobe dodaju pojedinačno. Sadržaj imenika može da se ispiše na glavnom izlazu računara, jedna osoba po redu.

Napisati na jeziku C++ **program** koji napravi imenik s nekoliko osoba i ispiše ga na glavnom izlazu računara. Koristiti fiksne parametre – nije potrebno ništa učitavati s glavnog ulaza.

Rešenje:

```
// jmbg.h - Definicija klase jedinstvenih matičnih brojeva građana.

#include <cstring>
#include <iostream>
using namespace std;

class JMBG {
    char jmbg[14];
public:
    JMBG(const char j[]) { strcpy(jmbg, j); } // Konstruktor.
    const char* dohvJMBG() const { return jmbg; } // Dohvatanje broja.
    friend bool veci(const JMBG& j1, const JMBG& j2) // Upoređivanje.
        { return strcmp(j1.jmbg, j2.jmbg) > 0; }
    void pisi() const { cout << jmbg; } // Pisanje.
};
```

// osoba.h - Definicija klase osoba.

```
#include "jmbg.h"
#include <cstring>
#include <iostream>
using namespace std;

class Osoba {
    char* ime;
    JMBG jmbg;
public:
    Osoba(const char* i, JMBG j): jmbg(j) // Stvaranje.
        { ime = new char [strlen(i)+1]; strcpy(ime, i); }
    Osoba(const Osoba& oso): jmbg(oso.jmbg) // Konstruktor kopije.
        { ime=new char [strlen(oso.ime)+1]; strcpy(ime, oso.ime); }
    ~Osoba() { delete [] ime; } // Destruktor.
    const char* dohvIme() const { return ime; } // Dohvatanje imena.
    JMBG dohvJMBG() const { return jmbg; } // Dohvatanje JMBG.
    void pisi() const // Pisanje.
        { cout << ime << ' '; jmbg.pisi(); cout << ' '; }
};
```

```
// imenik1.h - Definicija klase imenika.
```

```
#include "osoba1.h"
```

```
class Imenik {
    struct Elem {                // Element liste:
        Osoba oso;               // - sadržana osoba.
        Elem* sled;              // - pokazivač na sledeći element.
        Elem(const Osoba& o, Elem* s=0): // - konstruktor
            oso(o) { sled = s; }
    };
    Elem* prvi;                  // Pokazivač na početak liste.
public:
    Imenik() { prvi=0; }         // Stvaranje praznog imenika.
    Imenik(const Imenik& im);     // Konstruktor kopije.
    ~Imenik();                   // Destruktor.
    Imenik& dodaj(const Osoba& oso); // Dodavanje osobe.
    void pisi() const;           // Pisanje imenika.
};
```

```
// imenik1.C - Definicije metoda klase imenika.
```

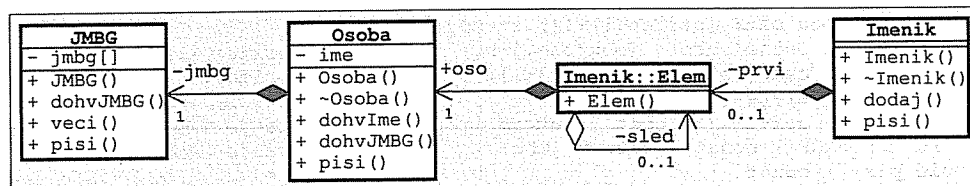
```
#include "imenik1.h"
#include <iostream>
using namespace std;
```

```
Imenik::Imenik(const Imenik& im) { // Konstruktor kopije.
    prvi = 0;
    for (Elem *tek=im.prvi, *posl=0; tek; tek=tek->sled)
        posl= (!prvi ? prvi : posl->sled) = new Elem(tek->oso);
}
```

```
Imenik::~Imenik() { // Destruktor.
    while (prvi) { Elem* stari = prvi; prvi = prvi->sled; delete stari; }
}
```

```
Imenik& Imenik::dodaj(const Osoba& oso) { // Dodavanje osobe.
    Elem *tek=prvi, *pret=0;
    while (tek && veci(oso.dohvJMBG(), tek->oso.dohvJMBG()))
        { pret = tek; tek = tek->sled; }
    (!pret ? prvi : pret->sled) = new Elem(oso, tek);
    return *this;
}
```

```
void Imenik::pisi() const { // Pisanje imenika.
    for (Elem* tek=prvi; tek; tek=tek->sled)
        { tek->oso.pisi(); cout<<endl; }
}
```



```
// imenik1t.C - Ispitivanje klase imenika.
```

```
#include "imenik1.h"
#include <iostream>
using namespace std;
```

```
int main() {
    Imenik im;
    im.dodaj(Osoba("Marko", JMBG("1205986110022")))
    .dodaj(Osoba("Milan", JMBG("3112969235052")))
    .dodaj(Osoba("Zoran", JMBG("1511990872035")))
    .dodaj(Osoba("Petar", JMBG("0207000345678")))
    .dodaj(Osoba("Stevo", JMBG("2503973123024")))
    .pisi();
    return 0;
}
```

```
% CC imenik1.C imenik1t.C -o imenik1t
```

```
% imenik1t
```

```
Petar(0207000345678)
Marko(1205986110022)
Zoran(1511990872035)
Stevo(2503973123024)
Milan(3112969235052)
```

3 Operatorske funkcije

Zadatak 3.1 Kompleksni brojevi

Napisati na jeziku C++ klasu kompleksnih brojeva. Predvideti:

- stvaranje kompleksnog broja,
- dohvaćanje delova kompleksnog broja,
- izračunavanje apsolutne vrednosti i argumenta kompleksnog broja,
- nalaženje konjugovanog kompleksnog broja,
- izvođenje aritmetičkih operacija,
- upoređivanje dva kompleksna broja,
- izračunavanje e^x , $\log x$ i x^i , i
- čitanje kompleksnog broja iz ulaznog toka i upisivanje u izlazni tok.

Koristiti operatorske funkcije kad god je to primereno.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:

```
// kompl1.h - Klasa kompleksnih brojeva.
#include <cmath>
#include <iostream>
using namespace std;

class Kompl {
    double re, im; // Realni i imaginarni deo.
public:
    Kompl(double r=0, double i=0) // Stvaranje kompleksnog broja.
    { re = r; im = i; }

    friend double real(const Kompl& a) // Realni deo.
    { return a.re; }
    friend double imag(const Kompl& a) // Imaginarni deo.
    { return a.im; }
    friend double abs(const Kompl& a) // Apsolutna vrednost.
    { return sqrt(a.re*a.re + a.im*a.im); }
    friend double arg(const Kompl& a) // Argument.
    { return (a.re!=0 || a.im!=0) ? atan2(a.im,a.re) : 0; }

    Kompl operator~() const // Konjugovan broj.
    { return Kompl(re, -im); }
    friend Kompl operator+(const Kompl& a, const Kompl& b) // a+b
    { return Kompl(a.re+b.re, a.im+b.im); }
    friend Kompl operator-(const Kompl& a, const Kompl& b) // a-b
    { return Kompl(a.re-b.re, a.im-b.im); }
    friend Kompl operator*(const Kompl& a, const Kompl& b) // a*b
    { return Kompl(a.re*b.re-a.im*b.im, a.im*b.re+a.re*b.im); }
    friend Kompl operator/(const Kompl& a, const Kompl& b) { // a/b
        double c = b.re*b.re + b.im*b.im;
        return Kompl((a.re*b.re+a.im*b.im)/c, (a.im*b.re-a.re*b.im)/c);
    }

    Kompl& operator+=(const Kompl& b) { return *this = *this + b; } // +=b
    Kompl& operator-=(const Kompl& b) { return *this = *this - b; } // -=b
    Kompl& operator*=(const Kompl& b) { return *this = *this * b; } // *=b
    Kompl& operator/=(const Kompl& b) { return *this = *this / b; } // /=b

    Kompl& operator++() { re++; return *this; } // ++a
    Kompl& operator--() { re--; return *this; } // --a
    Kompl operator++(int) { Kompl z=*this; re++; return z; } // a++
    Kompl operator--(int) { Kompl z=*this; re--; return z; } // a--
}
```

```
friend bool operator==(const Kompl& a, const Kompl& b) // a==b
{ return a.re==b.re && a.im==b.im; }
friend bool operator!=(const Kompl& a, const Kompl& b) // a!=b
{ return a.re!=b.re || a.im!=b.im; }

friend Kompl exp(const Kompl& a) { // exp(a)
    double abs = exp(a.re), arg = a.im;
    return Kompl(abs*cos(arg), abs*sin(arg));
}
friend Kompl log(const Kompl& a) // log(a)
{ return Kompl(log(abs(a)), arg(a)); }
friend Kompl pow(const Kompl& a, const Kompl& b) // pow(a, b)
{ return exp(b * log(a)); }

friend ostream& operator>>(ostream& ut, Kompl& z) // Čitanje.
{ return ut >> z.re >> z.im; }
friend ostream& operator<<(ostream& it, const Kompl& z) // Pisanje.
{ return it << '(' << z.re << ',' << z.im << ')'; }
};
```

```
// kompl1t.C - Ispitivanje klase kompleksnih brojeva
// (tabeliranje kompleksnog polinoma).
```

```
#include "kompl1.h"
#include <iostream>
using namespace std;

Kompl poli(const Kompl p[], int n, Kompl z) {
    Kompl s = p[n];
    for (int i=n-1; i>=0; (s*=z)+=p[i--]);
    return s;
}

int main() {
    cout << "Red polinoma? "; int n; cin >> n;
    Kompl* p = new Kompl [n+1];
    cout << "Koeficijenti? "; for (int i=n; i>=0; cin>>p[i--]);
    cout << "z0, dz, k? ";
    Kompl z, dz; int k; cin >> z >> dz >> k; cout << endl;
    for (int i=0; i<k; i++, z+=dz)
        cout << z << '\t' << poli(p,n,z) << endl;
    delete [] p;
    return 0;
}
```

```
% kompl1t
Red polinoma? 3
Koeficijenti? 1 1 -1 0 3 -2 5 0
z0, dz, k? 0 0 0.24 -0.48 12
```

```
(0,0) (5,0)
(0.24,-0.48) (4.75309,-1.81402)
(0.48,-0.96) (3.7735,-3.91373)
(0.72,-1.44) (0.982976,-7.04563)
```

```
(0.96,-1.92) (-4.69677,-11.9562)
(1.2,-2.4) (-14.344,-19.392)
(1.44,-2.88) (-29.037,-30.0995)
(1.68,-3.36) (-49.854,-44.8251)
(1.92,-3.84) (-77.8733,-64.3154)
(2.16,-4.32) (-114.173,-89.3169)
(2.4,-4.8) (-159.832,-120.576)
(2.64,-5.28) (-215.928,-158.839)
```

Zadatak 3.2 Vremenski intervali

Napisati na jeziku C++ klasu vremenskih intervala s rezolucijom od jedne sekunde. Predvideti:

- stvaranje vremena na osnovu broja časova, minuta i sekundi i na osnovu ukupnog broja sekundi,
- dohvaćanje delova vremena,
- sve aditivne aritmetičke operacije s vremenima,
- množenje i deljenje vremena celim brojem,
- upoređivanje dva vremena, i
- čitanje vremena iz ulaznog toka i upisivanje vremena u izlazni tok.

Koristiti operatorske funkcije kad god je to primereno.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:

```
// vreme.h - Klasa vremenskih intervala.

#include <iostream>
using namespace std;

class Vreme {
    long t; // Vreme u sekundama.
    typedef const Vreme CV; // Nepromenljivo vreme.
public:
    // Sastavljanje vremena.
    Vreme(int h, int m, int s) {
        t = (h * 60L + m) * 60 + s;
    }
    Vreme(long s=0) { t = s; } // Sekunde u vreme.
    friend int cas(CV& v) { return v.t / 3600; } // Časovi u vremenu.
    friend int min(CV& v) { return (v.t/60)%60; } // Minute u vremenu.
    friend int sek(CV& v) { return v.t % 60; } // Sekunde u vremenu.
    friend long Sek(CV& v) { return v.t; } // Ukupno sekundi.
    Vreme operator+ ()const { return *this; } // +t
    Vreme operator- ()const { return Vreme(-t); } // -t
    Vreme& operator++() { ++t; return *this; } // ++t
    Vreme& operator--() { --t; return *this; } // --t
    Vreme operator++(int) { Vreme w(*this); t++; return w; } // t++
    Vreme operator--(int) { Vreme w(*this); t--; return w; } // t--
    friend Vreme operator+(CV& v, CV& w) // v + w
    { return Vreme(v.t + w.t); }
    friend Vreme operator-(CV& v, CV& w) // v - w
    { return Vreme(v.t - w.t); }
    friend Vreme operator*(CV& v, int k) // v * k
    { return Vreme(v.t * k); }
    friend Vreme operator/(CV& v, int k) // v / k
    { return Vreme(v.t / k); }
    Vreme& operator+=(CV& v) { t += v.t; return *this; } // t += v
    Vreme& operator-=(CV& v) { t -= v.t; return *this; } // t -= v
    Vreme& operator*=(int k) { t *= k; return *this; } // t *= k
    Vreme& operator/=(int k) { t /= k; return *this; } // t /= k
    friend bool operator< (CV& v, CV& w) { return v.t < w.t; } // v < w
    friend bool operator<= (CV& v, CV& w) { return v.t <= w.t; } // v <= w
    friend bool operator> (CV& v, CV& w) { return v.t > w.t; } // v > w
    friend bool operator>= (CV& v, CV& w) { return v.t >= w.t; } // v >= w
    friend bool operator== (CV& v, CV& w) { return v.t == w.t; } // v == w
    friend bool operator!= (CV& v, CV& w) { return v.t != w.t; } // v != w
}
```

```
friend istream& operator>>(istream& ut, Vreme& v) // Čitanje.
{ int h, m, s; ut >> h >> m >> s; v = Vreme(h, m, s); return ut; }
friend ostream& operator<<(ostream& it, CV& v) { // Pisanje.
    Vreme w(v); if (w.t < 0) { it << '-'; w = -w; }
    return it << cas(w) << ':' << min(w) << ':' << sek(w);
}
};
```

// vremet.C - Ispitivanje klase vremenskih intervala.

```
#include "vreme.h"
#include <iostream>
using namespace std;

int main() {
    while (true) {
        int k; cin >> k;
        if (!k) break;
        Vreme t1, t2; cin >> t1 >> t2;
        long s1 = Sek(t1), s2 = Sek(t2);
        cout << "k = " << k << endl;
        cout << "t1, t2 = " << t1 << ", " << t2 << endl;
        cout << "s1 = " << s1 << " (" << Vreme(s1) << ")\n";
        cout << "s2 = " << s2 << " (" << Vreme(s2) << ")\n";
        cout << "- t1 = " << -t1 << endl;
        cout << "++ t1 = " << ++t1 << endl;
        cout << "-- t2 = " << t2-- << endl;
        cout << "t1 = " << t1 << " (" << Sek(t1) << ")\n";
        cout << "t2 = " << t2 << " (" << Sek(t2) << ")\n";
        cout << "t1 + t2 = " << t1 + t2 << endl;
        cout << "t1 - t2 = " << t1 - t2 << endl;
        cout << "t1 == t2 = " << (t1 == t2) << endl;
        cout << "t1 > t2 = " << (t1 > t2) << endl;
        cout << "t1 * k = " << t1 * k << endl;
        cout << "t2 / k = " << t2 / k << endl;
        cout << "s1+t2*k = " << Vreme(s1) + t2*k << " (" <<
            << s1 + Sek(t2*k) << ")\n";
    }
    return 0;
}
```

vremet.pod

```
3 2 30 40 -1 -20 -30
0
```

```
% vremet <vremet.pod
k = 3
t1, t2 = 2:30:40, -1:20:30
s1 = 9040 (2:30:40)
s2 = -4830 (-1:20:30)
- t1 = -2:30:40
++ t1 = 2:30:41
t2 -- = -1:20:30
```

```
t1 = 2:30:41 (9041)
t2 = -1:20:31 (-4831)
t1 + t2 = 1:10:10
t1 - t2 = 3:51:12
t1 == t2 = 0
t1 > t2 = 1
t1 * k = 7:32:3
t2 / k = -0:26:50
s1+t2*k = -1:30:53 (-5453)
```

Zadatak 3.3 Nizovi kompleksnih brojeva

Napisati na jeziku C++ klasu nizova kompleksnih brojeva. Predvideti:

- stvaranje niza zadate dužine (podrazumevano 10) sa slučajnim sadržajem,
- stvaranje niza kao kopiju drugog niza,
- uništavanje niza,
- dodelu vrednosti jednog niza drugom,
- dohvaćanje dužine niza, i
- pristup elementu niza.

Koristiti operatorske funkcije kad god je to primereno.

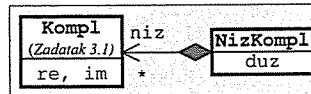
Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:

```
// nizkompl.h - Klasa nizova
// kompleksnih brojeva.
```

```
#include "kompl.h" → Zadatak 3.1
```

```
class NizKompl {
    int n; Kompl* niz; // Dužina i pokazivač na elemente.
    void kopiraj(const NizKompl&); // Kopiranje.
    void brisi() { delete [] niz; } // Uništavanje.
public:
    explicit NizKompl(int d=10) // Konstruktori:
    { niz = new Kompl [n = d]; } // - podrazumevani,
    NizKompl(const NizKompl& nk) // - kopije.
    { kopiraj(nk); }
    ~NizKompl() { brisi(); } // Destruktor.
    NizKompl& operator=(const NizKompl& nk) { // Dodela vrednosti.
        if (this != &nk) { brisi(); kopiraj(nk); }
        return *this;
    }
    int duz() const { return n; } // Dužina niza.
    Kompl& operator[](int i) // Element niza:
    { return niz[i]; } // - promenljivog,
    const Kompl& operator[](int i) const // - nepromenljivog.
    { return niz[i]; }
};
```



```
// nizkompl.C - Metoda klase nizova kompleksnih brojeva.
```

```
#include "nizkompl.h"
```

```
void NizKompl::kopiraj(const NizKompl& nk) {
    niz = new Kompl [n = nk.n];
    for (int i=0; i<n; i++) niz[i] = nk.niz[i];
}
```

```
// nizkompl.C - Ispitivanje klase nizova kompleksnih brojeva.
```

```
#include "nizkompl.h"
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
    while (true) {
        int n; cout<<"n? "; cin>>n;
        if (n <= 0) break;
        NizKompl a(n); cout << "a? ";
        for (int i=0; i<n; cin>>a[i++]); // Ne sme: *(a+i++)
        Kompl s = 0; for (int i=0; i<a.duz(); s+=a[i++]);
        cout << "s= " << s << endl;
    } // Tu se niz uništi destruktorem (na kraju svakog prolaza kroz ciklus).
    return 0;
}
```

```
% nizkompl
```

```
n? 3
```

```
a? 1 2 3 4 5 6
```

```
s= (9,12)
```

```
n? 5
```

```
a? 0 9 8 7 6 5 4 3 2 1
```

```
s= (20,25)
```

```
n? 0
```

Zadatak 3.4 Kvadri s automatski generisanim identifikacionim brojevima

Napisati na jeziku C++ klasu kvadara kod koje svaki objekat mora da ima različit, automatski generisan celobrojan identifikator. Predvideti:

- stvaranje kvadra zadatih dimenzija i kao kopiju drugog kvadra (novi kvadar u oba slučaja mora da dobije nov identifikator),
- dodelu vrednosti jednog kvadra drugom (oba kvadra moraju zadržati svoje identifikatore),
- izračunavanje zapremine kvadra,
- upoređivanje da li jedan kvadar ima manju zapreminu od drugog,
- čitanje kvadra iz ulaznog toka, i
- upisivanje kvadra u izlazni tok u obliku $K_i(a, b, c)$, gde su: i – identifikator broj kvadra, a, b, c – dimenzije kvadra.

Koristiti operatorske funkcije kad god je to primereno.

Napisati na jeziku C++ program koji pročitati niz kvadara s glavnog ulaza, ispiše pročitani niz na glavnom izlazu, pronađe kvadar najmanje zapremine i ispiše pronađeni kvadar na glavnom izlazu.

Rešenje:

```
// kvadar2.h - Klasa numerisanih kvadara.
```

```
#include <iostream>
using namespace std;
```

```
class Kvadar {
    static int posId;    // Poslednji identifikator.
    int id;              // Identifikator kvadra.
    double a, b, c;      // Dužine ivica.
public:
    explicit Kvadar(double aa=1, double bb=1, double cc=1) // Novom kvadru
    { id = ++posId; a = aa; b = bb; c = cc; }              // nov broj.
    Kvadar(const Kvadar& k)                                // Kopiji nov broj.
    { id = ++posId; a = k.a; b = k.b; c = k.c; }
    Kvadar& operator=(const Kvadar& k)                      // Levom operandu
    { a = k.a; b = k.b; c = k.c; return *this; }           // se ne menja broj.
    double V() const { return a * b * c; }                 // Zapremina.
    friend bool operator<(const Kvadar& k1, const Kvadar& k2) // k1 < k2
    { return k1.V() < k2.V(); }
    friend istream& operator>>(istream& ut, Kvadar& k)      // Čitanje.
    { return ut >> k.a >> k.b >> k.c; }
    friend ostream& operator<<(ostream& it, const Kvadar& k) { // Pisanje.
        return it << 'K' << k.id << '(' << k.a << ', '
            << k.b << ', ' << k.c << ')';
    }
};
```

```
// kvadar2.C - Statičko polje klase numerisanih kvadara.
```

```
#include "kvadar2.h"
```

```
int Kvadar::posId = 0;
```

```
// kvadar2t.C - Ispitivanje klase numerisanih kvadara.
```

```
#include "kvadar2.h"
#include <iostream>
using namespace std;
```

```
int main() {
    Kvadar niz[20];
    cout << "n? "; int n; cin >> n;           // Tu se stvara 20 kvadara.
    for (int i=0; i<n; i++)
        { cout << "niz[" << i << "]? "; cin >> niz[i]; }
    cout << "\nProcitano:\n";
    for (int i=0; i<n; i++)
        cout << niz[i] << "\t V=" << niz[i].V() << endl;
    Kvadar min = niz[0];
    for (int i=1; i<n; i++) if (niz[i] < min) min = niz[i]; // Tu se stvara jos jedan kvadar.
    cout << "\nmin= " << min << endl;
    return 0;
}
```

```
% kvadar2t
```

```
n? 5
```

```
niz[0]? 7 3 5
```

```
niz[1]? 2 6 4
```

```
niz[2]? 4 5 6
```

```
niz[3]? 2 8 5
```

```
niz[4]? 3 6 3
```

```
Procitano:
```

```
K1(7,3,5) V=105
```

```
K2(2,6,4) V=48
```

```
K3(4,5,6) V=120
```

```
K4(2,8,5) V=80
```

```
K5(3,6,3) V=54
```

```
min= K2(2,6,4)
```

Zadatak 3.5 Polinomi s realnim koeficijentima

Napisati na jeziku C++ klasu polinoma s realnim koeficijentima i realnim argumentom. Predvideti:

- stvaranje praznog polinoma, polinoma od niza koeficijenata i kao kopiju drugog polinoma,
- uništavanje polinoma,
- dodelu vrednosti jednog polinoma drugom,
- smeštanje sadržaja polinoma u niz,
- dohvatanje reda polinoma,
- izračunavanje vrednosti polinoma,
- pristup datom koeficijentu polinoma,
- izračunavanje zbira, razlike i proizvoda dva polinoma bez i s bočnim efektima, i
- čitanje polinoma iz ulaznog toka i upisivanje polinoma u izlazni tok.

Koristiti operatorske funkcije kad god je to primereno.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:

```
// pol1.h - Klasa polinoma.

#include <iostream>
using namespace std;

class Poli {
    int n; double* a;          // Red polinoma i pokazivač na koeficijente.
    typedef const Poli CP;     // Nepromenljiv polinom.
    void kopiraj(const double* a, int n); // Kopiranje u tekući objekat.
    void brisi() { delete[] a; a=0; n=-1; } // Pražnjenje polinoma.
public:
    Poli() { a = 0; n = -1; } // - prazan polinom
    Poli(const double* a, int n) { kopiraj(a, n); } // - od niza
    Poli(CP& p) { kopiraj(p.a, p.n); } // - od polinoma
    ~Poli() { brisi(); } // Destruktor.
    Poli& operator=(CP& p) { // Dodela vrednosti.
        if (this != &p) { brisi(); kopiraj(p.a, p.n); }
        return *this;
    }
    void niz(double* a, int& n) const; // Smeštanje polinoma u niz.
    int red() const { return n; } // Red polinoma.
    double operator()(double x) const; // Vrednost polinoma.
    double& operator[](int ind); // Pristupanje koeficijentu.
    const double& operator[](int ind) const;
    friend Poli operator+(CP& p1, CP& p2); // p1 + p2
    friend Poli operator-(CP& p1, CP& p2); // p1 - p2
    friend Poli operator*(CP& p1, CP& p2); // p1 * p2
    Poli& operator+=(CP& p2) { return *this = *this + p2; } // p1 += p2
    Poli& operator-=(CP& p2) { return *this = *this - p2; } // p1 -= p2
    Poli& operator*=(CP& p2) { return *this = *this * p2; } // p1 *= p2
    friend istream& operator>>(istream& ut, Poli& p); // Čitanje polinoma.
    friend ostream& operator<<(ostream& it, CP& p); // Pisanje polinoma.
private:
    enum Greska { GRED, GIND }; // Šifre grešaka.
    static void greska(Greska g); // Obrada greške.
};
```

// pol1.C - Metode i funkcije uz klasu polinoma.

```
#include "pol1.h"
#include <cstdlib>
using namespace std;

void Poli::greska(Greska g) { // Obrada greške.
    const char* poruke[] = { "Nedozvoljen red polinoma",
                              "Nedozvoljen indeks koeficijenta polinoma"
    };
    cout << "*** " << poruke[g] << "!\n";
    exit(g+1);
}

void Poli::kopiraj(const double* aa, int nn) { // Kopiranje u tekući objekat
    if ((n = nn) < -1) greska(GRED);
    a = new double [nn+1];
    for (int i=0; i<=nn; i++) a[i] = aa[i];
}

void Poli::niz(double* aa, int& nn) const // Smeštanje polinoma u niz.
{ for (int i=0; i<=n; i++) aa[i] = a[i]; nn = n; }

double Poli::operator()(double x) const // Vrednost polinoma.
{ double s = 0; for (int i=n; i>=0; (s*=x)+=a[i--]); return s; }

double& Poli::operator[](int ind) { // Pristupanje koeficijentu.
    if (ind < 0 || ind > n) greska(GIND);
    return a[ind];
}

const double& Poli::operator[](int ind) const {
    if (ind < 0 || ind > n) greska(GIND);
    return a[ind];
}

Poli operator+(const Poli& p1, const Poli& p2) { // Zbir dva polinoma.
    int n;
    if (p1.n == p2.n) for (n=p1.n; n>=0 && p1.a[n]+p2.a[n]==0; n--);
    else n = (p1.n > p2.n) ? p1.n : p2.n;
    Poli p; p.n = n; p.a = new double [n+1];
    for (int i=0; i<=n; i++)
        p.a[i] = (i > p2.n) ? p1.a[i] :
                  (i > p1.n) ? p2.a[i] :
                  p1.a[i] + p2.a[i];
    return p;
}

Poli operator-(const Poli& p1, const Poli& p2) { // Razlika dva polinoma.
    int n;
    if (p1.n == p2.n) for (n=p1.n; n>=0 && p1.a[n]-p2.a[n]==0; n--);
    else n = (p1.n > p2.n) ? p1.n : p2.n;
    Poli p; p.n = n; p.a = new double [n+1];
    for (int i=0; i<=n; i++)
        p.a[i] = (i > p2.n) ? p1.a[i] :
                  (i > p1.n) ? -p2.a[i] :
                  p1.a[i] - p2.a[i];
    return p;
}
```

```

Poli operator*(const Poli& p1, const Poli& p2){ // Proizvod dva polinoma.
    int n = (p1.n>=0 && p2.n>=0) ? p1.n+p2.n : -1;
    Poli p; p.n = n; p.a = new double [n+1];
    for (int i=0; i<=n; p.a[i++]=0);
    for (int i=0; i<=p1.n; i++)
        for (int j=0; j<=p2.n; j++)
            p.a[i+j] += p1.a[i] * p2.a[j];
    return p;
}

istream& operator>>(istream& ut, Poli& p) { // Čitanje polinoma.
    p.brisi();
    ut >> p.n; if (p.n < -1) Poli::greska(Poli::GRED);
    p.a = new double [p.n+1];
    for (int i=p.n; i>=0; ut >> p.a[i--]);
    return ut;
}

ostream& operator<<(ostream& it, const Poli& p){ // Pisanje polinoma.
    it << "p[";
    for (int i=p.n; i>=0; i--) it << p.a[i] << (i ? ", " : "");
    it << ']';
    return it;
}

```

// polilt.C - Ispitivanje klase polinoma.

```

#include "polil.h"
#include <iostream>
#include <iomanip>
using namespace std;

```

```

int main() {
    // Definisanje tri polinoma i inicijalizacija kao prazni polinomi.
    Poli p1, p2, p3;

    // Čitanje polinoma, aritmetičke operacije i dodela vrednosti.
    cin >> p1; cout << "p1 = " << p1 << endl;
    cin >> p2; cout << "p2 = " << p2 << endl;
    p3 = p1 + p2; cout << "p1+p2 = " << p3 << endl;
    p3 = p1 - p2; cout << "p1-p2 = " << p3 << endl;
    p3 = p2 - p1; cout << "p2-p1 = " << p3 << endl;
    p3 = p1 - p1; cout << "p1-p1 = " << p3 << endl;
    p3 = p1 * p2; cout << "p1*p2 = " << p3 << endl;

    // Konverzija polinoma u niz i obrnuto.
    double* a = new double [p1.red() + 1]; int n; p1.niz(a, n);
    cout << "a = ";
    for (int i=n; i>=0; i--) cout << a[i] << ' '; cout << endl;
    p3 = Poli(a, n); cout << "p(a,n) = ";
    for (int i=p3.red(); i>=0; i--) cout << p3[i] << ' '; cout << endl;
    delete [] a;

    // Korišćenje pokazivača na polinom.
    Poli* pp4 = new Poli(p1);
    cout << "*pp4 = " << *pp4 << endl;
    delete pp4;
}

```

```

// Tabeliranje polinoma.
double xmin, xmax, dx; cin >> xmin >> xmax >> dx;
cout << "\nxmin, xmax, dx = "
    << xmin << ", " << xmax << ", " << dx << endl;
cout << "\n x      p1(x)\n===== \n" << fixed;
for (double x=xmin; x<=xmax; x+=dx)
    cout << setprecision(2) << setw(6) << x
        << setprecision(6) << setw(12) << p1(x) << endl;
return 0;
}

```

polilt.pod

```

5 1 2 3 4 5 6
4 5 4 3 2 1
-1 1 .25

```

```

% polilt <polilt.pod
p1 = p[1,2,3,4,5,6]
p2 = p[5,4,3,2,1]
p1+p2 = p[1,7,7,7,7,7]
p1-p2 = p[1,-3,-1,1,3,5]
p2-p1 = p[-1,3,1,-1,-3,-5]
p1-p1 = p[]
p1*p2 = p[5,14,26,40,55,70,50,32,17,6]
a = 1 2 3 4 5 6
p(a,n) = 1 2 3 4 5 6
*pp4 = p[1,2,3,4,5,6]

xmin, xmax, dx = -1, 1, 0.25

```

x	p1(x)
-1.00	3.000000
-0.75	3.629883
-0.50	4.218750
-0.25	4.959961
0.00	6.000000
0.25	7.555664
0.50	10.031250
0.75	14.135742
1.00	21.000000

Zadatak 3.6 *Studenti koji ne smeju da se kopiraju*

Podatke o studentima čine prezime i ime (niska promenljive dužine), broj indeksa (dugačak ceo broj po šemi ggggrrrrr, gde su: g – godina upisa i r – registarski broj), broj ocena na ispitima i niz ocena zadatog kapaciteta. Napisati na jeziku C++ klasu za studente. Predvideti:

- stvaranje objekta zadatim imenom, brojem indeksa i dozvoljenim brojem ocena (podrazumevano 40),
- uništavanje objekta,
- sprečavanje kopiranja objekta inicijalizacijom i dodelom vrednosti,
- dohvaćanje i promenu imena i broja indeksa,
- dodavanje nove ocene (operator +=),
- dohvaćanje broja ocena,
- dohvaćanje ocene sa zadatim rednim brojem (operator []), i
- izračunavanje srednje ocene.

Napisati na jeziku C++ program koji pročita podatke o određenom broju studenata s glavnog ulaza i posle toga ispiše njihova imena i srednje ocene na glavnom izlazu po opadajućem redosledu srednjih ocena.

Rešenje:

```
// student1.h - Klasa studenata.

class Student {
    char* ime;           // Ime studenta.
    long ind;            // Broj indeksa (ggggrrrrr).
    int* ocene;          // Niz ocena.
    int kap, brOc;       // Kapacitet niza i broj ocena.
    Student(const Student&) {} // Ne sme da se kopira.
    void operator=(const Student&) {} // Ne sme da se dodeljuje.
public:
    Student(const char* iime, long iind, int kkap=40); // Konstruktor.
    ~Student() { delete [] ime; delete [] ocene; } // Destruktor.
    const char* dohvatiIme() const { return ime; } // Ime studenta.
    long dohvatiInd() const { return ind; } // Broj indeksa.
    Student& promeniIme(const char* iime); // Promena imena.
    Student& promeniInd(long iind) // Promena broja indeksa.
    { ind = iind; return *this; }
    Student& operator+=(int ocena); // Dodavanje ocene.
    int brOcena() const { return brOc; } // Broj ocena.
    int& operator[](int i); // Dohvaćanje ocene.
    int operator[](int i) const;
    double srOcena() const; // Srednja ocena.
};
```

```
// student.C - Metode klase studenata.

#include "student1.h"
#include <cstring>
#include <cstdlib>
using namespace std;

Student::Student(const char* iime, long iind, int kkap) { // Konstruktor.
    ime = new char [strlen(iime)+1];
    strcpy(ime, iime);
    ind = iind;
    ocene = new int [kap = kkap];
    brOc = 0;
}

Student& Student::promeniIme(const char* iime) { // Promena imena.
    delete [] ime;
    ime = new char [strlen(iime)+1];
    strcpy(ime, iime);
    return *this;
}

Student& Student::operator+=(int ocena) { // Dodavanje ocene.
    if (brOc == kap) exit(1);
    ocene[brOc++] = ocena;
    return *this;
}

int& Student::operator[](int i) { // Dohvaćanje ocene.
    if (i<0 || i>=brOc) exit(2);
    return ocene[i];
}

int Student::operator[](int i) const {
    if (i<0 || i>=brOc) exit(2);
    return ocene[i];
}

double Student::srOcena() const { // Srednja ocena.
    double s = 0;
    for (int i=0; i<brOc; s+=ocene[i++]);
    return brOc ? s/brOc : 0;
}
```

```
// student1t.C - Ispitivanje klase studenata.
```

```
#include "student1.h"
#include <iostream>
using namespace std;

int main() {
    int n; cin >> n;
    Student** s = new Student* [n];
    for (int i=0; i<n; i++) {

        // Čitanje podataka i stvaranje sledećeg studenta.
        char ime[50]; long ind; cin >> ime >> ind;
        Student* t = new Student(ime, ind);
        int brOc; cin >> brOc;
        for (int j=0; j<brOc; j++) { int oc; cin >> oc; *t += oc; }

        // Umetanje u uređeni niz.
        int k = i;
        while (k>0 && s[k-1]->srOcena()<t->srOcena()) { s[k] = s[k-1]; k--; }
        s[k] = t;
    }

    // Ispisivanje uređenog niza.
    for (int i=0; i<n; i++)
        cout << s[i]->dohvatiIme() << '\t' << s[i]->srOcena() << endl;
    return 0;
}
```

```
studentt.pod
```

```
5
Markovic_Marko 20000055 7 8 9 6 7 10 6 8
Petrovic_Petar 20010123 5 9 9 9 9
Zoranovic_Zoran 20010222 4 6 7 8 9
Mirkovic_Mirko 20010049 3 7 8 9
Jovanovic_Jovan 20030153 0
```

```
% student1t <student1t.pod
```

```
Petrovic_Petar 9
Mirkovic_Mirko 8
Markovic_Marko 7.71429
Zoranovic_Zoran 7.5
Jovanovic_Jovan 0
```

Zadatak 3.7 Redovi brojeva neograničenih kapaciteta

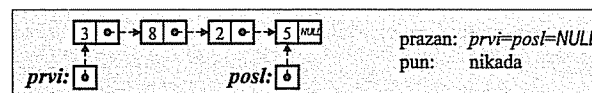
Napisati na jeziku C++ klasu redova celih brojeva neograničenih kapaciteta. Predvideti:

- stvaranje praznog reda, reda koji sadži jedan zadati broj i kao kopiju drugog reda,
- uništavanje reda,
- dodelu vrednosti jednog reda drugom,
- dohvaćanje dužine reda,
- ispitivanje da li je red prazan,
- dodavanje jednog podatka i sadržaja celog reda na kraj reda,
- uzimanje podatka s početka reda (uz izbacivanje) i dohvaćanje podatka koji je na početku reda (bez izbacivanja),
- izbacivanje iz reda svih podataka koji imaju zadatu vrednost, i
- pražnjenje reda.

Pojedine operacije ostvariti što prikladnije odabranim operatorskim funkcijama.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:



```
// red3.h - Klasa redova neograničenog kapaciteta.
```

```
#include <cstdlib>
using namespace std;

class Red {
    struct Elem {
        int pod; // Element reda:
        Elem* sled; // - sadržaj elementa,
        Elem(int p, Elem* s=0) { pod = p; sled = s; } // - sledeći element;
    }; // - konstruktor.
    Elem* prvi, *posl; // Početak i kraj reda.
    int duz; // Dužina reda.
    void kopiraj(const Red& r); // Kopiranje reda.
    void brisi(); // Brisanje svih elemenata.
public:
    Red() { prvi = posl = 0; duz = 0; } // Konstruktor praznog reda.
    Red(int a) { prvi = posl = new Elem(a); duz = 1; } // Konverzija int u Red.
    Red(const Red& r) { kopiraj(r); } // Konstruktor kopije.
    ~Red() { brisi(); } // Destruktor.
    Red& operator=(const Red& r) { // Dodela vrednosti.
        if (this != &r) { brisi(); kopiraj(r); }
        return *this;
    }
    int duzina() const { return duz; } // Dužina reda.
    bool prazan() const { return !duz; } // Da li je red prazan?
    Red& operator+=(const Red& r); // Dodavanje podatka ili reda.
    Red operator+ (const Red& r) const { // Spoj dva reda.
        return Red(*this) += r;
    }
};
```



```

int uzmi(); // Uzimanje podatka s početka.
int vodeci() const { // Vrednost podatka na početku.
    if (!prvi) exit(1);
    return prvi->pod;
}
Red operator- (int k) const; // Red bez elemenata jednakih k.
Red& operator-=(int k) // Brisanje svih elemenata jednakih k.
{ return *this = *this - k; }
Red& operator~() // Brisanje svih elemenata.
{ brisi(); return *this; }
};

```

// red3.C - Metode i funkcije uz klasu redova.

```
#include "red3.h"
```

```

void Red::kopiraj(const Red& r) { // Kopiranje reda.
    prvi = posl = 0; duz = r.duz;
    for (Elem* tek=r.prvi; tek; tek=tek->sled)
        posl = (posl ? posl->sled : prvi) = new Elem(tek->pod);
}

```

```

void Red::brisi() { // Brisanje svih elemenata reda.
    while (prvi) { Elem* stari = prvi; prvi = prvi->sled; delete stari; }
    posl = 0; duz = 0;
}

```

```
Red& Red::operator+=(const Red& r) { // Dodavanje reda.
```

```

    if (r.prvi) {
        Red s(r);
        (posl ? posl->sled : prvi) = s.prvi;
        posl = s.posl; duz += s.duz;
        s.prvi = s.posl = 0;
        s.duz = 0;
    }
    return *this;
}

```

bez ovoga

// Uzimanje podatka s početka.

```

int Red::uzmi() {
    if (!prvi) exit(1);
    int pod = prvi->pod;
    Elem* stari = prvi;
    prvi = prvi->sled;
    delete stari;
    if (!prvi) posl = 0;
    duz--;
    return pod;
}

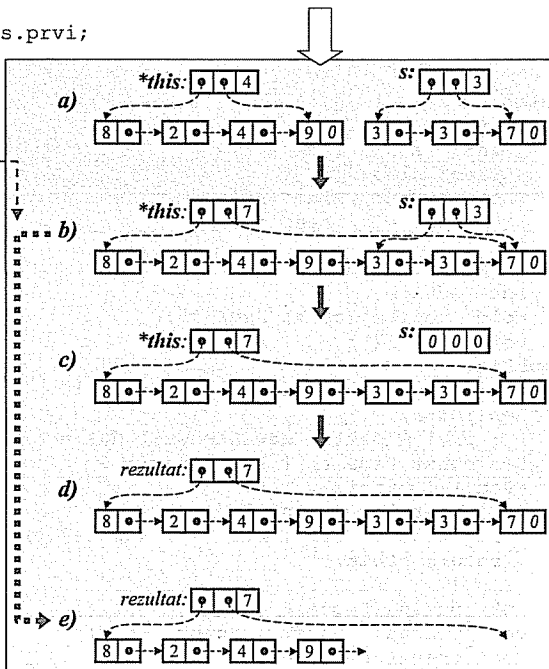
```

// Red bez elemenata jednakih k.

```

Red Red::operator-(int k) const {
    Red r;
    for (Elem* tek=prvi; tek;
         tek=tek->sled)
        if (tek->pod != k)
            r += tek->pod;
    return r;
}

```



// red3t.C - Ispitivanje klase redova neograničenog kapaciteta.

```

#include "red3.h"
#include <iostream>
using namespace std;

```

```

Red citaj(const char* nasl) {
    Red r; int n, a, i; cin >> n;
    cout << nasl << " :";
    for (i=0; i<n; i++) { cin >> a; cout << ' ' << a; r += a; } cout << endl;
    return r;
}

```

```

void pisi(const char* nasl, const Red& red) {
    Red r = red;
    cout << nasl << " :";
    while (r.duzina()) cout << ' ' << r.uzmi(); cout << endl;
}

```

```

int main() {
    Red r1 = citaj("niz1 ");
    pisi("r1 ", r1 );
    Red r2 = citaj("niz2 ");
    pisi("r2 ", r2 );
    pisi("r1+r2 ", r1+r2);
    pisi("r1+=r2", r1+=r2);
    pisi("vodeci", r1.vodeci()); // Konverzija int u Red.
    pisi("uzmi ", r1.uzmi()); // Konverzija int u Red.
    pisi("r1 ", r1);
    int a; cin >> a;
    pisi("a ", a); // Konverzija int u Red.
    pisi("r1-a ", r1-a );
    pisi("r1-=a ", r1-=a);
    pisi("~r1 ", ~r1 );
    return 0;
}

```

red3t.pod

```

6 2 4 1 3 2 5
5 1 2 2 3 2
2

```

```
% red3t <red3t.pod
```

```

niz1 : 2 4 1 3 2 5
r1 : 2 4 1 3 2 5
niz2 : 1 2 2 3 2
r2 : 1 2 2 3 2
r1+r2 : 2 4 1 3 2 5 1 2 2 3 2
r1+=r2 : 2 4 1 3 2 5 1 2 2 3 2
vodeci : 2
uzmi : 2
r1 : 4 1 3 2 5 1 2 2 3 2
a : 2
r1-a : 4 1 3 5 1 3
r1-=a : 4 1 3 5 1 3
~r1 :

```

Zadatak 3.8 Tekstovi

Napisati na jeziku C++ klasu tekstova. Predvideti:

- stvaranje praznog teksta, teksta od običnog niza znakova i kao kopiju drugog teksta,
- uništavanje teksta,
- dodelu vrednosti jednog teksta drugom,
- dohvatanje dužine teksta,
- spajanje dva teksta i dodavanje jednog teksta na kraj drugog,
- pristupanje znaku u tekstu,
- izdvajanje podniza između dve pozicije,
- nalaženje mesta prvog pojavljivanja podniza u tekstu, i
- čitanje teksta iz ulaznog toka i upisivanje teksta u izlazni tok.

Pojedine operacije ostvariti što prikladnije odabranim operatorskim funkcijama.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:

```
// tekst1.h - Klasa tekstova.
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class Tekst {
    char* niz; // Pokazivač na sam tekst.
    void kopiraj(const char* n); // Kopiranje teksta.
    void brisi() { delete [] niz; } // Uništavanje teksta.
public: // Inicijalizacija:
    Tekst() { niz = 0; } // - kao prazan tekst,
    Tekst(const char* n) { kopiraj(n); } // - niskom,
    Tekst(const Tekst& t) { kopiraj(t.niz); } // - drugim tekstom.
    ~Tekst() { brisi(); } // Uništavanje.
    Tekst& operator=(const Tekst& t) { // Dodela vrednosti.
        if (this != &t) { brisi(); kopiraj(t.niz); }
        return *this;
    }
    friend int len(const Tekst& t) // Dužina teksta.
    { return t.niz ? strlen(t.niz) : 0; }
    friend Tekst operator+ // Spajanje tekstova.
    (const Tekst&, const Tekst&);
    Tekst& operator+=(const Tekst& t) // Dodavanje teksta.
    { return *this = *this + t; }
    char& operator[](int i) { // Znak u promenljivom tekstu.
        if (i<0 || len(*this)<=i) exit(1);
        return niz[i];
    }
    const char& operator[](int i) const { // Znak u nepromenljivom tekstu.
        if (i<0 || len(*this)<=i) exit(1);
        return niz[i];
    }
    Tekst operator()(int, int) const; // Podniz teksta.
    friend int operator%(const Tekst&, const Tekst&); // Mesto podniza.
    friend ostream& operator<<(ostream& it, const Tekst& t) // Pisanje teksta.
    { if (t.niz) it << t.niz; return it; }
    friend istream& operator>>(istream&, Tekst&); // Čitanje teksta.
};
```

```
// tekst1.C - Metode i funkcije uz klasu tekstova.
#include "tekst1.h"

void Tekst::kopiraj(const char* n) { // Kopiranje teksta.
    if (n && strlen(n)) {
        niz = new char [strlen(n)+1]; strcpy(niz, n);
    } else niz = 0;
}

Tekst operator+(const Tekst& t1, const Tekst& t2) { // Spajanje tekstova.
    if (!t1.niz) return t2;
    if (!t2.niz) return t1;
    Tekst t; t.niz = new char [len(t1) + len(t2) + 1];
    strcpy(t.niz, t1.niz); strcat(t.niz, t2.niz); return t;
}

Tekst Tekst::operator()(int i, int j) const { // Podniz teksta.
    if (i<0 || j<i || len(*this)<=j) exit(1);
    Tekst t; int d = j - i + 1;
    t.niz = new char [d+1]; strncpy(t.niz, niz+i, d); t.niz[d] = 0;
    return t;
}

int operator%(const Tekst& t1, const Tekst& t2) { // Mesto podniza.
    int d1 = len(t1), d2 = len(t2);
    if (!d1 || !d2) return -1;
    for (int i=0; i<d1-d2+1; i++) {
        int j;
        for (j=0; j<d2 && t1.niz[i+j]==t2.niz[j]; j++);
        if (j == d2) return i;
    }
    return -1;
}

istream& operator>>(istream& ut, Tekst& t) { // Čitanje teksta.
    char* n = new char [81]; ut >> n; t = n; delete [] n;
    return ut;
}
```

// tekst1t.C - Ispitivanje klase tekstova.

```
#include "tekst1.h"
#include <iostream>
using namespace std;

int main(int barg, char* varg[]) {
    Tekst t;
    while (--barg) { t += *varg; if (barg > 1) t += " "; }
    cout << len(t) << " - " << t << endl;
    int k = 1; for (int i=0; i<len(t); i++) if (t[i] == ' ') k++;
    cout << "Broj reci: " << k << endl;
    int i;
    while ((i = t % " ") >= 0) {
        Tekst r = t(0, i-1);
        cout << len(r) << " - " << r << endl;
        t = t(i+1, len(t)-1);
    }
    cout << len(t) << " - " << t << endl;
    return 0;
}
```

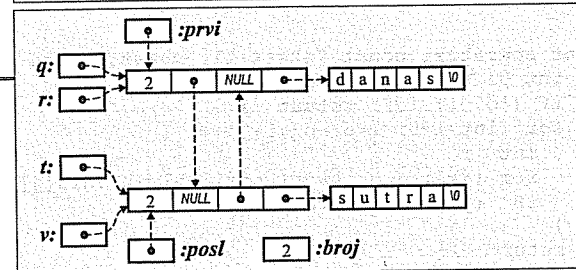
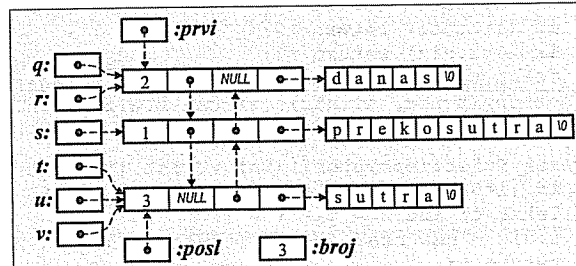
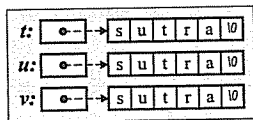
```
% tekst1t Danas je divan dan!
19 - Danas je divan dan!
Broj reci: 4
5 - Danas
2 - je
5 - divan
4 - dan!
```

Zadatak 3.9 Tekstovi s uštedom memorije

Napisati na jeziku C++ klasu tekstova iz prethodnog zadatka tako da se ista niska nikada ne nalazi više puta u memoriji.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:



```
// tekst2.h - Štedljiva
// klasa tekstova.
```

```
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;
```

```
class Tekst {
    struct Tks {                // Element liste svih tekstova:
        int kor;                // - broj korišćenja teksta,
        Tks *sled, *pret;       // - pokazivači na sledeći i prethodni tekst,
        char* niz;              // - pokazivač na nisku.
    };
    static Tks *prvi, *posl;    // Pokazivači na prvi i poslednji tekst.
    static int broj;            // Ukupan broj tekstova.

    Tks* tks;                   // Pokazivač na tekst.
    void kopiraj(const Tekst& t) // Kopiranje teksta.
    { if ((tks = t.tks) != 0) tks->kor++; }
    void brisi();               // Uništavanje teksta.

public:
    Tekst() { tks = 0; }        // Inicijalizacija:
    Tekst(const char*);         // - kao prazan tekst,
    Tekst(const Tekst& t) { kopiraj(t); } // - nizom znakova,
    ~Tekst() { brisi(); }        // - drugim tekstom.
    Tekst& operator=(const Tekst& t) { // Uništavanje.
        if (this != &t) { brisi(); kopiraj(t); } // Dodela vrednosti.
        return *this;
    }
    friend int len(const Tekst& t) // Dužina teksta.
    { return t.tks ? strlen(t.tks->niz) : 0; }
    friend Tekst operator+ // Spajanje tekstova.
    (const Tekst&, const Tekst&);
};
```

```
Tekst& operator+=(const Tekst& t) // Dodavanje teksta.
{ return *this = *this + t; }
const char& operator[](int i) const { // Znak u tekstu
    if (i<0 || len(*this)<=i) exit(1); // (ne može da se promeni).
    return tks->niz[i];
}
Tekst operator()(int, int) const; // Podniz teksta.
friend int operator%(const Tekst&, const Tekst&); // Mesto podniza.
friend ostream& operator<<(ostream& it, const Tekst& t) // Pisanje teksta.
{ if (t.tks) it << t.tks->niz; return it; }
friend istream& operator>>(istream&, Tekst&); // Čitanje teksta.
static int ukupno() { return broj; } // Ukupan broj tekstova.
static void spisak(); // Spisak svih tekstova.
};
```

```
// tekst2.C - Metode, funkcije i statička polja
// uz štedljivu klasu tekstova.
```

```
#include "tekst2.h"
```

```
void Tekst::brisi() { // Uništavanje teksta.
    if (tks && --tks->kor==0) {
        (tks->pret ? tks->pret->sled : prvi) = tks->sled;
        (tks->sled ? tks->sled->pret : posl) = tks->pret;
        delete [] tks->niz;
        delete tks;
        broj--;
    }
}

Tekst::Tekst(const char* n) { // Inicijalizacija niskom.
    if (n==0 || strlen(n)==0) { // PRAZAN TEKST.
        tks = 0;
    } else { // NIJE PRAZAN TEKST.
        int f; Tks* tek;
        for (tek=prvi; tek && ((f=strcmp(tek->niz,n))<0); tek=tek->sled);
        if (tek && f == 0) // Tekst već postoji u memoriji.
            (tks = tek)->kor++;
        else { // Tekst još ne postoji u memoriji:
            tks = new Tks; tks->kor = 1;
            tks->niz = new char [strlen(n)+1]; strcpy(tks->niz, n);
            broj++;
            if (!prvi) { // - jedini u listi,
                prvi = posl = tks; tks->sled = tks->pret = 0;
            } else if (!tek) { // - iza poslednjeg,
                tks->sled = 0; tks->pret = posl;
                if (posl) posl->sled = tks; posl = tks;
            } else if (!tek->pret) { // - ispred prvog,
                tks->pret = 0; tks->sled = prvi;
                if (prvi) prvi->pret = tks; prvi = tks;
            } else { // - usred liste.
                tks->sled = tek->pret->sled; tek->pret->sled = tks;
                tks->pret = tek->pret; tek->pret = tks;
            }
        }
    }
}
```

```

Tekst operator+(const Tekst& t1, const Tekst& t2) { // Spajanje tekstova.
    if (!t1.tks) return t2;
    if (!t2.tks) return t1;
    char* niz = new char [len(t1) + len(t2) + 1];
    strcpy(niz, t1.tks->niz); strcat(niz, t2.tks->niz);
    Tekst t(niz);
    delete [] niz;
    return t;
}

Tekst Tekst::operator()(int i, int j) const { // Podniz teksta.
    if (i<0 || j<i || len(*this)<=j) exit(1);
    Tekst t; int d = j - i + 1;
    t.tks->niz = new char [d+1]; strncpy(t.tks->niz, tks->niz+i, d);
    t.tks->niz[d] = 0;
    return t;
}

int operator%(const Tekst& t1, const Tekst& t2) { // Mesto podniza.
    int d1 = len(t1), d2 = len(t2);
    if (!d1 || !d2) return -1;
    for (int i=0; i<d1-d2+1; i++) {
        int j;
        for (j=0; j<d2 && t1.tks->niz[i+j]==t2.tks->niz[j]; j++);
        if (j == d2) return i;
    }
    return -1;
}

istream& operator>>(istream& ut, Tekst& t) { // Čitanje teksta.
    char* n = new char [81]; ut >> n; if (ut) t = n; delete [] n;
    return ut;
}

void Tekst::spisak() { // Spisak svih tekstova.
    int poz = 0, duz;
    for (Tks* tek=prvi; tek; tek=tek->sled) {
        if (poz + (duz = strlen(tek->niz) + 2) < 72) poz += duz;
        else { cout << endl; poz = duz; }
        cout << " " << tek->niz;
    }
    cout << endl;
}

Tekst::Tks *Tekst::prvi=0, *Tekst::posl=0; // Inicijalizacija statičkih
int Tekst::broj = 0; // polja.

```

```
// tekst2t.C - Ispitivanje štedljive klase tekstova.
```

```

#include "tekst2.h"
#include <iostream>
using namespace std;

int main() {
    struct Elem { Tekst tekst; Elem* sled; };
    Elem *prvi=0, *posl=0, *novi;
    Tekst t;
    while (true) {
        cin >> t;
        if (!cin) break;
        novi = new Elem; novi->tekst = t; novi->sled = 0;
        posl = (!prvi ? prvi : posl->sled) = novi;
    }
    Tekst::spisak();
    return 0;
}

```

```

% tekst2t <tekst2.h
!= #include &t) ((tks (const (i<0 (ne (t.tks) (this *posl;
*pret; *prvi, *sled, *this *this; + - // 0) 0; : <<
<cstdlib> <cstring> <iostream> = ? Citanje Dodavanje Dodela
Duzina Element Inicijalizacija: Kopiranje Mesto Pisanje Podniz
Pokazivac Pokazivaci Spajanje Spisak Stedljiva Tekst Tekst&
Tekst&); Tekst&, Tekst() Tekst(const Tks Tks* Ukupan
Unistavanje. Uništavanje Znak brisi(); broj broj; char& char*
char*); class const const; da drugim exit(1); friend i i)
if int int) istream& it it, it; kao klasa kopiraj(const
kopiraj(t); kor; koriscenja len(*this)<=i) len(const liste moze
na namespace nisku. niz; nizom operator%(const operator()(int,
operator+ operator+=(const operator<<(ostream& operator=(const
operator>>(istream&, operator[])(int ostream& podniza. pokazivac
pokazivaci poslednji prazan prethodni promeni). prvi public:
return se sledeci spisak(); static std; strlen(t.tks->niz)
struct svih t) t)// t.tks t.tks) t.tks->niz; t; tekst,
tekst. tekst2.h teksta, teksta. tekstom. tekstova. tekstova:
tekstu tks tks->kor++; tks->niz[i]; tks; u ukupno() using
void vrednosti. znakova, { || } ); ~Tekst()

```

Zadatak 3.10 Karte i predstave

Karta za predstavu sadrži broj reda, broj sedišta u redu i cenu (realan broj). Napisati na jeziku C++ klasu karta. Predvideti:

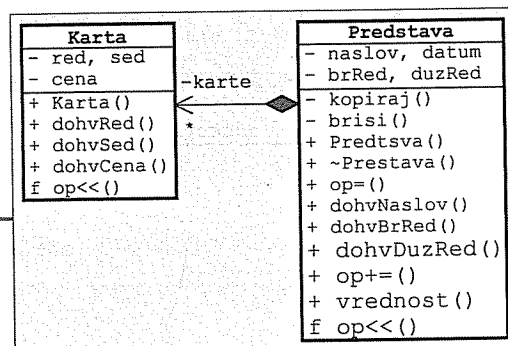
- inicijalizaciju karte zadatim brojem reda, brojem sedišta i cene (ispravnost podataka ne treba proveravati),
- dohvatanje broja reda, broja sedišta u redu i cene karte, *i*
- upisivanje karte u izlazni tok (`it<<karta`) u obliku (*red, sedišta, cena*).

Predstava ima naslov, datum (dugačak ceo broj po šemi ggggmmdd), broj redova u sali, dužinu redova (svaki red ima isti broj sedišta) i matricu pokazivača na prodane karte. Napisati na jeziku C++ klasu za predstavu. Predvideti:

- inicijalizaciju prazne predstave (bez prodanih karata) zadatog naslova, datuma, broja i dužine redova,
- inicijalizaciju predstave drugom predstavom,
- uništavanje predstave,
- dodeljivanje predstavi vrednost druge predstave (`predst1=predst2`),
- dohvatanje naslova, datuma, broja redova i dužine redova,
- dodavanje prodane karte na odgovarajućem mestu (`predst+=karta`; vrednost funkcije je indikator uspeha; neuspeh je ako je broj reda ili sedišta u karti izvan opsega, odnosno ako se karta odnosi na već popunjeno mesto),
- izračunavanje vrednosti svih prodanih karata, *i*
- upisivanje predstave u izlazni tok (`it<<predst`; piše se matricni prikaz slobodnih ('_') i zauzetih ('#') mesta).

Napisati na jeziku C++ **program** koji napravi jednu predstavu, dodaje nekoliko prodanih karata, ispiše na glavnom izlazu predstavu i vrednost svih prodanih karata. Koristiti konstantne parametre (ne treba ništa učitavati s glavnog ulaza).

Rešenje:



```
// karta.h - Klasa karata.
```

```
#include <iostream>
using namespace std;
```

```
class Karta {
    int red, sed;
    float cena;
public:
    Karta(int r, int s, float c) // Stvaranje.
    { red = r; sed = s; cena = c; }
    int dohvRed() const { return red; } // Dohvatanje broja reda.
    int dohvSed() const { return sed; } // Dohvatanje broja sedista.
    float dohvCena() const { return cena; } // Dohvatanje cene.
    friend ostream& operator<<(ostream& it, const Karta& k) { // Pisanje.
        return it << '(' << k.red << ',' << k.sed << ',' << k.cena << ')'; }
};
```

```
// predstava.h - Klasa predstava.
```

```
#include "karta.h"
#include <iostream>
using namespace std;

class Predstava {
    char* naslov; // Naslov.
    long datum; // Datum održavanja.
    Karta** karte; // Matrica karata.
    int brRed, duzRed; // Broj redova i dužina redova.
    void kopiraj(const Predstava& p); // Kopiranje u objekat.
    void brisi(); // Uništavanje objekta.
public:
    // Konstruktori:
    Predstava(const char* nas, long dat, // - novog objekta,
              int brR, int duzR);
    Predstava(const Predstava& p) // - kopije.
    { kopiraj(p); }
    ~Predstava() { brisi(); } // Destruktor.
    Predstava& operator=(const Predstava& p) {
        if (this != &p) { brisi(); kopiraj(p); }
        return *this;
    }
    // Dohvatanje:
    const char* dohvNaslov() const { return naslov; } // - naslova,
    long dohvDatum() const { return datum; } // - datuma,
    int dohvBrRed() const { return brRed; } // - broja redova,
    int dohvDuzRed() const { return duzRed; } // - dužine redova.
    bool operator+=(const Karta& k); // Dodavanje karte.
    float vrednost() const; // Vrednost svih karata.
    friend ostream& operator<<(ostream& it, const Predstava& p); // Pisanje.
};
```

```
// predstava.C - Metode i funkcije uz klasu predstava.
```

```
#include "predstava.h"
```

```
void Predstava::kopiraj(const Predstava& p) { // Kopiranje u objekat.
    naslov = new char [strlen(p.naslov)+1];
    naslov = strcpy(naslov, p.naslov);
    datum = p.datum; brRed = p.brRed;
    duzRed = p.duzRed;
    karte = new Karta** [brRed];
    for (int i=0; i<brRed; i++) {
        karte[i] = new Karta* [duzRed];
        for (int j=0; j<duzRed; j++)
            karte[i][j] = p.karte[i][j] ? new Karta(*p.karte[i][j]) : 0;
    }
}

void Predstava::brisi() { // Uništavanje objekta.
    delete [] naslov;
    for (int i=0; i<brRed; i++) {
        for (int j=0; j<duzRed; delete karte[i][j++]);
        delete [] karte[i];
    }
    delete [] karte;
}
```

```

Predstava::Predstava(const char* nas, long dat, int brR, int duzR) {
    naslov = new char [strlen(nas)+1]; // Konstruktor novog objekta.
    strcpy(naslov, nas);
    datum = dat; brRed = brR; duzRed = duzR;
    karte = new Karta** [brRed];
    for (int i=0; i<brRed; i++) {
        karte[i] = new Karta* [duzRed];
        for (int j=0; j<duzRed; karte[i][j++]=0);
    }
}

bool Predstava::operator+=(const Karta& k) { // Dodavanje karte.
    int red = k.dohvRed(), sed = k.dohvSed();
    if (red<0 || red>=brRed || sed<0 || sed>=duzRed || karte[red][sed])
        return false;
    karte[red][sed] = new Karta(k);
    return true;
}

float Predstava::vrednost() const { // Vrednost svih karata.
    float v = 0;
    for (int i=0; i<brRed; i++)
        for (int j=0; j<duzRed; j++)
            if (karte[i][j]) v += karte[i][j]->dohvCena();
    return v;
}

ostream& operator<<(ostream& it, const Predstava& p) { // Pisanje.
    for (int i=0; i<p.brRed; i++) {
        for (int j=0; j<p.duzRed; j++)
            it << (p.karte[i][j] ? '#' : '_') << ' ';
        it << endl;
    }
    return it;
}

```

// predstavat.C - Ispitivanje klasa predstava i karata.

```

#include "predstava.h"
#include <iostream>
using namespace std;

int main() {
    Predstava p("Naslov", 20090308, 4, 6);
    p += Karta(0, 3, 800);
    p += Karta(3, 5, 200);
    p += Karta(2, 1, 500);
    cout << p << endl << p.vrednost() << endl;
    return 0;
}

```

% predstavat

```

- - - # - - -
- - - - - - -
- # - - - - -
- - - - - #

```

1500

Zadatak 3.11 Zapisi artikala i inventari

Zapis o artiklu ima naziv artikla, količinu, oznaku jedinice mere (više znakova) i cenu. Napisati na jeziku C++ klasu za zapise. Predvideti:

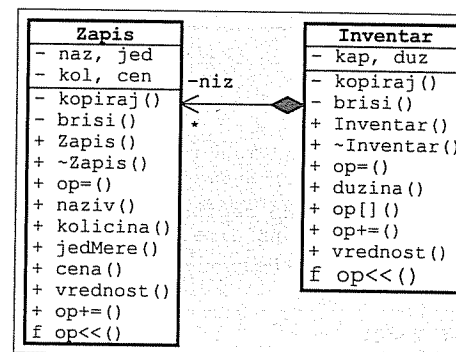
- inicijalizaciju zapisa nazivom artikla, oznakom jedinice mere i cenom (početna količina je 0);
- inicijalizaciju zapisa drugim zapisom;
- uništavanje zapisa;
- dodeljivanje zapisu vrednost drugog zapisa (zap1=zap2);
- dohvatanje naziva artikla, količine, oznake jedinice mere i cene;
- određivanje vrednosti količine artikla predstavljene zapisom;
- menjanje količine artikla za zadati iznos (zap+=promena; povratna vrednost je indikator uspeha – neuspeh je ako bi količina postala negativna);
- upisivanje zapisa u izlazni tok (it<<zap; piše se naziv artikla, količina s jedinicom mere, cena i vrednost s dodatkom *din*).

Inventar sadrži niz pokazivača na zapise određenog kapaciteta. Napisati na jeziku C++ klasu za inventare. Predvideti:

- inicijalizaciju praznog inventara (bez zapisa u njemu) i zadatog kapaciteta (podrazumevano 10);
- inicijalizaciju inventara drugim inventarom;
- uništavanje inventara;
- dodeljivanje inventaru vrednost drugog inventara (inv1=inv2);
- dohvatanje broja zapisa u inventaru;
- dohvatanje pokazivača na zapis sa zadatim nazivom artikla (inv[naz]; povratna vrednost je pokazivač na pronađeni zapis ili 0 ako takav zapis ne postoji);
- dodavanje zapisa inventaru pomoću adrese (inv+=&zap; povratna vrednost je indikator uspeha – neuspeh je ako se prekorači kapacitet inventara ili ako u inventaru već postoji zapis istog naziva);
- određivanje ukupne vrednosti artikala u zapisima u inventaru;
- upisivanje sadržaja inventara u izlazni tok (it<<inv; pišu se zapisi u inventaru red po red i na kraju ukupna vrednost artikala).

Napisati na jeziku C++ **program** koji napravi inventar kapaciteta 8, dodaje tri zapisa za različite artikle, u svakom zapisu postavi količinu i ispiše sadržaj inventara na glavnom izlazu. Koristiti konstantne parametre (ne treba ništa učitavati s glavnog ulaza).

Rešenje:



```
// zapis.h - Klasa zapisa o artiklu.
```

```
#include <cstring>
#include <iostream>
using namespace std;

class Zapis {
    char *naz, *jed; // Naziv i jedinica mere.
    double kol, cen; // Količina i cena.
    void kopiraj(const Zapis& z) { // Kopiranje u objekat.
        naz = new char [strlen(z.naz)+1]; strcpy(naz, z.naz);
        jed = new char [strlen(z.jed)+1]; strcpy(jed, z.jed);
        kol = z.kol; cen = z.cen;
    }
    void brisi() { delete [] naz; delete [] jed; } // Brisanje objekta.
public:
    Zapis(const char* n, const char* j, double c) { // Stvaranje objekta.
        naz = new char [strlen(n)+1]; strcpy(naz, n);
        jed = new char [strlen(j)+1]; strcpy(jed, j);
        kol = 0; cen = c;
    }
    Zapis(const Zapis& z) { kopiraj(z); } // Konstruktor kopije.
    ~Zapis() { brisi(); } // Destruktor.
    Zapis& operator=(const Zapis& z) { // Dodela vrednosti.
        if (this != &z) { brisi(); kopiraj(z); }
        return *this;
    }
    // Dohvatanje:
    const char* naziv() const { return naz; } // - naziva,
    double kolicina() const { return kol; } // - količine,
    const char* jedMere() const { return jed; } // - jedinice mere,
    double cena() const { return cen; } // - cene.
    double vrednost() const { return cen * kol; } // Izračunavanje vrednosti
    bool operator+=(double dk) { // Promena količine.
        if (kol + dk < 0) return false;
        kol += dk; return true;
    }
    friend ostream& operator<<(ostream& it, const Zapis& z) { // Pisanje.
        return it << z.naz << ": " << z.kol << z.jed << " x "
            << z.cen << "din = " << z.vrednost() << "din";
    }
};
```

```
// inventar.h - Klasa inventara.
```

```
#include "zapis.h"
#include <iostream>
using namespace std;

class Inventar {
    Zapis** niz; // Niz pokazivača na zapise.
    int kap, duz; // Kapacitet i dužina niza.
    void kopiraj(const Inventar& inv); // Kopiranje u objekat.
    void brisi(); // Brisanje objekta.
public:
    explicit Inventar(int k=10) // Stvaranje objekta.
    { niz = new Zapis* [kap = k]; duz = 0; }
    Inventar(const Inventar& inv) { kopiraj(inv); } // Konstruktor kopije.
    ~Inventar() { brisi(); } // Destruktor.
};
```

```
Inventar& operator=(const Inventar& inv) { // Dodela vrednosti.
    if (this != &inv) { brisi(); kopiraj(inv); }
    return *this;
}
int duzina() const { return duz; } // Dužina niza.
Zapis* operator[](const char* naziv); // Dohvatanje zapisa:
const Zapis* operator[](const char* naziv) const { // - iz promenljivog,
    (return const_cast<Inventar*>(*this)[naziv]); } // objekta.
bool operator+=(Zapis* z) { // Dodavanje zapisa.
    if (duz==kap || (*this)[z->naziv()]) return false;
    niz[duz++] = z; return true;
}
double vrednost() const; // Vrednost svih artikala.
friend ostream& operator<<(ostream& it, const Inventar& inv); // Pisanje.
};
```

```
// inventar.C - Metode i funkcije uz klasu inventara.
```

```
#include "inventar.h"
#include <cstring>
using namespace std;

void Inventar::kopiraj(const Inventar& inv) { // Kopiranje u objekat.
    niz = new Zapis* [kap = inv.kap];
    duz = inv.duz;
    for (int i=0; i<duz; i++) niz[i] = new Zapis(*inv.niz[i]);
}

void Inventar::brisi() { // Uništavanje objekta.
    for (int i=0; i<duz; delete niz[i++]);
    delete [] niz;
}

Zapis* Inventar::operator[](const char* naziv) { // Dohvatanje zapisa
    for (int i=0; i<duz; i++) // promenljivog objekta.
        if (strcmp(niz[i]->naziv(), naziv) == 0)
            return niz[i];
    return 0;
}

double Inventar::vrednost() const { // Vrednost svih artikala.
    double v = 0;
    for (int i=0; i<duz; v+=niz[i++]->vrednost());
    return v;
}

ostream& operator<<(ostream& it, const Inventar& inv) { // Pisanje.
    for (int i=0; i<inv.duz; it<<*inv.niz[i++]<<endl);
    return it << "=====\n"
        << "UKUPNO: " << inv.vrednost() << "din" << endl;
}
```

```
// inventart.C - Ispitivanje klasa zapisa i inventara.
```

```
#include "inventar.h"
#include <iostream>
using namespace std;

int main() {
    Inventar inv(8);
    inv += new Zapis("kruska", "kg", 95);
    inv += new Zapis("mleko", "l", 75);
    inv += new Zapis("kabl", "m", 120);
    *inv["kruska"] += 8;
    *inv["mleko"] += 35;
    *inv["kabl"] += 155;
    cout << inv;
    return 0;
}
```

```
% inventart
kruska: 8kg x 95din = 760din
mleko: 35l x 75din = 2625din
kabl: 155m x 120din = 18600din
=====
UKUPNO: 21985din
```

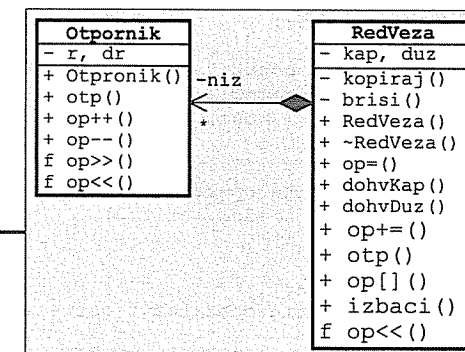
Zadatak 3.12 Otpornici i redne veze otpornika

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorom i operatorom za dodelu vrednosti koji su potrebni za bezbedno korišćenje klase):

- Otpornost promenljivog **otpornika** može skokovito da se poveća ili smanjuje. Stvara se sa zadatom početnom vrednošću (≥ 0 , podrazumevano $1k\Omega$) i korakom promene vrednosti (>0 , podrazumevano 1Ω). Može da se dohvati trenutna vrednost otpornosti, da se otpornost poveća ($++r$) ili smanji ($--r$) za jedan korak, da se otpornik pročita iz ulaznog toka ($ut \gg r$) i da se upiše u izlazni tok ($it \ll r$) u obliku $R(r, \Delta r)$, gde su: r – trenutna vrednost otpornosti i Δr – iznos koraka promene. Parametre prilikom stvaranja otpornika ne treba proveravati. Ako bi otpornost pri smanjivanju trebalo da postane negativna, postavlja se na nulu.
- Redna veza** otpornika može da sadrži zadati broj otpornika. Stvara se prazna zadatog kapaciteta (podrazumevano 2), posle čega se otpornici dodaju pojedinačno ($veza += r$; u slučaju neuspeha program se prekida). Može da se dohvati kapacitet i broj popunjenih mesta, da se izračuna otpornost veze kao zbir otpornosti sadržanih otpornika, da se pristupi otporniku zadatog rednog broja ($veza[i]$, nedozvoljen indeks prekida program), da se izbace otpornici nulte vrednosti i da se veza upiše u izlazni tok ($it \ll veza$) u obliku $r+r+...+r$, gde je: r – rezultat pisanja jednog sadržanog otpornika.

Napisati na jeziku C++ **program** koji, čitajući podatke s glavnog ulaza, prvo napravi rednu vezu otpornika od nekoliko otpornika. Zatim, sve dok u rednoj vezi postoji bar jedan otpornik, ispiše vezu i otpornost veze na glavnom izlazu, smanji vrednost svakog sadržanog otpornika za jedan korak i izbaci iz veze sve otpornike čije su otpornosti postale nula.

Rešenje:



```
// otpornik.h - Klasa otpornika.
```

```
#include <iostream>
using namespace std;
```

```
class Otpornik {
    double r, dr;
public:
    Otpornik(double rr=1000, double ddr=1) // Stvaranje objekta.
    { r = rr; dr = ddr; }
    double otp() const { return r; } // Dohvatanje otpornosti.
    Otpornik& operator++() // Povećanje otpornosti.
    { r += dr; return *this; }
    Otpornik& operator--() { // Smanjivanje otpornosti.
        r -= dr; if (r < 0) r = 0;
        return *this;
    }
    friend istream& operator>>(istream& ut, Otpornik& r) // Čitanje.
    { return ut >> r.r >> r.dr; }
    friend ostream& operator<<(ostream& it, const Otpornik& r) // Pisanje
    { return it << "R(" << r.r << ',' << r.dr << ')'; }
};
```

// Otpornost i korak promene.


```
// redveza.h - Klasa rednih veza otpornika.
```

```
#include "otpornik.h"
#include <iostream>
#include <cstdlib>
using namespace std;

class RedVeza {
    Otpornik* niz;           // Niz otpornika.
    int kap, duz;           // Kapacitet i dužina niza.
    void kopiraj(const RedVeza& rv); // Kopiranje u objekat.
    void brisi() { delete [] niz; } // Brisanje objekta.
public:
    explicit RedVeza (int k=2) { // Stvaranje objekta.
        niz = new Otpornik [kap = k];
        duz = 0;
    }
    RedVeza(const RedVeza& rv) { kopiraj(rv); } // Konstruktor kopije.
    ~RedVeza() { brisi(); } // Destruktor.
    RedVeza& operator=(const RedVeza& rv) { // Dodela vrednosti.
        if (this != &rv) { brisi(); kopiraj(rv); }
        return *this;
    }
    // Dohvatanje:
    int dohvKap() const { return kap; } // - kapaciteta veze,
    int dohvDuz() const { return duz; } // - dužine veze.
    RedVeza& operator+=(const Otpornik& r) { // Dodavanje otpornika.
        if (duz == kap) exit(1);
        niz[duz++] = r;
        return *this;
    }
    // Otpornost veze.
    double otp() const; // Pristup elementu veze:
    Otpornik& operator[](int i) { // - promenljivog objekta,
        if (i<0 || i>=duz) exit(2);
        return niz[i];
    }
    // - nepromenljivog objekta.
    const Otpornik& operator[](int i) const {
        if (i<0 || i>=duz) exit(2);
        return niz[i];
    }
    RedVeza& izbaci(); // Izbacivanje nultih otpornosti.
    friend ostream& operator<<(ostream& it, const RedVeza& rv); // Pisanje.
};
```

```
// redveza.C - Metode i funkcije uz klasu rednih veza otpornika.
```

```
#include "redveza.h"

void RedVeza::kopiraj(const RedVeza& rv) { // Kopiranje u objekat.
    niz = new Otpornik [kap = rv.kap];
    duz = rv.duz;
    for (int i=0; i<duz; i++) niz[i] = rv.niz[i];
}

double RedVeza::otp() const { // Otpornost veze.
    double r = 0;
    for (int i=0; i<duz; i++) r+=niz[i].otp();
    return r;
}
```

```
RedVeza& RedVeza::izbaci() { // Izbacivanje nultih otpornosti.
    int j=0;
    for (int i=0; i<duz; i++)
        if (niz[i].otp()) niz[j++] = niz[i];
    duz = j;
    return *this;
}

ostream& operator<<(ostream& it, const RedVeza& rv) { // Pisi.
    for (int i=0; i<rv.duz; i++) {
        if (i) it << '+';
        it << rv.niz[i];
    }
    return it;
}
```

```
// redvezat.C - Ispitivanje klase rednih veza otpornika.
```

```
#include "redveza.h"
#include <iostream>
using namespace std;

int main() {
    cout << "n? "; int n; cin >> n;
    RedVeza rv(n);
    for (int i=0; i<n; i++) {
        cout << "r,d? "; Otpornik r; cin >> r; rv += r;
    }
    while (rv.dohvDuz()) {
        cout << rv << '=' << rv.otp() << endl;
        for (int i=0; i<rv.dohvDuz(); --rv[i++]);
        rv.izbaci();
    }
}
```

```
% redvezat
n? 4
r,d? 4 1
r,d? 8 3
r,d? 5 2
r,d? 10 3
R(4,1)+R(8,3)+R(5,2)+R(10,3)=27
R(3,1)+R(5,3)+R(3,2)+R(7,3)=18
R(2,1)+R(2,3)+R(1,2)+R(4,3)=9
R(1,1)+R(1,3)=2
```

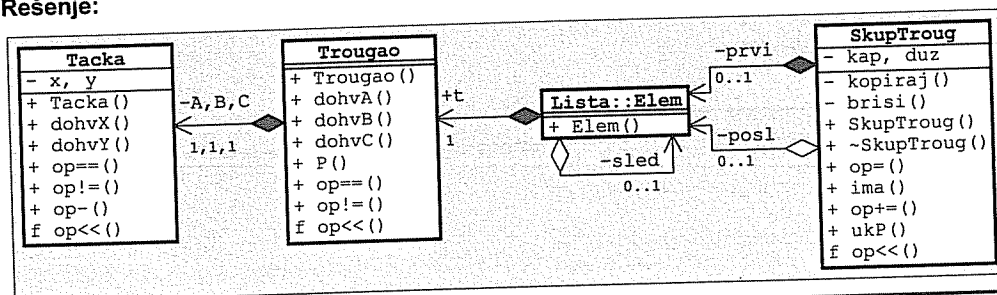
Zadatak 3.13 Tačke, trouglovi, skupovi trouglova u ravni

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorom i operatorom za dodelu vrednosti, koji su potrebni za bezbedno korišćenje klasa):

- **Tacka** u ravni zadaje se realnim koordinatama x i y , podrazumevano $(0,0)$. Mogu da se dohvate koordinate tačke, da se ispita da li se dve tačke poklapaju ($T1==T2$) ili ne ($T1!=T2$), da se izračuna rastojanje između dve tačke ($T1-T2$; $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$) i da se tačka upiše u izlazni tok ($it << T$) u obliku (x,y) .
- **Trougao** u ravni zadaje se se tri tačke A, B i C koje čine temena trougla, podrazumevano $((0,0), (1,0)$ i $(0,1)$). Mogu da se dohvate temena trougla, da se izračuna površina trougla ($P = \sqrt{s(s-a)(s-b)(s-c)}$; $s = (a+b+c)/2$; gde su: a, b i c – dužine stranica trougla), da se ispita da li su dva trougla podudarna ($t1==t2$) ili ne ($t1!=t2$; podudarna su samo ako im se poklapaju istoimena temena) i da se trougao upiše u izlazni tok ($it << t$) u obliku $[A, B, C]$, dge su A, B i C – rezultati upisivanja temena trougla.
- **Skup trouglova** u ravni može da sadrži proizvoljan broj nepodudarnih trouglova. Stvara se prazan, posle čega se trouglovi dodaju pojedinačno ($s+=t$). Pokušaj dodavanja podudarnog trougla nekom trouglu u skupu se ignoriše. Može da se ispita da li se zadati trougao nalazi u skupu, da se izračuna ukupna površina sadržanih trouglova i da se skup upiše u izlazni tok ($it << s$) u obliku $\{t, t, \dots, t\}$, gde je t – rezultat upisivanja jednog trougla.

Napisati na jeziku C++ **program** koji napravi jedan skup, doda mu nekoliko trouglova i ispiše na glavnom izlazu dobijeni skup i ukupnu površinu trouglova u skupu. Koristiti fiksne parametre (ne treba ništa učitavati s glavnog ulaza).

Rešenje:



// tacka3.h – Klasa tačkaka u ravni.

```

#include <cmath>
#include <iostream>
using namespace std;

class Tacka {
public:
    double x, y; // Koordinate tačke.
    explicit Tacka(double xx=0, double yy=0) // Stvaranje tačke.
    { x = xx; y = yy; }
    double dohvX() const { return x; } // Dohvatanje koordinata.
    double dohvY() const { return y; }
    bool operator==(const Tacka& T2) const // Da li se poklapaju?
    { return x==T2.x && y==T2.y; }
    bool operator!=(const Tacka& T2) const // Da li se ne poklapaju?
    { return x!=T2.x || y!=T2.y; }
};
  
```

```

double operator-(const Tacka& T2) const // Rastojanje dve tačke.
{ return sqrt(pow(x-T2.x, 2) + pow(y-T2.y, 2)); }
friend ostream& operator<<(ostream& it, const Tacka& T) // Pisanje.
{ return it << '(' << T.x << ',' << T.y << ')'; }
};
  
```

// trougao3.h – Klasa trouglova u ravni.

```

#include "tacka3.h"
#include <iostream>
using namespace std;

class Trougao {
public:
    Tacka A, B, C; // Temena trougla.
    explicit Trougao( // Stvaranje trougla.
        const Tacka& AA=Tacka(),
        const Tacka& BB=Tacka(1,0),
        const Tacka& CC=Tacka(0,1)
    ): A(AA), B(BB), C(CC) {}
    Tacka dohvA() const { return A; } // Dohvatanje temena.
    Tacka dohvB() const { return B; }
    Tacka dohvC() const { return C; }
    double P() const { // Računanje površine.
        double a = B-C, b = C-A, c = A-B;
        double s = (a + b + c) / 2;
        return sqrt(s * (s-a) * (s-b) * (s-c));
    }
    bool operator==(const Trougao& t2) const // Da li su podudarni?
    { return A == t2.A && B == t2.B && C == t2.C; }
    bool operator!=(const Trougao& t2) const // Da li su nepodudarni?
    { return !(*this == t2); }
    friend ostream& operator<<(ostream& it, const Trougao& t) // Pisanje.
    { return it << '[' << t.A << ',' << t.B << ',' << t.C << ']'; }
};
  
```

// skuptroug.h – Klasa skupova trouglova u ravni.

```

#include "trougao3.h"
#include <iostream>
using namespace std;

class SkupTroug {
public:
    struct Elem { // Element liste.
        Trougao t; Elem* sled;
        Elem(const Trougao& tt): t(tt), sled(0) {}
    };
    Elem *prvi, *posl; // Početak i kraj liste.
    void kopiraj(const SkupTroug& s); // Kopiranje u objekat.
    void brisi(); // Brisanje objekta.
    SkupTroug() { prvi = posl = 0; } // Stvaranje praznog skupa.
    SkupTroug(const SkupTroug& s) // Konstruktor kopije.
    { kopiraj(s); }
    ~SkupTroug() { brisi(); } // Destruktor.
    SkupTroug& operator=(const SkupTroug& s) { // Dodela vrednosti.
        if (this != &s) { brisi(); kopiraj(s); }
        return *this;
    }
    bool ima(const Trougao& t) const; // Da li postoji u skupu?
    SkupTroug& operator+=(const Trougao& t); // Dodavanje trougla.
    double ukP() const; // Ukupna površina trouglova.
};
  
```

```

    friend ostream& operator<<(ostream& it, const SkupTroug& s); // Pisanje.
};

// skuptroug.C - Metode i funkcije uz klasu trouglova u ravni.
#include "skuptroug.h"
void SkupTroug::kopiraj(const SkupTroug& s) { // Kopiranje u objekat.
    prvi = posl = 0;
    for (Elem *tek=s.prvi; tek; tek=tek->sled)
        posl = (!prvi ? prvi : posl->sled) = new Elem(tek->t);
}
void SkupTroug::brisi() { // Brisanje objekta.
    while (prvi) { Elem* stari = prvi; prvi = prvi->sled; delete stari; }
    posl = 0;
}
bool SkupTroug::ima(const Trougao& t) const { // Da li postoji u skupu?
    for (Elem* tek=prvi; tek; tek=tek->sled)
        if (tek->t == t) return true;
    return false;
}
SkupTroug& SkupTroug::operator+=(const Trougao& t) { // Dodavanje trougla.
    if (!ima(t)) posl = (!prvi ? prvi : posl->sled) = new Elem(t);
    return *this;
}
double SkupTroug::ukP() const { // Ukupna površina trouglova.
    double p = 0;
    for (Elem* tek=prvi; tek; tek=tek->sled) p += tek->t.P();
    return p;
}
ostream& operator<<(ostream& it, const SkupTroug& s) { // Pisanje.
    it << '{';
    for (SkupTroug::Elem* tek=s.prvi; tek; tek=tek->sled) {
        it << tek->t;
        if (tek->sled) it << ',';
    }
    return it << '}';
}

```

```

// skuptroug.C - Ispitivanje klase skupova trouglova u ravni.
#include "skuptroug.h"
#include <iostream>
using namespace std;
int main() {
    SkupTroug s;
    s += Trougao(Tacka( 1, 1), Tacka( 2, 5), Tacka( 4, 3));
    s += Trougao(Tacka(-1, 1), Tacka(-2, 5), Tacka(-4, 3));
    s += Trougao(Tacka( 1, 1), Tacka( 2, 5), Tacka( 4, 3));
    s += Trougao(Tacka(-1, 1), Tacka(-2, 5), Tacka(-4, 3));
    cout << s << endl << s.ukP() << endl;
    return 0;
}

```

```

% skuptroug
{[(1,1),(2,5),(4,3)], [(-1,1), (-2,5), (-4,3)]}
10

```

4 Izvedene klase

Zadatak 4.1 Valjci i kante

Valjak se zadaje poluprečnikom i visinom. Napisati na jeziku C++ klasu valjaka. Predvideti:

- stvaranje valjka zadatog poluprečnika i visine,
- dohvaćanje poluprečnika i visine,
- izračunavanje zapremine, i
- upisivanje valjka u izlazni tok (operator <<).

Kanta je valjak u koji može da se sipa tečnost. Napisati na jeziku C++ klasu kanti kao izvedenu klasu iz klase valjaka. Pored mogućnosti osnovne klase predvideti:

- stvaranje kante zadatog poluprečnika, visine i početne popunjenosti,
- dohvaćanje količine tečnosti u kanti i određivanje koliko tečnosti može još da se dolije,
- ispitivanje da li je kanta skroz puna i da li je skroz prazna,
- dolivanje određene količine tečnosti (ako se kanta prepuni, višak tečnosti se prosipa – operator +=),
- odlivanje određene količine tečnosti (operator -=),
- presipanje moguće najveće količine tečnosti iz jedne kante u drugu (operator =), i
- upisivanje kante u izlazni tok (operator <<).

Napisati na jeziku C++ program za ispitivanje prethodnih klasa.

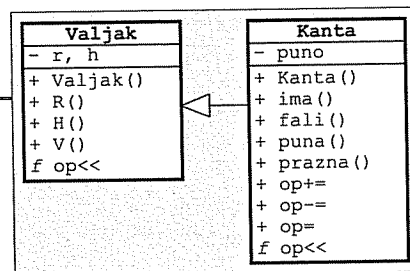
Rešenje:

// valjak1.h - Klasa valjaka.

```
#include <iostream>
using namespace std;
```

```
const double PI = 3.1415926535897932;
```

```
class Valjak {
    double r, h;
public:
    explicit Valjak(double rr=1, double hh=1): // Inicijalizacija.
        r(rr), h(hh) {}
    double R() const { return r; } // Poluprečnik.
    double H() const { return h; } // Visina.
    double V() const { return r * r * PI * h; } // Zapremina.
    friend ostream& operator<<(ostream& it, const Valjak& v) // Pisanje.
    { return it << '[' << v.r << ',' << v.h << ']' << '\n'; }
};
```



// kanta.h - Klasa kanti.

```
#include "valjak1.h"
```

```
class Kanta: public Valjak {
    double puno;
    Kanta(const Kanta&) {} // Napunjeni deo.
public:
    Kanta(double rr=1, double hh=1, double pp=0): // Inicijali-
        Valjak(rr, hh), puno(pp<=V() ? pp : V()) {} // zacija.
    double ima() const { return puno; } // Koliko ima tečnosti?
    double fali() const { return V() - puno; } // Koliko tečnosti fali?
    bool puna() const { return puno == V(); } // Da li je puna?
    bool prazna() const { return puno == 0; } // Da li je prazna?
};
```

```
Kanta& operator+=(double dopuna) { // Dolivanje.
    puno = (puno+dopuna <= V()) ? puno+dopuna : V();
    return *this;
}
Kanta& operator-=(double odliv) { // Odlivanje.
    if (puno-odliv > 0) puno -= odliv; else puno = 0;
    return *this;
}
Kanta& operator=(Kanta& k) { // Presipanje.
    if (this != &k) {
        double prazno = fali();
        if (prazno >= k.puno) { puno += k.puno; k.puno = 0; }
        else { puno = V(); k.puno -= prazno; }
    }
    return *this;
}
friend ostream& operator<<(ostream& it, const Kanta& k) { // Pisanje.
    return it << '[' << static_cast<const Valjak&>(k) << ',' <<
        << k.puno << ']' << '\n';
}
};
```

// valjak1t.C - Ispitivanje klase valjaka i kanti.

```
#include "kanta.h"
#include <iostream>
using namespace std;
```

```
int main() {
    Valjak v1(2, 3); cout << "v1 : " << v1 << '\n' << v1.V() << endl;
    Kanta k1(1, 3, 10); cout << "k1(10): " << k1 << '\n' << k1.V() << endl;
    cout << "k1-=5 : " << (k1-=5) << endl;
    cout << "k1+=4 : " << (k1+=4) << endl;
    Kanta k2(0.6, 5); cout << "k2(0) : " << k2 << '\n' << k2.V() << endl;
    cout << "k2=k1 : " << (k2=k1) << endl;
    cout << "k1 : " << k1 << endl;
    Valjak& uv = k1; cout << "uv : " << uv << '\n' << uv.V() << endl;
    Valjak* pv = &k1; cout << "*pv : " << *pv << '\n' << pv->V() << endl;
    Kanta& uk = k1; cout << "uk : " << uk << '\n' << uk.V() << endl;
    return 0;
}
```

```
% valjak1t
v1 : [2,3] 37.6991
k1(10): {[1,3],9.42478} 9.42478
k1-=5 : {[1,3],4.42478}
k1+=4 : {[1,3],8.42478}
k2(0) : {[0.6,5],0} 5.65487
k2=k1 : {[0.6,5],5.65487}
k1 : {[1,3],2.76991}
uv : [1,3] 9.42478
*pv : [1,3] 9.42478
uk : {[1,3],2.76991} 9.42478
```

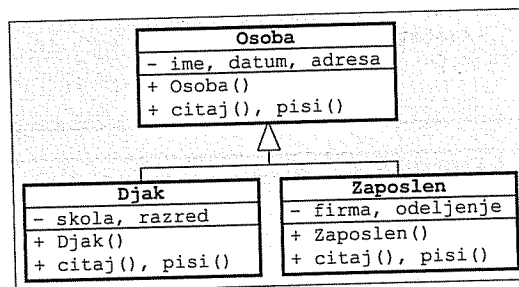
Zadatak 4.2 Osobe, đaci i zaposleni

Podatke o *osobi* čine ime, datum rođenja i adresa stanovanja. *Dak* je osoba za koju se dodatno znaju naziv škole i razred koji pohađa. *Zaposlen* je osoba za koju se dodatno znaju naziv firme i naziv odeljenja u kome radi.

Napisati na jeziku C++ klase koje omogućavaju unificiranu obradu podataka o nabrojanim kategorijama osoba. Sva polja su niske fiksne dužine. Predvideti inicijalizaciju primeraka klasa praznim niskama, unošenje podataka u njih čitanjem s glavnog ulaza i prikazivanje njihovog sadržaja pisanjem na glavnom izlazu.

Napisati na jeziku C++ program za ispitivanje prethodnih klasa.

Rešenje:



// osobe.h - Klase osoba, daka i zaposlenih.

```

class Osoba {
    char ime[31], datum[11], adresa[31];
public:
    Osoba() { ime[0] = datum[0] = adresa[0] = 0; }
    virtual void citaj();
    virtual void pisi() const;
};

class Djak: public Osoba {
    char skola[31], razred[7];
public:
    Djak(): Osoba() { skola[0] = razred[0] = 0; }
    void citaj();
    void pisi() const;
};

class Zaposlen: public Osoba {
    char firma[31], odeljenje[31];
public:
    Zaposlen(): Osoba() { firma[0] = odeljenje[0] = 0; }
    void citaj();
    void pisi() const;
};
  
```

// osobe.C - Metode klase osoba, daka i zaposlenih.

```

#include "osobe.h"
#include <iostream>
using namespace std;

void Osoba::citaj() {
    cout << "Ime i prezime? "; cin.getline(ime, 31);
    cout << "Datum rođenja? "; cin.getline(datum, 11);
    cout << "Adresa stanovanja? "; cin.getline(adresa, 31);
}

void Osoba::pisi() const {
    cout << "Ime i prezime: " << ime << endl;
    cout << "Datum rođenja: " << datum << endl;
    cout << "Adresa stanovanja: " << adresa << endl;
}

void Djak::citaj() {
    Osoba::citaj();
    cout << "Naziv škole? "; cin.getline(skola, 31);
    cout << "Razred? "; cin.getline(razred, 7);
}

void Djak::pisi() const {
    Osoba::pisi();
    cout << "Naziv škole: " << skola << endl;
    cout << "Razred: " << razred << endl;
}

void Zaposlen::citaj() {
    Osoba::citaj();
    cout << "Naziv firme? "; cin.getline(firma, 31);
    cout << "Naziv odeljenja? "; cin.getline(odeljenje, 31);
}

void Zaposlen::pisi() const {
    Osoba::pisi();
    cout << "Naziv firme: " << firma << endl;
    cout << "Naziv odeljenja: " << odeljenje << endl;
}
  
```

// osobet.C - Ispitivanje klase osoba, daka i zaposlenih.

```

#include "osobe.h"
#include <iostream>
using namespace std;

int main() {
    Osoba* ljudi[20]; int n=0;

    cout << "Citanje podataka o ljudima\n";
    while (true) {
        cout << "\nIzbor (O=osoba, D=djak, Z=zaposlen, K=kraj)? ";
        char izbor; cin >> izbor; cin.ignore(100, '\n');
        if (izbor=='K' || izbor=='k') break;
    }
  
```

```

ljudi[n] = 0;
switch (izbor) {
    case 'O': case 'o': ljudi[n] = new Osoba;    break;
    case 'D': case 'd': ljudi[n] = new Djak;    break;
    case 'Z': case 'z': ljudi[n] = new Zaposlen; break;
}
if (ljudi[n]) ljudi[n++] -> citaj();
}

cout << "\nPrikaz procitanih podataka\n";
for (int i=0; i<n; i++) { cout << endl; ljudi[i] -> pisi(); }

return 0;
}

```

```

% osobet
Citanje podataka o ljudima

Izbor (O=osoba, D=djak, Z=zaposlen, K=kraj)? o
Ime i prezime?      Marko Markovic
Datum rođenja?      13.5.1984
Adresa stanovanja?  Avalska 33, Beograd

Izbor (O=osoba, D=djak, Z=zaposlen, K=kraj)? d
Ime i prezime?      Zoran Zoranic
Datum rođenja?      1.1.1985
Adresa stanovanja?  Backa 132, Beograd
Naziv skole?        O.S. "P. P. Njegos"
Razred?              III-1

Izbor (O=osoba, D=djak, Z=zaposlen, K=kraj)? z
Ime i prezime?      Petar Petrovic
Datum rođenja?      23.10.1965
Adresa stanovanja?  Pancevacki put 37a, Beograd
Naziv firme?        "IMT Rakovica", Beograd
Naziv odeljenja?    Nabavna sluzba

Izbor (O=osoba, D=djak, Z=zaposlen, K=kraj)? k

Prikaz procitanih podataka

Ime i prezime:      Marko Markovic
Datum rođenja:      13.5.1984
Adresa stanovanja:  Avalska 33, Beograd

Ime i prezime:      Zoran Zoranic
Datum rođenja:      1.1.1985
Adresa stanovanja:  Backa 132, Beograd
Naziv skole:        O.S. "P. P. Njegos"
Razred:              III-1

Ime i prezime:      Petar Petrovic
Datum rođenja:      23.10.1965
Adresa stanovanja:  Pancevacki put 37a, Beograd
Naziv firme:        "IMT Rakovica", Beograd
Naziv odeljenja:    Nabavna sluzba

```

Zadatak 4.3 Neuređene i uređene liste celih brojeva

Napisati na jeziku C++ klasu neuređenih listi celih brojeva. Predvideti:

- stvaranje prazne liste; liste od jednog broja i kao kopiju druge liste,
- uništavanje liste,
- dodeljivanje vrednosti jedne liste drugoj,
- dohvatjanje dužine liste,
- dodavanje jednog broja i cele liste na kraj liste (operator +=), i
- upisivanje liste u izlazni tok (operator <<).

Napisati na jeziku C++ klasu uređenih listi celih brojeva kao izvedenu klasu iz klase neuređenih listi. Prilikom dodavanja novih elemenata voditi računa o uređenosti liste.

Napisati na jeziku C++ program za ispitivanje prethodnih klasa.

Rešenje:

```

// lista5.h - Klasa neuređenih
//              listi celih brojeva.

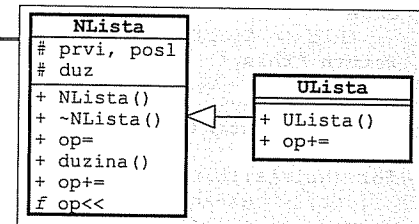
#ifndef lista5_h_
#define lista5_h_

#include <iostream>
using namespace std;

class NLista {
protected:
    struct Elem {                // ELEMENT LISTE:
        int broj; Elem* sled;    // - sadržaj i pokazivač na sledeći element,
        Elem(int b, Elem* s=0)  // - konstruktor.
        { broj = b; sled = s; }
    };
    Elem *prvi, *posl;           // Prvi i poslednji element liste.
    int duz;                     // Dužina liste.
private:
    void kopiraj(const NLista& lst); // Kopiranje druge liste.
    void brisi();                  // Pražnjenje liste.
public:
    NLista() { prvi = posl = 0; duz = 0; } // Konstruktori:
    NLista(int b) { prvi = posl = new Elem(b); duz = 1; } // - prazna lista,
    NLista(const NLista& lst) { kopiraj(lst); } // - konverzija,
    ~NLista() { brisi(); } // - kopiranje.
    NLista& operator=(const NLista& lst) { // Uništavanje.
        if (this != &lst) { brisi(); kopiraj(lst); } // Dodela vrednosti.
        return *this;
    }
    int duzina() const { return duz; } // Dužina liste.
    virtual NLista& operator+=(int b) { // Dodavanje broja.
        Elem* novi = new Elem(b); duz++;
        posl = (!prvi ? prvi : posl->sled) = novi;
        return *this;
    }
    NLista& operator+=(const NLista& lst); // Dodavanje liste.
    friend ostream& operator<<(ostream&, const NLista&); // Pisanje.
};

#endif

```



```
// lista5.C - Metode i funkcije uz klasu neuređenih listi.

#include "lista5.h"

void NLista::kopiraj(const NLista& lst) { // Kopiranje druge liste.
    prvi = posl = 0; duz = 0;
    for (Elem* tek=lst.prvi; tek; tek=tek->sled) *this += tek->broj;
}

void NLista::brisi() { // Pražnjenje liste.
    while (prvi) { Elem* stari = prvi; prvi = prvi->sled; delete stari; }
    posl = 0; duz = 0;
}

NLista& NLista::operator+=(const NLista& lst) { // Dodavanje liste.
    if (this != &lst)
        for (Elem* tek=lst.prvi; tek; tek=tek->sled) *this += tek->broj;
    else
        *this += NLista(lst); // lst += lst
    return *this;
}

ostream& operator<<(ostream& it, const NLista& lst) { // Pisanje.
    it << '(';
    for (NLista::Elem* tek=lst.prvi; tek; tek=tek->sled)
        { it << tek->broj; if (tek->sled) it << ','; }
    return it << ')';
}
```

```
// lista6.h - Klasa uređenih listi celih brojeva.
```

```
#ifndef _lista6_h_
#define _lista6_h_

#include "lista5.h"

class ULista: public NLista {
public:
    ULista() {} // Konstruktori:
    ULista(int b): NLista(b) {} // - prazna lista,
    ULista& operator+=(int b); // - konverzija.
    ULista& operator+=(const NLista& lst) // Dodavanje broja.
    { NLista::operator+=(lst); return *this; } // Dodavanje liste.
};

#endif
```

```
// lista5.C - Metode klase uređenih listi celih brojeva.
```

```
#include "lista6.h"

ULista& ULista::operator+=(int b) { // Dodavanje broja.
    Elem *tek = prvi, *pret = 0;
    while (tek && tek->broj < b) { pret = tek; tek = tek->sled; }
    Elem* novi = new Elem(b, tek);
    (!pret ? prvi : pret->sled) = novi;
    if (!tek) posl = novi;
    return *this;
    duz++;
}
```

```
// lista6t.C - Ispitivanje klase neuređenih i uređenih listi.
```

```
#include "lista5.h"
#include "lista6.h"
#include <iostream>
using namespace std;

int main() {
    NLista lst1; ULista lst2; cout << "Niz? ";
    do { int b; cin >> b; lst1 += b; lst2 += b; } while (cin.get() != '\n');
    cout << "Neuredjeno= " << lst1 << endl;
    cout << "Uredjeno = " << lst2 << endl;
    NLista lst3; cout << "Niz? ";
    do { int b; cin >> b; lst3 += b; } while (cin.get() != '\n');
    cout << "Neuredjeno= " << (lst1+=lst3) << endl;
    cout << "Uredjeno = " << (lst2+=lst3) << endl;
    cout << "Neuredjeno= " << (lst1+=lst1) << endl;
    cout << "Uredjeno = " << (lst2+=lst2) << endl;
    return 0;
}
```

```
% lista6t
```

```
Niz? 5 9 3 7 1
Neuredjeno= (5,9,3,7,1)
Uredjeno = (1,3,5,7,9)
Niz? 6 2 8 4 0
Neuredjeno= (5,9,3,7,1,6,2,8,4,0)
Uredjeno = (0,1,2,3,4,5,6,7,8,9)
Neuredjeno= (5,9,3,7,1,6,2,8,4,0,5,9,3,7,1,6,2,8,4,0)
Uredjeno = (0,0,1,1,2,2,3,3,4,4,5,5,6,6,7,7,8,8,9,9)
```

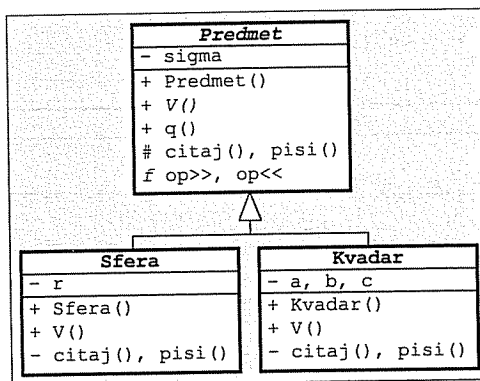
Zadatak 4.4 Predmeti, sfere i kvadri

Predmet ima specifičnu težinu, **sfera** je predmet zadat poluprečnikom, a **kvadar** je predmet zadat dužinama ivica.

Napisati na jeziku C++ klase koje omogućavaju unificiranu obradu nabrojanih vrsta predmeta. Predvideti inicijalizaciju zadatim vrednostima parametara (podrazumevano 1), izračunavanje zapremine, izračunavanje težine, čitanje podataka o predmetu iz ulaznog toka (operator >>) i upisivanje podataka o predmetu u izlazni tok (operator <<).

Napisati na jeziku C++ **program** koji s glavnog ulaza pročita podatke o određenom broju predmeta i posle toga na glavnom izlazu ispiše podatke o predmetima čije su težine iznad prosečne.

Rešenje:



// predmet1.h - Apstraktna klasa predmeta.

```

#ifndef _predmet1_h_
#define _predmet1_h_

#include <iostream>
using namespace std;

class Predmet {
    double sigma;           // Specifična težina.
public:
    Predmet(double ss=1) { sigma = ss; }           // Konstruktor.
    virtual double V() const =0;                   // Zapremina.
    double q() const { return V() * sigma; }        // Težina.
protected:
    virtual void citaj(istream& ut) { ut >> sigma; } // Čitanje.
    friend istream& operator>>(istream& ut, Predmet& p)
    { p.citaj(ut); return ut; }
    virtual void pisi(ostream& it) const { it << sigma; } // Pisanje.
    friend ostream& operator<<(ostream& it, const Predmet& p)
    { p.pisi(it); return it; }
};

#endif
  
```

// sferal.h - Klasa sfera.

```

#ifndef _sferal_h_
#define _sferal_h_

#include "predmet1.h"

class Sfera: public Predmet {
    double r;
public:
    Sfera(double ss=1, double rr=1)           // Konstruktor.
    : Predmet(ss) { r = rr; }
    double V() const { return 4./3 * r*r*r * 3.14159; } // Zapremina.
private:
    void citaj(istream& ut)                     // Čitanje.
    { Predmet::citaj(ut); ut >> r; }
    void pisi(ostream& it) const                // Pisanje.
    { it << "sfera ["; Predmet::pisi(it); it << ',' << r << ']'>> }
};

#endif
  
```

// kvadar3.h - Klasa kvadara.

```

#ifndef _kvadar3_h_
#define _kvadar3_h_

#include "predmet1.h"

class Kvadar: public Predmet {
    double a, b, c;
public:
    Kvadar(double ss=1, double aa=1, double bb=1, double cc=1) // Konstr.
    : Predmet(ss) { a = aa; b = bb; c = cc; }
    double V() const { return a * b * c; }           // Zapremina.
private:
    void citaj(istream& ut)                     // Čitanje.
    { Predmet::citaj(ut); ut >> a >> b >> c; }
    void pisi(ostream& it) const                // Pisanje.
    { it << "kvadar["; Predmet::pisi(it);
      it << ',' << a << ',' << b << ',' << c << ']'>> }
};

#endif
  
```



```
// predmet1t.C - Ispitivanje klasa predmeta.
```

```
#include "sferal.h"
#include "kvadar3.h"
#include <iostream>
using namespace std;

int main() {
    Predmet* p[100] = {0}; double q = 0; int n = 0;
    while (true) {
        char tip; cin >> tip;
        if (tip == '.') break;
        switch (tip) {
            case 's': case 'S': p[n] = new Sfera; break;
            case 'k': case 'K': p[n] = new Kvadar; break;
        }
        if (p[n]) {
            cin >> *p[n];
            cout << *p[n] << " (q=" << p[n]->q() << ")\n";
            q += p[n]->q();
        }
        if (n) q /= n;
        cout << "\nqsr= " << q << "\n\n";
        for (int i=0; i<n; i++)
            if (p[i]->q() > q) cout << *p[i] << " (q=" << p[i]->q() << ")\n";
        return 0;
    }
}
```

```
predmet1t.pod
```

```
k 0.5 1 2 3
s 2.8 1
s 1.8 0.75
k 1.5 2 2 2
.
```

```
% predmet1t <predmet1t.pod
kvadar[0.5,1,2,3] (q=3)
sfera [2.8,1] (q=11.7286)
sfera [1.8,0.75] (q=3.18086)
kvadar[1.5,2,2,2] (q=12)

qsr= 7.47737

sfera [2.8,1] (q=11.7286)
kvadar[1.5,2,2,2] (q=12)
```

Zadatak 4.5 Geometrijske figure, krugovi, kvadrati i trouglovi u ravni

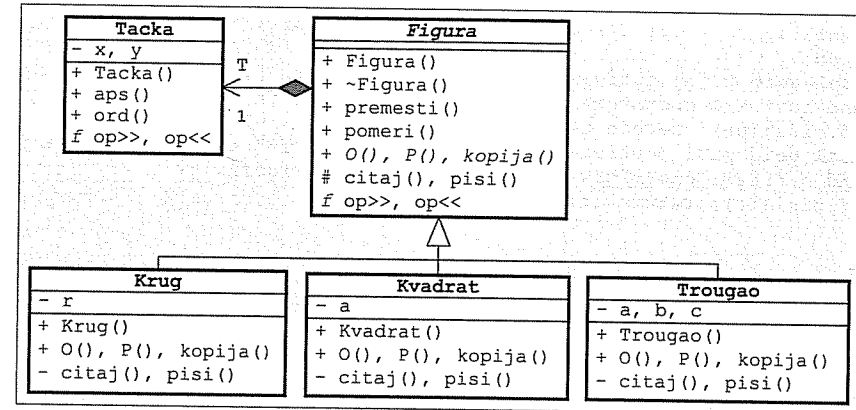
Napisati na jeziku C++ klasu geometrijskih figura u ravni. Predvideti:

- stvaranje figure s težištem u zadatoj tački,
- uništavanje figure,
- premeštanje figure u novu tačku i pomeranje figure za određeni pomak,
- izračunavanje obima i površine figure,
- stvaranje kopije figure, i
- čitanje figure iz ulaznog toka i upisivanje figure u izlazni tok (operatori >> i <<).

Napisati na jeziku C++ klase krugova, kvadrata i trouglova u ravni kao izvedene klase iz klase figura.

Napisati na jeziku C++ program za proveru ispravnosti prethodnih klasa.

Rešenje:



```
// tacka4.h - Klasa tačaka u ravni.
```

```
#ifndef _tacka4_h_
#define _tacka4_h_

typedef double Real; // Tip za realne vrednosti.

#include <iostream>
using namespace std;

class Tacka {
    Real x, y; // Koordinate.
public:
    Tacka(Real xx=0, Real yy=0) { x = xx; y = yy; } // Konstruktor.
    Real aps() const { return x; } // Apscisa.
    Real ord() const { return y; } // Ordinata.
    friend istream& operator>>(istream& ut, Tacka& t) // Čitanje.
    { return ut >> t.x >> t.y; }
    friend ostream& operator<<(ostream& it, const Tacka& t) // Pisanje.
    { return it << '(' << t.x << ',' << t.y << ')'; }
};

const Tacka ORG; // Koordinatni početak.

#endif
```

```
// figur1.h - Klasa geometrijskih figura u ravni.
```

```
#ifndef _figur1_h_
#define _figur1_h_
```

```
#include "tacka4.h"
```

```
class Figura {
    Tacka T; // Težište figure.
public:
    Figura(const Tacka& tt=ORG): T(tt) {} // Konstruktor.
    virtual ~Figura() {} // Destruktor.
    Figura& premesti(Real x, Real y) // Premešanje figure.
    { T = Tacka(x, y); return *this; }
    Figura& pomeri(Real dx, Real dy) // Pomeranje figure.
    { T = Tacka(T.aps()+dx, T.ord()+dy); return *this; }
    virtual Real O() const =0; // Obim.
    virtual Real P() const =0; // Površina.
    virtual Figura* kopija() const =0; // Kopiranje.
protected:
    virtual void citaj(istream& ut) { ut >> T; } // Čitanje.
    friend istream& operator>>(istream& ut, Figura& f)
    { f.citaj(ut); return ut; }
    virtual void pisi (ostream& it) const { it << "T=" << T; } // Pisanje.
    friend ostream& operator<<(ostream& it, const Figura& f)
    { f.pisi(it); return it; }
};
```

```
#endif
```

```
// krug2.h - Klasa krugova u ravni.
```

```
#ifndef _krug2_h_
#define _krug2_h_
```

```
#include "figur1.h"
```

```
const Real PI = 3.1415926535897932;
```

```
class Krug: public Figura {
    Real r; // Poluprečnik.
public:
    Krug(Real rr=1, const Tacka& tt=ORG) // Konstruktor.
    : Figura(tt) { r = rr; }
    Real O() const { return 2 * r * PI; } // Obim.
    Real P() const { return r * r * PI; } // Površina.
    Krug* kopija() const // Kopiranje.
    { return new Krug(*this); }
private:
    void citaj(istream& ut) // Čitanje.
    { Figura::citaj(ut); ut >> r; }
    void pisi (ostream& it) const // Pisanje.
    { it << "krug ["; Figura::pisi(it);
      it << ", r=" << r << ", O=" << O() << ", P=" << P() << ']' ;
    }
};
```

```
#endif
```

```
// kvadrat.h - Klasa kvadrata u ravni.
```

```
#ifndef _kvadrat_h_
#define _kvadrat_h_
```

```
#include "figur1.h"
```

```
class Kvadrat: public Figura {
    Real a; // Stranica.
public:
    Kvadrat(Real aa=1, const Tacka& tt=ORG) // Konstruktor.
    : Figura(tt) { a = aa; }
    Real O() const { return 4 * a; } // Obim.
    Real P() const { return a * a; } // Površina.
    Kvadrat* kopija() const // Dinamička kopija.
    { return new Kvadrat(*this); }
private:
    void citaj(istream& ut) // Čitanje.
    { Figura::citaj(ut); ut >> a; }
    void pisi (ostream& it) const // Pisanje.
    { it << "kvadrat ["; Figura::pisi(it);
      it << ", a=" << a << ", O=" << O() << ", P=" << P() << ']' ;
    }
};
```

```
#endif
```

```
// trougao4.h - Klasa trouglova u ravni.
```

```
#ifndef _trougao4_h_
#define _trougao4_h_
```

```
#include "figur1.h"
```

```
#include <cmath>
```

```
using namespace std;
```

```
class Trougao: public Figura {
    Real a, b, c; // Stranice.
public:
    Trougao(Real aa=1, const Tacka& tt=ORG) // Konstruktori:
    : Figura(tt) { a = b = c = aa; } // jednakokranični,
    Trougao(Real aa, Real bb, const Tacka& tt=ORG) // jednakokraki,
    : Figura(tt) { a = aa; b = c = bb; }
    Trougao(Real aa, Real bb, Real cc, const Tacka& tt=ORG) // opšti.
    : Figura(tt) { a = aa; b = bb; c = cc; }
    Real O() const { return a + b + c; } // Obim.
    Real P() const // Površina.
    { Real s = (a + b + c) / 2; return sqrt(s*(s-a)*(s-b)*(s-c)); }
    Trougao* kopija() const // Kopiranje.
    { return new Trougao(*this); }
private:
    void citaj(istream& ut) // Čitanje.
    { Figura::citaj(ut); ut >> a >> b >> c; }
    void pisi (ostream& it) const // Pisanje.
    { it << "trougao ["; Figura::pisi(it);
      it << ", a=" << a << ", b=" << b << ", c=" << c
      << ", O=" << O() << ", P=" << P() << ']' ;
    }
};
```

```
#endif
```

// figuralt.C - Ispitivanje klasa geometrijskih figura.

```
#include "krug2.h"
#include "kvadrat.h"
#include "trougao4.h"
#include <iostream>
using namespace std;
```

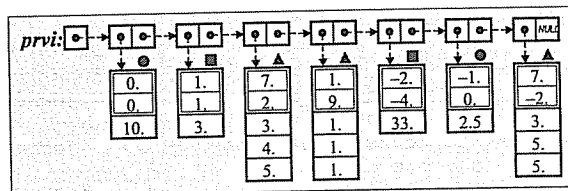
```
int main() {
```

// Element liste figura.

```
struct Elem {
    Figura* fig; Elem* sled;
    Elem(Figura* ff, Elem* ss=0) { fig = ff; sled = ss; }
    ~Elem() { delete fig; }
};
```

// Stvaranje liste figura čitajući s glavnog ulaza.

```
Elem *prvi = 0, *posl = 0;
while (true) {
    Figura* pf = 0;
    char vrsta; cin >> vrsta;
    switch (vrsta) {
        case 'o': pf = new Krug; break;
        case 'k': pf = new Kvadrat; break;
        case 't': pf = new Trougao; break;
    }
    if (!pf) break;
    cin >> *pf;
    Elem* novi = new Elem(pf);
    posl = (!prvi ? prvi : posl->sled) = novi;
}
```



```
// Prikazivanje sadržaja liste na glavnom izlazu.
for (Elem* tek=prvi; tek; tek=tek->sled) cout << *tek->fig << endl;
```

```
// Čitanje pomeraja za pomeranje figura.
Real dx, dy; cin >> dx >> dy;
cout << "\ndx, dy= " << dx << ", " << dy << "\n\n";
```

```
// Stvaranje kopije liste uz pomeranje figura.
Elem *poc = 0, *kraj = 0;
for (Elem* tek=prvi; tek; tek=tek->sled) {
    Elem* novi = new Elem(tek->fig->kopija());
    novi->fig->pomeri(dx, dy);
    kraj = (!poc ? poc : kraj->sled) = novi;
}
```

// Uništavanje prve liste.

```
while (prvi) { Elem* stari=prvi; prvi=prvi->sled; delete stari; }
posl = 0;
```

// Prikazivanje sadržaja kopirane liste na glavnom izlazu.

```
for (Elem* tek=poc; tek; tek=tek->sled) cout << *tek->fig << endl;
```

```
return 0;
```

figuralt.pod

```
o 0 0 10
k 1 1 3
t 7 2 3 4 5
t 1 9 1 1 1
k -2 -4 33
o -1 0 2.5
t 7 -2 3 5 5
?
3 5
```

% figuralt <figuralt.pod

```
krug [T=(0,0), r=10, O=62.8319, P=314.159]
kvadrat [T=(1,1), a=3, O=12, P=9]
trougao [T=(7,2), a=3, b=4, c=5, O=12, P=6]
trougao [T=(1,9), a=1, b=1, c=1, O=3, P=0.433013]
kvadrat [T=(-2,-4), a=33, O=132, P=1089]
krug [T=(-1,0), r=2.5, O=15.708, P=19.635]
trougao [T=(7,-2), a=3, b=5, c=5, O=13, P=7.15454]
```

dx, dy= 3, 5

```
krug [T=(3,5), r=10, O=62.8319, P=314.159]
kvadrat [T=(4,6), a=3, O=12, P=9]
trougao [T=(10,7), a=3, b=4, c=5, O=12, P=6]
trougao [T=(4,14), a=1, b=1, c=1, O=3, P=0.433013]
kvadrat [T=(1,1), a=33, O=132, P=1089]
krug [T=(2,5), r=2.5, O=15.708, P=19.635]
trougao [T=(10,3), a=3, b=5, c=5, O=13, P=7.15454]
```

Zadatak 4.6 Vektori, brzine i pokretni objekti i tačke u prostoru

Napisati na jeziku C++ sledeće klase:

- **Vektor** u trodimenzionalnom prostoru predstavlja se komponentama u pravcu x, y, z ose (podrazumevane vrednosti su $(0,0,0)$). Može da se izračuna intenzitet vektora, zbir dva vektora ($\text{vekt1} + \text{vekt2}$) i proizvod vektora i skalarnog realnog broja ($\text{vekt} * \text{skal}$). Vektor se, izrazom $\text{tok} << \text{vekt}$ piše u obliku (x, y, z) .
- **Brzina** je vektor koji se piše u obliku $\mathbf{v}(x, y, z)$.
- Apstraktni **pokretni** objekti mogu da promene svoj položaj u prostoru u zavisnosti od dužine proteklog vremenskog intervala ($\text{p.proteklo}(\text{dt})$).
- **Tačka** je pokretan objekat koji ima jedinstven, automatski generisan celobrojan identifikator, vektor položaja i brzinu kretanja. Može da se dohvati trenutni vektor položaja tačke i da se izračuna rastojanje između dve tačke. Tačka se izrazom $\text{it} << \text{tačka}$ piše u izlazni tok u obliku $Ti(x, y, z)$, gde je: i – identifikator tačke.

Napisati na jeziku C++ program koji pročita niz tačaka s glavnog ulaza, a zatim u zadatom broju vremenskih intervala, sa zadatim korakom dt ispisuje na glavnom izlazu tačku koja je na kraju svakog intervala najbliža koordinatnom početku i njeno rastojanje od koordinatnog početka.

Rešenje:

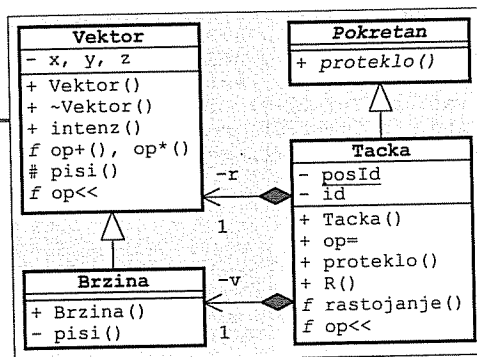
```
// vektor1.h - Klasa vektora
// u prostoru.
```

```
#ifndef _vektor1_h_
#define _vektor1_h_
```

```
#include <iostream>
#include <cmath>
using namespace std;
```

```
class Vektor {
    double x, y, z; // Komponente vektora.
public:
    Vektor() { x = y = z = 0; } // Konstruktori.
    Vektor(double xx, double yy, double zz) { x = xx; y = yy; z = zz; }
    virtual ~Vektor() {} // Destruktor.
    double intenz() const { return sqrt(x*x+y*y+z*z); } // Intenzitet.
    friend Vektor operator+(const Vektor& v1, const Vektor& v2) // v1 + v2
    { return Vektor(v1.x+v2.x, v1.y+v2.y, v1.z+v2.z); }
    friend Vektor operator*(const Vektor& v1, double s) // v1 * s
    { return Vektor(v1.x*s, v1.y*s, v1.z*s); }
protected:
    virtual void pisi(ostream& it) const // Pisanje.
    { it << '(' << x << ', ' << y << ', ' << z << ')'; }
    friend ostream& operator<<(ostream& it, const Vektor& v)
    { v.pisi(it); return it; }
};

#endif
```



```
// brzina.h - Klasa brzina.
```

```
#ifndef _brzina_h_
#define _brzina_h_
```

```
#include "vektor1.h"
```

```
class Brzina: public Vektor {
    void pisi(ostream& it) const { it << 'v'; Vektor::pisi(it); } // Pisanje.
public:
    Brzina(): Vektor() {} // Konstruktori.
    Brzina(double x, double y, double z): Vektor(x, y, z) {}
};

#endif
```

```
// pokretan.h - Apstraktna klasa pokretnih objekata.
```

```
#ifndef _pokretan_h_
#define _pokretan_h_
```

```
class Pokretan {
public:
    virtual Pokretan& proteklo(double dt) = 0; // Proteklo je vreme dt.
};

#endif
```

```
// tacka5.h - Klasa pokretnih tačaka.
```

```
#ifndef _tacka5_h_
#define _tacka5_h_
```

```
#include "pokretan.h"
#include "vektor1.h"
#include "brzina.h"
```

```
class Tačka: public Pokretan {
    static int posId; // Poslednji identifikator.
    const int id; // Identifikator tačke.
    Vektor r; // Vektor položaja.
    Brzina v; // Vektor brzine.
public:
    Tačka(const Vektor& rr=Vektor(), const Brzina& vv=Brzina()) // Nova tačka dobija nov
    { r(rr), v(vv), id(++posId) {} // identifikator.
    Tačka(const Tačka& T) // Kopija tačke dobija nov
    { r(T.r), v(T.v), id(++posId) {} // identifikator.
    Tačka& operator=(const Tačka& T) // Levom operandu se ne
    { r = T.r; v = T.v; return *this; } // menja identifikator.
    Pokretan& proteklo(double dt) // Promena položaja zbog
    { r = r + v * dt; return *this; } // proteklog vremena.
    Vektor R() const { return r; } // Trenutni položaj.
    friend double rastojanje(const Tačka& T1, const Tačka& T2) // Rastojanje
    { return (T1.r + T2.r * -1).intenz(); } // dve tačke
    friend ostream& operator<<(ostream& it, const Tačka& T) // Pisanje.
    { return it << 'T' << T.id << T.r; }
};

#endif
```

```
// tacka5.C - Statičko polje klase pokretnih tačaka.
```

```
#include "tacka5.h"
```

```
int Tacka::posId = 0;
```

```
// pokretant.C - Ispitivanje klase pokretnih tačaka.
```

```
#include "tacka5.h"
#include "vektor1.h"
#include "brzina.h"
#include <iostream>
using namespace std;
```

```
int main() {
    cout << "Broj tacaka? "; int n; cin >> n;
    Tacka** tacke = new Tacka* [n];
    for (int i=0; i<n; i++) {
        cout << "Koordinate tmena " << i << "? ";
        double x, y, z; cin >> x >> y >> z;
        cout << "Komponente brzine? ";
        double vx, vy, vz; cin >> vx >> vy >> vz;
        tacke[i] = new Tacka(Vektor(x,y,z), Brzina(vx,vy,vz));
    }

    cout << "\nBroj koraka? "; int k; cin >> k;
    cout << "Trajanje koraka? "; double dt; cin >> dt;
    const Tacka org;
    cout << "ORG " << org << "\n\n";

    for (int i=0; i<k; i++) {
        for (int j=0; j<n; tacke[j++]>proteklo(dt));
        double min = rastojanje(org, *tacke[0]); int m = 0;
        for (int j=1; j<n; j++) {
            double d = rastojanje(org, *tacke[j]);
            if (d < min) { min = d; m = j; }
        }
        cout << i << " ' ' << *tacke[m] << ' ' << min << endl;
    }
    return 0;
}
```

```
% pokretant
Broj tacaka? 3
Koordinate tmena 0? 2 2 2
Komponente brzine? -1 -1.2 -1.3
Koordinate tmena 1? -2 2 0
Komponente brzine? .5 -.6 .2
Koordinate tmena 2? 0 0 0
Komponente brzine? .3 .3 .3
```

```
Broj koraka? 10
Trajanje koraka? 1
ORG T4(0,0,0)
```

```
0 T3(0.3,0.3,0.3) 0.519615
1 T1(0,-0.4,-0.6) 0.72111
2 T2(-0.5,0.2,0.6) 0.806226
3 T2(0,-0.4,0.8) 0.894427
4 T2(0.5,-1,1) 1.5
5 T2(1,-1.6,1.2) 2.23607
6 T2(1.5,-2.2,1.4) 3.00832
7 T2(2,-2.8,1.6) 3.79473
8 T2(2.5,-3.4,1.8) 4.58803
9 T3(3,3,3) 5.19615
```

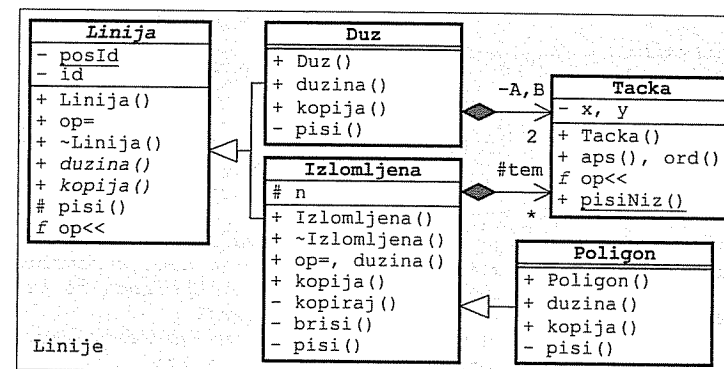
Zadatak 4.7 Tačke, linije, duži, izlomljene linije i poligoni u ravni

Napisati na jeziku C++ sledeće klase:

- **Tačka** u ravni zadaje se pomoću realnih koordinata (podrazumevano (0,0)), mogu da joj se dohvate vrednosti koordinata i može da se piše u izlazni tok (<<). U klasi postoji i uslužna metoda za upisivanje niza tačaka u izlazni tok.
- Apstraktna **linija** u ravni ima jedinstven, automatski generisan celobrojan identifikator, može da joj se izračuna dužina, može da se napravi kopija i može da se piše u izlazni tok (<<, tekst sadrži identifikator broj linije).
- **Duž** je linija koja je zadata krajnjim tačkama (podrazumevano (-1,-1) i (1,1)). Prilikom pisanja u izlazni tok, pored identifikatora, piše se i reč *duž* i koordinate krajnjih tačaka.
- **Izlomljena** linija je linija koja se sastoji od niza tačaka koje predstavljaju temena linije. Prilikom pisanja u izlazni tok, pored identifikatora, piše se i reč *izlomljena* i niz koordinata temena.
- **Poligon** je zatvorena izlomljena linija koja se dobija spajanjem prvog i poslednjeg temena. Prilikom pisanja u izlazni tok koristi se reč *poligon*.

Napisati na jeziku C++ **funkciju** za čitanje jedne linije s glavnog ulaza i **program** koji s glavnog ulaza pročita niz raznovrsnih linija, ispiše sve pročitane podatke na glavnom izlazu (uključujući i dužine pojedinih linija), pronade liniju koja ima najveću dužinu i ispiše na glavnom izlazu podatke o pronađenoj liniji. Dužina niza linija zadaje se kao parametar glavne funkcije.

Rešenje:



```
// tacka6.h - Klasa tačaka u ravni.
```

```
#ifndef _tacka6_h_
#define _tacka6_h_
```

```
#include <iostream>
using namespace std;
```

```
namespace Linije {
    class Tacka {
        double x, y;
    public:
        Tacka(double xx=0, double yy=0) { x = xx; y = yy; } // Konstruktor.
        double aps() const { return x; } // Apscisa.
        double ord() const { return y; } // Ordinata.
    };
}
```

```

friend ostream& operator<<(ostream& it, const Tacka& t) // Pisanje.
{ return it << '(' << t.x << ',' << t.y << ')'; }
static void pisiNiz(ostream& it, const Tacka* niz, int duz);
// Pisanje niza tačaka.
};
}
#endif

```

// tacka6.C - Metode klase tačaka u ravni.

```

#include "tacka6.h" // Pisanje niza tačaka.
void Linije::Tacka::pisiNiz(ostream& it, const Tacka* niz, int duz) {
    it << '(';
    for (int i=0; i<duz; i++) {
        if (i) it << ',';
        it << niz[i];
    }
    it << ')';
}

```

// linija.h - Apstraktna klasa linija u ravni.

```

#ifndef _linija_h_
#define _linija_h_

#include <iostream>
using namespace std;

namespace Linije {
    class Linija {
        static int posId; // Poslednji identifikator.
        int id; // Identifikator linije.
    public:
        Linija(): id(++posId) {} // Nova linija dobija nov broj.
        Linija(const Linija&): id(++posId) {} // Kopija dobija nov broj.
        Linija& operator=(const Linija&) // Levom operandu se ne menja
        { return *this; } // broj.
        virtual ~Linija() {} // Virtuelan destruktor.
        virtual double duzina() const =0; // Dužina linije.
        virtual Linija* kopija() const =0; // Kopiranje.
    protected:
        virtual void pisi(ostream& it) const { it << id; } // Pisanje.
        friend ostream& operator<<(ostream& it, const Linija& lin)
        { lin.pisi(it); return it; }
    };
}
#endif

```

// linija.C - Statičko polje apstraktne klase linija u ravni.

```

#include "linija.h"
int Linije::Linija::posId = 0;

```

// duzl.h - Klasa duži u ravni.

```

#ifndef _duzl_h_
#define _duzl_h_

#include "linija.h"
#include "tacka6.h"
#include <cmath>
using namespace std;

namespace Linije {
    class Duz: public Linija {
        Tacka A, B; // Krajnje tačke.
    public:
        explicit Duz(const Tacka& P=Tacka(-1,-1), // Konstruktor.
                     const Tacka& Q=Tacka(1,1)): A(P), B(Q) {}
        double duzina() const // Dužina.
        { return sqrt(pow(A.aps()-B.aps(),2) + pow(A.ord()-B.ord(),2)); }
        Duz* kopija() const // Kopiranje.
        { return new Duz(*this); }
    private:
        void pisi(ostream& it) const // Pisanje.
        { Linija::pisi(it); it << "[duz: A" << A << ", B" << B << ']' ; }
    };
}
#endif

```

// izlomljena.h - Klasa izlomljenih linija u ravni.

```

#ifndef _izlomljena_h_
#define _izlomljena_h_

#include "linija.h"
#include "tacka6.h"

namespace Linije {
    class Izlomljena: public Linija {
    protected:
        Tacka* tem; int n; // Niz i broj temena.
    private:
        void kopiraj(const Tacka*, int); // Kopiranje u objekat.
        void brisi() { delete [] tem; tem=0; } // Uništavanje.
    public:
        Izlomljena(const Tacka* niz, int k) // Inicijalizacija nizom tačaka.
        { kopiraj(niz, k); }
        Izlomljena(const Izlomljena& lin) // Inicijalizacija linijom.
        : Linija(lin) { kopiraj(lin.tem, lin.n); }
        ~Izlomljena() { brisi(); } // Uništavanje.
        Izlomljena& operator=(const Izlomljena& lin) { // Dodela vrednosti.
            if (this != &lin)
                { brisi(); Linija::operator=(lin); kopiraj(lin.tem, lin.n); }
            return *this;
        }
        double duzina() const; // Dužina.
        Izlomljena* kopija() const // Kopiranje.
        { return new Izlomljena(*this); }
    };
}

```

```
private:
    virtual void pisi(ostream& it) const {           // Pisanje.
        Linija::pisi(it); it << "[izlomljena: ";
        Tacka::pisiNiz(it, tem, n); it << ']'
    }
};
#endif
```

// izlomljena.C - Metode klase izlomljenih linija u ravni.

```
#include "izlomljena.h"
#include "duzl.h"

void Linije::Izlomljena::kopiraj(const Tacka* niz, int k) {
    tem = new Tacka [n = k];           // Kopiranje u objekat.
    for (int i=0; i<n; i++) tem[i] = niz[i];
}

double Linije::Izlomljena::duzina() const {       // Dužina.
    double d = 0;
    for (int i=0; i<n-1; i++) d += Duz(tem[i],tem[i+1]).duzina();
    return d;
}
```

// poligon.h - Klasa poligona u ravni.

```
#ifndef _poligon_h_
#define _poligon_h_

#include "izlomljena.h"

namespace Linije {
    class Poligon: public Izlomljena {
    public:
        Poligon(const Tacka* niz, int k)           // Inicijalizacija nizom tačaka.
            : Izlomljena(niz, k) {}
        double duzina() const                       // Dužina.
        { return Izlomljena::duzina() + Duz(tem[0], tem[n-1]).duzina(); }
        Poligon* kopija() const                     // Kopiranje.
        { return new Poligon(*this); }
    private:
        virtual void pisi(ostream& it) const {       // Pisanje.
            Linija::pisi(it); it << "[poligon: ";
            Tacka::pisiNiz(it, tem, n); it << ']'
        }
    };
}

#endif
```

// linijac.C - Ispitivanje klase linija u ravni.

```
#include "tacka6.h"
#include "linija.h"
#include "duzl.h"
#include "izlomljena.h"
#include "poligon.h"
#include <iostream>
#include <cstdlib>
using namespace std;
using namespace Linije;

Linija* citaj() {                               // Čitanje jedne linije.
    cout << "Vrsta (D, I, P)? "; char vrs; cin >> vrs;
    switch (vrs) {
        case 'd': case 'D': {
            cout << "A? "; double xA, yA; cin >> xA >> yA;
            cout << "B? "; double xB, yB; cin >> xB >> yB;
            return new Duz(Tacka(xA, yA), Tacka(xB, yB));
        }
        case 'i': case 'I':
        case 'p': case 'P': {
            cout << "Broj temena? "; int n; cin >> n;
            Tacka* niz = new Tacka [n];
            for (int i=0; i<n; i++) {
                cout << i+1 << ". teme? "; double x, y; cin >> x >> y;
                niz[i] = Tacka(x, y);
            }
            Linija* lin = (vrs=='i' || vrs=='I') ? new Izlomljena(niz, n)
                : new Poligon (niz, n);

            delete [] niz;
            return lin;
        }
        default: return 0;
    }
}

int main(int, const char** varg) {               // Glavna funkcija.
    int n = atoi(varg[1]);
    Linija** niz = new Linija* [n];
    for (int i=0; i<n; i++)
        if (Linija* lin = citaj()) niz[i++] = lin;
        else cout << "*** Nepoznata vrsta linije! ***\n";
    cout << "\nProcitano:\n";
    double max = niz[0]->duzina(); int imax = 0;
    for (int i=0; i<n; i++) {
        double d = niz[i]->duzina();
        cout << *niz[i] << " d=" << d << endl;
        if (d > max) { max = d; imax = i; }
    }
    cout << "\nNajduza: " << *niz[imax] << endl;
    for (int i=0; i<n; delete niz[i++]);
    delete [] niz;
    return 0;
}
```

```

% linijati 3
Vrsta (D, I, P)? d
A? 1 1
B? 2 2
Vrsta (D, I, P)? i
Broj temena? 3
1. teme? 0 0
2. teme? 0 2
3. teme? 2 0
Vrsta (D, I, P)? x
*** Nepoznata vrsta linije! ***
Vrsta (D, I, P)? p
Broj temena? 3
1. teme? 0 0
2. teme? 1 0
3. teme? 1 1

Procitano:
1[duz: A(1,1), B(2,2)] d=1.41421
2[izlomljena: ((0,0),(0,2),(2,0))] d=4.82843
3[poligon: ((0,0),(1,0),(1,1))] d=3.41421

Najduza: 2[izlomljena: ((0,0),(0,2),(2,0))]

```

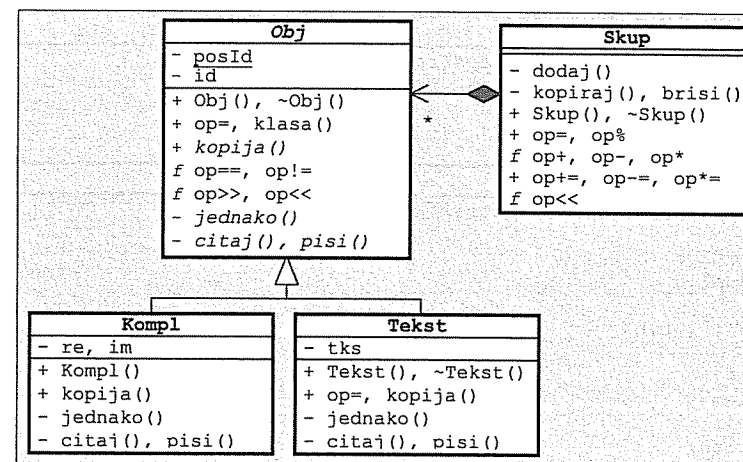
Zadatak 4.8 Objekti, skupovi objekata, kompleksni brojevi i tekstovi

Napisati na jeziku C++ sledeće klase (sve klase opremiti potrebnim konstruktorima, destruktorom i operatorom za dodelu vrednosti):

- Apstraktan *objekat* ima jedinstven, automatski generisan celobrojan identifikator. Može da se dohvati ime klase objekta, da se stvara kopija objekta, da se ispita da li su dva objekta jednaka ili su različita. Objekat može da se čita iz ulaznog toka (ut>>obj) i da se upiše u izlazni tok (it<<obj). Za označavanje objekta piše se prvo slovo imena klase i identifikator objekta.
- Za *skup* apstraktnih objekata može da se ispita da li sadrži neki objekat (skup%obj), da se pronađe unija (s1+s2, s1+=s2), razlika (s1-s2, s1-=s2) i presek (s1*s2, s1*=s2) dva skupa i da se sadržaj skupa upiše u izlazni tok (it<<skup).
- *Kompleksan broj* i *tekst* su objekti.

Napisati na jeziku C++ *program* za ispitivanje prethodnih klasa.

Rešenje:



// objekat.h - Apstraktna klasa objekata.

```

#ifndef _objekat_h_
#define _objekat_h_

#include <iostream>
#include <typeinfo>
using namespace std;

class Obj {
    static int posId;        // Poslednji identifikator.
    const int id;           // Identifikator objekta.
    virtual bool jednako(const Obj& obj) const = 0;        // Da li su jednaki?
    virtual void pisi(ostream& it) const = 0;             // Pisanje.
    virtual void citaj(istream& ut) = 0;                  // Čitanje.
};

```



```

public:
    Obj(): id(++posId) {} // Nov objekat dobija nov identifikator.
    Obj(const Obj&): id(++posId) {} // Kopija dobija nov identifikator.
    Obj& operator=(const Obj&) // Levom operandu se ne menja identifik.
    { return *this; };
    virtual ~Obj() {} // Uništavanje.
    const char* klasa() const // Ime klase.
    { return typeid(*this).name(); }
    virtual Obj* kopija() const = 0; // Kopiranje.
    friend bool operator==(const Obj& o1, const Obj& o2) // o1 == o2
    { return typeid(o1)==typeid(o2) && o1.jednako(o2); }
    friend bool operator!=(const Obj& o1, const Obj& o2) // o1 != o2
    { return !(o1 == o2); }
    friend ostream& operator<<(ostream& it, const Obj& obj) // it << obj
    { it << obj.klasa()[0] << obj.id; obj.pisi(it); return it; }
    friend istream& operator>>(istream& ut, Obj& obj) // ut >> obj
    { obj.citaj(ut); return ut; }
};

#endif

```

// objekat.C - Statičko polje apstraktne klase objekata.

```
#include "objekat.h"
```

```
int Obj::posId = 0;
```

// skup2.h - Klasa skupova apstraktnih objekata.

```
#ifndef _skup2_h_
#define _skup2_h_
```

```
#include "objekat.h"
```

```

class Skup {
    struct Elem { // Element skupa:
        Obj* obj; // - sadržaj,
        Elem* sled; // - sledeći,
        Elem(Obj* ob, Elem* sl=0) { obj=obj; sled=sl; } // - konstruktor,
        ~Elem() { delete obj; } // - destruktor.
    };

    Elem *prvi, *posl; // Elementi skupa.
    void dodaj(const Obj& obj) { // Dodavanje objekta.
        Elem* novi = new Elem(obj.kopija());
        posl = (posl ? posl->sled : prvi) = novi;
    }

    void kopiraj(const Skup& s); // Kopiranje skupa.
    void brisi(); // Uništavanje skupa.

public:
    Skup() { prvi = posl = 0; } // Prazan skup.
    Skup(const Obj& obj) // Konverzija.
    { prvi = posl = new Elem(obj.kopija()); }
    Skup(const Skup& s) { kopiraj(s); } // Kopija.
    ~Skup() { brisi(); } // Uništavanje.
    Skup& operator=(const Skup& s) { // Dodela.
        if (this != &s) { brisi(); kopiraj(s); }
        return *this;
    }
};

```

```

bool operator%(const Obj& obj) const; // Element skupa?
friend Skup operator+(const Skup& s1, const Skup& s2) // Unija.
{ return Skup(s1) += s2; }
friend Skup operator-(const Skup& s1, const Skup& s2); // Razlika.
friend Skup operator*(const Skup& s1, const Skup& s2); // Presek.
Skup& operator+=(const Skup& s); // Unija.
Skup& operator-=(const Skup& s) { return *this = *this - s; } // Razlika.
Skup& operator*=(const Skup& s) { return *this = *this * s; } // Presek.
friend ostream& operator<<(ostream& it, const Skup& s); // Pisanje.
};

#endif

```

// skup2.C - Metode i funkcije uz klasu skupova apstraktnih objekata.

```
#include "skup2.h"
```

```

void Skup::kopiraj(const Skup& s) { // Kopiranje skupa.
    prvi = posl = 0;
    for (Elem* tek=s.prvi; tek; tek=tek->sled) dodaj(*tek->obj);
}

```

```

void Skup::brisi() { // Uništavanje.
    while (prvi) { Elem* stari = prvi; prvi = prvi->sled; delete stari; }
    posl = 0;
}

```

```

bool Skup::operator%(const Obj& obj) const { // Element skupa?
    for (Elem* tek=prvi; tek; tek=tek->sled)
        if (*tek->obj == obj) return true;
    return false;
}

```

```

Skup operator-(const Skup& s1, const Skup& s2) { // Razlika.
    Skup s;
    for (Skup::Elem* tek=s1.prvi; tek; tek=tek->sled)
        if (!(s2 % *tek->obj)) s.dodaj(*tek->obj);
    return s;
}

```

```

Skup operator*(const Skup& s1, const Skup& s2) { // Presek.
    Skup s;
    for (Skup::Elem* tek=s1.prvi; tek; tek=tek->sled)
        if (s2 % *tek->obj) s.dodaj(*tek->obj);
    return s;
}

```

```

Skup& Skup::operator+=(const Skup& s) { // Unija.
    for (Elem* tek=s.prvi; tek; tek=tek->sled)
        if (!(this % *tek->obj)) dodaj(*tek->obj);
    return *this;
}

```

```

ostream& operator<<(ostream& it, const Skup& s) { // Pisanje.
    it << '{';
    for (Skup::Elem* tek=s.prvi; tek; tek=tek->sled) {
        it << *tek->obj << (tek->sled ? ", " : "");
    }
    return it << '}';
}

```

```
// kompl2.h - Klasa kompleksnih brojeva.
```

```
#ifndef _kompl2_h_
#define _kompl2_h_

#include "objekat.h"

class Kompl: public Obj {
    double re, im; // Sadržaj objekta.
    bool jednako(const Obj& obj) const // Da li su jednaki?
    { return re==(Kompl&obj).re && im==(Kompl&obj).im; }
    void pisi (ostream& it) const // Pisanje.
    { it << '(' << re << ',' << im << ')'; }
    void citaj(istream& ut) // Čitanje.
    { char zn; ut >> zn >> re >> zn >> im >> zn; }
public:
    Kompl(double x=0, double y=0): re(x), im(y) {} // Inicijalizacija.
    Kompl* kopija() const // Kopiranje.
    { return new Kompl(*this); }
};

#endif
```

```
// tekst3.h - Klasa tekstova.
```

```
#ifndef _tekst3_h_
#define _tekst3_h_

#include "objekat.h"
#include <cstring>
using namespace std;

class Tekst: public Obj {
    char* tks; // Sadržaj objekta.
    bool jednako(const Obj& obj) const // Da li su jednaki?
    { return strcmp(tks, ((Tekst&obj).tks) == 0; }
    void pisi (ostream& it) const // Pisanje.
    { it << '[' << (tks ? tks : "") << ']'; }
    void citaj(istream& ut); // Čitanje.
    void kopiraj(const char* niz); // Kopiranje.
public:
    Tekst(const char* niz="") { kopiraj(niz); } // Inicijalizacija nizom.
    Tekst(const Tekst& t): Obj(t) // Inicijalizacija tekstom.
    { kopiraj(t.tks); }
    ~Tekst() { delete [] tks; } // Uništavanje.
    Tekst& operator=(const Tekst& t) { // Dodela vrednosti.
        if (this != &t) { delete [] tks; Obj::operator=(t); kopiraj(t.tks); }
        return *this;
    }
    Tekst* kopija() const { return new Tekst(*this); } // Kopiranje.
};

#endif
```

```
// tekst3.C - Metode klase tekstova.
```

```
#include "tekst3.h"

void Tekst::kopiraj(const char* niz) { // Kopiranje.
    if (niz && niz[0]) {
        tks = new char [strlen(niz) + 1]; strcpy(tks, niz);
    } else tks = 0;
}
```

```
void Tekst::citaj(istream& ut) { // Čitanje.
    char niz[81], zn; ut >> zn; ut.getline(niz, 81, ' ');
    delete [] tks; kopiraj(niz);
}
```

```
// skup2t.C - Ispitivanje klase skupova apstraktnih objekata.
```

```
#include "skup2.h"
#include "tekst3.h"
#include "kompl2.h"
#include <iostream>
using namespace std;

Skup citaj() {
    Skup s;
    cout << "Ulaz: ";
    for (bool prvi=true;;) {
        char tip; cin >> tip;
        if (tip == '.') break;
        if (!prvi) cout << ", "; prvi = false;
        switch (tip) {
            case 't': case 'T': { Tekst t; cin >> t; cout << t; s += t; break; }
            case 'k': case 'K': { Kompl z; cin >> z; cout << z; s += z; break; }
        }
    }
    cout << endl;
    return s;
}

int main() {
    Skup s1(citaj()), s2(citaj());
    cout << "s1 = " << s1 << endl;
    cout << "s2 = " << s2 << endl;
    cout << "s1+s2 = " << s1+s2 << endl;
    cout << "s1-s2 = " << s1-s2 << endl;
    cout << "s1*s2 = " << s1*s2 << endl;
    cout << "s1-s1 = " << s1-s1 << endl;
    cout << "s1*{} = " << s1*Skup() << endl;
    return 0;
}
```

```
skup2t.pod
```

```
t [alfa] k (1,2) k (2,3) t [alfa] t [beta] k (1,2).
k (2,3) t [gama] k (1,3) t [alfa]
```

```
% skup2t <skup2t.pod
```

```
Ulaz: T1[alfa], K4(1,2), K7(2,3), T10[alfa], T12[beta], K15(1,2)
Ulaz: K25(2,3), T28[gama], K31(1,3), T34[alfa]
s1 = {T21[alfa], K22(1,2), K23(2,3), T24[beta]}
s2 = {K41(2,3), T42[gama], K43(1,3), T44[alfa]}
s1+s2 = {T51[alfa], K52(1,2), K53(2,3), T54[beta], T55[gama], K56(1,3)}
s1-s2 = {K59(1,2), T60[beta]}
s1*s2 = {T63[alfa], K64(2,3)}
s1-s1 = {}
s1*{} = {}
```

Zadatak 4.9 Geometrijska tela, sfere, valjci i redovi tela

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorom i operatorom za dodelu vrednosti, koji su potrebni za bezbedno korišćenje klasa):

- Apstraktnom geometrijskom *telu* može da se dohvati jednoslovnna oznaka vrste tela, da se izračuna zapremina, da se napravi kopija i upiše u izlazni tok (`it<<telo`).
- *Sfera* je predmet zadat poluprečnikom (podrazumevano 1). Oznaka vrste predmeta je *S*. U izlazni tok upisuje se u obliku *S(r)*.
- *Valjak* je predmet zadat poluprečnikom osnovice i visinom (podrazumevano 1 i 1). Oznaka vrste predmeta je *V*. U izlazni tok upisuje se u obliku *V(r,h)*.
- *Red tela* ograničenog kapaciteta stvara se prazan zadatog kapaciteta (podrazumevano 5). Može da se ispita da li je red pun i da li je prazan, da se stavi jedno telo na kraj reda (`red+=telo`; pokušaj stavljanja u pun red prekida program), da se uzme telo s početka reda (pokušaj uzimanja iz praznog reda prekida program) i da se sadržaj reda upiše u izlazni tok (`it<<red`) u obliku `red[t,t,...,t]`, gde je *t* – rezultat upisivanja jednog sadržanog tela.

Napisati na jeziku C++ *program* koji napravi red kapaciteta koji pročita s glavnog ulaza, čita raznovrsna tela s glavnog ulaza i stavlja ih u red dok se red ne napuni, ispiše sadržaj reda na glavnom izlazu uzimajući tela iz reda dok se red ne isprazni, izračuna srednju zapreminu svih tela u redu, ispiše dobijeni rezultat na glavnom izlazu i ponavlja prethodne korake dok za kapacitet reda ne pročita nedozvoljenu vrednost.

Rešenje:

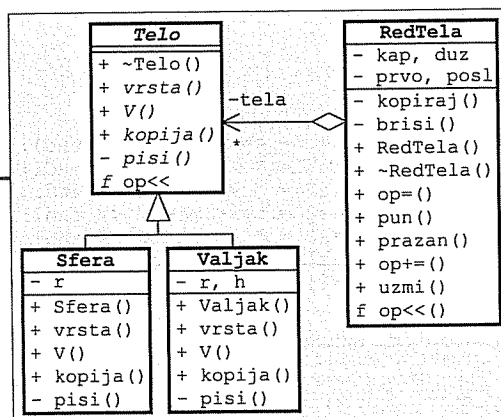
```
// telo1.h - Apstraktna klasa
// geometrijskih tela.
```

```
#ifndef _telo1_h_
#define _telo1_h_

#include <iostream>
using namespace std;
```

```
class Telo {
public:
    virtual ~Telo() {} // Virtuelan destruktor.
    virtual char vrsta() const =0; // Oznaka vrste.
    virtual double V() const =0; // Zapremina.
    virtual Telo* kopija() const =0; // Kopiranje.
private:
    virtual void pisi(ostream&) const =0; // Pisanje.
    friend ostream& operator<<(ostream& it, const Telo& t)
    { t.pisi(it); return it; }
};

#endif
```



```
// sfera2.h - Klasa sfera.
```

```
#ifndef _sfera2_h_
#define _sfera2_h_

#include "telo1.h"

class Sfera: public Telo {
    double r; // Poluprečnik
public:
    explicit Sfera(double rr=1) { r = rr; } // Konstruktor.
    char vrsta() const { return 'S'; } // Oznaka vrste.
    double V() const { return 4 * r*r*r * 3.14159 / 3; } // Zapremina.

    Sfera* kopija() const { return new Sfera(*this); } // Kopiranje.
private:
    void pisi(ostream& it) const { it << "S(" << r << ')'; } // Pisanje.
};

#endif
```

```
// valjak2.h - Klasa valjaka.
```

```
#ifndef _valjak2_h_
#define _valjak2_h_

#include "telo1.h"

class Valjak: public Telo {
    double r, h; // Poluprečnik i visina.
public:
    Valjak(double rr=1, double hh=1) { r = rr; h = hh; } // Konstruktor.
    char vrsta() const { return 'V'; } // Oznaka vrste.
    double V() const { return r*r * 3.14159 * h; } // Zapremina.
    Valjak* kopija() const { return new Valjak(*this); } // Kopiranje.
private:
    void pisi(ostream& it) const // Pisanje.
    { it << "V(" << r << ', ' << h << ')'; }
};

#endif
```

```
// redtela.h - Klasa redova tela.
```

```
#ifndef _redtela_h_
#define _redtela_h_

#include "telo1.h"
#include <iostream>
#include <cstdlib>
using namespace std;

class RedTela {
    Telo** tela; // Niz tela.
    int kap, duz; // Kapacitet i dužina reda.
    int prvo, posl; // Prvo i poslednje telo.
    void kopiraj(const RedTela&); // Kopiranje u objekat.
    void brisi(); // Brisanje objekta.
};
```

```

public:
    explicit RedTela(int k=5);           // Konstruktori.
    RedTela(const RedTela& r) { kopiraj(r); }
    ~RedTela() { brisi(); }             // Destruktor.
    RedTela& operator=(const RedTela& r) { // Dodela vrednoszi.
        if (this != &r) { brisi(); kopiraj(r); }
        return *this;
    }
    bool pun() { return duz == kap; }    // Da li je pun?
    bool prazan() { return duz == 0; }   // Da li je prazan?
    RedTela& operator+=(const Telo& t) { // Dodvanje tela na kraj.
        if (duz == kap) exit(1);
        tela[posl++] = t.kopija(); if (posl == kap) posl = 0;
        duz++;
        return *this;
    }
    Telo* uzmi() {                       // Uzimanje tela sa početka.
        if (duz == 0) exit(2);
        Telo* t = tela[prvo];
        tela[prvo++] = 0; if (prvo == kap) prvo = 0;
        duz--;
        return t;
    }
    friend ostream& operator<<(ostream&, const RedTela&); // Pisanje.
};

#endif

```

// redtela.C - Metode i funkcije uz klasu redova tela.

```

#include "redtela.h"

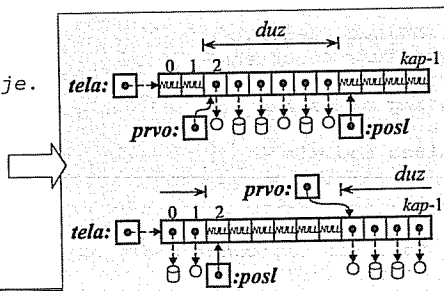
RedTela::RedTela(int k) {               // Konstruktor praznog reda.
    tela = new Telo* [kap = k];
    for (int i=0; i<kap; tela[i++]=0);
    prvo = posl = duz = 0;
}

void RedTela::kopiraj(const RedTela& r) { // Kopiranje u objekat.
    tela = new Telo* [kap = r.kap];
    for (int i=0; i<kap; i++)
        tela[i] = r.tela[i] ? r.tela[i]->kopija() : 0;
    prvo = r.prvo; posl = r.posl; duz = r.duz;
}

void RedTela::brisi() { // Brisanje objekta.
    for (int i=0; i<kap; delete tela[i++]);
    delete [] tela;
}

ostream& operator<<(ostream& it, // Pisanje.
                    const RedTela& r) {
    it << "red[";
    for (int i=0; i<r.duz; i++) {
        if (i) it << ',';
        it << *r.tela[(r.prvo+i)%r.kap];
    }
    return it << ']';
}

```



// redtelat.C - Ispitivanje klase redova tela.

```

#include "sfera2.h"
#include "valjak2.h"
#include "redtela.h"
#include <iostream>
using namespace std;

int main() {
    while (true) {
        cout << "kap? "; int k; cin >> k;
        if (k <= 0) break;
        RedTela red(k);
        while (!red.pun()) {
            cout << "Vrsta (S,V)? "; char vr; cin >> vr;
            double r, h;
            switch (vr) {
                case 's': case 'S':
                    cout << "r? "; cin >> r;
                    red += Sfera(r);
                    break;
                case 'v': case 'V':
                    cout << "r,h? "; cin >> r >> h;
                    red += Valjak(r, h);
                    break;
            }
        }
        cout << red << endl;
        double V = 0; int n = 0;
        while (!red.prazan()) {
            Telo* t = red.uzmi();
            V += t->V(); n++;
            delete t;
        }
        if (n) V /= n;
        cout << "Vsr= " << V << endl;
    }
    return 0;
}

```

```

% redtelat
kap? 4
Vrsta (S,V)? s
r? 13
Vrsta (S,V)? v
r,h? 1 5
Vrsta (S,V)? v
r,h? 5 3
Vrsta (S,V)? s
r? 8
red[S(13),V(1,5),V(5,3),S(8)]
Vsr= 2899.69
kap? 0

```

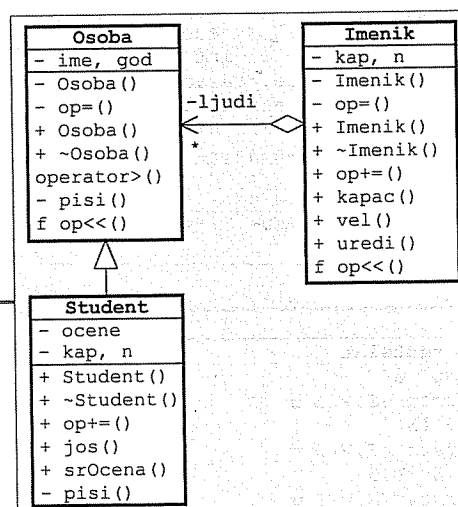
Zadatak 4.10 Osobe, studenti i imenici

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorom i operatorom za dodelu vrednosti, koji su potrebni za bezbedno korišćenje klasa):

- **Osoba** ima ime i godine starosti. Može da se ispita da li je jedna osoba starija od druge (osoba1 > osoba2) i da se osoba upiše u izlazni tok (it << osoba) u obliku *ime (god)*, gde su: *ime* – ime osobe i *god* – godine starosti. Osoba ne sme da se kopira ni na koji način.
- **Student** je osoba koja može da polaže najviše zadati broj ispita (podrazumevano 20) za koje se pamte samo ocene. Stvara se bez ijedne ocene, posle čega se ocene dodaju pojedinačno (student += ocena). Pokušaj dodavanja previše ocena prekida program. Može da se ispita koliko ocena može još da se doda, da se izračuna srednja vrednost položenih ispita. U izlazni tok se piše u obliku *ime (god) / sred*, gde je *sred* – srednja ocena položenih ispita.
- **Imenik** može da sadrži zadat broj (podrazumevano 10) osoba proizvoljne vrste. Stvara se prazan, posle čega se osobe dodaju pojedinačno (imenik += osoba). Prepunjavanje imenika prekida program. Može da se dohvati kapacitet imenika, broj osoba u imeniku, da se imenik uredi po opadajućem redosledu starosti osoba u njemu i da se upiše u izlazni tok (it << imenik), tako što se svaka osoba piše u posebnom redu. Imenik ne sme da se kopira ni na koji način.

Mapisati na jeziku C++ **program** koji čitajući potrebne podatke s glavnog ulaza napravi imenik koji sadrži izvestan broj osoba proizvoljne vrste, uredi imenik po opadajućem redosledu starosti osoba u njemu i ispiše imenik na glavnom izlazu.

Rešenje:



// osoba2.h - Klasa osoba.

```
#ifndef _osoba2_h_
#define _osoba2_h_

#include <cstring>
#include <iostream>
using namespace std;
```

```
class Osoba {
    char* ime;           // Ime.
    int god;             // Godine starosti.
    Osoba(const Osoba&) {} // Ne sme da se kopira.
    void operator=(const Osoba&) {} // Ne sme da se dodlejuje.
public:
    Osoba(const char* ii, int gg) { // Konstruktor.
        ime = new char [strlen(ii) + 1];
        strcpy(ime, ii);
        god = gg;
    }
    virtual ~Osoba() { delete [] ime; } // Virtuelan destruktor.
```

```
friend bool operator>(const Osoba& o1, // Da li je starija?
                      const Osoba& o2)
{ return o1.god > o2.god; }

protected:
virtual void pisi(ostream& it) const // Pisanje.
{ it << ime << '(' << god << ')'; }
friend ostream& operator<<(ostream& it, const Osoba& oso)
{ oso.pisi(it); return it; }
};

#endif
```

// student2.h - Klasa studenata.

```
#ifndef _student2_h_
#define _student2_h_
```

```
#include "osoba2.h"
#include <cstdlib>
#include <iostream>
using namespace std;
```

```
class Student: public Osoba {
    int* ocene;           // Niz ocena.
    int kap, n;           // Kapacitet niza i broj ocena.
public:
    // Konstruktor.
    Student(const char* ime, int god, int k=20): Osoba(ime, god)
    { ocene = new int [kap = k]; n = 0; }
    ~Student() { delete [] ocene; } // Destruktor.
    Student& operator+=(int oc) { // Dodavanje ocene.
        if (n == kap) exit(1);
        ocene[n++] = oc;
        return *this;
    }
    int jos() const { return kap - n; } // Broj slobodnih mesta.
    double srOcena() const;           // Srednje ocena.
private:
    void pisi(ostream& it) const // Pisanje.
    { Osoba::pisi(it); it << '/' << srOcena(); }
};

#endif
```

// student2.C - Metoda klase studenata.

```
#include "student2.h"
```

```
double Student::srOcena() const { // Srednja ocena.
    double s = 0; int k = 0;
    for (int i=0; i<n; i++)
        if (ocene[i] > 5) { s += ocene[i]; k++; }
    if (k) s /= k;
    return s;
}
```

```
// imenik2.h - Klasa imenika osoba.
```

```
#ifndef _imenik2_h_
#define _imenik2_h_

#include "osoba2.h"
#include <cstdlib>
#include <iostream>
using namespace std;

class Imenik {
    Osoba** ljudi;           // Niz osoba.
    int kap, n;              // Kapacitet niza i broj osoba.
    Imenik(const Imenik&) {} // Ne sme da se kopira.
    void operator=(const Imenik&) {} // Ne sme da se dodeljuje.
public:
    explicit Imenik(int k=10) { // Konstruktor.
        ljudi = new Osoba* [kap = k];
        n = 0;
    }
    ~Imenik();               // Destruktor.
    Imenik& operator+=(Osoba* oso) {
        if (n == kap) exit(2);
        ljudi[n++] = oso;
        return *this;
    }
    int kapac() const { return kap; } // Kapacitet imenika.
    int vel() const { return n; }    // Broj osoba u imeniku.
    Imenik& uredi();                // Uredivanje.
    friend ostream& operator<<(ostream& it, const Imenik& imen); // Pisanje.
};

#endif
```

```
// imenik2.C - Metode i funkcije uz klasu imenika osoba.
```

```
#include "imenik2.h"

Imenik::~Imenik() { // Destruktor.
    for (int i=n; i>0; delete ljudi[--i]);
    delete [] ljudi;
}

Imenik& Imenik::uredi() { // Uredivanje.
    for (int i=0; i<n-1; i++)
        for (int j=i+1; j<n; j++)
            if (*ljudi[j] > *ljudi[i])
                { Osoba* pom = ljudi[i]; ljudi[i] = ljudi[j]; ljudi[j] = pom; }
    return *this;
}

ostream& operator<<(ostream& it, const Imenik& imen) { // Pisanje.
    for (int i=0; i<imen.n; i++) it << *imen.ljudi[i] << endl;
    return it;
}
```

```
// imenik2t.C - Ispitivanje klase imenika osoba.
```

```
#include "osoba2.h"
#include "student2.h"
#include "imenik2.h"
#include <iostream>
using namespace std;

int main() {
    Imenik imen;
    while (true) {
        cout << "Vrsta (O, S, .)? "; char vrs; cin >> vrs;
        if (vrs == '.') break;
        cout << "Ime? "; char ime[30]; cin >> ime;
        cout << "Godine? "; int god; cin >> god;
        if (vrs == 'o')
            imen += new Osoba(ime, god);
        else {
            Student* stud = new Student(ime, god);
            while (true) {
                cout << "Ocena? "; int oce; cin >> oce;
                if (oce==0) break;
                *stud += oce;
            }
            imen += stud;
        }
    }
    cout << endl << imen.uredi();
    return 0;
}
```

```
% imenik2t
Vrsta (O, S, .)? o
Ime? Marko
Godine? 35
Vrsta (O, S, .)? s
Ime? Zoran
Godine? 22
Ocena? 7
Ocena? 5
Ocena? 8
Ocena? 9
Ocena? 0
Vrsta (O, S, .)? o
Ime? Stevan
Godine? 28
Vrsta (O, S, .)? .

Marko(35)
Stevan(28)
Zoran(22)/8
```

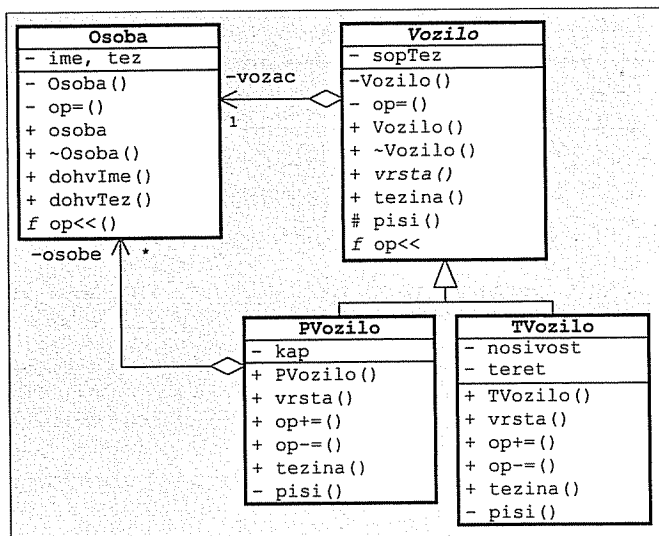
Zadatak 4.11 Osobe, vozila, teretna vozila i putnička vozila

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorom i operatorom za dodelu vrednosti koji su potrebni za bezbedno korišćenje klasa):

- **Osoba** ima zadatu ime proizvoljne dužine i zadatu težinu. Može da se dohvati ime i težina i da se osoba upiše u izlazni tok (`it<<osoba`) u obliku `ime (težina)`. Osoba ne sme da se kopira ni na koji način.
- Apstraktno **vozilo** ima zadatog vozača (osobu) i zadatu sopstvenu težinu. Vozač se zadaje pomoću adrese i ne postaje vlasništvo vozila. Može da se dohvati jednoslovna oznaka vrste vozila, da se odredi ukupna težina vozila i da se vozilo upiše u izlazni tok (`it<<vozilo`) u obliku `vrsta [vozač, sopstvenaTežina]`. Vozilo ne sme da se kopira ni na koji način.
- **Teretno vozilo** je vozilo koje ima zadatu nosivost. Može da se utovari (`vozilo+=tezina`) i da se istovari (`vozilo-=tezina`) teret zadate težine. Povratna vrednost obe radnje je indikator uspeha (neuspeh je ako bi se vozilo preopteretilo pri utovaru, odnosno ako u vozilu nema dovoljno tereta pri istovaru). U izlazni tok upisuje se u obliku `vrsta [vozač, sopstvenaTežina] {tovar/nosivost}`.
- **Putničko vozilo** je vozilo koje ima zadat maksimalan broj putnika (osoba). Može da u vozilo ulazi zadati putnik na prvo slobodno mesto (`vozilo+=putnik`; putnik se zadaje pomoću adrese i ne postaje vlasništvo vozila) i da iz vozila izlazi zadati putnik (`vozilo-=putnik`). Povratna vrednost obe radnje je indikator uspeha (neuspeh je ako u vozilu nema mesta za novog putnika pri ulasku, odnosno ako se vozilo u ne nalazi zadati putnik pri izlasku). U izlazni tok upisuje se u obliku `vrsta [vozač, sopstvenaTežina] {putnik|...|putnik}`.

Napisati na jeziku C++ **program** koji s glavnog ulaza pročita podatke o nekoliko vozila i posle toga na glavnom izlazu ispiše podatke o vozilima koja mogu da pređu preko mosta zadate nosivosti.

Rešenje:



```

// osoba.h - Klasa osoba.

#ifndef _osoba_h_
#define _osoba_h_

#include <cstring>
#include <iostream>
using namespace std;

class Osoba {
    char* ime; // Ime osobe.
    float tez; // Težina osobe.
    Osoba(const Osoba&); // Ne sme da se kopira.
    Osoba& operator=(const Osoba&); // Ne sme da se dodeljuje.
public:
    explicit Osoba(const char* iime, float ttez) { // Konstruktor
        ime = strcpy(new char [strlen(iime)+1], iime);
        tez = ttez;
    }
    ~Osoba() { delete [] ime; } // Destruktor.
    const char* dohvIme() const { return ime; } // Dohvatanje imena.
    float dohvTez() const { return tez; } // Dohvatanje težine.
    friend ostream& operator<<(ostream& it, const Osoba& osoba) // Pisanje.
    { return it << osoba.ime << '(' << osoba.tez << ')'; }
};

#endif

```

```

// vozilo.h - Apstraktna klasa vozila.

#ifndef _vozilo_h_
#define _vozilo_h_

#include "osoba3.h"
#include <iostream>
using namespace std;

class Vozilo {
    float sopTez; // Sopstvena težina.
    const Osoba *vozac; // Vozač vozila.
    Vozilo(const Vozilo&) {} // Ne sme da se kopira.
    void operator=(const Vozilo&) {} // Ne sme da se dodeljuje.
public:
    Vozilo(const Osoba* vozac, float sopTez) // Konstruktor.
    { this->vozac = vozac; this->sopTez=sopTez; }
    virtual ~Vozilo() {} // Destruktor.
    virtual char vrsta() const =0; // Oznaka vrste vozila.
    virtual float tezina() const // Ukupna težina.
    { return sopTez + vozac->dohvTez(); }
protected:
    virtual void pisi(ostream& it) const // Pisanje.
    { it << vrsta() << '[' << *vozac << ',' << sopTez << ']'; }
    friend ostream& operator<<(ostream& it, const Vozilo& v)
    { v.pisi(it); return it; }
};

#endif

```

```
// tvozilo.h - Klasa teretnih vozila.

#ifndef _tvozilo_h_
#define _tvozilo_h_

#include "vozilo1.h"

class TVozilo: public Vozilo {
    float nosivost, tovar; // Nosivost i teжина tovara.
public:
    TVozilo(const Osoba* vozac, float sopTez, float nosivost) // Konstruktor.
        : Vozilo(vozac, sopTez) {
        { this->nosivost=nosivost; tovar=0; }
        char vrsta() const { return 'T'; } // Oznaka vrste vozila.
        bool operator+=(float tez) { // Stavljanje tovara.
            if (tovar+tez > nosivost) return false;
            tovar += tez; return true;
        }
        bool operator-=(float tez) { // Skidanje tovara.
            if (tovar-tez < 0) return false;;
            tovar -= tez; return true;
        }
        float tezina() const // Ukupna težina.
            { return Vozilo::tezina() + tovar; }
private:
        void pisi(ostream& it) const // Pisanje.
            {
                Vozilo::pisi(it);
                it << '(' << tovar << '/' << nosivost << ')';
            }
};

#endif
```

```
// pvozilo.h - Klasa putničkih vozila.
```

```
#ifndef _pvozilo_h_
#define _pvozilo_h_

#include "vozilo1.h"

class PVozilo: public Vozilo {
    const Osoba** osobe; // Niz osoba.
    int kap; // Kapacitet vozila.
public:
    PVozilo(const Osoba* vozac, float sopTez, int k); // Konstruktor.
    ~PVozilo() { delete [] osobe; } // Destruktor.
    char vrsta() const { return 'P'; } // Oznaka vrste vozila.
    bool operator+=(const Osoba* putnik); // Ulazak putnika.
    bool operator-=(const Osoba* putnik); // Izlazak putnika.
    float tezina() const;
private:
    void pisi(ostream& it) const; // Pisanje.
};

#endif
```

```
// pvozilo.C - Metode klase putničkih vozila.
```

```
#include "pvozilo.h"

PVozilo::PVozilo(const Osoba* vozac, float sopTez, int k) // Konstruktor.
    : Vozilo(vozac, sopTez) {
    osobe = new const Osoba* [kap = k];
    for (int i=0; i<kap; osobe[i++]=0);
}

bool PVozilo::operator+=(const Osoba* putnik) { // Ulazak putnika.
    int i = 0; while (i<kap && osobe[i]) i++;
    if (i == kap) return false;
    osobe[i] = putnik; return true;
}

float PVozilo::tezina() const {
    float t = Vozilo::tezina();
    for (int i=0; i<kap; i++)
        if (osobe[i]) t += osobe[i]->dohvTez();
    return t;
}

bool PVozilo::operator-=(const Osoba* putnik) { // Izlazak putnika.
    int i = 0; while (i<kap && osobe[i]!=putnik) i++;
    if (i == kap) return false;
    osobe[i] = 0; return true;
}

void PVozilo::pisi(ostream& it) const { // Pisanje.
    Vozilo::pisi(it);
    it << '{';
    bool prvi = true;
    for (int i=0; i<kap; i++)
        if (osobe[i]) {
            if (!prvi) cout << '|';
            if (osobe[i]) it << *osobe[i];
            prvi = false;
        }
    it << '}';
}
```

```
// vozilat.C - Ispitivanje klase vozila.
```

```
#include "tvozilo.h"
#include "pvozilo.h"
#include <iostream>
using namespace std;

int main() {
    Vozilo* vozila[100]; int brVozila = 0;
    Osoba* osobe[100]; int brOsoba = 0;
    while (true) {
        cout << "\nVrsta vozila (T,P,*)? "; char vrs; cin >> vrs;
        if (vrs == '*') break;
        cout << "Ime vozaca? "; char ime[30]; cin >> ime;
        cout << "Težina vozaca? "; float tezina; cin >> tezina;
        Osoba* vozac = osobe[brOsoba++] = new Osoba(ime, tezina);
        cout << "Težina vozila? "; float tezVozila; cin >> tezVozila;
        switch (vrs) {
```



```

case 't': case 'T': {
    cout << "Nosivost vozila? "; float nosivost; cin >> nosivost;
    TVozilo* tv = new TVozilo(vozac, tezVozila, nosivost);
    cout << "Tezina tereta? "; float teret; cin >> teret;
    *tv += teret;
    vozila[brVozila++] = tv;
    break;
}
case 'p': case 'P': {
    cout << "Broj mesta? "; int brMesta; cin >> brMesta;
    PVozilo* pv = new PVozilo(vozac, tezVozila, brMesta);
    while (true) {
        cout << "Ime putnika? "; char putnik[30]; cin >> putnik;
        if (putnik[0] == '*') break;
        cout << "Tezina putnika? "; float tezina; cin >> tezina;
        *pv += osobe[brOsoba++] = new Osoba(putnik, tezina);
    }
    vozila[brVozila++] = pv;
    break;
}
default:
    cout << "**** Nepoznata vrsta vozila! ****\n";
}
}
cout << "\nNosivost mosta? "; double nosivost; cin >> nosivost;
cout << "\nMogu da predju most:\n";
for (int i=0; i<brVozila; i++)
    if (vozila[i]->tezina() <= nosivost)
        cout << *vozila[i] << " - " << vozila[i]->tezina() << endl;
for (int i=0; i<brVozila; delete vozila[i++]);
for (int i=0; i<brOsoba; delete osobe [i++]);
return 0;
}

```

```

% vozilat
Vrsta vozila (T,P,*)? t
Ime vozaca? Marko
Tezina vozaca? 75
Tezina vozila? 1500
Nosivost vozila? 5000
Tezina tereta? 2000

```

```

Broj mesta? 6
Ime putnika? Mirko
Tezina putnika? 75
Ime putnika? Milan
Tezina putnika? 68
Ime putnika? Petar
Tezina putnika? 82
Ime putnika? *

```

```

Vrsta vozila (T,P,*)? p
Ime vozaca? Zoran
Tezina vozaca? 89
Tezina vozila? 800

```

```

Vrsta vozila (T,P,*)? t
Ime vozaca? Jovan
Tezina vozaca? 72
Tezina vozila? 2000
Nosivost vozila? 10000
Tezina tereta? 8000

```

```

Vrsta vozila (T,P,*)? *

```

```

Nosivost mosta? 8000

```

```

Mogu da predju most:

```

```

T[Marko(75),1500](2000/5000) - 3575

```

```

P[Zoran(89),800](Mirko(75)|Milan(68)|Petar(82)) - 1114

```

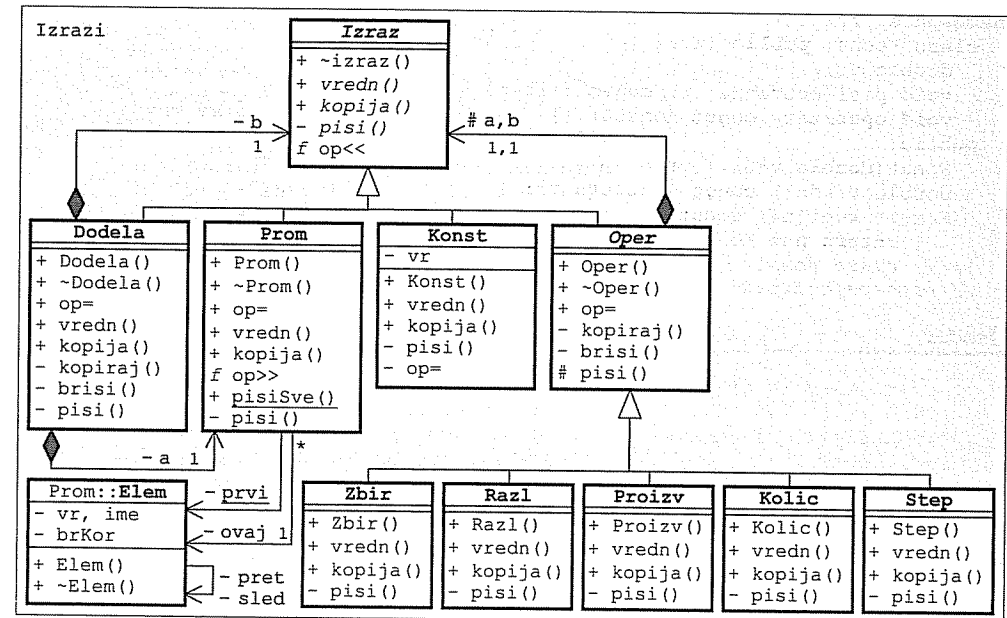
Zadatak 4.12 Izrazi, konstante, promenljive, dodele vrednosti i aritmetičke operacije

Napisati na jeziku C++ sledeće klase (sve klase opremiti potrebnim konstruktorima, destruktorom i operatorom za dodelu vrednosti):

- Apstraktnom **izrazu** može da se izračuna vrednost realnog tipa, da se stvara kopija i da se njegov algebarski izraz upiše u izlazni tok (`it<<izr`).
- **Konstanta** je izraz čija vrednost ne može da se promeni posle inicijalizacije. U izlazni tok se upisuje vrednost konstante.
- **Promenljiva** je izraz koji ima ime i realnu vrednost (podrazumevano 0) koja može da se promeni i posle inicijalizacije. U izlazni tok se upisuje ime promenljive. Više promenljivih s istim imenom predstavljaju isti podatak. Vrednost promenljive može da se čita iz ulaznog toka (`tok>>prom`).
- **Dodela** je izraz koji se inicijalizuje jednom promenljivom (na primer: a) i jednim izrazom (na primer: b). Vrednost izraza b prilikom izračunavanja dodeljuje se promenljivoj a . Vrednost izraza dodele jednaka je vrednosti koja je stavljena u promenljivu. Objekat dodele piše se u obliku ($a=b$), gde su: a i b – rezultati pisanja operanada.
- **Zbir, razlika, proizvod, količnik i stepen** su izrazi koji se inicijalizuju s dva izraza (na primer: a i b) i čije su vrednosti jednake $a+b$, $a-b$, ab i a/b . Pišu se u obliku ($a+b$), ($a-b$), ($a*b$), (a/b) i (a^b), gde su: a i b – rezultati pisanja operanada. Objekti istih tipova mogu međusobno da se dodeljuju (na primer: `zbir1=zbir2`). Tada se u odredišni objekat kopira sadržaj izvorišnog objekta.

Napisati na jeziku C++ **program** koji stvara objekat promenljive x i objekat za izraz x^3-2x , ispiše algebarski oblik tog izraza na glavnom izlazu i posle tabelira vrednost tog izraza na glavnom izlazu za sve vrednosti $x_{\min} \leq x \leq x_{\max}$ s korakom Δx . Parametri tabeliranja dostavljaju se kao parametri programa.

Rešenje:



```
// izraz.h - Apstraktna klasa izraza.
```

```
#ifndef _izraz_h_
#define _izraz_h_
```

```
#include <iostream>
using namespace std;
```

```
namespace Izrazi {
    class Izraz {
    public:
        virtual ~Izraz() {} // Virtuelan destruktor.
        virtual double vredn() const =0; // Vrednost izraza.
        virtual Izraz* kopija() const =0; // Kopiranje.
    private:
        virtual void pisi(ostream& it) const =0; // Pisanje.
        friend ostream& operator<<(ostream& it, const Izraz& izr)
        { izr.pisi(it); return it; }
    }; // class Izraz
} // namespace Izrazi
```

```
#endif
```

```
// konst.h - Klasa konstanti.
```

```
#ifndef _konst_h_
#define _konst_h_
```

```
#include "izraz.h"
```

```
namespace Izrazi {
    class Konst: public Izraz {
    public:
        double vr; // Vrednost.
        void pisi(ostream& it) const { it << vr; } // Pisanje.
        void operator=(const Konst&) {} // Ne sme da se menja.
    private:
        Konst(double v=0) { vr = v; } // Stvaranje.
        double vredn() const { return vr; } // Vrednost.
        Konst* kopija() const // Kopiranje.
        { return new Konst(*this); }
    }; // class Konst
} // namespace Izrazi
```

```
#endif
```

```
// prom.h - Klasa promenljivih.
```

```
#ifndef _prom_h_
#define _prom_h_
```

```
#include "izraz.h"
#include <cstring>
using namespace std;
```

```
namespace Izrazi {
    class Prom: public Izraz {
    public:
        struct Elem { // Element liste svih promenljivih:
            char* ime; // - ime,
            double vr; // - vrednost,
            int brKor; // - broj korišćenja,
            Elem *pret, *sled; // - prethodni i sledeći u listi,
            Elem(const char* i, double v, // - konstruktor,
                Elem* p=0, Elem* s=0) {
                ime = new char [strlen(i)+1]; strcpy(ime, i);
                vr = v; pret = p; sled = s; brKor = 1;
            }
            ~Elem() { delete [] ime; } // - destruktor.
        }; // struct Elem

        static Elem* prvi; // Početak liste svih promenljivih.
        Elem* ovaj; // Element pridružen ovom objektu.
        void pravi(const char* ime, double vr); // Stvaranje objekta.
        void pisi(ostream& it) const { it << ovaj->ime; } // Pisanje.
    public:
        Prom(const char* ime, double vr=0) // Inicijalizacija:
        { pravi(ime, vr); } // - imenom i brojem,
        Prom(const char* ime, const Izraz& i) // - imenom i izrazom,
        { pravi(ime, i.vredn()); }
        Prom(const Prom& p) // - kopijom promenljive.
        { ovaj = p.ovaj; ovaj->brKor++; }
        ~Prom() { // Uništavanje.
            if (--ovaj->brKor == 0) {
                (ovaj->pret ? ovaj->pret->sled : prvi) = ovaj->sled;
                if (ovaj->sled) ovaj->sled->pret = ovaj->pret;
                delete ovaj;
            }
        }
        Prom& operator=(double v) // Dodela vrednosti:
        { ovaj->vr = v; return *this; } // - realnog broja,
        Prom& operator=(const Izraz& i) // - izraza,
        { ovaj->vr = i.vredn(); return *this; }
        Prom& operator=(const Prom& p) // - druge promenljive.
        { ovaj->vr = p.vredn(); return *this; }
        double vredn() const { return ovaj->vr; } // Vrednost promenljive.
        Prom* kopija() const { return new Prom(*this); } // Kopiranje.
        friend istream& operator>>(istream& ut, Prom& p) // Čitanje.
        { return ut >> p.ovaj->vr; }
        static void pisiSve(ostream& it); // Spisak svih promenljivih.
    }; // class Prom
} // namespace Izrazi
```

```
#endif
```

```
// prom.C - Statičko polje i metode klase promenljivih.
```

```
#include "prom.h"
```

```
Izrazi::Prom::Elem* Izrazi::Prom::prvi = 0; // Lista svih promenljivih.
```

```
void Izrazi::Prom::pravi(const char* ime, double vr) { // Stvaranje
    Elem *tek = prvi, *pret=0; int p; // objekta.
    while (tek && (p = strcmp(tek->ime,ime))<0)
    { pret = tek; tek = tek->sled; }
    if (tek && p==0) { // Ime vec postoji.
        ovaj = tek; ovaj->brKor++;
    } else { // Novo ime.
        Elem* novi = new Elem(ime, vr, pret, tek);
        if (tek) tek->pret = novi;
        (!pret ? prvi : pret->sled) = novi;
        ovaj = novi;
    }
}
```

```
void Izrazi::Prom::pisiSve(ostream& it) {
    for (Elem* tek=prvi; tek; tek=tek->sled)
        it << tek->ime << ' ';
    it << endl;
}
```

```
// dodela.h - Klasa dodele vrednosti.
```

```
#ifndef _dodela_h_
#define _dodela_h_
```

```
#include "izraz.h"
#include "prom.h"
```

```
namespace Izrazi {
    class Dodela: public Izraz {
    private:
        Prom* a; Izraz* b; // Operandi.
        void pisi(ostream& it) const // Pisanje.
        { it << '(' << *a << '=' << *b << ')'; }
        void kopiraj(const Dodela& d) // Kopiranje u objekat.
        { a = d.a->kopija(); b = d.b->kopija(); }
        void brisi() { delete a; delete b; } // Oslobađanje memorije.
    public:
        Dodela(const Prom& x, const Izraz& y) // Inicijalizacija izrazima.
        { a = x.kopija(); b = y.kopija(); }
        Dodela(const Dodela& op) { kopiraj(op); } // Konstruktor kopije.
        ~Dodela() { brisi(); } // Destruktor.
        Dodela& operator=(const Dodela& y) { // Dodela vrednosti.
            if (this != &y) { brisi(); kopiraj(y); }
            return *this;
        }
        double vredn() const { return (*a = *b).vredn(); } // Vrednost.
        Dodela* kopija() const { return new Dodela(*this); } // Kopiranje.
    }; // class Dodela
} // namespace Izrazi;

#endif
```

```
// oper.h - Klase operatora.
```

```
#ifndef _oper_h_
#define _oper_h_
```

```
#include "izraz.h"
#include <cmath>
using namespace std;
```

```
namespace Izrazi {
    class Oper: public Izraz { // APSTRAKTNA KLASA BINARNIH OPERATORA.
    protected:
        Izraz *a, *b; // Operandi.
        void pisi(ostream& it, char op) const // Pisanje.
        { it << '(' << *a << op << *b << ')'; }
    private:
        void kopiraj(const Oper& op) // Kopiranje u objekat.
        { a = op.a->kopija(); b = op.b->kopija(); }
        void brisi() { delete a; delete b; } // Oslobađanje memorije.
    public:
        Oper(const Izraz& x, const Izraz& y) // Inicijalizacija izrazima.
        { a = x.kopija(); b = y.kopija(); }
        Oper(const Oper& op) { kopiraj(op); } // Konstruktor kopije.
        ~Oper() { brisi(); } // Destruktor.
        Oper& operator=(const Oper& y) { // Dodela vrednosti.
            if (this != &y) { brisi(); kopiraj(y); }
            return *this;
        }
    }; // class Oper

    class Zbir: public Oper { // KLASA ZBIROVA.
        void pisi(ostream& it) const { Oper::pisi(it, '+'); } // Pisanje.
    public:
        Zbir(const Izraz& x, const Izraz& y): Oper(x, y) {} // Stvaranje.
        double vredn() const { return a->vredn() + b->vredn(); } // Vrednost.
        Zbir* kopija() const { return new Zbir(*this); } // Kopiranje.
    }; // class Zbir

    class Razl: public Oper { // KLASA RAZLIKA.
        void pisi(ostream& it) const { Oper::pisi(it, '-'); } // Pisanje.
    public:
        Razl(const Izraz& x, const Izraz& y): Oper(x, y) {} // Stvaranje.
        double vredn() const { return a->vredn() - b->vredn(); } // Vrednost.
        Razl* kopija() const { return new Razl(*this); } // Kopiranje.
    }; // class Razl

    class Proizv: public Oper { // KL. PROIZVODA.
        void pisi(ostream& it) const { Oper::pisi(it, '*'); } // Pisanje.
    public:
        Proizv(const Izraz& x, const Izraz& y): Oper(x, y) {} // Stvaranje.
        double vredn() const { return a->vredn() * b->vredn(); } // Vrednost.
        Proizv* kopija() const { return new Proizv(*this); } // Kopiranje.
    }; // class Proizv

    class Kolic: public Oper { // KL. KOLIČNIKA.
        void pisi(ostream& it) const { Oper::pisi(it, '/'); } // Pisanje.
    public:
        Kolic(const Izraz& x, const Izraz& y): Oper(x, y) {} // Stvaranje.
        double vredn() const { return a->vredn() / b->vredn(); } // Vrednost.
        Kolic* kopija() const { return new Kolic(*this); } // Kopiranje.
    }; // class Kolic
```

```

class Step: public Oper { // KLASA STEPENA.
    void pisi(ostream& it) const { Oper::pisi(it, '^'); } // Pisanje.
public:
    Step(const Izraz& x, const Izraz& y): Oper(x, y) {} // Stvaranje.
    double vredn() const // Vrednost.
    { return pow(a->vredn(), b->vredn()); }
    Step* kopija() const { return new Step(*this); } // Kopiranje.
}; // class Step

// POMOĆNE FUNKCIJE ZA OMOGUĆAVANJE PISANJA ČITLJIVIH IZRAZA.
inline Zbir operator+ (const Izraz& a, const Izraz& b) // a + b
{ return Zbir (a, b); }
inline Razl operator- (const Izraz& a, const Izraz& b) // a - b
{ return Razl (a, b); }
inline Proizv operator* (const Izraz& a, const Izraz& b) // a * b
{ return Proizv(a, b); }
inline Kolic operator/ (const Izraz& a, const Izraz& b) // a / b
{ return Kolic (a, b); }
inline Step operator^ (const Izraz& a, const Izraz& b) // a ^ b
{ return Step (a, b); }
inline bool operator==(const Izraz& a, const Izraz& b) // a == b
{ return a.vredn() == b.vredn(); }
inline bool operator!=(const Izraz& a, const Izraz& b) // a != b
{ return a.vredn() != b.vredn(); }
inline bool operator< (const Izraz& a, const Izraz& b) // a < b
{ return a.vredn() < b.vredn(); }
inline bool operator<= (const Izraz& a, const Izraz& b) // a <= b
{ return a.vredn() <= b.vredn(); }
inline bool operator> (const Izraz& a, const Izraz& b) // a > b
{ return a.vredn() > b.vredn(); }
inline bool operator>= (const Izraz& a, const Izraz& b) // a >= b
{ return a.vredn() >= b.vredn(); }

} // namespace Izrazi;

#endif

```

```

// izrazt.C - Ispitivanje klase za izraze.

#include "konst.h"
#include "prom.h"
#include "oper.h"
using namespace Izrazi;
#include <iostream>
#include <cstdlib>
using namespace std;

int main(int, const char** varg) {
    Prom x("x"), xmin("xmin", atof(varg[1])),
        xmax("xmax", atof(varg[2])), dx("dx", atof(varg[3]));
    const Izraz& izr = (x ^ Konst(3)) - Konst(2) * x;
    cout << xmin << '=' << xmin.vredn() << ", "
        << xmax << '=' << xmax.vredn() << ", "
        << dx << '=' << dx.vredn() << endl;
    cout << '\n' << x << '\t' << izr
        << "\n=====\\n";
    for (x=xmin; x<=xmax; x=x+dx)
        cout << x.vredn() << '\t' << izr.vredn() << endl;
    return 0;
}

```

```

% izrazt -5 5 1
xmin=-5, xmax=5, dx=1

```

x	((x^3)-(2*x))
-5	-115
-4	-56
-3	-21
-2	-4
-1	1
0	0
1	-1
2	4
3	21
4	56
5	115

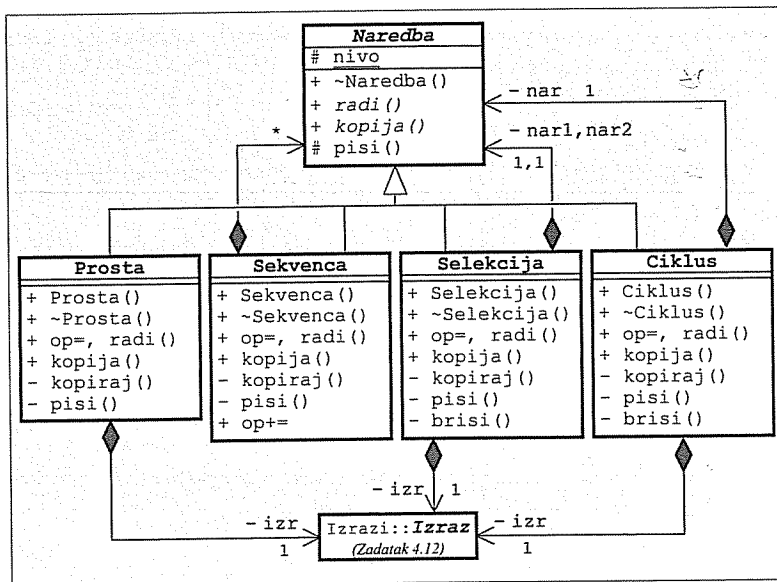
Zadatak 4.13 Naredbe, proste naredbe, sekvence, selekcije i ciklusi

Korišćenjem klase za izraze iz prethodnog zadatka, napisati na jeziku C++ sledeće klase (sve klase opremiti potrebnim konstruktorima, destruktorom i operatorom za dodelu vrednosti):

- Apstraktna **naredba** može da se izvrši, može da se stvara kopija i da se njen tekstualni oblik upiše u izlazni tok (`it<<nar`).
- **Prosta** naredba je naredba koja sadrži jedan izraz, može da bude i prazna. Izvršavanje se sastoji od izračunavanja izraza. Piše se u obliku `izr;`, gde je: `izr` – rezultat pisanja sadržanog izraza.
- **Sekvenca** je naredba koja sadrži proizvoljan broj naredbi. Stvara se prazna, a sadržane naredbe se dodaju jedna po jedna (`sekv+=nar`) i izvršavaju se po redosledu dodavanja. Sekvenca se piše u obliku `{nar ... nar}`, gde je: `nar` – rezultat jedne sadržane naredbe.
- **Selekcija** je naredba koja sadrži jedan izraz i dve naredbe. Pri izvršavanju izračuna se vrednost izraza i ako je $\neq 0$, izvršava se prva sadržana naredba, a ako je $=0$, izvršava se druga sadržana naredba. Piše se u obliku `if (izr) nar1 else nar2`, gde su: `izr` – rezultat pisanja sadržanog izraza, a `nar1` i `nar2` – rezultati pisanja sadržanih naredbi.
- **Ciklus** je naredba koja sadrži jedan izraz i jednu naredbu. Pri izvršavanju sadržana naredba se izvršava sve dok je vrednost sadržanog izraza $\neq 0$. Piše se u obliku `while (izr) nar`, gde su: `izr` – rezultat pisanja sadržanog izraza, a `nar` – rezultat pisanja sadržane naredbe.

Napisati na jeziku C++ **program** koji stvara objekat naredbe koja predstavlja program za izračunavanje $n!$, ispiše naredbu na glavnom izlazu i posle toga čita vrednost za n s glavnog ulaza, izračunava $n!$ i ispisuje dobijeni rezultat na glavnom izlazu, sve dok za n ne pročitati negativnu vrednost.

Rešenje:



// naredba.h - Apstraktna klasa naredbi.

```

#ifndef _naredba_h_
#define _naredba_h_

#include <iostream>
#include <iomanip>
using namespace std;

namespace Naredbe {
    class Naredba {
    public:
        virtual ~Naredba() {} // Virtuelan destruktor.
        virtual void radi() const = 0; // Izvršavanje.
        virtual Naredba* kopija() const = 0; // Kopiranje.
    protected:
        static int nivo; // Nivo naredbe pri pisanju.
        virtual void pisi(ostream& it) const // Pisanje.
        { it << setw(nivo*2) << " "; }
        friend ostream& operator<<(ostream& it, const Naredba& nar)
        { nar.pisi(it); return it; }
    }; // class Naredba
} // namespace Naredbe

#endif

```

// naredba.C - Statičko polje apstraktne klase naredbi.

```
#include "naredba.h"
```

```
int Naredbe::Naredba::nivo = 0; // Nivo naredbe pri pisanju.
```

// prosta.h - Klasa prostih naredbi.

```

#ifndef _prosta_h_
#define _prosta_h_

```

```
#include "naredba.h"
```

```
#include "izraz.h" // Zadatak 4.12
```

```
using namespace Izrazi;
```

```

namespace Naredbe {
    class Prosta: public Naredba {
        Izraz* izr; // Izraz koji se izračunava.
        void pisi(ostream& it) const { // Pisanje.
            Naredba::pisi(it);
            if (izr) it << *izr;
            it << ";\n";
        }
        void kopiraj(const Prosta& p) // Kopiranje u objekat.
        { izr = p.isr ? p.isr->kopija() : 0; }
    public:
        Prosta() { izr = 0; } // Prazna naredba.
        Prosta(const Izraz& i) { izr = i.kopija(); } // Inic. izrazom.
        Prosta(const Prosta& p) { kopiraj(p); } // Konstruktor kopije.
        ~Prosta() { delete izr; izr = 0; } // Uništavanje.
    };
}

```

```

Prosta& operator=(const Prosta& p) { // Dodela vrednosti
    if (this != &p) { delete izr; kopiraj(p); }
    return *this;
}
void radi() const { if (izr) izr->vredn(); } // Izvršavanje.
Prosta* kopija() const { return new Prosta(*this); } // Kopiranje.
}; // class Prosta
} // namespace Naredbe

#endif

```

```
// sekvenca.h - Klasa sekvenci.
```

```

#ifndef _sekvenca_h_
#define _sekvenca_h_

#include "naredba.h"

namespace Naredbe {
    class Sekvenca: public Naredba {
        struct Elem { // Element sekvence:
            Naredba* nar; // - sadržana naredba,
            Elem* sled; // - sledeći element liste,
            Elem(Naredba* n) { nar = n; sled = 0; } // - konstruktor,
            ~Elem() { delete nar; delete sled; } // - destruktor.
        }; // struct Elem

        Elem* prva, *posl; // Prva i poslednja naredba.
        void kopiraj(const Sekvenca&); // Kopiranje u objekat.
        void pisi(ostream&) const; // Pisanje.

    public:
        Sekvenca() { prva = posl = 0; } // Prazna sekvenca.
        Sekvenca(const Sekvenca& s) { kopiraj(s); } // Konstruktor kopije.
        ~Sekvenca() { delete prva; } // Uništavanje.
        Sekvenca& operator=(const Sekvenca& s) { // Dodela vrednosti.
            if (this != &s) { delete prva; kopiraj(s); }
            return *this;
        }
        Sekvenca& operator+=(const Naredba& n) { // Dodavanje naredbe.
            posl = (!prva ? prva : posl->sled) = new Elem(n.kopija());
            return *this;
        }
        void radi() const; // Izvršavanje.
        Sekvenca* kopija() const { return new Sekvenca(*this); } // Kopiranje.
    }; // class Sekvenca
} // namespace Naredbe

#endif

```

```
// sekvenca.C - Metode klase sekvenci.
```

```

#include "sekvenca.h"

namespace Naredbe {
    void Sekvenca::kopiraj(const Sekvenca& s) { // Kopiranje u objekat.
        prva = 0; Elem* posl = 0;
        for (Elem* tek=s.prva; tek; tek=tek->sled)
            posl = (!prva ? prva : posl->sled) = new Elem(tek->nar->kopija());
    }

    void Sekvenca::pisi(ostream& it) const { // Pisanje.
        Naredba::pisi(it); it << "\n";
        nivo++;
        for (Elem* tek=prva; tek; tek=tek->sled) it << *tek->nar;
        nivo--;
        Naredba::pisi(it); it << "\n";
    }

    void Sekvenca::radi() const { // Izvršavanje.
        for (Elem* tek=prva; tek; tek=tek->sled) tek->nar->radi();
    }
} // namespace Naredbe

```

```
// selekcija.h - Klasa selekcija.
```

```

#ifndef _selekcija_h_
#define _selekcija_h_

#include "izraz.h" // Zadatak 4.12
using namespace Izrazi;
#include "naredba.h"

namespace Naredbe {
    class Selekcija: public Naredba {
        Izraz* izr; // Uslov selekcije.
        Naredba *nar1, *nar2; // Uslovno izvršavanje naredbe.
        void kopiraj(const Selekcija& s) { // Kopiranje u objekat.
            izr = s.izr->kopija();
            nar1 = s.nar1->kopija();
            nar2 = s.nar2->kopija();
        }
        void brisi() // Oslobađanje memorije.
        { delete izr; delete nar1; delete nar2; }
        void pisi(ostream& it) const { // Pisanje.
            Naredba::pisi(it); it << "if(" << *izr << ")\n";
            nivo++; it << *nar1; nivo--;
            Naredba::pisi(it); it << "else\n";
            nivo++; it << *nar2; nivo--;
        }
    public:
        // Inicijalizacija.
        Selekcija(const Izraz& i, const Naredba& n1, const Naredba& n2)
        { izr = i.kopija(); nar1 = n1.kopija(); nar2 = n2.kopija(); }
        Selekcija(const Selekcija& s) { kopiraj(s); } // Konstruktor kopije.
        ~Selekcija() { brisi(); } // Uništavanje.
    };
}

```

```

Selekcija& operator=(const Selekcija& s) {           // Dodela vrednosti.
    if (this != &s) { brisi(); kopiraj(s); }
    return *this;
}
void radi() const                                   // Izvršavanje.
{ if (izr->vredn()) nar1->radi(); else nar2->radi(); }
Selekcija* kopija() const { return new Selekcija(*this); } // Poli. kop.
}; // class Selekcija
} // namespace Naredbe

#endif

```

// ciklus.h - Klasa ciklusa.

```

#ifndef _ciklus_h_
#define _ciklus_h_

#include "izraz.h"  ➡ Zadatak 4.12
using namespace Izrazi;
#include "naredba.h"

namespace Naredbe {
    class Ciklus: public Naredba {
        Izraz* izr;           // Uslov ciklusa.
        Naredba* nar;         // Sadržaj ciklusa.
        void kopiraj(const Ciklus& c) // Kopiranje u objekat.
        { izr = c.izr->kopija(); nar = c.nar->kopija(); }
        void brisi() { delete izr; delete nar; } // Oslobađanje memorije.
        void pisi(ostream& it) const { // Pisanje.
            Naredba::pisi(it); it << "while(" << *izr << ")\n";
            nivo++; it << *nar; nivo--;
        }
    public:
        Ciklus(const Izraz& i, const Naredba& n) // Inicijalizacija.
        { izr = i.kopija(); nar = n.kopija(); }
        Ciklus(const Ciklus& s) { kopiraj(s); } // Konstruktor kopije.
        ~Ciklus() { brisi(); } // Uništavanje.
        Ciklus& operator=(const Ciklus& s) { // Dodela vrednosti.
            if (this != &s) { brisi(); kopiraj(s); }
            return *this;
        }
        void radi() const; // Izvršavanje.
        Ciklus* kopija() const { return new Ciklus(*this); } // Kopiranje.
    }; // class Ciklus
} // namespace Naredbe

#endif

```

// ciklus.C - Metoda klase ciklusa.

```

#include "ciklus.h"

void Naredbe::Ciklus::radi() const // Izvršavanje.
{ while (izr->vredn()) nar->radi(); }

```

// naredbat.C - Ispitivanje klase naredbi.

```

#include "konst.h"
#include "prom.h"
#include "oper.h" ➡ Zadatak 4.12
#include "dodela.h"
using namespace Izrazi;
#include "prosta.h"
#include "sekvenca.h"
#include "selekcija.h"
#include "ciklus.h"
using namespace Naredbe;
#include <iostream>
using namespace std;

int main() {
    Prom n("n"), i("i"), f("f");
    Sekvenca telo;
    telo += Prosta(Dodela(i, Zbir(i, Konst(1))));
    telo += Prosta(Dodela(f, Proizv(f, i)));
    Ciklus cikl(Razl(n, i), telo);
    Sekvenca prog;
    prog += Prosta(Dodela(i, Konst(0)));
    prog += Prosta(Dodela(f, Konst(1)));
    prog += cikl;
    cout << "Program:\n" << prog << endl;
    while (true) {
        cout << "n? "; cin >> n;
        if (n.vredn() < 0) break;
        prog.radi();
        cout << "f= " << f.vredn() << endl;
    }
    return 0;
}

```

```

% naredbat
Program:
{
    (i=0);
    (f=1);
    while((n-i))
    {
        (i=(i+1));
        (f=(f*i));
    }
}

n? 0
f= 1
n? 4
f= 24
n? 8
f= 40320
n? 10
f= 3.6288e+06
n? -1

```

5 Izuzeci

Zadatak 5.1 Vektori realnih brojeva sa zadatim opsezima indeksa

Napisati na jeziku C++ klasu vektora realnih brojeva sa zadatim opsezima indeksa. Predvideti:

- stvaranje i uništavanje vektora,
- dodelu vrednosti jednog vektora drugom,
- pristup komponenti vektora,
- izračunavanje skalarnog proizvoda dva vektora, i
- dohvaćanje graničnih vrednosti indeksa.

Za razrešavanje konfliktnih situacija koristiti mehanizam izuzetaka.

Napisati na jeziku C++ program za proveru ispravnosti prethodne klase.

Rešenje:

```
// vektor2.h - Klasa vektora realnih brojeva.

#ifndef _vektor2_h_
#define _vektor2_h_

class Vekt {
    // Polja:
    int min, max;           // - granice indeksa,
    int duz;                // - broj komponentata,
    double* niz;            // - niz komponentata.
    // Pomoćne metode:
    void pravi(int poc, int kra); // - stvaranje vektora,
    void kopiraj(const Vekt& v);  // - kopiranje vektora,
    void brisi() { delete[] niz; niz=0; } // - uništavanje vektora.
public:
    enum Greska { OPSEG,      // - neispravan opseg indeksa,
                 PRAZAN,     // - vektor je prazan,
                 INDEKS,     // - indeks je izvan opsega,
                 DUZINA };   // - neusaglašene dužine vektora.
    // Konstruktori:
    Vekt() { niz = 0; }      // - prazan niz,
    explicit Vekt(int kra) { pravi(1, kra); } // - podraz. početni indeks,
    Vekt(int poc, int kra) { pravi(poc, kra); } // - sa zadatim indeksima,
    Vekt(const Vekt& v) { kopiraj(v); } // - kopija vektora.
    ~Vekt() { brisi(); }     // Destruktor.
    Vekt& operator=(const Vekt& v) { // Dodela vrednosti.
        if (this != &v) { brisi(); kopiraj(v); }
        return *this;
    }
    // Pristup komponenti:
    double& operator[](int ind) { // - promenljivog vektora,
        if (niz == 0) throw PRAZAN;
        if (ind<min || ind>max) throw INDEKS;
        return niz[ind-min];
    }
    // - nepromenljivog vektora.
    const double& operator[](int ind) const { return const_cast<Vekt&>(*this)[ind]; }
    friend double operator* // Skalarni proizvod.
        (const Vekt& v, const Vekt& w);
    int minInd() const { return min; } // Najmanji indeks.
    int maxInd() const { return max; } // Najveći indeks.
};

#endif
```

```
// vektor2.C - Metode klase vektora realnih brojeva.

#include "vektor2.h"

void Vekt::pravi(int poc, int kra) { // Formiranje vektora.
    if ((min=poc) > (max=kra)) throw OPSEG;
    niz = new double [duz=kra-poc+1];
    for (int i=0; i<duz; niz[i++]=0);
}

void Vekt::kopiraj(const Vekt& v) { // Kopiranje vektora.
    if (v.niz) {
        niz = new double [duz=(max=v.max)-(min=v.min)+1];
        for (int i=0; i<duz; i++) niz[i] = v.niz[i];
    } else niz = 0;
}

double operator*(const Vekt& v, const Vekt& w) { // Skalarni proizvod.
    if (v.niz==0 || w.niz==0) throw Vekt::PRAZAN;
    if (v.duz != w.duz) throw Vekt::DUZINA;
    double s = 0;
    for (int i=0; i<v.duz; i++) s += v.niz[i] * w.niz[i];
    return s;
}
```

```
// vektor2t.C - Ispitivanje klase vektora realnih brojeva.

#include "vektor2.h"
#include <iostream>
#include <new>
using namespace std;

int main() {
    while (true) {
        try {
            int min, max;
            cout << "Opseg indeksa prvog vektora? "; cin >> min >> max;
            if (min==0 && max==0) break;
            Vekt v1(min, max); cout << "Komponente prvog vektora? ";
            for (int i=min; i<=max; i++) cin >> v1[i];
            cout << "Opseg indeksa drugog vektora? "; cin >> min >> max;
            if (min==0 && max==0) break;
            Vekt v2(min, max); cout << "Komponente drugog vektora? ";
            for (int i=min; i<=max; i++) cin >> v2[i];
            cout << "Skalarni proizvod = " << v1 * v2 << "\n\n";
        } catch (Vekt::Greska g) {
            char* poruke[] = { "Neispravan opseg indeksa",
                              "Vektor je prazan",
                              "Indeks je izvan opsega",
                              "Neusaglasene dužine nizova"
                            };
            cout << "\n*** " << poruke[g] << "! ***\a\n\n";
        } catch (bad_alloc) {
            cout << "\n*** Dodela memorije nije uspela! ***\a\n\n";
        }
    }
    return 0;
}
```

```
% vektor2t
Opseg indeksa prvog vektora? 1 5
Komponente prvog vektora? 1 2 3 4 5
Opseg indeksa drugog vektora? -5 -1
Komponente drugog vektora? 5 4 3 2 1
Skalarni proizvod = 35

Opseg indeksa prvog vektora? 5 1

*** Neispravan opseg indeksa! ***

Opseg indeksa prvog vektora? 10 12
Komponente prvog vektora? 11 22 33
Opseg indeksa drugog vektora? -2 2
Komponente drugog vektora? 10 20 30 40 50

*** Neusaglasene duzine nizova! ***

Opseg indeksa prvog vektora? -2000000000 2000000000

*** Dodela memorije nije uspela! ***

Opseg indeksa prvog vektora? 0 0
```

Zadatak 5.2 Racionalni brojevi

Za tačno predstavljanje racionalnih brojeva koriste se dva cela broja čiji količnik predstavlja vrednost racionalnog broja. Napisati na jeziku C++ klasu racionalnih brojeva sa svim mogućnostima rada s brojačnim podacima.

Pokušaj stvaranja racionalnog broja čiji je delilac jednak nuli prijaviti izuzetkom tipa specijalne jednostavne klase.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:

```
// racbroj.h - Klasa racionalnih brojeva.

#ifndef _racbroj_h_
#define _racbroj_h_

#include <iostream>
#include <cstdlib>
using namespace std;

namespace Aritmetika {
    class GRacBroj {}; // Klasa grešaka: Imenilac jednak nuli.

    class RacBroj {
        typedef const RacBroj CRB; // Nepromenljiv racionalan broj.
        long x, y; // Brojilac i delilac.
        static long nzd(long a, long b); // Najveći zajednički delilac.
    public:
        RacBroj(long a = 0, long b = 1) { // Inicijalizacija.
            x = a; y = b;
            if (y == 0) throw GRacBroj();
            if (y < 0) { x = -x; y = -y; }
            long z = nzd(labs(x), y); x /= z; y /= z;
        }
        RacBroj operator-() const { return RacBroj(-x, y); } // -r
        RacBroj operator+() const { return *this; } // +r
        RacBroj operator!() const { return RacBroj(y, x); } // 1 / r

        friend RacBroj operator+(CRB& r, CRB& s) // r + s
        { return RacBroj(r.x*s.y+s.x*r.y, r.y*s.y); }
        friend RacBroj operator-(CRB& r, CRB& s) // r - s
        { return RacBroj(r.x*s.y-s.x*r.y, r.y*s.y); }
        friend RacBroj operator*(CRB& r, CRB& s) // r * s
        { return RacBroj(r.x*s.x, r.y*s.y); }
        friend RacBroj operator/(CRB& r, CRB& s) // r / s
        { return RacBroj(r.x*s.y, r.y*s.x); }

        RacBroj& operator+=(CRB& s) { return *this = *this + s; } // r += s
        RacBroj& operator-=(CRB& s) { return *this = *this - s; } // r -= s
        RacBroj& operator*=(CRB& s) { return *this = *this * s; } // r *= s
        RacBroj& operator/=(CRB& s) { return *this = *this / s; } // r /= s
    };
}
```

```

RacBroj& operator++() { return *this += 1; } // ++r
RacBroj& operator--() { return *this -= 1; } // --r
RacBroj operator++(int) { RacBroj a(*this); ++*this; return a; } // r++
RacBroj operator--(int) { RacBroj a(*this); --*this; return a; } // r--

friend bool operator==(CRB& r, CRB& s) // r == s
{ return r.x==s.x && r.y==s.y; }
friend bool operator!=(CRB& r, CRB& s) // r != s
{ return r.x!=s.x || r.y!=s.y; }
friend bool operator<(CRB& r, CRB& s) // r < s
{ return r.x*s.y < r.y*s.x; }
friend bool operator<=(CRB& r, CRB& s) // r <= s
{ return r.x*s.y <= r.y*s.x; }
friend bool operator>(CRB& r, CRB& s) // r > s
{ return r.x*s.y > r.y*s.x; }
friend bool operator>=(CRB& r, CRB& s) // r >= s
{ return r.x*s.y >= r.y*s.x; }

friend istream& operator>>(istream& ut, RacBroj& r); // Čitanje.
friend ostream& operator<<(ostream& it, CRB& r); // Pisanje.
};

#endif

```

// racbroj.C - Metode i funkcije uz klasu racionalnih brojeva.

```

#include "racbroj.h"
#include <cstdlib>
using namespace std;

namespace Aritmetika {
    long RacBroj::nzd(long a, long b) { // Najveći zajednički delilac.
        while (b != 0) { long c = a % b; a = b; b = c; }
        return a;
    }

    istream& operator>>(istream& ut, RacBroj& r) { // Čitanje.
        long a, b = 1; ut >> a;
        if (ut.peek() == '/') { ut.get(); ut >> b; }
        r = RacBroj(a, b);
        return ut;
    }

    ostream& operator<<(ostream& it, const RacBroj& r) { // Pisanje.
        it << r.x;
        if (r.y != 1) it << '/' << r.y;
        return it;
    }
}

```

// racbrojt.C - Ispitivanje klase racionalnih brojeva.

```

#include "racbroj.h"
using namespace Aritmetika;
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    while (true) {
        try {
            RacBroj a, b; cout << "\na,b? "; cin >> a >> b;
            cout << "a,b= " << setw(12) << a << setw(12) << b << endl;
            cout << "a+b= " << setw(12) << (a + b) << endl;
            cout << "a-b= " << setw(12) << (a - b) << endl;
            cout << "a*b= " << setw(12) << (a * b) << endl;
            cout << "a/b= " << setw(12) << (a / b) << endl;
            cout << "1/a= " << setw(12) << (1/a) << endl;
            cout << "-a = " << setw(12) << (-a) << endl;
            cout << "+a = " << setw(12) << (+a) << endl;
            cout << "a++= " << setw(12) << (a++) << endl;
            cout << "++a= " << setw(12) << (++a) << endl;
            cout << "a--= " << setw(12) << (a--) << endl;
            cout << "--a= " << setw(12) << (--a) << endl;
        } catch (GRacBroj) {
            cout << "\nGreska: Delilac jednak nuli.\n";
        }
    }
}

```

% racbrojt

```

a,b? 1/2 3/4
a,b=          1/2          3/4
a+b=          5/4
a-b=         -1/4
a*b=          3/8
a/b=          2/3
1/a=           2
-a =         -1/2
+a =          1/2
a++=          1/2
++a=          5/2
a--=          5/2
--a=          1/2

```

a,b? 3/2 1/0

Greska: Delilac jednak nuli.

a,b? 0/1 2/3

```

a,b=          0          2/3
a+b=          2/3
a-b=         -2/3
a*b=           0
a/b=           0

```

Greska: Delilac jednak nuli.

a,b? ctrl/C

Zadatak 5.3 Matrice racionalnih brojeva

Korišćenjem klase racionalnih brojeva iz prethodnog zadatka, napisati na jeziku C++ klasu matrica racionalnih brojeva. Predvideti:

- stvaranje matrice zadatih dimenzija i popunjavanje elemenata zadatom vrednošću (podrazumevano nulama),
- inicijalizaciju matrice kopijom druge matrice,
- uništavanje matrice,
- dodelu vrednosti jedne matrice drugoj,
- dohvatanje broja vrsta i kolona matrice,
- pristupanje zadatom elementu matrice,
- nalaženje transponovane i inverzne matrice,
- izračunavanje determinante matrice,
- aritmetičke operacije između matrica i između matrice i racionalnog broja,
- ispitivanje da li su dve matrice jednake ili različite,
- čitanje elemenata matrice iz ulaznog toka i upisivanje elemenata matrice u izlazni tok.

Konfliktne situacije prijavljivati izuzecima tipa specijalnih jednostavnih klasa.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:

```
// racmatr.h - Klasa matrica racionalnih brojeva.

#ifndef _racmatr_h_
#define _racmatr_h_

#include "racbroj.h" // Zadatak 5.2
#include <iostream>
using namespace std;

namespace Aritmetika {
    class GRacMatInd {}; // Klasa grešaka: Indeks izvan opsega.
    class GRacMatDim {}; // Klasa grešaka: Neusaglašene dimenzije.
    class GRacMatInv {}; // Klasa grešaka: Ne postoji inverzna matrica.
    class GRacMatKvd {}; // Klasa grešaka: Matrica nije kvadratna.

    class RacMatr {
        typedef const RacBroj CRB; // Nepromenljiv racionalan broj.
        typedef const RacMatr CRM; // Nepromenljiva matrica.
        RacBroj** niz; int m, n; // Elementi i dimenzije matrice.
        void pravi(int v, int k, CRB& r); // Dodela memorije.
        void kopiraj(CRM& a); // Kopiranje.
        void brisi(); // Oslobađanje memorije.
        RacMatr inv_det(RacBroj& det) const; // Inverzna matrica i determ.
    public:
        RacMatr(int v, int k, RacBroj r=0) // - pravougaona matr.,
            { pravi(v, k, r); }
        explicit RacMatr(int v=5, RacBroj r=0) // - kvadratna matrica,
            { pravi(v, v, r); }
        RacMatr(CRM& a) { kopiraj(a); } // - kopija matrice.
        ~RacMatr() { brisi(); } // Uništavanje.
        RacMatr& operator=(CRM& b) { // A = B
            if (this != &b) { brisi(); kopiraj(b); }
            return *this;
        }
    };
}
```

```
int vrs() const { return m; } // Broj vrsta.
int kol() const { return n; } // Broj kolona.
const RacBroj*const& operator[](int i) const { // A[i] (pok. na vrstu):
    if (i<0 || i>=m) throw GRacMatInd(); // - nepromenljiva matr,
    return niz[i];
}
RacBroj*const& operator[](int i) // - promenljiva matr.
{ return (RacBroj*const&)((const RacMatr&)(*this))[i]; }

RacMatr operator+(const RacMatr& a, const RacMatr& b) const { return *this + b; } // +A
RacMatr operator-(const RacMatr& a, const RacMatr& b) const { return *this - b; } // -A
friend RacMatr T (CRM& a); // Transponovana matrica.
friend RacMatr inv(CRM& a) const { return *this * a.inv_det(det); } // Inverzna matrica.
friend RacBroj det(CRM& a) const { return a.inv_det(det); } // Determinanta.
friend RacBroj det(CRM& a) const { return a.inv_det(det); return det; }

RacMatr& operator+=(CRM& b); // A += B
RacMatr& operator-=(CRM& b); // A -= B
RacMatr operator+(CRM& b) const { return RacMatr(*this)+=b; } // A + B
RacMatr operator-(CRM& b) const { return RacMatr(*this)-=b; } // A - B

RacMatr operator*(CRM& b) const; // A * B
RacMatr operator/(CRM& b) const { return *this * inv(b); } // A / B
RacMatr& operator*=(CRM& b) { return *this = *this * b; } // A *= B
RacMatr& operator/=(CRM& b) { return *this = *this / b; } // A /= B

RacMatr& operator+=(CRB& r); // A += r
RacMatr& operator-=(CRB& r) { return *this += -r; } // A -= r
RacMatr& operator*=(CRB& r); // A *= r
RacMatr& operator/=(CRB& r) { return *this *= !r; } // A /= r

friend RacMatr operator+(CRB& r, CRM& a) const { return RacMatr(a) += r; } // r + A
friend RacMatr operator-(CRB& r, CRM& a) const { return RacMatr(a) -= r; } // r - A
friend RacMatr operator*(CRB& r, CRM& a) const { return RacMatr(a) * r; } // r * A
friend RacMatr operator/(CRB& r, CRM& a) const { return RacMatr(a) / r; } // r / A

RacMatr operator+(CRB& r) const { return r + *this; } // A + r
RacMatr operator-(CRB& r) const { return -r + *this; } // A - r
RacMatr operator*(CRB& r) const { return r * *this; } // A * r
RacMatr operator/(CRB& r) const { return !r * *this; } // A / r

bool operator==(CRM& b) const; // A == B
bool operator!=(CRM& b) const { return !(*this == b); } // A != B

friend istream& operator>>(istream& ut, RacMatr& a); // Čitanje.
friend ostream& operator<<(ostream& it, CRM& a); // Pisanje.
};

#endif
```

// racmatr.C - Metode i funkcije uz klasu matrica racionalnih brojeva.

#include "racmatr.h"

#include <iomanip>

using namespace std;

namespace Aritmetika {

void RacMatr::pravi(int v, int k, CRB& r) { // Dodela memorije.

m = v; n = k;

niz = new RacBroj* [m];

for (int i=0; i<m; i++) {

niz[i] = new RacBroj [n];

for (int j=0; j<n; niz[i][j++]=r);

}

}

void RacMatr::kopiraj(CRB& a) { // Kopiranje.

m = a.m; n = a.n;

niz = new RacBroj* [m];

for (int i=0; i<m; i++) {

niz[i] = new RacBroj [n];

for (int j=0; j<n; j++) niz[i][j] = a.niz[i][j];

}

}

void RacMatr::brisi() { // Oslobađanje memorije.

for (int i=0; i<m; delete [] niz[i++]);

delete [] niz; niz = 0; m = n = 0;

}

RacMatr RacMatr::operator-() const { // -A

RacMatr a(*this);

for (int i=0; i<m; i++)

for (int j=0; j<n; j++)

a.niz[i][j] = -a.niz[i][j];

return a;

}

RacMatr T(const RacMatr& a) { // Transponovana matrica.

RacMatr b(a.n, a.m);

for (int i=0; i<a.m; i++)

for (int j=0; j<a.n; j++) b.niz[j][i] = a.niz[i][j];

return b;

}

1. Početna matrica

2	4	3
1	5	2
3	4	1

2. Proširena matrica:

2	4	3	1	0	0
1	5	2	0	1	0
3	4	1	0	0	1

3. Normiranje prve vrste:

1	2	3/2	1/2	0	0
1	5	2	0	1	0
3	4	1	0	0	1

4. Stvaranje nula u prvoj koloni nadole:

1	2	3/2	1/2	0	0
0	3	1/2	-1/2	1	0
0	-2	-7/2	-3/2	0	1

5. Normiranje druge vrste:

1	2	3/2	1/2	0	0
0	1	1/6	-1/6	1/3	0
0	-2	-7/2	-3/2	0	1

6. Stvaranje nula u drugoj koloni nadole:

1	2	3/2	1/2	0	0
0	1	1/6	-1/6	1/3	0
0	0	-19/6	-11/6	2/3	1

7. Normiranje treće vrste:

1	2	3/2	1/2	0	0
0	1	1/6	-1/6	1/3	0
0	0	1	11/19	-4/19	-6/19

8. Stvaranje nula u trećoj koloni nagore:

1	2	0	-7/19	6/19	9/19
0	1	0	-5/19	7/19	1/19
0	0	1	11/19	-4/19	-6/19

9. Stvaranje nula u drugoj koloni nagore:

1	0	0	3/19	-8/19	7/19
0	1	0	-5/19	7/19	1/19
0	0	1	11/19	-4/19	-6/19

10. Rezultantna matrica:

3/19	-8/19	7/19			
-5/19	7/19	1/19			
11/19	-4/19	-6/19			

Det = 2.3 $\frac{-19}{6}$

RacMatr RacMatr::inv det(RacBroj& det) const { // Inverzna matrica i
if (m != n) throw GRacMatKvd(); // determinata.

RacMatr b(m, 2*m); RacBroj d(1);

// Stvaranje proširene matrice:

for (int i=0; i<m; i++)

for (int j=0; j<m; j++)

{ b.niz[i][j] = niz[i][j]; b.niz[i][j+m] = i==j; }

// Eliminacija unapred:

for (int il=0; il<m; il++) {

// obezbeđivanje nenulte vrednosti na dijagonali:

if (b.niz[il][il] == 0) {

int j=il+1; while (j<n && b.niz[j][il]==0) j++;

if (j == n) throw GRacMatInv();

RacBroj* p = b.niz[il]; b.niz[il] = b.niz[j]; b.niz[j] = p;

}

// normiranje vrste:

if (b.niz[il][il] != 1) {

d *= b.niz[il][il];

for (int j=2*m-1; j>=il; j--) b.niz[il][j] /= b.niz[il][il];

}

// stvaranje nula u koloni il nadole:

for (int i2=il+1; i2<m; i2++)

if (b.niz[i2][il] != 0)

for (int j=2*m-1; j>=il; j--)

b.niz[i2][j] -= b.niz[il][j] * b.niz[i2][il];

}

// Zamenjivanje unazad (stvaranje nula u kolonama nagore):

for (int il=m-1; il>0; il--)

for (int i2=il-1; i2>=0; i2--)

if (b.niz[i2][il] != 0)

for (int j=2*m-1; j>=il; j--)

b.niz[i2][j] -= b.niz[il][j] * b.niz[i2][il];

}

// Vraćanje rezultata:

RacMatr a(m, m);

for (int i=0; i<m; i++)

for (int j=0; j<m; j++) a.niz[i][j] = b.niz[i][j+m];

det = d;

return a;

}

```

RacMatr& RacMatr::operator+=(CRM& b) { // A += B
    if (m!=b.m || n!=b.n) throw GRacMatDim();
    for (int i=0; i<m; i++)
        for (int j=0; j<n; j++) niz[i][j] += b.niz[i][j];
    return *this;
}

RacMatr& RacMatr::operator-=(CRM& b) { // A -= B
    if (m!=b.m || n!=b.n) throw GRacMatDim();
    for (int i=0; i<m; i++)
        for (int j=0; j<n; j++) niz[i][j] -= b.niz[i][j];
    return *this;
}

RacMatr RacMatr::operator*(CRM& b) const { // A * B
    if (n != b.m) throw GRacMatDim();
    RacMatr c(m, b.n);
    for (int i=0; i<m; i++)
        for (int k=0; k<b.n; k++)
            for (int j=0; j<n; j++)
                c.niz[i][k] += niz[i][j] * b.niz[j][k];
    return c;
}

RacMatr& RacMatr::operator+=(CRB& r) { // A += r
    for (int i=0; i<m; i++)
        for (int j=0; j<n; j++) niz[i][j] += r;
    return *this;
}

RacMatr& RacMatr::operator*=(CRB& r) { // A *= r
    for (int i=0; i<m; i++)
        for (int j=0; j<n; j++) niz[i][j] *= r;
    return *this;
}

bool RacMatr::operator==(CRM& b) const { // A == B
    if (m!=b.m || n!=b.n) return false;
    for (int i=0; i<m; i++)
        for (int j=0; j<n; j++)
            if (niz[i][j] != b.niz[i][j]) return false;
    return true;
}

istream& operator>>(istream& ut, RacMatr& a) { // Čitanje.
    for (int i=0; i<a.m; i++)
        for (int j=0; j<a.n; j++) ut>>a.niz[i][j++];
    return ut;
}

ostream& operator<<(ostream& it, const RacMatr& a) { // Pisanje.
    int sir = it.width(1);
    for (int i=0; i<a.m; i++) {
        it << '[';
        for (int j=0; j<a.n; j++) it << setw(sir) << a.niz[i][j++];
        it << ']' << endl;
    }
    return it;
}

```

// racmatrt.C - Ispitivanje klase matrica racionalnih brojeva.

```

#include "racmatr.h"
using namespace Aritmetika;
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    while (true) {
        try {
            cout << "nm,n? "; int m, n; cin >> m >> n;
            if (m<=0 || n<=0) break;
            RacMatr a(m, n); cout << "Matrica? "; cin >> a;
            cout << "nMatrica:\n" << setw(7) << a;
            cout << "nInverzna:\n" << setw(7) << inv(a);
            cout << "nA * inv(A):\n" << setw(7) << a * inv(a);
            cout << "nDterminanta: " << det(a) << endl;
            cout << "nTransponovana:\n" << setw(7) << T(a);
            cout << "nA + T(A):\n" << setw(7) << a + T(a);
        } catch (GRacMatInd) { cout << "nGreska: Indeks izvan opsega.\n";
        } catch (GRacMatDim) { cout << "nGreska: Neusaglasene dimenzije.\n";
        } catch (GRacMatInv) {
            cout << "nGreska: Ne postoji inverzna matrica.\n";
        } catch (GRacMatKvd) { cout << "nGreska: Matrica nije kvadratna.\n";
        } catch (GRacBroj) { cout << "nGreska: Imenilac jednak nuli.\n";
        }
    }
    return 0;
}

```

% racmatrt

```

m,n ? 3 3
Matrica? 2 4 3
          1 5 2
          3 4 1

Matrica:
[ 2 4 3]
[ 1 5 2]
[ 3 4 1]

Inverzna:
[ 3/19 -8/19 7/19]
[ -5/19 7/19 1/19]
[ 11/19 -4/19 -6/19]

A * inv(A):
[ 1 0 0]
[ 0 1 0]
[ 0 0 1]

Dterminanta: -19

Transponovana:
[ 2 1 3]
[ 4 5 4]
[ 3 2 1]

```

```

A + T(A):
[ 4 5 6]
[ 5 10 6]
[ 6 6 2]

```

```

m,n? 3 3
Matrica? 1 2 3
          2 3 4
          3 4 5

```

```

Matrica:
[ 1 2 3]
[ 2 3 4]
[ 3 4 5]

```

Greska: Ne postoji inverzna matrica.

```

m,n? 3 2
Matrica? 1 2 3
          2 3 4

```

```

Matrica:
[ 1 2]
[ 3 2]
[ 3 4]

```

Greska: Matrica nije kvadratna.

```

m,n? 0 0

```

Zadatak 5.4 Nizovi, funkcije i verižni razlomci

Napisati na jeziku C++ sledeće klase:

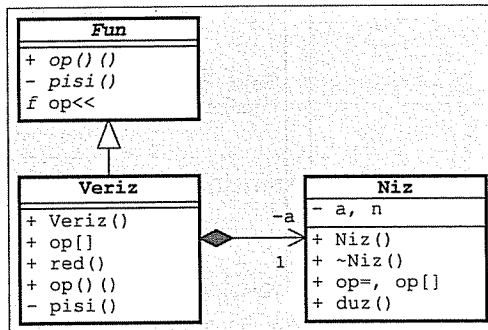
- **Niz** realnih brojeva može da se inicijalizuje zadatom dužinom (podrazumevano 10 elemenata, vrednosti elemenata su proizvoljne) i drugim nizom, da se uništi, da se dodeljuje vrednost jednog niza drugom (`niz1=niz2`), da se pristupi elementu sa zadatim indeksom (`niz[ind]`) i da se dohvati dužina niza. Nedoizvoljen indeks se prijavljuje izuzetkom celobrojnog tipa s vrednošću jednakom tom nedozvoljenom indeksu.

- **Funkcija** je apstraktna klasa u kojoj je predviđeno izračunavanje vrednosti realne funkcije s jednim realnim argumentom (`fun(x)`) i upisivanje simboličkog oblika te funkcije (na primer formule) u izlazni tok (`it<<fun`).

- **Verižni** razlomak je *funkcija* koja sadrži niz realnih koeficijenata i može da izračuna vrednost priložene funkcije $v(x)$. Može da se inicijalizuje zadatim redom n tako da predstavlja funkciju $v_0(x)$, da se pristupi zadatom koeficijentu (`ver[ind]`) i da se dohvati red razlomka. Verižan razlomak se piše u obliku niza njegovih koeficijenata, na primer: **Veriz** [$a_0, a_1, \dots, a_{n-1}$]. Pokušaj deljenja nulom prijavljuje se izuzetkom celobrojnog tipa s vrednošću nula.

Napisati na jeziku C++ **program** koji pročita red verižnog razlomka n s glavnog ulaza, stvori jedan verižni razlomak s podrazumevanim koeficijentima, promeni vrednost nekih od koeficijenata čitajući s glavnog ulaza parove $i - a_i$ sve dok ne pročita nedozvoljen indeks, pročita vrednosti $x_{\min}$, $x_{\max}$ i Δx , i na kraju tabelira vrednosti verižnog razlomka na glavnom izlazu za svako $x_{\min} \leq x \leq x_{\max}$ s korakom Δx .

Rešenje:



// niz1.h - Klasa nizova realnih brojeva.

```

#ifndef _niz1_h_
#define _niz1_h_

class Niz {
public:
    double* a; int n; // Niz i dužina niza.
    void kopiraj(const Niz& niz); // Kopiranje niza.

    explicit Niz(int k=10) { a = new double [n = k]; } // Stvaranje niza.
    Niz(const Niz& niz) { kopiraj(niz); } // Inicijalizacija nizom.
    ~Niz() { delete [] a; } // Uništavanje.
    Niz& operator=(const Niz& niz) { // Dodela vrednosti.
        if(this != &niz) { delete [] a; kopiraj(niz); }
        return *this;
    }
    double& operator[](int i) { // Pristup elementu
        if (i<0 || i>=n) throw i; // - promenljivog niza,
        return a[i];
    }
    const double& operator[](int i) const { // - nepromenljivog niza.
        if (i<0 || i>=n) throw i;
        return a[i];
    }
    int duz() const { return n; } // Dužina niza.
};

#endif
  
```

// niz1.C - Metode klase nizova realnih brojeva.

```

#include "niz1.h"

void Niz::kopiraj(const Niz& niz) { // Kopiranje niza.
    a = new double [n = niz.n];
    for (int i=0; i<n; i++) a[i] = niz.a[i];
}
  
```

// fun1.h - Apstraktna klasa funkcija.

```

#ifndef _fun1_h_
#define _fun1_h_

#include <iostream>
using namespace std;

class Fun {
public:
    virtual ~Fun() {} // Virtuelan destruktork.
    virtual double operator()(double x) const =0; // Vrednost funkcije.
private:
    virtual void pisi(ostream& it) const =0; // Pisanje.
    friend ostream& operator<<(ostream& it, const Fun& f)
    { f.pisi(it); return it; }
};

#endif
  
```

```
// veriz.h - Klasa verižnih razlomaka.
```

```
#ifndef _veriz_h_
#define _veriz_h_
```

```
#include "fun1.h"
#include "niz1.h"
```

```
class Veriz: public Fun {
    Niz a; // Koeficijenti razlomka.
    void pisi(ostream& it) const; // Pisanje.
public:
    explicit Veriz(int n); // Stvaranje razlomka.
    double& operator[](int i) { return a[i]; } // Dohvatanje koeficijenta.
    const double& operator[](int i) const { return a[i]; }
    int red() const { return a.duz(); } // Red razlomka.
    double operator()(double x) const; // Vrednost funkcije.
};

#endif
```

```
// veriz.C - Metode klase verižnih razlomaka.
```

```
#include "veriz.h"
```

```
Veriz::Veriz(int n): a(n) // Stvaranje razlomka.
{ for (int i=0; i<n; i++) a[i] = i+1; }
```

```
void Veriz::pisi(ostream& it) const { // Pisanje.
    it << "Veriz[";
    for (int i=0; i<a.duz(); i++) { if (i) it << ','; it << a[i]; }
    it << ']';
}
```

```
double Veriz::operator()(double x) const { // Vrednost funkcije.
    double s = 0;
    for (int i=a.duz()-1; i>=0; i--) {
        if (x + s == 0) throw 0;
        s = a[i] / (x + s);
    }
    return s;
}
```

```
// verizt.C - Ispitivanje klase verižnih razlomaka.
```

```
#include "veriz.h"
#include <iostream>
using namespace std;
```

```
int main() {
    cout << "n? "; int n; cin >> n; Veriz v(n);
    try {
        while (true) { cout << "i, a[i]? "; int i; cin >> i; cin >> v[i]; }
    } catch (int) {}
    cout << endl << v << endl << endl;
    cout << "xmin, xmax, dx? ";
    double xmin, xmax, dx; cin >> xmin >> xmax >> dx; cout << endl;
    for (double x=xmin; x<=xmax; x+=dx) {
        cout << x << '\t';
        try { cout << v(x) << endl; } catch (int) { cout << "Ne moze!\n"; }
    }
    return 0;
}
```

```
% verizt
n? 5
```

```
i, a[i]? 0 5
i, a[i]? 1 4
i, a[i]? 2 3
i, a[i]? 3 2
i, a[i]? 4 1
i, a[i]? 5
```

```
Veriz[5,4,3,2,1]
```

```
xmin, xmax, dx? -1 1 0.25
```

```
-1      -1.92308
-0.75   -2.1302
-0.5    -2.59786
-0.25   -4.32392
0        Ne moze!
0.25     4.32392
0.5      2.59786
0.75     2.1302
1        1.92308
```

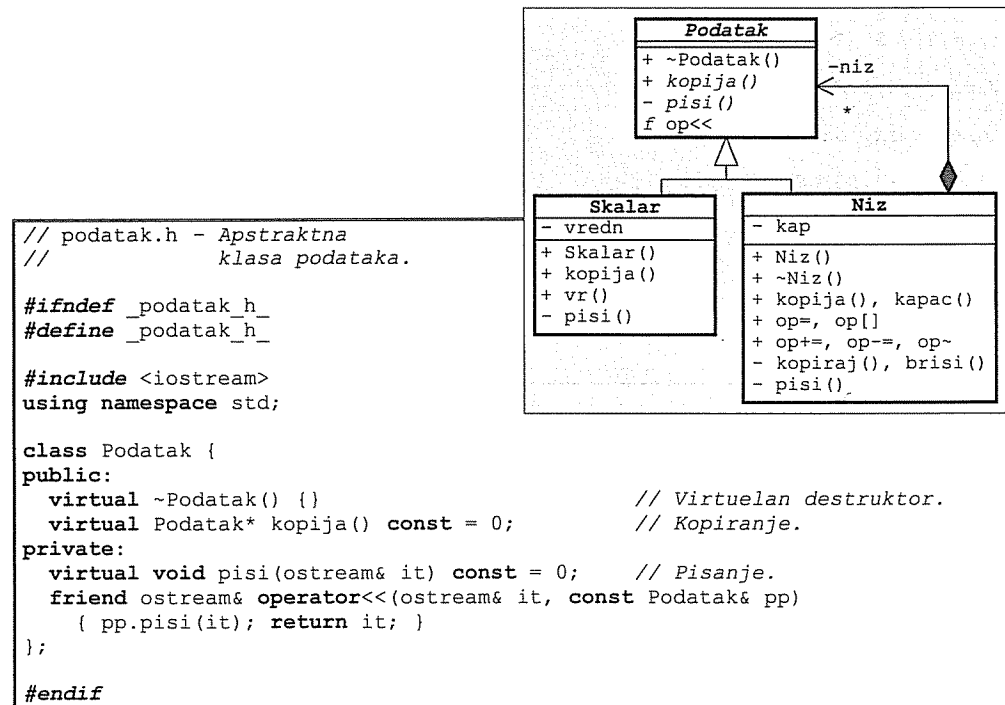

Zadatak 5.5 Podaci, skalarni podaci i nizovi

Napisati na jeziku C++ sledeće klase:

- Apstraktan *podatak* može da se upiše u izlazni tok (`it<<pod`), da mu se stvara kopija i da se uništi.
- *Skalar* je podatak koji ima realnu vrednost, može da se inicijalizuje običnom realnom vrednošću i da se dohvati njegova vrednost (`+skal`).
- *Niz* je podatak koji može da sadrži izvestan broj raznovrsnih podataka (uključujući i nizove), može da se inicijalizuje kao prazan niz zadatog kapaciteta ili da se inicijalizuje drugim nizom. Pored toga, postoji mogućnost dodele vrednosti jednog niza drugom (`niz1=niz2`), dohvaćanje kapaciteta niza (`+niz`), dohvaćanje vrednosti nekog elementa niza (`niz[ind]`; greška je ako je indeks izvan dozvoljenog opsega ili ako je dato mesto prazno), stavljanje podatka na prvo slobodno mesto u nizu (`niz+=pod`; greška je ako u nizu nema slobodnog mesta), izbacivanje podatka s datog mesta u nizu (`niz-=ind`; greška je ako je indeks izvan dozvoljenog opsega) i pražnjenje sadržaja niza (`~niz`). Greške prijavljivati *izuzecima*.

Napisati na jeziku C++ *program* koji pročita kapacitet niza s glavnog ulaza, stvori prazan niz tog kapaciteta, napuni niz podacima koje čita s glavnog ulaza (skalarnim podacima ili nizovima koji sadrže samo skalarnu podatke), ispiše sadržaj dobijenog složenog niza na glavnom izlazu i ponavlja prethodne korake sve dok za dužinu niza ne pročita negativnu vrednost.

Rešenje:



// skalar.h - Klasa skalarnih podataka.

```

#ifndef _skalar_h_
#define _skalar_h_

#include "podatak.h"

class Skalar: public Podatak {
    double vredn; // Vrednost podatka.
public:
    Skalar(double v=0) { vredn = v; } // Konstruktor.
    Skalar* kopija() const { return new Skalar(*this); } // Kopiranje.
    double vr() const { return vredn; } // Vrednost.
private:
    void pisi(ostream& it) const { it << vredn; } // Pisanje.
};

#endif
  
```

// niz2gr.h - Klase grešaka.

```

#ifndef _niz2gr_h_
#define _niz2gr_h_

#include <iostream>
using namespace std;

class GPun {}; // GREŠKA: Niz je pun.

class GPrazno { // GREŠKA: Mesto u nizu je prazno.
public:
    friend ostream& operator<<(ostream& it, const GPrazno&)
    { return it << "*** GRESKA: Pristup praznom mestu u nizu! ***\n"; }
};

class GIndeks { // GREŠKA: Indeks je izvan opsega.
    int ind, duz; // Pogrešan indeks i dužina niza.
public:
    GIndeks(int i, int d) { ind = i; duz = d; }
    int dohvInd() const { return ind; } // Dohvaćanje pogrešnog indeksa.
    int dohvDuz() const { return duz; } // Dohvaćanje dužine niza.
    friend ostream& operator<<(ostream& it, const GIndeks& g) {
        return it << "*** GRESKA: Nedoovoljen indeks " << g.ind
            << " u nizu dužine " << g.duz << "! ***\n";
    }
};

#endif
  
```

```
// niz2.h - Klasa nizova apstraktnih podataka.
```

```
#ifndef _niz2_h_
#define _niz2_h_
```

```
#include "podatak.h"
#include "niz2gr.h"
```

```
class Niz: public Podatak { // Polja:
    Podatak** niz; // - niz pokazivača na podatke,
    int kap; // - kapacitet niza.

    // Pomoćne metode:
    void kopiraj(const Niz& n); // - kopiranje niza,
    void brisi() { ~*this; delete [] niz; } // - uništavanje niza,
    void pisi(ostream& it) const; // - pisanje niza.
public: // Konstruktori:
    explicit Niz(int kk=10); // - praznog niza,
    Niz(const Niz& nn) { kopiraj(nn); } // - kopije.
    ~Niz() { brisi(); } // Destruktor.
    Niz* kopija() const // Kopiranje.
    { return new Niz(*this); }
    Niz& operator=(const Niz& n) { // Dodela vrednosti.
        if (this != &n) { brisi(); kopiraj(n); }
        return *this;
    }
    int kapac() const { return kap; } // Kapacitet niza.
    // Pristup komponenti:
    Podatak& operator[](int ind) { // - promenljivog niza,
        if (ind < 0 || ind >= kap) throw GIndeks(ind, kap);
        if (niz[ind] == 0) throw GPrazno();
        return *niz[ind];
    }
    const Podatak& operator[](int ind) const // - nepromenljivog niza.
    { return const_cast<Niz&>(*this)[ind]; }
    Niz& operator+=(const Podatak& p); // Umetanje podatka.
    Niz& operator-=(int ind) { // Izbacivanje podatka.
        if (ind < 0 || ind >= kap) throw GIndeks(ind, kap);
        delete niz[ind]; niz[ind] = 0;
        return *this;
    }
    Niz& operator~(); // Pražnjenje niza.
};

#endif
```

```
// niz2.C - Metode klase nizova apstraktnih podataka.
```

```
#include "niz2.h"
```

```
Niz::Niz(int k) { // Stvaranje praznog niza.
    niz = new Podatak* [kap = k];
    for (int i=0; i<kap; niz[i++]=0);
}

void Niz::kopiraj(const Niz& n) { // Inicijalizacija nizom.
    niz = new Podatak* [kap = n.kap];
    for (int i=0; i<kap; i++)
        niz[i] = n.niz[i] ? n.niz[i]->kopija() : 0;
}

void Niz::pisi(ostream& it) const { // Pisanje.
    it << '[';
    for (int i=0; i<kap; i++) {
        if (niz[i]) it << *niz[i];
        if (i < kap-1) it << ',';
    }
    it << ']';
}

Niz& Niz::operator+=(const Podatak& p) { // Umetanje podatka.
    int i = 0; while (i<kap && niz[i]) i++;
    if (i == kap) throw GPun();
    niz[i] = p.kopija();
    return *this;
}

Niz& Niz::operator~() { // Pražnjenje niza.
    for (int i=0; i<kap; i++) { delete niz[i]; niz[i] = 0; }
    return *this;
}
```

```
// podatakt.C - Ispitivanje klasa za obradu podataka.
```

```
#include "skalar.h"
#include "niz2.h"
#include <iostream>
using namespace std;

int main() {
    while (true) {
        try {
            cout << "\nBroj podataka? "; int n; cin >> n;
            if (n < 0) break;
            Niz a(n);
            for (int i=0; i<n; i++) {
                cout << "Vrsta podatka (Skalar, Niz)? ";
                char vrs; cin >> vrs;
                switch (vrs) {
                    case 's': case 'S': {
                        cout << "Vrednost? "; double s; cin >> s;
                        a += Skalar(s); break;
                    }
                    case 'n': case 'N': {
                        cout << "Kapacitet? ";
                        int n; cin >> n; Niz b(n);
                        cout << "Vrednosti? ";
                        do {
                            double s; cin >> s; b += Skalar(s);
                        } while (cin.get() != '\n'); // do kraja reda
                        a += b; break;
                    }
                }
            }
            cout << "Niz podataka: " << a << endl;
        } catch (GPun) { cout << "\n*** GRESKA: Niz je pun! ***\n"; }
        catch (GPrazno g) { cout << endl << g; }
        catch (GIndeks g) { cout << endl << g; }
    }
    return 0;
}
```

```
% podatakt
```

```
Broj podataka? 5
Vrsta podatka (Skalar, Niz)? s
Vrednost? 9
Vrsta podatka (Skalar, Niz)? n
Kapacitet? 3
Vrednosti? 1 2 3
Vrsta podatka (Skalar, Niz)? n
Kapacitet? 5
Vrednosti? 1 1 1 1 1
Vrsta podatka (Skalar, Niz)? s
Vrednost? 5
Vrsta podatka (Skalar, Niz)? s
```

```
Vrednost? 0
Niz podataka:
[9,[1,2,3],[1,1,1,1],5,0]
```

```
Broj podataka? 10
Vrsta podatka (Skalar, Niz)? n
Kapacitet? 5
Vrednosti? 1 2 3 4 5 6
```

```
*** GRESKA: Niz je pun! ***
```

```
Broj podataka? -1
```

Zadatak 5.6 Funkcije i greške; izračunavanje određenog integrala

Napisati na jeziku C++ sledeće klase:

- Apstraktnoj **funkciji** s jednim argumentom, za datu vrednost realne nezavisne promenljive x , može da se izračuna vrednost ($\text{fun}(x)$) i vrednost neodređenog integrala (integraciona konstanta se uzima da je 0). Ako ne postoji algebarski izraz za neodređeni integral prijavljuje se izuzetak.
- $\sin x$, $e^{-0.1x} \sin x$, $\log_e x$, i polinom $p(x)$ su funkcije.
- **Greška** može da sadrži tekst poruke koja se upisuje u izlazni tok izrazom oblika `it<<gr`. Koristi se za prijavljivanje izuzetaka. Za posebne potrebe mogu da se definišu klase izvedene iz klase **greška**.

Napisati na jeziku C++ **funkciju** za izračunavanje određenog integrala zadate funkcije na datom intervalu tačno, na osnovu neodređenog integrala, ako postoji algebarski izraz za to, odnosno po približnoj formuli ukoliko ne postoji takav izraz.

Napisati na jeziku C++ **program** za ispitivanje prethodnih klasa i funkcije.

Rešenje:

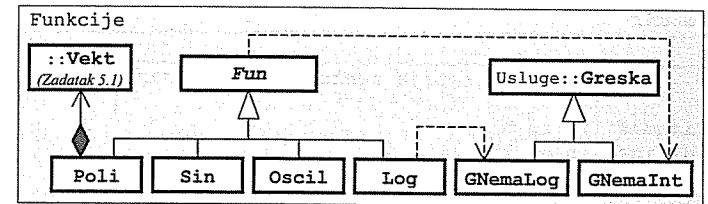
```
// greska.h - Klasa greška.
```

```
#ifndef _greska_h_
#define _greska_h_

#include <iostream>
#include <cstring>
using namespace std;

namespace Usluge {
    class Greska {
        char* poruka;
        void kopiraj(const char* p) { // Tekst poruke.
            if (p) { poruka = new char [strlen(p)+1]; // Kopiranje u objekat.
                strcpy(poruka, p);
            } else poruka = 0;
        }
        void brisi() { delete [] poruka; poruka = 0; } // Pražnjenje objekta.
    public:
        Greska() { poruka = 0; } // Konstruktor:
        Greska(const char* por) { kopiraj(por); } // - podrazumevani,
        Greska(const Greska& g) { kopiraj(g.poruka); } // - konverzije,
        virtual ~Greska() { brisi(); } // - kopije.
        Greska& operator=(const Greska& g) { // Destruktor.
            if (this != &g) { brisi(); kopiraj(g.poruka); } // Dodela vrednosti.
            return *this;
        }
    protected:
        virtual void pisi(ostream& it) const { // Pisanje poruke.
            it << "\n*** " << (poruka ? poruka : "Greska") << " ***\n";
        }
        friend ostream& operator<<(ostream& it, const Greska& g)
        { g.pisi(it); return it; }
        bool imaPoruke() const { return poruka != 0; } // Ima li poruke?
    }; // class Greska
} // namespace Usluge

#endif
```



// fun2.h - Apstraktna klasa funkcija.

```
#ifndef _fun_h_
#define _fun_h_

#include "greska.h"
using Usluge::Greska;

namespace Funkcije {
    class GNemaInt: public Greska {                // KLASA GREŠAKA:
    public:
        GNemaInt(): Greska("Funkcija nema integral") {} // Konstruktor.
    }; // class GNemaInt

    class Fun {                                    // KLASA FUNKCIJA:
    public:
        virtual ~Fun() {}                        // Virtuelan destr.
        virtual double operator()(double x) const =0; // Računanje funkcije.
        virtual double I(double x) const          // Računanje integrala.
        { throw GNemaInt(); return 0; }
    }; // class Fun
} // namespace Funkcije

#endif
```

// integ.h - Deklaracija funkcije za računanje određenog integrala.

```
#ifndef _integ_h_
#define _integ_h_

#include "fun2.h"

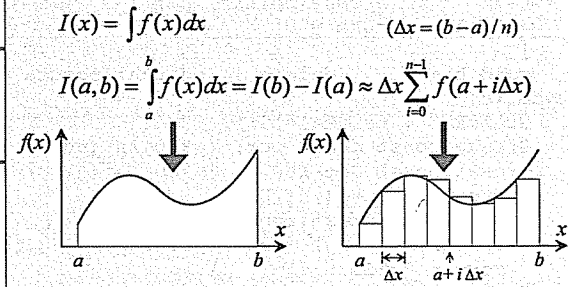
namespace Funkcije {
    double integral(const Fun& f, double a, double b);
} // namespace Funkcije

#endif
```

// integ.C - Definicija
// funkcije za računanje
// određenog integrala.

```
#include "integ.h"

double Funkcije::integral(const Fun& f, double a, double b) {
    const int MAX_INTERVAL = 30;
    try {
        return f.I(b) - f.I(a);                // Pokušaj tačnog računanja.
    } catch (GNemaInt) {
        double dx = (b - a) / MAX_INTERVAL, s = 0; // Približno računanje.
        for (int i=0; i<MAX_INTERVAL; i++) s += f(a+i*dx);
        return s * dx;
    }
}
```



// sin.h - Klasa sinusioda.

```
#ifndef _sin_h_
#define _sin_h_

#include "fun2.h"
#include <cmath>
using namespace std;

namespace Funkcije {
    class Sin: public Fun {
    public:
        double operator()(double x) const { return sin(x); } // Funkcija.
        double I(double x) const { return -cos(x); } // Integral.
    }; // class Sin
} // namespace Funkcije

#endif
```

$$\int \sin x dx = -\cos x$$

// oscil.h - Klasa prigušenih oscilacija.

```
#ifndef _oscil_h_
#define _oscil_h_

#include "fun2.h"
#include <cmath>
using namespace std;

namespace Funkcije {
    class Oscil: public Fun {
    public:
        // Računanje funkcije.
        double operator()(double x) const { return exp(-0.1*x)*sin(x); }
    }; // class Oscil
} // namespace Funkcije

#endif
```

$$\int e^{-0.1x} \sin x dx = ???$$

// log.h - Klasa logaritama.

```
#ifndef _log_h_
#define _log_h_

#include "fun2.h"
#include <cmath>
using namespace std;
#include "greska.h"
using Usluge::Greska;

namespace Funkcije {
    class GNemaLog: public Greska {                // KLASA GREŠAKA:
    public:
        double x;                                  // Nedozvoljena vrednost.
        GNemaLog(): Greska("Logaritam negativnog broja") {} // Konstruktori.
        GNemaLog(double x): Greska() { this->x = x; }
        double dohvX() const                        // Uzimanje nedozvoljene
        { return imaPoruke() ? 1 : x; }              // vrednosti.
    };
}
```

$$\int \log x dx = ???$$

```
private:
    void pisi(ostream& it) const {          // Pisanje poruke.
        if (imaPoruke()) Greska::pisi(it);
        else it << "\n*** Ne postoji logaritam od " << x << " ***\n\n";
    }
}; // class GNemaLog

class Log: public Fun {                    // KLASA LOGARITAMA:
public:
    double operator()(double x) const {    // Računanje funkcije.
        if (x <= 0) throw GNemaLog(x);
        return log(x);
    }
}; // class Log
} // namespace Funkcije

#endif
```

```
// poli2.h - Klasa polinoma.
```

```
#ifndef _poli2_h_
#define _poli2_h_

#include "fun2.h"
#include "vektor2.h" ➡ Zadatak 5.1

namespace Funkcije {
    class Poli: public Fun {
    public:
        Vekt a; // Vektor koeficijenata.

        Poli(): a(0, 5) {} // Konstruktori.
        explicit Poli(int n): a(0, n) {}
        double& operator[](int i) { return a[i]; } // Dohvatanje koeficijenta.
        double operator[](int i) const { return a[i]; }
        int red() const { return a.maxInd(); } // Red polinoma.
        double operator()(double x) const; // Vrednost polinoma.
        double I(double x) const; // Vrednost integrala.
    }; // class Poli
} // namespace Funkcije

#endif
```

```
// poli2.C - Metode klase polinoma.
```

```
#include "poli2.h"

double Funkcije::Poli::operator()(double x) const { // Vrednost polinoma.
    double s = 0;
    for (int i=a.maxInd(); i>=0; (s*=x)+=a[i--]);
    return s;
}

double Funkcije::Poli::I(double x) const { // Vrednost integrala.
    double s = 0;
    for (int i=a.maxInd(); i>=0; i--) (s*=x)+=a[i]/(i+1);
    return s * x;
}
```

$$p(x) = \sum_{i=0}^n a_i x^i$$

$$\int p(x) dx = \sum_{i=0}^n a_i \frac{x^{i+1}}{i+1} = x \sum_{i=0}^n \frac{a_i}{i+1} x^i$$

```
// integ.C - Ispitivanje računanja integrala.
```

```
#include "fun2.h"
#include "sin.h"
#include "oscil.h"
#include "poli2.h"
#include "log.h"
#include "integ.h"
using namespace Funkcije;
#include "greska.h"
#include "vektor2.h"
#include <iostream>
#include <new>
using namespace std;

int main() {
    while (true) {
        Fun* fun = 0;
        try {
            cout << "\nFunkcija (S, O, L, P, .)? ";
            char izb; cin >> izb;
            switch (izb) {
                case 's': case 'S': fun = new Sin; break;
                case 'o': case 'O': fun = new Oscil; break;
                case 'l': case 'L': fun = new Log; break;
                case 'p': case 'P': {
                    int n; cout << "Red polinoma? "; cin >> n;
                    Poli* p = new Poli(n);
                    while (true) {
                        cout << "Indeks i vrednost nenultog koeficijenta? ";
                        int i; double k; cin >> i >> k;
                        if (i<0) break;
                        (*p)[i] = k;
                    }
                    fun = p;
                    break;
                }
                case '.': break;
                default: throw "Nedozvoljen izbor";
            }
        }
        if (!fun) break;
        double a, b; cout << "Interval? "; cin >> a >> b;
        cout << "Integral= " << integral(*fun,a,b) << endl;
    } catch (const Usluge::Greska& g) {
        cout << g;
    } catch (Vekt::Greska g) {
        cout << "\n*** Greska Vekt: " << g << " ***\n\n";
    } catch (const char* g) {
        cout << "\n*** " << g << " ***\n\n";
    } catch (bad_alloc) {
        cout << "\n*** Dodela memorije nije uspela ***\n\n";
    }
    delete fun;
}
```

```
% integt
```

```
Funkcija (S, O, L, P, .)? s
Interval? 0 3.14
Integral= 2
```

```
Funkcija (S, O, L, P, .)? o
Interval? 0 3.14
Integral= 1.71163
```

```
Funkcija (S, O, L, P, .)? l
Interval? 1 10
Integral= 13.6737
```

```
Funkcija (S, O, L, P, .)? p
Red polinoma? 3
Indeks i vrednost koeficijenta? 3 -1
Indeks i vrednost koeficijenta? 2 2
Indeks i vrednost koeficijenta? 1 -3
Indeks i vrednost koeficijenta? 0 4
Indeks i vrednost koeficijenta? -1 0
Interval? -5 5
Integral= 206.667
```

```
Funkcija (S, O, L, P, .)? k
```

```
*** Nedoizvoljen izbor ***
```

```
Funkcija (S, O, L, P, .)? l
Interval? -1 1
```

```
*** Ne postoji logaritam od -1 ***
```

```
Funkcija (S, O, L, P, .)? p
Red polinoma? 3
Indeks i vrednost koeficijenta? 6 1
```

```
*** Greska Vekt::2 ***
```

```
Funkcija (S, O, L, P, .)? p
Red polinoma? 2000000000
```

```
*** Dodela memorije nije uspela ***
```

```
Funkcija (S, O, L, P, .)? .
```

Zadatak 5.7 Predmeti, celi brojevi, zbirke i nizovi predmeta

Napisati na jeziku C++ sledeće klase (sve klase opremiti potrebnim konstruktorima, destruktorom i operatorom =):

- Apstraktan *predmet* ima svoj jedinstven, automatski generisan celobrojan identifikator koji može da se dohvati. Predvideti stvaranje kopije i upisivanje predmeta u izlazni tok (`it<<pr`).
- *Ceo* broj je predmet koji sadrži jednu celobroju vrednost. U izlazni tok se piše vrednost broja.
- Apstraktnoj *zbirci* predmeta može da se dohvati naziv vrste zbirke, da se dohvati broj predmeta u zbirci, da se dodaje kopija predmeta (`zb+=pr`), da se dohvati predmet s datim rednim brojem u zbirci (`zb[i]`), da se zbirka isprazni (`~zb`) i da se sadržaj zbirke upiše u izlazni tok (`it<<zb`). Piše se naziv vrste zbirke i vrednosti pojedinih predmeta unutar para uglastih zagrada, međusobno razdvojenih zarezima. Nedoizvoljen indeks prijavljuje se izuzetkom pomoću objekta tipa specijalne jednostavne klase.
- *Niz* je zbirka koja predmete skladišti u dinamički niz. Naziv vrste zbirke je "Niz". Stvara se prazan, zadatog kapaciteta (podrazumevano 5). Novi predmeti dodaju se iza poslednjeg popunjenog mesta. Kapacitet niza se, po potrebi, automatski povećava za 10% trenutnog kapaciteta niza, ali najmanje za 10 mesta.

Napisati na jeziku C++ *program* koji od celih brojeva koje pročita s glavnog ulaza napravi jedan niz. Unos brojeva se završava sa 9999. Po završetku čitanja program ispiše sadržaj niza na glavnom izlazu. Na kraju čitajući indekse s glavnog ulaza, ispisuje na glavnom izlazu identifikatore i vrednosti odabranih elemenata niza sve dok se za indeks ne unese 9999.

Rešenje:

```
// predmet2.h - Apstraktna klasa predmeta.
```

```
#ifndef _predmet2_h_
#define _predmet2_h_
```

```
#include <iostream>
using namespace std;
```

```
namespace Zbirke {
```

```
class Predmet {
```

```
static int posId;
```

```
int id;
```

```
public:
```

```
Predmet() { id = ++posId; }
```

```
Predmet(const Predmet&)
```

```
{ id = ++posId; }
```

```
virtual ~Predmet() {}
```

```
Predmet& operator=(const Predmet&)
```

```
{ return *this; }
```

```
int idBr() const { return id; }
```

```
virtual Predmet* kopija() const =0;
```

```
private:
```

```
virtual void pisi(ostream& it) const =0; // Pisanje.
```

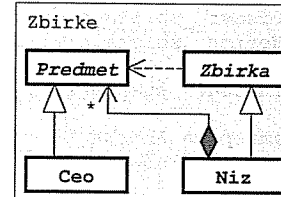
```
friend ostream& operator<<(ostream& it, const Predmet& p)
```

```
{ p.pisi(it); return it; }
```

```
};
```

```
} // namespace
```

```
#endif
```



```
// Poslednji identifikator.
```

```
// Identifikator predmeta.
```

```
// Nov predmet dobija nov broj.
```

```
// Kopija dobija nov broj.
```

```
// Virtuelan destruktor.
```

```
// Levom operandu se
```

```
// ne menja broj.
```

```
// Identifikacioni broj.
```

```
// Kopiranje.
```

```
// predmet2.C - Statičko polje apstraktne klase predmeta.
```

```
#include "predmet2.h"
```

```
int Zbirke::Predmet::posId = 0;
```

```
// ceo.h - Klasa celih brojeva.
```

```
#ifndef _ceo_h_
#define _ceo_h_
```

```
#include "predmet2.h"
```

```
namespace Zbirke {
    class Ceo: public Predmet {
        int vr; // Vrednost broja.
        void pisi(ostream& it) const { it << vr; } // Pisanje.
    public:
        Ceo(int v=0) { vr = v; } // Konstruktor.
        int vredn() const { return vr; } // Dohvatanje vrednosti.
        Ceo* kopija() const { return new Ceo(*this); } // Kopiranje.
    };
} // namespace

#endif
```

```
// zbirka.h - Apstraktna klasa zbirki apstraktnih predmeta.
```

```
#ifndef _zbirka_h_
#define _zbirka_h_
```

```
#include "predmet2.h"
```

```
namespace Zbirke {
    class GIndeks {}; // GREŠKA: Indeks izvan opsega.

    class Zbirka {
    public:
        virtual ~Zbirka() {} // Virtuelan destruktor.
        virtual const char* vrsta() const =0; // Ime vrste zbirke.
        virtual int vel() const =0; // Broj predmeta u zbirki.
        virtual Zbirka& operator+=(const Predmet& p) =0; // Dodavanje predm.
        virtual Predmet* operator[](int i) =0; // Dohvatanje elementa
        virtual const Predmet* const& operator[](int i) const =0; // (pokaz.!).
        virtual Zbirka& operator~() =0; // Pražnjenje zbirke.
        friend ostream& operator<<(ostream& it, const Zbirka& z); // Pisanje.
    };
} // namespace

#endif
```

```
// zbirka.C - Funkcija uz apstraktnu klasu zbirki apstraktnih predmeta.
```

```
#include "zbirka.h"
```

```
namespace Zbirke {
    ostream& operator<<(ostream& it, const Zbirka& z) { // Pisanje.
        it << z.vrsta() << '[';
        for (int i=0; i<z.vel(); i++) { if (i) it << ','; it << *z[i]; }
        return it << ']';
    }
} // namespace
```

```
// niz3.h - Klasa nizova apstraktnih predmeta.
```

```
#ifndef _niz3_h_
#define _niz3_h_
```

```
#include "zbirka.h"
#include "predmet2.h"
```

```
namespace Zbirke {
    class Niz: public Zbirka { // Polja:
        Predmet** niz; // - niz predmeta,
        int duz, kap; // - dužina i kapacitet.
        // Pomoćne metode:
        void kopiraj(const Niz& n); // - kopiranje u niz,
        void brisi(); // - uništavanje niza.
        { ~this; delete[] niz; }
    public:
        explicit Niz(int k=5) // Konstruktori:
        { niz = new Predmet* [kap = k]; duz = 0; } // - praznog niza,
        Niz(const Niz& n) { kopiraj(n); } // - kopije.
        ~Niz() { brisi(); } // Destruktor.
        Niz& operator=(const Niz& n) { // Dodela vrednosti.
            if (this != &n) { brisi(); Zbirka::operator=(n); kopiraj(n); }
            return *this;
        }
        const char* vrsta() const { return "Niz"; } // Ime vrste zbirke.
        Niz& operator+=(const Predmet& p); // Dodavanje predmeta.
        Predmet* operator[](int i) { // Dohvatanje elementa:
            if (i<0 || i>=duz) throw GIndeks(); // - promenljivog niza,
            return niz[i];
        }
        const Predmet* const& operator[](int i) const { // - nepromenljivog niza.
            if (i<0 || i>=duz) throw GIndeks();
            return niz[i];
        }
        int vel() const { return duz; } // Dužina niza.
        Niz& operator~(); // Pražnjenje niza.
    };
} // namespace

#endif
```

```
// niz3.C - Metode klase nizova apstraktnih predmeta.
```

```
#include "niz3.h"

namespace Zbirke {
    void Niz::kopiraj(const Niz& n) { // Kopiranje u objekat.
        niz = new Predmet* [kap = n.kap]; duz = n.duz;
        for (int i=0; i<duz; i++)
            niz[i] = n.niz[i]->kopija();
    }

    Niz& Niz::operator+=(const Predmet& p) { // Dodavanje predmeta.
        if (duz == kap) {
            kap += kap<100 ? 10 : kap/10;
            Predmet** pom = new Predmet* [kap];
            for (int i=0; i<duz; i++) pom[i] = niz[i];
            delete [] niz; niz = pom;
        }
        niz[duz++] = p.kopija();
        return *this;
    }

    Niz& Niz::operator~() { // Pražnjenje niza.
        for (int i=0; i<duz; delete niz[i++]);
        duz = 0;
        return *this;
    }
} // namespace
```

```
// zbirkat.C - Ispitivanje klase
// zbirkat.
```

```
#include "niz3.h"
#include "ceo.h"
#include <iostream>
using namespace Zbirke;
using namespace std;

int main() {
    Niz niz(20); cout << "Niz? ";
    while (true) {
        int k; cin >> k;
        if (k == 9999) break;
        niz += Ceo(k);
    }
    cout << niz << endl;
    while (true) {
        int i; cout << "Indeks? "; cin >> i;
        if (i == 9999) break;
        try {
            cout << "id=" << niz[i]->idBr() <<
                ", vr=" << *niz[i] << endl;
        } catch (GIndeks) {
            cout << "**** Neispravan indeks! ***\n";
        }
    }
    return 0;
}
```

```
% zbirkat
Niz? 0 1 2 3 4 5 6 7 8 9 9999
Niz[0,1,2,3,4,5,6,7,8,9]
Indeks? 0
id=2, vr=0
Indeks? 1
id=4, vr=1
Indeks? 2
id=6, vr=2
Indeks? 7
id=16, vr=7
Indeks? 8
id=18, vr=8
Indeks? 9
id=20, vr=9
Indeks? 10
*** Neispravan indeks! ***
Indeks? -1
*** Neispravan indeks! ***
Indeks? 9999
```

Zadatak 5.8 Vektori, figure, tačke i mnogouglovi u prostoru

Napisati na jeziku C++ sledeće klase (sve klase opremiti potrebnim konstruktorima, destruktorom i operatorom =):

- **Vektor** u prostoru zadaje se komponentama u pravcu x, y i z ose. Može da se izračuna intenzitet vektora, da se vektoru doda drugi vektor ($\text{vekt1} += \text{vekt2}$), da se vektor pomnoži realnim brojem ($\text{vekt} *= \text{broj}$) i da se vektor upiše u izlazni tok ($\text{it} << \text{vekt}$) u obliku (x, y, z) .
- Apstraktnoj geometrijskoj **figuri** može da se napravi kopija, da se pomeri za određeni pomak ($\text{fig} += \text{vekt}$), da se odredi vektor položaja težišta i da se upiše u izlazni tok ($\text{it} << \text{fig}$).
- **Tačka** je figura zadata vektorom položaja, podrazumevano $(0,0,0)$. Težište tačke poklapa se s položajem tačke. Tačka se piše u obliku $T(x, z, y)$.
- **Mnogougao** je figura koja se zadaje nizom vektora položaja temena. Stvara se zadatim brojem temena (podrazumevano 3) čije se vrednosti naknadno podešavaju pristupajući njima indeksiranjem ($\text{mnog}[\text{ind}]$). Vektor položaja težišta mnogougla je aritmetička srednja vrednost vektora položaja temena. Mnogougao se piše u obliku $M[t_0, t_1, \dots, t_{n-1}]$, gde su: t_i – rezultati pisanja vektora položaja temena.

Konfliktne situacije prijavljivati izuzecima tipa specijalnih jednostavnih klasa.

Napisati na jeziku C++ **program** koji pročita niz figura s glavnog ulaza, pronalazi figuru čije je težište najbliže koordinatnom početku i pomera sve figure za isti pomak, tako da težište prethodno pronađene figure dođe u koordinatni početak. Rezultate ispisivati na glavnom izlazu.

Rešenje:

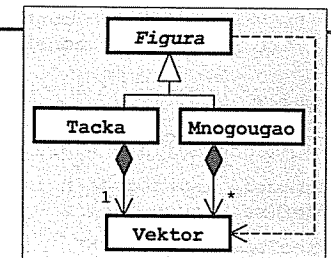
```
// vektor3.h - Klasa vektora u prostoru.
```

```
#ifndef _vektor_h_
#define _vektor_h_

#include <iostream>
#include <cmath>
using namespace std;

class Vektor {
    double x, y, z; // Komponente vektora.
public:
    explicit Vektor(double xx=0, double yy=0, double zz=0) // Konstruktor.
        { x = xx; y = yy; z = zz; }
    double intenz() const { return sqrt(x*x+y*y+z*z); } // Intenzitet.
    Vektor& operator+=(const Vektor& v2) // v1 += v2
        { x+=v2.x; y+=v2.y; z+=v2.z; return *this; }
    Vektor& operator*=(double s) // v1 *= s
        { x*=s; y*=s; z*=s; return *this; }
    friend ostream& operator<<(ostream& it, const Vektor& v) // Pisanje.
        { return it << '(' << v.x << ',' << v.y << ',' << v.z << ')'; }
};

#endif
```



```
// figura2.h - Apstraktna klasa figura u prostoru.
```

```
#ifndef _figura2_h_
#define _figura2_h_

#include "vektor3.h"
#include <iostream>
using namespace std;
```



```

class Figura {
public:
    virtual ~Figura() {} // Virt. destruktor.
    virtual Figura* kopija() const =0; // Kopiranje.
    virtual Figura& operator+=(const Vektor& v) =0; // Pomeranje figure.
    virtual Vektor teziste() const =0; // Težiste figure.
private:
    virtual void pisi(ostream& it) const =0; // Pisanje.
    friend ostream& operator<<(ostream& it, const Figura& f)
    { f.pisi(it); return it; }
};
#endif

```

// tacka7.h - Klasa tačaka u prostoru.

```

#ifndef _tacka7_h_
#define _tacka7_h_
#include "figura2.h"
#include "vektor3.h"
class Tacka: public Figura {
    Vektor p; // Vektor položaja.
    void pisi(ostream& it) const { it << 'T' << p; } // Pisanje.
public:
    Tacka(const Vektor& v=Vektor()): p(v) {} // Konstruktor.
    Tacka* kopija() const { return new Tacka(*this); } // Kopiranje.
    Tacka& operator+=(const Vektor& v) { p += v; return *this; } // T += v
    Vektor teziste() const { return p; } // Težiste.
};
#endif

```

// mnogougao1.h - Klasa mnogouglova u prostoru.

```

#ifndef _mnogougao1_h_
#define _mnogougao1_h_
#include "figura2.h"
#include "vektor3.h"
class GIndeks {}; // Klasa grešaka: Nedozvoljen indeks.
class Mnogougao: public Figura {
    Vektor* niz; // Temena mnogougla.
    int n; // Broj temena.
    void pisi(ostream& it) const; // Pisanje.
    void kopiraj(const Mnogougao& m); // Kopiranje u objekat.
public:
    explicit Mnogougao(int k=3) // Stvaranje mnogougla.
    { niz = new Vektor [n = k]; }
    Mnogougao(const Mnogougao& m) { kopiraj(m); } // Konstruktor kopije.
    ~Mnogougao() { delete [] niz; } // Dstruktor.
    Mnogougao& operator=(const Mnogougao& m) { // Dodela vrednosti.
        if (this != &m) { delete [] niz; kopiraj(m); }
        return *this;
    }
    Mnogougao* kopija() const // Kopiranje.
    { return new Mnogougao(*this); }
    Mnogougao& operator+=(const Vektor& v); // Pomeranje mnogougla.
    Vektor teziste() const; // Težište.

```

```

Vektor& operator[](int i) { // Pristupanje temenu:
    if (i<0 || i>=n) throw GIndeks(); // - promenljivog objekta,
    return niz[i];
}
const Vektor& operator[](int i) const { // - nepromenljivog objekta.
    if (i<0 || i>=n) throw GIndeks();
    return niz[i];
}
};
#endif

```

// mnogougao1.C - Metode klase mnogouglova u prostoru.

```

#include "mnogougao1.h"
void Mnogougao::pisi(ostream& it) const { // Pisanje mnogougla.
    it << "M[";
    for (int i=0; i<n; i++) {
        if (i > 0) it << ',';
        it << niz[i];
    }
    it << ']';
}
void Mnogougao::kopiraj(const Mnogougao& m) { // Kopiranje u objekat.
    niz = new Vektor [n = m.n];
    for (int i=0; i<n; i++) niz[i] = m.niz[i];
}
Mnogougao& Mnogougao::operator+=(const Vektor& v) { // Pomeranje mnogougla.
    for (int i=0; i<n; niz[i++] += v);
    return *this;
}
Vektor Mnogougao::teziste() const { // Težište mnogougla.
    Vektor t;
    for (int i=0; i<n; t+=niz[i++]);
    return t *=(1. / n);
}

```

// figura2t.C - Ispitivanje klase figura u prostoru.

```

#include "tacka7.h"
#include "mnogougao1.h"
#include <iostream>
using namespace std;
int main() {
    try {
        cout << "Broj figura? "; int n; cin >> n;
        Figura** niz = new Figura* [n];
        for (int i=0; i<n; i++) {
            cout << "Vrsta (T, M)? "; char vrs; cin >> vrs;
            switch (vrs) {
                case 't': case 'T': {
                    double x, y, z; cout << "Koordinate tacke? "; cin >> x >> y >> z;
                    niz[i++] = new Tacka(Vektor(x, y, z));
                    break;
                }
            }
        }
    }
}

```

```

case 'm': case 'M': {
    cout << "Broj temena? "; int n; cin >> n;
    Mnogougao* m(new Mnogougao(n));
    for (int j=0; j<n; j++) {
        cout << "Koordinate temena " << j << "? ";
        double x, y, z; cin >> x >> y >> z;
        (*m)[j] = Vektor(x, y, z);
    }
    niz[i++] = m;
    break;
}
default: cout << "*** Neispravan izbor! ***\n";
}
}
cout << "\nProcitano:\n";
for (int i=0; i<n; i++) cout<<niz[i++]<<endl;
double min = niz[0]->teziste().intenz(); int imin = 0;
for (int i=1; i<n; i++)
    if (niz[i]->teziste().intenz() < min)
        { min = niz[i]->teziste().intenz(); imin = i; }
cout << "\nNajblize=" << *niz[imin]
    << " teziste=" << niz[imin]->teziste() << endl;
Vektor pomak = niz[imin]->teziste() *=-1;
for (int i=0; i<n; i++) *niz[i++] += pomak;
cout << "\nPomereno:\n";
for (int i=0; i<n; i++) cout<<*niz[i++]<<endl;
} catch (...) { cout << "*** Neocekivana greska! ***\n"; }
}

```

```

% figura2t
Broj figura? 5
Vrsta (T, M)? t
Koordinate tacke? 3 3 3
Vrsta (T, M)? m
Broj temena? 4
Koordinate temena 0? 0 0 0
Koordinate temena 1? 0 1 0
Koordinate temena 2? 1 1 0
Koordinate temena 3? 1 0 0

```

```

Procitano:
T(3,3,3)
M[(0,0,0), (0,1,0), (1,1,0), (1,0,0)]
M[(-1,-1,-1), (-1,2,2), (0,0,0)]
T(2,1,0)
T(-1,1,0)

Najblize=M[(0,0,0), (0,1,0), (1,1,0), (1,0,0)] teziste=(0.5,0.5,0)

Pomereno:
T(2.5,2.5,3)
M[(-0.5,-0.5,0), (-0.5,0.5,0), (0.5,0.5,0), (0.5,-0.5,0)]
M[(-1.5,-1.5,-1), (-1.5,1.5,2), (-0.5,-0.5,0)]
T(1.5,0.5,0)
T(-1.5,0.5,0)

```

```

Vrsta (T, M)? m
Broj temena? 3
Koordinate temena 0? -1 -1 -1
Koordinate temena 1? -1 2 2
Koordinate temena 2? 0 0 0
Vrsta (T, M)? t
Koordinate tacke? 2 1 0
Vrsta (T, M)? x
*** Neispravan izbor! ***
Vrsta (T, M)? t
Koordinate tacke? -1 1 0

```

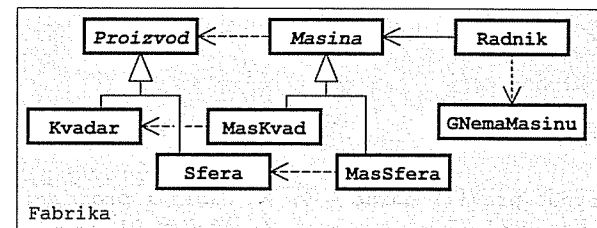
Zadatak 5.9 Proizvodi, sferè, kvadri, mašine i radnici

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorom i operatorom za dodelu vrednosti koji su potrebni za bezbedno korišćenje klasa; greške prijavljivati izuzecima tipa jednostavnih klasa koje su opremljene ispisivanjem poruke):

- Apstraktan **proizvod** ima jednoslovnu oznaku vrste i jedinstven, automatski generisan celobrojan identifikator. Može da se dohvati vrsta proizvoda i identifikator i da se izračuna zapremina. Pri upisivanju u izlazni tok (`it<<p`) piše se vrsta i identifikator proizvoda.
- Apstraktna **mašina** može da proizvodi apstraktan proizvod i zna se koliko proizvoda je ta mašina proizvela. Može da se dohvati oznaka vrste proizvoda koje data mašina proizvodi i broj proizvedenih proizvoda. Ne može da se stvara kopija mašine.
- **Kvadar** je proizvod zadat dužinama ivica. Oznaka vrste proizvoda je **K**. Pri pisanju navode se i dimenzije kvadra.
- **Sfera** je proizvod zadat poluprečnikom. Oznaka vrste proizvoda je **S**. Pri pisanju navodi se i poluprečnik sfere.
- **Mašina za kvadre** je mašina koja može da proizvodi kvadre zadatih dimenzija. Parametri mašine (dimenzije kvadra koje proizvodi) ne mogu da se promene, ali mogu da se dohvate.
- **Mašina za sfere** je mašina koja može da proizvodi sfere zadatog poluprečnika. Parametar mašine (poluprečnik sfere koje proizvodi) ne može da se promeni, ali može da se dohvati.
- **Radnik** ima ime i jedinstven, automatski generisan celobrojan identifikator. Izrađuje proizvode određene vrste pomoću odgovarajuće mašine. Mašina na kojoj radnik radi može da se promeni (`r+=m`). Zna se koliko je proizvoda izradio na mašini na kojoj trenutno radi. Radnik može da ne bude raspoređen ni na jednu mašinu. U tom slučaju, pokušaj izrade proizvoda ili dohvaćanja broja izrađenih proizvoda prijavljuje se izuzetkom tipa specijalne klase. Pri pisanju u izlazni tok (`it<<r`) piše se ime i identifikator radnika. U slučaju da je radnik raspoređen na neku mašinu piše se i vrsta proizvoda koje trenutno izrađuje i broj proizvoda koje je izradio na toj mašini od početka rada na njoj.

Napisati na jeziku C++ **program** za ispitivanje prethodnih klasa.

Rešenje:



```
// proizvod1.h - Apstraktna klasa proizvoda.
```

```
#ifndef _proizvod1_h_
#define _proizvod1_h_

#include <iostream>
using namespace std;

namespace Fabrika {
    class Proizvod {
        static int posId; // Poslednji identifikator.
        int id; // Identifikator objekta.
    public:
        Proizvod() { id = ++posId; } // Nov objekat dobija nov broj.
        Proizvod(const Proizvod&) { id = ++posId; } // Kopija dobija nov broj.
        virtual ~Proizvod() {} // Virtuelan destruktor.
        Proizvod& operator=(const Proizvod&) // Levom operandu se ne menja br.
        { return *this; }
        int dohvId() const { return id; } // Dohvatanje identifikatora.
        virtual char vrsta() const =0; // Oznaka vrste proizvoda.
        virtual double V() const =0; // Zapremina proizvoda.
    protected:
        virtual void pisi(ostream& it) const { it << vrsta() << id; } // Pisanje.
        friend ostream& operator<<(ostream& it, const Proizvod& p)
        { p.pisi(it); return it; }
    }; // class Proizvod
} // namespace Fabrika

#endif
```

```
// proizvod1.C - Statičko polje apstraktne klase proizvoda.
```

```
#include "proizvod1.h"

int Fabrika::Proizvod::posId = 0; // Poslednji identifikator.
```

```
// masina.h - Apstraktna klasa mašina.
```

```
#ifndef _masina_h_
#define _masina_h_

#include "proizvod1.h"

namespace Fabrika {
    class Masina {
        int br; // Broj napravljenih proizvoda.
        virtual Proizvod* pravi() const = 0; // Napravi proizvod.
        Masina(const Masina&) {} // Ne sme da se kopira.
        void operator=(const Masina&) {} // Ne sme da se dodeljuje.
    public:
        Masina() { br = 0; } // Inicijalizacija.
        virtual char vrsta() const =0; // Vrsta pravljenih proizvoda.
        Proizvod* napravi() // Napravi proizvod.
        { br++; return pravi(); }
        int broj() const { return br; } // Broj napravljenih proizvoda.
    }; // class Masina
} // namespace Fabrika

#endif
```

```
// kvadar4.h - Klasa kvadara.
```

```
#ifndef _kvadar4_h_
#define _kvadar4_h_

#include "proizvod1.h"

namespace Fabrika {
    class Kvadar: public Proizvod {
        double a, b, c; // Dimenzije kvadra.
        void pisi(ostream& it) const // Pisanje kvadra.
        { Proizvod::pisi(it); it << '(' << a << ',' << b << ',' << c << ')'; }
    public:
        enum { VR = 'K' }; // Oznaka vrste proizvoda.
        Kvadar(double aa, double bb, double cc) // Inicijalizacija.
        { a = aa; b = bb; c = cc; }
        char vrsta() const { return VR; } // Vrsta proizvoda.
        double V() const { return a*b*c; } // Zapremina kvadra.
    }; // class Kvadar
} // namespace Fabrika

#endif
```

```
// sfera3.h - Klasa sfera.
```

```
#ifndef _sfera3_h_
#define _sfera3_h_

#include "proizvod1.h"

namespace Fabrika {
    class Sfera: public Proizvod {
        double r; // Poluprečnik sfere.
        void pisi(ostream& it) const // Pisanje kvadra.
        { Proizvod::pisi(it); it << '(' << r << ')'; }
    public:
        enum { VR = 'S' }; // Oznaka vrste proizvoda.
        Sfera(double rr) { r = rr; } // Inicijalizacija.
        char vrsta() const { return VR; } // Vrsta proizvoda.
        double V() const // Zapremina sfere.
        { return 4 * r*r*r * 3.14159 / 3; }
    }; // class Sfera
} // namespace Fabrika

#endif
```

```
// maskvad.h - Klasa mašina za kvadre.

#ifndef _maskvad_h_
#define _maskvad_h_

#include "masina.h"
#include "kvadar4.h"

namespace Fabrika {
    class Maskvad: public Masina {
        double a, b, c; // Dimenzije pravljenih kvadara.
        Kvadar* pravi() const // Napravi kvadar.
        { return new Kvadar(a, b, c); }
    public:
        Maskvad(double aa, double bb, double cc) // Inicijalizacija.
        { a = aa; b = bb; c = cc; }
        char vrsta() const { return Kvadar::VR; } // Vrsta pravljenih proizv.
        double dohvA() const { return a; } // Dohvatanje dimenzija
        double dohvB() const { return b; } // pravljenih kvadara.
        double dohvC() const { return c; }
    }; // class Maskvad
} // namespace Fabrika

#endif
```

```
// massfera.h - Klasa mašina za sfere.

#ifndef _massfera_h_
#define _massfera_h_

#include "masina.h"
#include "sfera3.h"

namespace Fabrika {
    class MasSfera: public Masina {
        double r; // Poluprečnik pravljenih sfera.
        Sfera* pravi() const // Napravi sferu.
        { return new Sfera(r); }
    public:
        MasSfera(double rr) { r = rr; } // Inicijalizacija.
        char vrsta() const { return Sfera::VR; } // Vrsta pravljenih proizv.
        double dohvR() const { return r; } // Dohvatanje poluprečnika
    }; // class MasSfera // pravljenih sfera.
} // namespace Fabrika

#endif
```

```
// radnikl.h - Klasa radnika.

#ifndef _radnikl_h_
#define _radnikl_h_

#include "proizvodl.h"
#include "masina.h"
#include <iostream>
#include <cstring>
using namespace std;

namespace Fabrika {
    class GNemaMasinu { // KLASA GREŠAKA: Radnik nema mašinu.
        char* ime; int id; // Ime i identifikator problematičnog radnika.
        void operator=(const GNemaMasinu&) {} // Ne sme da se dodeljuje.
    public:
        GNemaMasinu(const char* iime, int iid) { // Inicijalizacija.
            ime = strcpy(new char [strlen(iime)+1], iime);
            id = iid;
        }
        GNemaMasinu(const GNemaMasinu& g) { // Konstruktor kopije.
            ime = strcpy(new char [strlen(g.ime)+1], g.ime);
            id = g.id;
        }
        ~GNemaMasinu() { delete [] ime; } // Destruktor.
        friend ostream& operator<<(ostream& it, const GNemaMasinu& g) {
            return it << "\n*** Radniku " << g.ime << ", id " << g.id
                << ", nije dodeljena masina ***\n\n";
        }
    }; // class GNemaMasinu

    class Radnik { // KLASA RADNIKA:
        static int posId; // Poslednji identifikator.
        int id; char* ime; // Identifikator i ime radnika.
        Masina* mas; // Mašina koju radnik koristi.
        int br; // Broj napravljenih proizvoda na masini.
        Radnik(const Radnik&) {} // Ne sme da se kopira.
        void operator=(const Radnik&) {} // Ne sme da se dodeljuje.
    public:
        Radnik(const char* iime) { // Inicijalizacija.
            id = ++posId; mas=0;
            ime = new char [strlen(iime)+1]; strcpy(ime,iime);
        }
        ~Radnik() { delete [] ime; } // Uništavanje (radnik nije vlasnik mašine!)
        const char* dohvIme() const { return ime; } // Dohvatanje imena.
        int dohvId() const { return id; } // Dohvatanje identifikatora
        Radnik& operator+=(Masina* m) // Pridruživanje mašine.
        { mas = m; br = 0; return *this; }
        int broj() const { // Broj napravljenih proizvoda.
            if (!mas) throw GNemaMasinu(ime, id);
            return br;
        }
        Proizvod* napravi() { // Napravi proizvod.
            if (!mas) throw GNemaMasinu(ime, id);
            br++; return mas->napravi();
        }
    };
```

```

friend ostream& operator<<(ostream& it, const Radnik& r) { // Pisanje.
    it << r.ime << '/' << r.id;
    if (r.mas) it << '(' << r.mas->vrsta() << ',' << r.br << ')';
    return it;
}
}; // class Radnik
} // namespace Fabrika

#endif

```

```

// radnik1.C - Statičko polje klase radnika.

```

```

#include "radnik1.h"

```

```

int Fabrika::Radnik::posId = 0; // Poslednji identifikator.

```

```

// masinat.C - Ispitivanje klase mašina i radnika.

```

```

#include "maskvad.h"
#include "massfera.h"
#include "radnik1.h"
using namespace Fabrika;
#include <iostream>
using namespace std;

int main() {
    try {
        MasKvad m1(1, 2, 3);
        MasSfera m2(2);
        Radnik marko("Marko");
        cout << marko << endl;
        marko += &m1;
        for (int i=0; i<10; i++) delete marko.napravi();
        cout << marko << endl;
        marko += &m2;
        for (int i=0; i<5; i++) delete marko.napravi();
        cout << marko << endl;
        marko += 0;
        for (int i=0; i<12; i++) delete marko.napravi();
        cout << marko << endl;
    } catch (GNemaMasinu g) { cout << g; }
    return 0;
}

```

```

% masinat
Marko/1
Marko/1(K,10)
Marko/1(S,5)

*** Radniku Marko, id 1, nije dodeljena masina ***

```

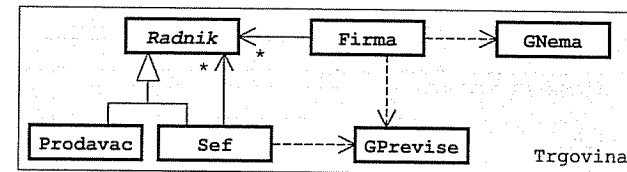
Zadatak 5.10 Radnici, prodavci, šefovi i firme

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorom i operatorom za dodelu vrednosti koji su potrebni za bezbedno korišćenje klasa; greške prijavljivati izuzecima tipa jednostavnih klasa koje su opremljene ispisivanjem poruke):

- Apstraktan **radnik** u trgovačkoj firmi ima ime i kao platu dobija zadati procenat od vrednosti ostvarenog prihoda. Može da se odredi vrednost ostvarenog prihoda, da se izračuna visina plate i da se radnik upiše u izlazni tok (`it<<radnik`) u obliku `ime/plata`. Radnik ne sme da se kopira ni na koji način.
- **Prodavac** je radnik koji može da prodaje robu. Može da prodaje robu u zadatoj vrednosti (vodi se računa samo o ukupnoj vrednosti pojedinačnih prodaja). Prihod prodavca čini ukupna vrednost prodate robe.
- **Šef** je radnik kome može biti potčinjen zadat broj drugih radnika. Radnici se dodeljuju pojedinačno na osnovu adrese i šef nije vlasnik svojih potčinjenih. Greška je ako se prekorači predviđeni broj potčinjenih. Prihod na osnovu kojeg šef dobija platu jednak je ukupnom prihodu svih njegovih potčinjenih.
- Trgovačka **firma** ima zadat broj radnih mesta i posluje sa zatom maržom (procentom od vrednosti prodate robe). Može da se zaposli jedan radnik koji se raspoređuje na prvo slobodno radno mesto, da se otpusti radnik s radnog mesta sa zadatim rednim brojem, da se odredi ukupna dobit firme i da se firma upiše u izlazni tok (`it<<firma`). Zapošljavanje se vrši na osnovu adrese radnika i firma je vlasnik zaposlenih radnika. Greška je ako pri zapošljavanju nema nijedno slobodno mesto, a pri otpuštanju ako je redni broj mesta izvan dozvoljenog opsega ili na odabranom mestu nema radnika. Ukupna dobit firme jednaka je razlici između ukupne zarade ($\text{prihod} \cdot \text{marža} / 100$) koju su ostvarili svi prodavci i ukupne plate svih radnika. U izlazni tok upisuje se po jedan radnik u svakom redu i ukupna dobit firme u poslednjem redu.

Napisati na jeziku C++ **program** koji napravi firmu s jednim šefom i dva prodavca koji su mu potčinjeni, obavi po jednu prodaju s oba prodavca i ispiše na glavnom izlazu podatke o firmi. Koristiti konstantne parametre (ne treba niša učitavati s glavnog ulaza).

Rešenje:



```

// firmagr.h - Klase grešaka.

```

```

#ifndef _firmagr_h_
#define _firmagr_h_

#include <iostream>
using namespace std;

namespace Trgovina {
    class GPreviše {}; // GREŠKA: Previše radnika.
    inline ostream& operator<<(ostream& it, const GPreviše&)
    { return it << "*** Previše radnika! ***"; }
}

```

```

class GNema {};           // GREŠKA: Nema traženog radnika.
inline ostream& operator<<(ostream& it, const GNema&)
{ return it << "*** Nema radnika! ***"; }
} // namespace

#endif

```

```
// radnik2.h - Apstraktna klasa radnika.
```

```

#ifndef _radnik2_h_
#define _radnik2_h_

#include <cstring>
#include <iostream>
using namespace std;

namespace Trgovina {
    class Radnik {
        char* ime;           // Ime radnika.
        double procenat;      // Procenat za platu.
        Radnik(const Radnik&) {} // Ne sme da se kopira.
        void operator=(const Radnik&) {} // Ne sme da se dodeljuje.
    public:
        Radnik(const char* iime, double pproc) { // Inicijalizacija.
            ime = strcpy(new char [strlen(iime)+1], iime);
            procenat = pproc;
        }
        virtual ~Radnik() { delete [] ime; } // Destruktor.
        virtual double prihod() const = 0; // Ostvareni prihod.
        double plata() const // Iznos plate.
        { return prihod() * procenat / 100; }
        friend ostream& operator<<(ostream& it, const Radnik& r) // Pisanje.
        { return it << r.ime << "/" << r.plata(); }
    }; // class
} // namespace

#endif

```

```
// prodavac.h - Klasa prodavaca.
```

```

#ifndef _prodavac_h_
#define _prodavac_h_

#include "radnik2.h"

namespace Trgovina {
    class Prodavac: public Radnik {
        double prih;           // Ostvareni prihod.
    public:
        Prodavac(const char* ime, double procenat): // Inicijalizacija.
            Radnik(ime, procenat) { prih = 0; }
        Prodavac& prodao(double vrednost) // Prodaja robe.
        { prih += vrednost; return *this; };
        double prihod() const { return prih; } // Ostvareni prihod.
    }; // class
} // namespace

#endif

```

```
// sef.h - Klasa šefova.
```

```

#ifndef _sef_h_
#define _sef_h_

#include "radnik2.h"
#include "firmagr.h"

namespace Trgovina {
    class Sef: public Radnik {
        Radnik** potcinjeni; // Niz potčinjenih radnika.
        int maxPot, brPot;    // Najveći i trenutni broj potčinjenih.
    public:
        Sef(const char* ime, double procenat, int k=3): // Inicijalizacija.
            Radnik(ime, procenat) {
                potcinjeni = new Radnik* [maxPot = k];
                brPot = 0;
            }
        ~Sef() { delete [] potcinjeni; } // Destruktor.
        Sef& dodeli(Radnik* r) { // Dodeljivanje potčinjenog.
            if (brPot == maxPot) throw GPreviše();
            potcinjeni[brPot++] = r;
            return *this;
        }
        double prihod() const; // Ostvareni prihod.
    }; // class
} // namespace

#endif

```

```
// sef.C - Metoda klase šefova.
```

```

#include "sef.h"

double Trgovina::Sef::prihod() const { // Ostvareni prihod.
    double p = 0;
    for (int i=0; i<brPot; p+=potcinjeni[i++]->prihod());
    return p;
}

```

```
// firma.h - Klasa firmi.
```

```

#ifndef _firma_h_
#define _firma_h_

#include "radnik2.h"
#include "firmagr.h"
#include <iostream>
using namespace std;

namespace Trgovina {
    class Firma {
        Radnik** radnici; // Niz zaposlenih.
        int kap;           // Broj radnih mesta.
        double marza;      // Iznos marže.
        Firma(const Firma&) {} // Ne sme da se kopira.
        void operator=(const Firma&) {} // Ne sme da se dodeljuje.
    };
}

```

```

public:
    Firma(double mar, int k=10);           // Inicijalizacija.
    ~Firma();                             // Destruktor.
    Firma& zaposli(Radnik* r);             // Zapošljavanje.
    Firma& otpusti(int i) {               // Opuštanje.
        if (i<0 || i>=kap || !radnici[i]) throw GNema();
        delete radnici[i]; radnici[i] = 0;
        return *this;
    }
    double dobit() const;                  // Dobit firme.
    friend ostream& operator<<(ostream& it, const Firma& fir); // Pisanje.
}; // class
} // namespace

#endif

```

// firma.C - Metode i funkcije uz klasu firmi.

```

#include "prodavac.h"
#include "firma.h"
#include <typeinfo>
using namespace std;

namespace Trgovina {
    Firma::Firma(double mar, int k) {      // Inicijalizacija.
        marza = mar;
        radnici = new Radnik* [kap = k];
        for (int i=0; i<kap; radnici[i++]=0);
    }

    Firma::~Firma() {                      // Destruktor.
        for (int i=0; i<kap; delete radnici[i++]);
        delete [] radnici;
    }

    Firma& Firma::zaposli(Radnik* r) {      // Zapošljavanje.
        int i=0; while (radnici[i]) i++;
        if (i == kap) throw GPreviše();
        radnici[i] = r;
        return *this;
    }

    double Firma::dobit() const {           // Dobit firme.
        double d = 0, p = 0;
        for (int i=0; i<kap; i++)
            if (radnici[i]) {
                if (typeid(*radnici[i]) == typeid(Prodavac))
                    d += radnici[i]->prihod();
                p += radnici[i]->plata();
            }
        return d * marza / 100 - p;
    }

    ostream& operator<<(ostream& it, const Firma& fir) { // Pisanje.
        for (int i=0; i<fir.kap; i++)
            if (fir.radnici[i])
                it << *fir.radnici[i] << endl;
        return it << "Dobit firme: " << fir.dobit() << endl;
    }
} // namespace

```

// firmat.C - Ispitivanje klasa trgovačkih firmi.

```

#include "firmagr.h"
#include "prodavac.h"
#include "sef.h"
#include "firma.h"
using namespace Trgovina;
#include <iostream>
using namespace std;

int main() {
    try {
        Firma firma(30, 5);
        Prodavac* marko = new Prodavac("Marko", 5);
        Prodavac* zoran = new Prodavac("Zoran", 4);
        Sef*      nenad = new Sef      ("Nenad", 6, 2);
        firma.zaposli(marko);
        firma.zaposli(zoran);
        firma.zaposli(nenad);
        nenad->dodeli(marko);
        nenad->dodeli(zoran);
        marko->prodao(5000);
        zoran->prodao(8000);
        cout << firma;
    } catch (GPreviše g) { cout << g << endl; }
    catch (GNema g) { cout << g << endl; }
}

```

```

% firmat
Marko/250
Zoran/320
Nenad/780
Dobit firme: 2550

```

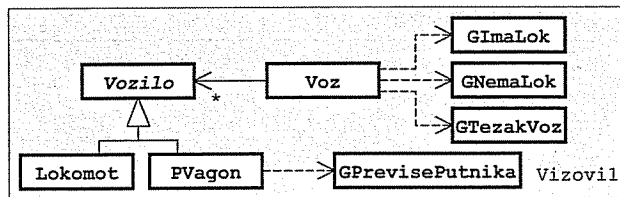
Zadatak 5.11 Vozila, lokomotive, putnički vagoni i vozovi

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorom i operatorom za dodelu vrednosti koji su potrebni za bezbedno korišćenje klase; greške prijavljivati izuzecima tipa jednostavnih klasa koje su opremljene ispisivanjem poruke):

- Apstraktno **vozilo** ima zadatu sopstvenu težinu i jedinstven, automatski generisan celoborjan identifikator. Može da se dohvati sopstvena težina, da se dohvati jednoslovna oznaka vrste vozila, da se odredi ukupna težina vozila, da se napravi kopija vozila i da se vozilo upiše u izlazni tok (it<<vozilo). Pišu se oznaka vrste, identifikator i sopstvena težina.
- **Lokomotiva** je vozilo zadate vučne snage koja može da se dohvati. Vučna snaga predstavlja najveću ukupnu težinu voza koji lokomotiva može da vuče. Oznaka vrste vozila je 'L'. U izlazni tok se piše i vučna snaga.
- **Putnički vagon** je vozilo koje može da prima zadat broj putnika, podrazumevano 50. Srednja težina putnika je osobina klase u celini i može da se postavlja i da se dohvati. Oznaka vrste vozila je 'P'. Može zadat broj putnika da uđe u vagon i da izađe iz vagona. Greška je ako se vagon prepuni. U izlazni tok upisuje se i trenutni broj putnika u vagonu i njihova ukupna težina.
- **Voz** ima zadato ime i može da se sastoji od proizvoljnog broja vozila od kojih samo prvo sme i mora da bude lokomotiva. Stvara se prazan posle čega se vozila dodaju pojedinačno (voz+=lokomotiva, voz+=vagon). Greške su ako se pokuša dodati druga lokomotiva, vagon a da koš nije dodata lokomotiva ili vagon kojim bi se prekoračila vučna snaga lokomotive. Može da se odredi ukupna težina voza i da se voz upiše u izlazni tok (it<<voz). U prvom redu se piše ime voza, ukupna težina voza i srednja težina putnika. Posle toga pišu se vozila u sastavu voza, jedno vozilo po redu.

Napisati na jeziku C++ **program** koji postavi srednju težinu putnika, napravi jedan voz, doda vozu jednu lokomotivu, doda vozu dva vagona sa po nekoliko putnika, doda vozu drugu lokomotivu i ispiše voz na glavnom izlazu. Koristiti proizvoljno odabrane konstantne parametre (ne treba ništa učitavati s glavnog ulaza).

Rešenje:



// vozilo2.h - Klasa vozila.

```

#ifndef _vozilo2_h_
#define _vozilo2_h_

#include <iostream>
using namespace std;

```

```

namespace Vozovil {
    class Vozilo {
        static int posId; // Poslednji identifikator.
        int id; // Identifikator vozila.
        float sopTez; // Sopstvena težina vozila.
    };
}

```

```

public:
    Vozilo(float tez) // Novom vozilu novi identifikator.
    { id = ++posId; sopTez = tez; }
    Vozilo(const Vozilo& v) // Kopiji novi identifikator
    { id = ++posId; sopTez = v.sopTez; }
    Vozilo& operator=(const Vozilo&v) // Levom operandu se ne menja ident.
    { sopTez = v.sopTez; }
    float sopTezina() const { return sopTez; } // Sopstvena težina.
    virtual char vrsta() const =0; // Oznaka vrste.
    virtual float ukTezina() const =0; // Ukupna težina.
    virtual Vozilo* kopija() const =0; // Kopiranje.
protected:
    virtual void pisi(ostream& it) const // Pisanje.
    { it << vrsta() << id << '[' << sopTez << ']' ; }
    friend ostream& operator<<(ostream& it,
        const Vozilo& v) { v.pisi(it); return it; }
}; // class
} // namespace

#endif

```

// vozilo2.C - Statičko polje klase vozila.

```
#include "vozilo2.h"
```

```
int Vozovil::Vozilo::posId = 0; // Poslednji identifikator.
```

// lokomot.h - Klasa lokomotiva.

```

#ifndef _lokomot_h_
#define _lokomot_h_

```

```

#include "vozilo2.h"
#include <iostream>
using namespace std;

```

```

namespace Vozovil {
    class Lokomot: public Vozilo {
        float vsnaga; // Vučna snaga.
        void pisi(ostream& it) const // Pisanje.
        { Vozilo::pisi(it); it << '(' << vsnaga << ')'; }
    public:
        static const char VR = 'L'; // Oznaka lokomotive.
        Lokomot(float sopTez, float vs): Vozilo( sopTez) // Sopstvena tež.
        { vsnaga = vs; }
        char vrsta() const { return VR; } // Oznaka vrste.
        float ukTezina() const { return sopTezina(); } // Ukupna težina.
        float snaga() const { return vsnaga; } // Vučna snaga.
        Lokomot* kopija() const {return new Lokomot(*this);} // Kopiranje.
    }; // class
} // namespace

#endif

```



```
// pvagon.h - Klasa putničkih vagona.

#ifndef _pvagon_h_
#define _pvagon_h_

#include "vozilo2.h"
#include <iostream>
using namespace std;

namespace Vozovil {
    class GPrevišePutnika {}; // KLASA GREŠAKA: Previše putnika.
    inline ostream& operator<<(ostream& it, const GPrevišePutnika&)
    { return it << "*** Previše putnika! ***"; }

    class PVagon: public Vozilo { // KLASA PUTNIČKIH VAGONA:
        static float srTezPut; // Srednja težina putnika.
        int kap, brPut; // Kapac. vagona i trenutni broj putn.
        void pisi(ostream& it) const { // Pisanje.
            Vozilo::pisi(it); it << '(' << brPut << ', '
                << brPut*srTezPut << ')';
        }
    public:
        static const char VR = 'P'; // Oznaka putničkih vagona.
        static void postaviSrTezPut(float stp) // Postavljanje sr. tež. putn.
        { srTezPut = stp; }
        static float dohvatiSrTezPut() // Dohvatanje sr. tež. putn.
        { return srTezPut; }
        PVagon(float sopTez, int k=50): Vozilo(sopTez) // Konstruktor.
        { kap = k; brPut = 0; }
        char vrsta() const { return VR; } // Oznaka vrste.
        float ukTezina() const // Ukupna težina.
        { return sopTezina() + brPut*srTezPut; }
        PVagon* kopija() const // Kopiranje.
        { return new PVagon(*this); }
        PVagon& ulazePutnici(int broj) { // Ulazak putnika.
            if (brPut+broj > kap) throw GPrevišePutnika();
            brPut += broj; return *this;
        }
        PVagon& izlazePutnici(int broj) { // Izlazak putnika.
            brPut = (broj <= brPut) ? brPut-broj : 0;
            return *this;
        }
    }; // class
} // namespace

#endif
```

```
// pvagon.C - Statičko polje klase putničkih vagona.
```

```
#include "PVagon.h"

float Vozovil::PVagon::srTezPut = 70; // Srednja težina putnika.
```

```
// voz1.h - Klasa vozova.

#ifndef _voz1_h_
#define _voz1_h_

#include "vozilo2.h"
#include "lokomot1.h"
#include "pvagon.h"
#include <iostream>
#include <cstring>
using namespace std;

namespace Vozovil {
    class GImaLok {}; // KLASA GREŠAKA: Već postoji lokomotiva.
    inline ostream& operator<<(ostream& it, const GImaLok)
    { return it << "*** Vec ima lokomotive! ***"; }

    class GNemaLok {}; // KLASA GREŠAKA: Još nema lokomotive.
    inline ostream& operator<<(ostream& it, const GNemaLok)
    { return it << "*** Jos nema lokomotive! ***"; }

    class GTezakVoz {}; // KLASA GREŠAKA: Lokomotiva je preopterećena.
    inline ostream& operator<<(ostream& it, const GTezakVoz)
    { return it << "*** Pretezak voz! ***"; }

    class Voz { // KLASA VOZOVA:
        char* ime; // Ime voza.
        struct Elem { // Element liste vozila.
            Vozilo* vozilo; Elem* sled;
            Elem(Vozilo* v) { vozilo = v; sled = 0; }
            ~Elem() { delete vozilo; }
        };
        Elem *prvi, *posl; // Prvi i poslednji element liste.
        void kopiraj(const Voz& v); // Kopiranje u objekat.
        void brisi(); // Brisanje objekta.
    public:
        Voz(const char* im) { // Inicijalizacija.
            prvi = posl = 0;
            ime = new char [strlen(im)+1];
            strcpy(ime, im);
        }
        Voz(const Voz& v) { kopiraj(v); } // Konstruktor kopije.
        ~Voz() { brisi(); } // Destruktor.
        Voz& operator=(const Voz&v) { // Dodela vrednosti.
            if (this != &v) { brisi(); kopiraj(v); }
            return *this;
        }
        Voz& operator+=(const Lokomot& lok); // Dodavanje lokomotive.
        Voz& operator+=(const PVagon& pvag); // Dodavanje putničkog vagona.
        float ukTezVoza() const; // Ukupna težina voza.
        friend ostream& operator<<(ostream& it, const Voz& v); // Pisanje.
    }; // class
} // namespace

#endif
```

```
// voz1.C - Metode i funkcije uz klasu vozova.
```

```
#include "voz1.h"
```

```
namespace Vozovil {
    void Voz::kopiraj(const Voz& v) {           // Kopiranje u objekat.
        ime = new char [strlen(v.ime)+1];
        strcpy(ime, v.ime);
        prvi = posl = 0;
        for (Elem* tek=v.prvi; tek; tek=tek->sled) {
            Elem* novi = new Elem(tek->vozilo->kopija());
            posl = (!prvi ? prvi : prvi->sled) = novi;
        }
    }

    void Voz::brisi() {                         // Brisanje objekta.
        while (prvi)
            {Elem* stari=prvi; prvi=prvi->sled; delete stari;}
        posl = 0; delete [] ime;
    }

    Voz& Voz::operator+=(const Lokomot& lok) { // Dodavanje lokomotive.
        if (prvi) throw GImaLok();
        posl = prvi = new Elem(lok.kopija());
        return *this;
    }

    Voz& Voz::operator+=(const PVagon& pvag) { // Dodavanje putn. vagona.
        if (!prvi) throw GNemaLok();
        if (ukTezVoza()+pvag.ukTezina() > ((Lokomot*)prvi->vozilo->snaga())
            throw GTezakVoz();
        posl = posl->sled = new Elem(pvag.kopija());
        return *this;
    }

    float Voz::ukTezVoza() const {             // Ukupna težina voza.
        float t = 0;
        for (Elem* tek=prvi; tek; tek=tek->sled) t += tek->vozilo->ukTezina();
        return t;
    }

    ostream& operator<<(ostream& it, const Voz& v) { // Pisanje.
        it << v.ime << '(' << v.ukTezVoza() << ','
            << PVagon::dohvatiSrTezPut() << ')' << endl;
        for (Voz::Elem* tek=v.prvi; tek; tek=tek->sled)
            it << *tek->vozilo << endl;
        return it;
    }
} // namespace
```

```
// voz1t.C - Ispitivanje klase vozova.
```

```
#include "voz1.h"
#include "lokomot1.h"
#include "pvagon.h"
using namespace Vozovil;
#include <iostream>
using namespace std;

int main() {
    PVagon::postaviSrTezPut(75);
    Voz voz("Avala ekspres");
    try {
        voz += Lokomot(500, 5000);
        voz += PVagon(300, 40).ulazePutnici(30);
        voz += PVagon(250, 30).ulazePutnici(10);
        voz += Lokomot(400, 4000);
    } catch (GPrevišePutnika g) { cout << g << endl;
    } catch (GNemaLok g) { cout << g << endl;
    } catch (GImaLok g) { cout << g << endl;
    } catch (GTezakVoz g) { cout << g << endl;
    }
    cout << voz;
    return 0;
}
```

```
% voz1t
*** Vec ima lokomotive! ***
Avala ekspres(4050,75)
L2[500](5000)
P4[300](30,2250)
P6[250](10,750)
```

Zadatak 5.12 Električni potrošači, uređaji, grupe uređaja i izvori.

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorom i operatorom za dodelu vrednosti koji su potrebni za bezbedno korišćenje klase; greške prijavljivati izuzecima tipa jednostavnih klasa koje su opremljene ispisivanjem poruke):

- Apstraktnom **potrošaču** električne energije može da se odredi snaga potrošnje, da se napravi kopija i da se upiše u izlazni tok (`it<<potr`).
- Električni **uređaj** je potrošač zadate snage i naziva vrste. U izlazni tok upisuje se u obliku *vrsta* (*snaga*).
- **Grupa** potrošača je potrošač koji se sastoji od proizvoljnog broja potrošača. Stvara se prazna posle čega se potrošači dodaju pojedinačno (`grupa+=potr`). Snaga grupe jednaka je zbiru snaga potrošača u grupi. U izlazni tok se piše u obliku [*potrošač*, ..., *potrošač*].
- **Izvor** električne energije može da razvije zadatu nazivnu snagu i sadrži zadati broj priključnih mesta za po jednog potrošača. Izvor se stvara bez priključenih potrošača i ne može da se kopira ni na koji način. Može da se dohvati nazivna snaga, trenutno opterećenje (snaga potrošnje priključenih potrošača) i broj priključnih mesta izvora, da se priključi potrošač na zadato priključno mesto (prenosi se pokazivač na potrošač i izvor postane vlasnik potrošača) i da se izvor upiše u izlazni tok (`it<<izv`). Greška je ako bi se priključivanjem potrošača izvor preoptereto ili ako je indeks izvan opsega. Ako se potrošač priključuje na zauzeto mesto, zatečeni potrošač na tom mestu se odveže od izvora. Ako se prilikom priključivanja umesto potrošača navodi 0 (NULL), oslobađa se odabrano priključno mesto izvora. Prilikom upisivanja u izlazni tok u prvom redu se piše nazivna snaga i trenutno opterećenje izvora, a u narednim redovima pišu se sadržani potrošači, po jedan red za svako priključno mesto.

Napisati na jeziku C++ **funkciju** za čitanje s glavnog ulaza jednog potrošača proizvoljnog tipa i **glavnu funkciju** koja čitajući podatke s glavnog ulaza napravi jedan izvor, dodaje potrošače dok se ne pročita "prazan" potrošač ili ne dođe do greške i ispiše na glavnom izlazu završno stanje izvora.

Rešenje:

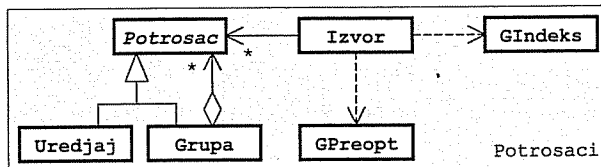
// potrosaci.h - Klasa električnih potrošača.

```
#ifndef _potrosaci_h_
#define _potrosaci_h_

#include <iostream>
using namespace std;
```

```
namespace Potrosaci {
    class Potrosac {
    public:
        virtual ~Potrosac() {} // Virtuelan destruktor.
        virtual double snaga() const =0; // Snaga potrošača.
        virtual Potrosac* kopija() const =0; // Kopiranje.
    private:
        virtual void pisi(ostream& it) const =0; // Pisanje.
        friend ostream& operator<<(ostream& it, const Potrosac& p)
        { p.pisi(it); return it; }
    }; // class
} // namespace

#endif
```



// uredjaj.h - Klasa električnih uređaja.

```
#ifndef _uredjaj_h_
#define _uredjaj_h_

#include "potrosaci.h"
#include <iostream>
#include <cstring>
using namespace std;

namespace Potrosaci {
    class Uredjaj: public Potrosac {
    public:
        char* naz; // Naziv uređaja.
        double sna; // Snaga uređaja.
        void kopiraj(const char* n, double s) { // Kopiranje u objekat.
            naz = new char [strlen(n)+1];
            strcpy(naz, n); sna = s;
        }
        void brisi() { delete [] naz; } // Brisanje objekta.
        void pisi(ostream&it) const // Pisanje.
        { it << naz << ' ' << sna << ' '; }
    public:
        Uredjaj(const char* n, double s) // Inicijalizacija.
        { kopiraj(n, s); }
        Uredjaj(const Uredjaj& u) // Konstruktor kopije.
        { kopiraj(u.naz,u.sna); }
        ~Uredjaj() { brisi(); } // Dstruktor.
        Uredjaj& operator=(const Uredjaj& u) { // Dodela vrednosti.
            if (this != &u) { brisi(); kopiraj(u.naz,u.sna); }
            return *this;
        }
        double snaga() const { return sna; } // Snaga uređaja.
        Uredjaj* kopija() const // Kopiranje.
        {return new Uredjaj(*this);}
    }; // class
} // namespace

#endif
```

// grupa.h - Klasa grupe električnih potrošača.

```
#ifndef _grupa_h_
#define _grupa_h_

#include "potrosaci.h"
#include <iostream>
using namespace std;

namespace Potrosaci {
    class Grupa: public Potrosac {
    public:
        struct Elem { // Element liste potrošača.
            Potrosac* potr; Elem* sled;
            Elem(Potrosac* p) { potr = p; sled = 0; }
            ~Elem() { delete potr; }
        };
        Elem *prvi, *posl; // Prvi i poslednji element.
        double sna; // Snaga grupe.
        void kopiraj(const Grupa& g); // Kopiranje u objekat.
        void brisi(); // Brisanje objekta.
    };
}
```

```

void pisi(ostream& it) const;           // Pisanje.
public:
    Grupa() { prvi = posl = 0; }        // Stvaranje prazne grupe.
    Grupa(const Grupa& g) { kopiraj(g); } // Konstruktor kopije.
    ~Grupa() { brisi(); }               // Destruktor.
    Grupa& operator=(const Grupa& g) {   // Dodela vrednosti.
        if (this != &g) { brisi(); kopiraj(g); }
        return *this;
    }
    Grupa& operator+=(const Potrosac& p) { // Dodavanje potrošača.
        Elem* novi = new Elem(p.kopija());
        posl = (!prvi ? prvi : posl->sled) = novi;
        sna += p.sna(); return *this;
    }
    double snaga() const { return sna; } // Snaga grupe.
    Grupa* kopija() const                // Kopiranje.
    { return new Grupa(*this); }
}; // class
} // namespace

#endif

```

// grupa.C - Metode klase grupa električnih potrošača.

```

#include "grupa.h"

namespace Potrosaci {
    void Grupa::kopiraj(const Grupa& g) { // Kopiranje u objekat.
        prvi = posl = 0;
        for (Elem* tek=g.prvi; tek; tek=tek->sled) {
            Elem* novi = new Elem(tek->potr->kopija());
            posl = (!prvi ? prvi : posl->sled) = novi;
        }
        sna = g.sna;
    }

    void Grupa::brisi() { // Brisanje objekta.
        while (prvi) { Elem* stari = prvi; prvi = prvi->sled; delete stari; }
        posl = 0;
    }

    void Grupa::pisi(ostream& it) const { // Pisanje.
        it << '[';
        for (Elem* tek=prvi; tek; tek=tek->sled)
            { it << *tek->potr; if (tek->sled) it << ','; }
        it << ']';
    }
} // namespace

```

// izvor.h - Klasa izvora električne energije.

```

#ifndef _izvor_h_
#define _izvor_h_

#include "potrosaci.h"
#include <iostream>
using namespace std;

```

```

namespace Potrosaci {
    class GIndeks{}; // KLASA GREŠAKA: Indeks izvan opsega.
    inline ostream& operator<<(ostream& it, const GIndeks&)
    { return it << "*** Nedoizvoljen indeks! ***"; }

    class GPreopt{}; // KLASA GREŠAKA: Preopterećen izvor.
    inline ostream& operator<<(ostream& it, const GPreopt&)
    { return it << "*** Preopterećen izvor! ***"; }

    class Izvor { // KLASA IZVORA:
        Potrosac** niz; // Niz potrošača.
        int kap; // Broj priključnih mesta.
        double sna, opt; // Nazivna snaga i trenutno opterećenje.
        Izvor(const Izvor&) {} // Ne sme da se kopira.
        void operator=(const Izvor&) {} // Ne sme da se dodeljuje.
    public:
        Izvor(double s, int k); // Inicijalizacija.
        ~Izvor(); // Destruktor.
        double snaga() const { return sna; } // Nazivna snaga izvora.
        double opterećenje() const { return opt; } // Trenutno opterećenje.
        int brMesta() const { return kap; } // Broj priključnih mesta.
        Izvor& prikljuci(Potrosac* p, int i); // Priključivanje potrošača.
        friend ostream& operator<<(ostream& it, const Izvor& izv); // Pisanje.
    }; // class
} // namespace

#endif

```

// izvor.C - Metode i funkcije uz klasu izvora električne energije.

```

#include "izvor.h"

namespace Potrosaci {
    Izvor::Izvor(double s, int k) { // Inicijalizacija.
        niz = new Potrosac* [kap = k];
        for (int i=0; i<kap; niz[i++]=0); sna = s; opt = 0;
    }

    Izvor::~Izvor() // Destruktor.
    { for (int i=0; i<kap; delete niz[i++]); delete [] niz; }

    Izvor& Izvor::prikljuci(Potrosac* p, int i) { // Priključivanje potrošača.
        if (i<0 || i>= kap) throw GIndeks();
        if (niz[i]) { opt-=niz[i]->snaga(); delete niz[i]; niz[i]=0; }
        if (p) {
            double s = p->snaga();
            if (opt+s > sna) throw GPreopt();
            niz[i] = p; opt += s;
        }
        return *this;
    }

    ostream& operator<<(ostream& it, const Izvor& izv) { // Pisanje.
        it << '\n' << izv.opt << '/' << izv.sna << endl;
        for (int i=0; i<izv.kap; i++) {
            it << i << ": ";
            if (izv.niz[i]) it << *izv.niz[i]; else it << "<< prazan >>";
            it << endl;
        }
        return it;
    }
} // namespace

```

```
// izvort.C - Ispitivanje klasa električnih potrošača i izvora.
#include "uredjaj.h"
#include "grupa.h"
#include "izvor.h"
using namespace Potrosaci;
#include <iostream>
using namespace std;

Potrosac* citaj() { // Čitanje potrošača:
    cout << "\nTip (U,G,..)? "; char izb; cin >> izb;
    switch (izb) {
        case 'u': case 'U': { // - uredjaj,
            cout << "Vrsta? "; char v[20]; cin >> v;
            cout << "Snaga? "; double s; cin >> s;
            return new Uredjaj(v, s);
        }
        case 'g': case 'G': { // - grupa uredjaja,
            Grupa* g = new Grupa();
            while (Potrosac* p=citaj()) { *g += *p; delete p; }
            return g;
        }
        default: return 0; // - "prazan" uredjaj.
    }
}

int main() { // Glavna funkcija.
    cout << "Snaga izvora? "; double s; cin >> s;
    cout << "Broj prikljucaka? "; int k; cin >> k;
    Izvor izv(s, k);
    try {
        while (true) {
            Potrosac* p = citaj();
            if (!p) break;
            cout << "Prikljucak? "; int i; cin >> i;
            izv.prikljuci(p, i);
        }
    } catch (GIndeks g) { cout << g << endl; }
    } catch (GPreopt g) { cout << g << endl; }
    }
    cout << izv;
    return 0;
}
```

```
% izvort
Snaga izvora? 1000
Broj prikljucaka? 5
```

```
Tip (U,G,..)? u
Vrsta? sijalica
Snaga? 100
Prikljucak? 3
```

```
Tip (U,G,..)? g
```

```
Tip (U,G,..)? u
Vrsta? tv
Snaga? 300
```

```
Tip (U,G,..)? u
Vrsta? radio
```

```
Snaga? 150
Tip (U,G,..)? .
Prikljucak? 1

Tip (U,G,..)? u
Vrsta? reso
Snaga? 1000
Prikljucak? 0
*** Preopterecen izvor! ***

550/1000
0: << prazan >>
1: [tv(300),radio(150)]
2: << prazan >>
3: sijalica(100)
4: << prazan >>
```

Zadatak 5.13 Funkcije za koje mogu da se stvaraju izvodi, delegati, monomi, eksponencijalne funkcije i zbrovi funkcija

Napisati na jeziku C++ sledeće klase (sve klase opremiti potrebnim konstruktorima, destruktorom i operatorom =):

- Za apstraktnu **funkciju** može da se izračuna vrednost za datu vrednost realne nezavisne promenljive x ($\text{fun}(x)$) i može da se stvori delegat koji sadrži njen prvi izvod. Predvideti stvaranje kopije funkcije i upisivanje algebarskog oblika funkcije u izlazni tok ($\text{it} \ll \text{fun}$).
- **Delegat** je funkcija koja sadrži pokazivač na funkciju u dinamičkoj zoni memorije, a koja je u isključivoj nadležnosti ove klase. Uloga klase je da omogući bezbedno uništavanje i kopiranje sadržane funkcije u dinamičkoj zoni memorije. Prvi izvod delegata sadrži prvi izvod funkcije u početnom delegatu. U izlazni tok upisuje se algebarski oblik sadržane funkcije.
- **Monom** je funkcija oblika ax^b (podrazumevano x), a prvi njen izvod je abx^{b-1} . Piše se u obliku $a*x^b$, gde su: a i b – vrednosti parametara monoma.
- **Eksponencijalna funkcija** je funkcija oblika ae^{bx} (podrazumevano e^x), a njen prvi izvod je abe^{bx} . Piše se u obliku $a*\exp(b*x)$, gde su: a i b – vrednosti parametara eksponencijalne funkcije.
- **Zbir funkcija** je funkcija koja može da sadrži zadat broj funkcija (podrazumevano 2). Stvara se prazan, posle čega funkcije mogu da se dodaju jedna po jedna ($\text{zbir} += \text{fun}$). Predvideti mogućnost stvaranja kopije funkcije, kao i preuzimanja pokazivača na funkciju. Pokušaj stavljanja funkcije u pun zbir funkcija prijavljuje se izuzetkom tipa specijalne jednostavne klase. Vrednost zbira funkcija je zbir vrednosti sadržanih funkcija. Prvi izvod zbira funkcija je zbir prvih izvoda sadržanih funkcija. Zbir funkcija piše se u obliku $(\text{fun}) + \dots + (\text{fun})$, gde je: fun – rezultat pisanja jedne sadržane funkcije.

Napisati na jeziku C++ **program** koji napravi jedan zbir funkcija kapaciteta koji se zadaje kao parametar programa, dodaje nekoliko prostih funkcija (monoma ili eksponencijalne funkcije) čitajući potrebne podatke s glavnog ulaza, ispiše algebarski oblik dobijenog zbira funkcija i njegovog prvog izvoda i posle tabelira vrednosti zbira funkcija i njegovog prvog izvoda za svako $x_{\min} \leq x \leq x_{\max}$ s korakom Δx . Parametri tabeliranja zadaju se kao parametri programa.

Rešenje:

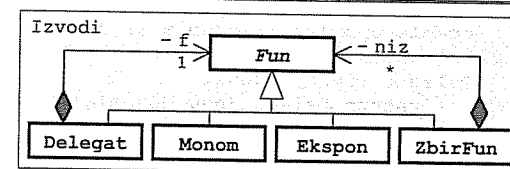
```
// fun3.h - Apstraktna
// klasa funkcija i
// klasa delegata.
```

```
#ifndef _fun3_h_
#define _fun3_h_
```

```
#include <iostream>
using namespace std;
```

```
namespace Izvodi {
    class Delegat; // Deklaracija klase delegata.
```

```
    class Fun { // DEFINICIJA KLASA FUNKCIJA:
    public:
        ~Fun() {} // Virtuelan destruktor.
        virtual double operator()(double x) const = 0; // Računanje vrednosti.
        virtual Delegat izvod() const = 0; // Stvaranje izvoda.
        virtual Fun* kopija() const = 0; // Kopiranje.
    private:
        virtual void pisi(ostream& it) const = 0; // Pisanje.
        friend ostream& operator<<(ostream& it, const Fun& f)
        { f.pisi(it); return it; }
    }; // class Fun
```



```

class Delegat: public Fun {           // DEFINICIJA KLASJE DELEGATA:
    Fun* f;                           // Sadržana funkcija.
    void pisi(ostream& it) const { it << *f; } // Pisanje.
public:
    Delegat(Fun* ff) { f = ff; }       // Inicijalizacija.
    Delegat(const Delegat& d) { f = d.f->kopija(); } // Konstruktor kopije.
    ~Delegat() { delete f; }           // Destruktor.
    Delegat& operator=(const Delegat& d) { // Dodela vrednosti.
        if (this != &d) { delete f; f = d.f->kopija(); }
        return *this;
    }
    double operator()(double x) const { return (*f)(x); } // Vrednost.
    Delegat izvod() const { return f->izvod(); } // Stvaranje izvoda.
    Delegat* kopija() const { return new Delegat(*this); } // Kopiranje.
}; // class Delegat
} // namespace Izvodi

#endif

```

```
// monom.h - Klasa monoma.
```

```

#ifndef _monom_h_
#define _monom_h_

#include "fun3.h"
#include <cmath>
using namespace std;

namespace Izvodi {
    class Monom: public Fun {
        double a, b; // Parametri monoma.
        void pisi(ostream& it) const // Pisanje.
        { it << a << "x^" << b; }
    public:
        Monom(double aa=1, double bb=1) // Inicijalizacija.
        { a = aa; b = bb; }
        double operator()(double x) const // Računanje vrednosti.
        { return a * pow(x, b); }
        Delegat izvod() const // Stvaranje izvoda.
        { return Delegat(new Monom(a*b, b-1)); }
        Monom* kopija() const // Kopiranje.
        { return new Monom(*this); }
    }; // class Monom
} // namespace Izvodi

#endif

```

```
// ekspon.h - Klasa eksponencijalnih funkcija.
```

```

#ifndef _ekspon_h_
#define _ekspon_h_

#include "fun3.h"
#include <cmath>
using namespace std;

namespace Izvodi {
    class Ekspon: public Fun {
        double a, b; // Parametri ekspon. funkcije.
        void pisi(ostream& it) const // Pisanje.
        { it << a << "exp(" << b << "x)"; }
    public:
        Ekspon(double aa=1, double bb=1) // Inicijalizacija.
        { a = aa; b = bb; }
        double operator()(double x) const // Računanje vrednosti.
        { return a * exp(b*x); }
        Delegat izvod() const // Stvaranje izvoda.
        { return Delegat(new Ekspon(a*b, b)); }
        Ekspon* kopija() const // Kopiranje.
        { return new Ekspon(*this); }
    }; // class Ekspon
} // namespace Izvodi

#endif

```

```
// zbifun.h - Klasa zbirova funkcija.
```

```

#ifndef _zbifun_h_
#define _zbifun_h_

#include "fun3.h"
#include <cmath>
using namespace std;

namespace Izvodi {
    class GZbirFunPun {}; // KLASA GREŠAKA: Zbir funkcija je pun.
    inline ostream& operator<<(ostream& it, GZbirFunPun)
    { return it << "*** Zbir funkcija je pun! ***\n"; }

    class ZbirFun: public Fun { // KLASA ZBIROVA FUNKCIJA:
        Fun** niz; // Niz funkcija.
        int kap, duz; // Kapacitet i broj popunjenih mesta.
        void pisi(ostream& it) const; // Pisanje.
        void kopiraj(const ZbirFun&); // Kopiranje u objekat.
        void brisi(); // Oslobađanje memorije.
    public:
        explicit ZbirFun(int n=2) // Inicijalizacija.
        { niz = new Fun* [kap = n]; duz = 0; }
        ZbirFun(const ZbirFun& z) { kopiraj(z); } // Konstruktor kopije.
        ~ZbirFun() { brisi(); } // Uništavanje.
        ZbirFun& operator=(const ZbirFun& z) { // Dodela vrednosti.
            if (this != &z) { brisi(); kopiraj(z); }
            return *this;
        }
        ZbirFun& operator+=(const Fun& f) { // Dodavanje funkcije
            if (duz == kap) throw GZbirFunPun(); // po vrednosti.
            niz[duz++] = f.kopija();
        }
    };
}

```

```

    return *this;
}
ZbirFun& operator+=(Fun* f) {           // Dodavanje funkcije
    if (duz == kap) throw GZbirFunPun(); // po adresi.
    niz[duz++] = f;
    return *this;
}
double operator()(double x) const;      // Računanje vrednosti.
Delegat izvod() const;                  // Stvaranje izvoda.
ZbirFun* kopija() const                  // Kopiranje.
{ return new ZbirFun(*this); }
}; // class ZbirFun
} // namespace Izvodi
#endif

```

// zbirfun.C - Metode klase zbirova funkcija.

```

#include "zbirfun.h"

namespace Izvodi {
    void ZbirFun::pisi(ostream& it) const { // Pisanje.
        for (int i=0; i<duz; i++) {
            if (i) it << '+';
            it << '(' << niz[i] << ')';
        }
    }

    void ZbirFun::kopiraj(const ZbirFun& z) { // Kopiranje u objekat.
        niz = new Fun* [kap = z.kap];
        duz = z.duz;
        for (int i=0; i<duz; i++) niz[i] = z.niz[i]->kopija();
    }

    void ZbirFun::brisi() { // Oslobođanje memorije.
        for (int i=0; i<duz; delete niz[i++]);
        delete [] niz;
    }

    double ZbirFun::operator()(double x) const { // Računanje vrednosti.
        double s = 0;
        for (int i=0; i<duz; s+=(*niz[i++])(x));
        return s;
    }

    Delegat ZbirFun::izvod() const { // Stvaranje izvoda.
        ZbirFun* z = new ZbirFun(kap);
        for (int i=0; i<duz; *z += niz[i++]->izvod());
        return Delegat(z);
    }
} // namespace Izvodi

```

// izvodt.C - Ispitivanje stvaranja izvoda.

```

#include "fun3.h"
#include "monom.h"
#include "ekspon.h"
#include "zbirfun.h"
using namespace Izvodi;
#include <iostream>
#include <cstdlib>
#include <new>
using namespace std;

int main(int, char** varg) {
    ZbirFun zbir(atoi(varg[1]));
    try {
        while (true) {
            double a, b; char vrs;
            cout << "Vrsta (M, E, *)? "; cin >> vrs;
            if (vrs == '*') break;
            switch (vrs) {
                case 'm': case 'M':
                    cout << "a,b? "; cin >> a >> b; zbir += Monom(a, b); break;
                case 'e': case 'E':
                    cout << "a,b? "; cin >> a >> b; zbir += Ekspon(a, b); break;
                default:
                    throw "*** Nedozvoljen izbor! ***\n";
            }
        }
        Delegat izvod = zbir.izvod();
        cout << "Fun= " << zbir << endl;
        cout << "Izv= " << izvod << endl;
        double xmin = atof(varg[2]), xmax = atof(varg[3]),
            dx = atof(varg[4]);
        for (double x=xmin; x<=xmax; x+=dx)
            cout << x << '\t' << zbir(x) << '\t' << izvod(x) << endl;
    } catch (GZbirFunPun) {
        cout << "*** Zbir funkcija je pun! ***\n";
    } catch (const char* g) {
        cout << g;
    } catch (bad_alloc) {
        cout << "*** Dodela memorije nije uspela! ***\n";
    }
    return 0;
}

```

```

% izvodt 5 -2 2 .5
Vrsta (M, E, *)? m
a,b? 4 2
Vrsta (M, E, *)? e
a,b? .5 -2
Vrsta (M, E, *)? m
a,b? 3 4
Vrsta (M, E, *)? *
Fun= (4*x^2)+(0.5*exp(-2*x))+(3*x^4)
Izv= (8*x^1)+(-1*exp(-2*x))+(12*x^3)

```

-2	91.2991	-166.598
-1.5	34.2303	-72.5855
-1	10.6945	-27.3891
-0.5	2.54664	-8.21828
0	0.5	-1
0.5	1.37144	5.13212
1	7.06767	19.8647
1.5	24.2124	52.4502
2	64.0092	111.982

6 Generičke funkcije i klase

Zadatak 6.1 Generička funkcija za fuziju uređenih nizova

Napisati na jeziku C++ generičku funkciju za nalaženje fuzije dva uređena niza objekata u treći, na isti način uređen niz.

Napisati na jeziku C++ program koji korišćenjem prethodne funkcije nalazi fuziju nizova:

- celih brojeva uređenih po vrednostima,
- tačaka u ravni uređenih po odstojanjima od koordinatnog početka, i
- pravougaonika uređenih po površinama.

Rešenje:

// fuzija.h - Generička funkcija za fuziju nizova objekata.

```
template <typename T>
void fuzija(const T a[], int na, const T b[], int nb, T c[], int& nc) {
    nc = 0;
    for (int ia=0, ib=0; ia<na || ib<nb; nc++)
        c[nc] = ia==na ? b[ib++] :
                ib==nb ? a[ia++] :
                a[ia]<b[ib] ? a[ia++] : b[ib++];
}
```

// tacka8.h - Klasa tačka.

```
#include <iostream>
using namespace std;

class Tacka {
    double x, y; // Koordinate.
public:
    friend istream& operator>>(istream& ut, Tacka& tt) // Čitanje.
    { return ut >> tt.x >> tt.y; }
    friend ostream& operator<<(ostream& it, const Tacka& tt) // Pisanje.
    { return it << "T(" << tt.x << ', ' << tt.y << ')'; }
    friend bool operator<(const Tacka& t1, const Tacka& t2) // Upoređivanje.
    { return t1.x*t1.x + t1.y*t1.y < t2.x*t2.x + t2.y*t2.y; }
};
```

// pravoug2.h - Klasa pravougaonika.

```
#include <iostream>
using namespace std;

class Pravoug {
    double a, b; // Dužine ivica.
public:
    friend istream& operator>>(istream& ut, Pravoug& pp) // Čitanje.
    { return ut >> pp.a >> pp.b; }
    friend ostream& operator<<(ostream& it, const Pravoug& pp) // Pisanje.
    { return it << "P[" << pp.a << ', ' << pp.b << ']'; }
    friend bool operator<(const Pravoug& p1, const Pravoug& p2) // Upoređivanje.
    { return p1.a*p1.b < p2.a*p2.b; }
};
```

// fuzijat1.C - Ispitivanje generičke funkcije za fuziju nizova.

```
#include <iostream>
using namespace std;
#include "fuzija.h"
#include "tacka8.h"
#include "pravoug2.h"

int main() {
    int na, nb, nc, i;

    {
        cout << "\nFuzija nizova celih brojeva:\n\n";
        cin >> na; int* a = new int [na]; cout << "Prvi niz:";
        for (i=0; i<na; i++) { cin >> a[i]; cout << ' ' << a[i]; } cout << endl;
        cin >> nb; int* b = new int [nb]; cout << "Drugi niz:";
        for (i=0; i<nb; i++) { cin >> b[i]; cout << ' ' << b[i]; } cout << endl;
        nc = na + nb; int* c = new int [nc]; cout << "Fuzija :";
        fuzija(a, na, b, nb, c, nc);
        for (i=0; i<nc; i++) { cout << ' ' << c[i]; } cout << endl;
        delete [] a; delete [] b; delete [] c;
    }

    {
        cout << "\nFuzija nizova tacaka:\n\n";
        cin >> na; Tacka* a = new Tacka [na]; cout << "Prvi niz:";
        for (i=0; i<na; i++) { cin >> a[i]; cout << ' ' << a[i]; } cout << endl;
        cin >> nb; Tacka* b = new Tacka [nb]; cout << "Drugi niz:";
        for (i=0; i<nb; i++) { cin >> b[i]; cout << ' ' << b[i]; } cout << endl;
        nc = na + nb; Tacka* c = new Tacka [nc]; cout << "Fuzija :";
        fuzija(a, na, b, nb, c, nc);
        for (i=0; i<nc; i++) { cout << ' ' << c[i]; } cout << endl;
        delete [] a; delete [] b; delete [] c;
    }

    {
        cout << "\nFuzija nizova pravougaonika:\n\n";
        cin >> na; Pravoug* a = new Pravoug [na]; cout << "Prvi niz:";
        for (i=0; i<na; i++) { cin >> a[i]; cout << ' ' << a[i]; } cout << endl;
        cin >> nb; Pravoug* b = new Pravoug [nb]; cout << "Drugi niz:";
        for (i=0; i<nb; i++) { cin >> b[i]; cout << ' ' << b[i]; } cout << endl;
        nc = na + nb; Pravoug* c = new Pravoug [nc]; cout << "Fuzija :";
        fuzija(a, na, b, nb, c, nc);
        for (i=0; i<nc; i++) { cout << ' ' << c[i]; } cout << endl;
        delete [] a; delete [] b; delete [] c;
    }

    return 0;
}
```

fuzijat.pod

```
8 -3 -1 0 2 2 5 6 6
5 -2 2 6 7 8
3 0 0 -2 -2 3 2
4 1 1 -1 2 2 -3 3 3
4 1 4 2 3 4 2 3 3
3 3 1 2 3 4 3
```

```
% fuzijat1 <fuzijat.pod

Fuzija nizova celih brojeva:

Prvi niz: -3 -1 0 2 2 5 6 6
Drugi niz: -2 2 6 7 8
Fuzija   : -3 -2 -1 0 2 2 2 5 6 6 6 7 8

Fuzija nizova tacaka:

Prvi niz: T(0,0) T(-2,-2) T(3,2)
Drugi niz: T(1,1) T(-1,2) T(2,-3) T(3,3)
Fuzija   : T(0,0) T(1,1) T(-1,2) T(-2,-2) T(2,-3) T(3,2) T(3,3)

Fuzija nizova pravougaonika:

Prvi niz: P[1,4] P[2,3] P[4,2] P[3,3]
Drugi niz: P[3,1] P[2,3] P[4,3]
Fuzija   : P[3,1] P[1,4] P[2,3] P[2,3] P[4,2] P[3,3] P[4,3]
```

```
// fuzijat2.C - Ispitivanje generičke funkcije za fuziju nizova.
```

```
#include <iostream>
using namespace std;
#include "fuzija.h"
#include "tacaka8.h"
#include "pravoug2.h"

template <typename T>
void radi(const char* naslov) {
    cout << naslov; int na, nb, nc, i;
    cin >> na; T* a = new T [na]; cout << "Prvi niz:";
    for (i=0; i<na; i++) { cin >> a[i]; cout << ' ' << a[i]; } cout << endl;
    cin >> nb; T* b = new T [nb]; cout << "Drugi niz:";
    for (i=0; i<nb; i++) { cin >> b[i]; cout << ' ' << b[i]; } cout << endl;
    nc = na + nb; T* c = new T [nc]; cout << "Fuzija   :";
    fuzija(a, na, b, nb, c, nc);
    for (i=0; i<nc; i++) { cout << ' ' << c[i]; } cout << endl;
    delete [] a; delete [] b; delete [] c;
}

int main() {
    radi<int> ("\\nFuzija nizova celih brojeva:\\n\\n");
    radi<Tacaka> ("\\nFuzija nizova tacaka:\\n\\n");
    radi<Pravoug> ("\\nFuzija nizova pravougaonika:\\n\\n");
    return 0;
}
```

Zadatak 6.2 Generički stekovi zadatih kapaciteta

Napisati na jeziku C++ generičku klasu stekova zadatih kapaciteta.

Napisati na jeziku C++ program za ispitivanje prethodne klase.

Rešenje:

```
// stek1.h - Generička klasa stekova zadatih kapaciteta.

template <typename Pod, int kap>
class Stek {
    Pod stek[kap]; // Stek u obliku niza.
    int vrh; // Indeks vrha steka.
public:
    Stek() { vrh = 0; } // Inicijalizacija.
    void stavi(const Pod& pod); // Smeštanje podatka.
    void uzmi(Pod& pod); // Uzimanje podatka.
    int vel() const { return vrh; } // Broj podataka na steku.
    void brisi() { vrh = 0; } // Pražnjenje steka.
    bool prazan() const { return vrh == 0; } // Indikator praznog steka.
    bool pun() const { return vrh == kap; } // Indikator punog steka.
};

// Definicije metoda klase Stek<>.

template <typename Pod, int kap>
void Stek<Pod,kap>::stavi(const Pod& pod) { // Smeštanje podatka.
    if (vrh == kap) throw "Stek je pun!";
    stek[vrh++] = pod;
}

template <typename Pod, int kap>
void Stek<Pod,kap>::uzmi(Pod& pod) { // Uzimanje podatka.
    if (vrh == 0) throw "Stek je prazan!";
    pod = stek[--vrh];
}
```

```
// steklt.C - Ispitivanje generičke klase stekova zadatih kapaciteta.
```

```
#include "stekl.h"
#include <iostream>
using namespace std;

int main() {
    Stek<int,10>   clb_stek; // Stek celih brojeva kapaciteta 10.
    Stek<double,3> dbl_stek; // Stek realnih brojeva kapaciteta 3.

    cout << "\nPunjenje celobrojnog steka:";
    int i = 0;
    while (!clb_stek.pun()) { clb_stek.stavi(i); cout << ' ' << i; i++; }
    cout << "\nPraznjenje celobrojnog steka:";
    while (!clb_stek.prazan()) { clb_stek.uzmi(i); cout << ' ' << i; }
    cout << endl;
    cout << "\nPunjenje realnog steka:";
    double d = 1.11;
    while (!dbl_stek.pun()) { dbl_stek.stavi(d); cout << ' ' << d; d+=1.11; }
    cout << "\nPraznjenje realnog steka:";
    while (!dbl_stek.prazan()) { dbl_stek.uzmi(d); cout << ' ' << d; }
    cout << endl;
    return 0;
}
```

```
% steklt
```

```
Punjenje celobrojnog steka: 0 1 2 3 4 5 6 7 8 9
Praznjenje celobrojnog steka: 9 8 7 6 5 4 3 2 1 0
```

```
Punjenje realnog steka: 1.11 2.22 3.33
Praznjenje realnog steka: 3.33 2.22 1.11
```

Zadatak 6.3 Generičke klase za upoređivanje podataka i uređivanje nizova podataka

Napisati na jeziku C++ generičke klase koje sadrže metodu za ispitivanje da li se podatak datog tipa nalazi ispred drugog podatka istog tipa po rastućem, odnosno po opadajućem poretку. Pretpostaviti da za odabrani tip podataka postoji operator < za upoređivanje podataka.

Napisati na jeziku C++ generičku klasu koja sadrži metodu za uređivanje niza podataka datog tipa. Strategija uređivanja treba da je parametar šablona. Podrazumevana strategija je po rastućem poretку.

Napisati na jeziku C++ program koji korišćenjem prethodnih klasa uređuje niz:

- celih brojeva uređenih po vrednostima,
- tačaka u ravni uređenih po odstojanjima od koordinatnog početka, i
- pravougaonika uređenih po površinama

po rastućem i po opadajućem redosledu.

Rešenje:

```
// raste.h - Generička klasa za rastuće upoređivanje podataka.
```

```
template <typename T>
class Raste {
public:
    static bool ispred(const T& a, const T& b) { return a < b; }
};
```

```
// opada.h - Generička klasa za opadajuće upoređivanje podataka.
```

```
template <typename T>
class Opada {
public:
    static bool ispred(const T& a, const T& b) { return b < a; }
};
```

```
// uredi2.h - Generička klasa za uređivanje niza podataka.
```

```
#include "raste.h"

template <typename T, class U=Raste<T> >
class Uredi { // Definicija klase.
public:
    static void uredi(T* a, int n);
};

template <typename T, class U>
void Uredi<T,U>::uredi(T* a, int n) { // Definicija metode.
    for (int i=0; i<n-1; i++)
        for (int j=i+1; j<n; j++)
            if (U::ispred(a[j],a[i]))
                { T b = a[i]; a[i] = a[j]; a[j] = b; }
}
```

// uredi2t.C - Ispitivanje generičke klase za uređivanje niza podataka.

```
#include "uredi2.h"
#include "opada.h"
#include "tacka8.h"
#include "pravoug2.h"
#include <iostream>
using namespace std;

template <typename T> // Generička funkcija za obradu datog tipa podataka.
void radi(const char* naslov) {
    cout << naslov;
    int n; cin >> n; T* a = new T [n];
    cout << "Pocetni niz:";
    for (int i=0; i<n; i++) { cin>>a[i]; cout<<' ' <<a[i]; } cout<<endl;
    Uredi<T>::uredi(a, n);
    cout << "Rastuci niz:";
    for (int i=0; i<n; i++) cout << ' ' << a[i]; cout << endl;
    Uredi<T,Opada<T>>::uredi(a, n);
    cout << "Opadajuci niz:";
    for (int i=0; i<n; i++) cout << ' ' << a[i]; cout << endl;
    delete [] a;
}

int main() { // Glavna funkcija.
    radi<int> ("\\nUredjivanje niza celih brojeva:\\n\\n");
    radi<Tacka> ("\\nUredjivanje niza tacaka:\\n\\n");
    radi<Pravoug> ("\\nUredjivanje niza parvougaonika:\\n\\n");
    return 0;
}
```

uredi2t.pod

```
13 2 5 -3 7 -1 6 0 8 2 6 -2 2 6
7 0 0 -2 -2 2 -3 3 3 3 2 1 1 -1 2
7 1 4 2 3 3 3 1 2 3 4 2 3 3 4
```

% uredi2t <uredi2t.pod

Uredjivanje niza celih brojeva:

```
Pocetni niz: 2 5 -3 7 -1 6 0 8 2 6 -2 2 6
Rastuci niz: -3 -2 -1 0 2 2 2 5 6 6 6 7 8
Opadajuci niz: 8 7 6 6 6 5 2 2 2 0 -1 -2 -3
```

Uredjivanje niza tacaka:

```
Pocetni niz: T(0,0) T(-2,-2) T(2,-3) T(3,3) T(3,2) T(1,1) T(-1,2)
Rastuci niz: T(0,0) T(1,1) T(-1,2) T(-2,-2) T(2,-3) T(3,2) T(3,3)
Opadajuci niz: T(3,3) T(3,2) T(2,-3) T(-2,-2) T(-1,2) T(1,1) T(0,0)
```

Uredjivanje niza parvougaonika:

```
Pocetni niz: P[1,4] P[2,3] P[3,3] P[1,2] P[3,4] P[2,3] P[3,4]
Rastuci niz: P[1,2] P[1,4] P[2,3] P[2,3] P[3,3] P[3,4] P[3,4]
Opadajuci niz: P[3,4] P[3,4] P[3,3] P[2,3] P[2,3] P[1,4] P[1,2]
```

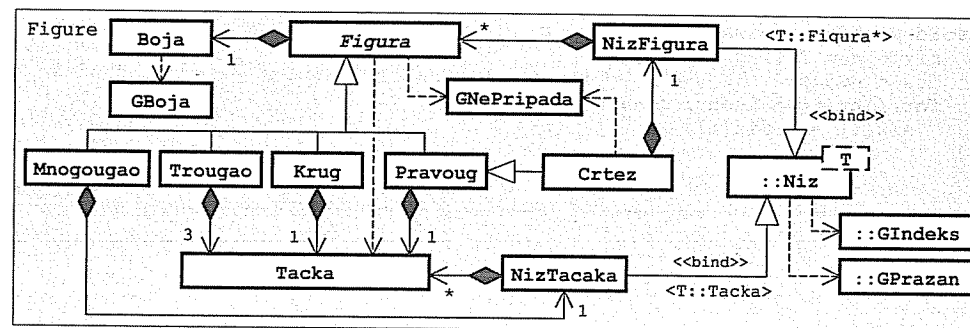
Zadatak 6.4 Generički nizovi, boje, tačke, obojene figure, krugovi, pravougaonici, trouglovi, mnogouglovi i crteži u ravni

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorom i operatorom za dodelu vrednosti koji su potrebni za bezbedno korišćenje klase; greške prijavljivati izuzecima tipa jednostavnih klasa koje su opremljene pisanjem teksta poruke):

- Generički **niz** stvara se prazan zadatog kapaciteta (podrazumevano 10) posle čega se elementi dodaju jedan po jedan na kraju niza (ako se niz prepuni, kapacitet se povećava za 5 mesta). Može da se dohvati broj elemenata niza, da se uzme poslednji element iz niza (greška je ako je niz prazan; ako broj praznih mesta postane veći od 10, kapacitet se smanji za 5 mesta) i da se pristupa elementu sa datim rednim brojem (niz[ind]; greška je ako je indeks izvan opsega).
- **Boja** se zadaje pomoću realnih intenziteta crvene, zelene i plave boje u opsegu od 0 do 1 (podrazumevano (1,1,1), tj. bela boja). Greška je ako je vrednost intenziteta neke od boja izvan navedenog opsega. Boja može da se upiše u izlazni tok (it<<b) u obliku (c,z,p), gde su: c, z i p – intenziteti komponentnih boja.
- **Tačka** u ravni zadaje se realnim koordinatama (podrazumevano (0,0)). Mogu da se dohvate koordinate tačke, da se izračuna rastojanje tačke od zadate tačke (podrazumevano od koordinatnog početka).
- Apstraktna popunjena **figura** u ravni ima jedinstven, automatski generisan celobrojan identifikator i boju. Može da se dohvati njen identifikator i boja, da joj se napravi kopija, da se ispita da li joj pripada neka tačka (smatra se da tačka pripada figuri ako leži u unutrašnjosti ili na ivici figure) i da se sazna koje je boje zadata tačka koja pripada figuri (greška je ako navedena tačka ne pripada figuri).
- **Krug** je figura koja se zadaje pomoću tačke koja predstavlja centar kruga i poluprečnikom (podrazumevano (0,0) i 1).
- **Pravougaonik** je figura koja ima ivice paralelne koordinatnim osama i zadaje se tačkom koja predstavlja donje levo teme pravougaonika i širinom i visinom (podrazumevano (0,0), 1 i 1).
- **Trougao** je figura koja se zadaje pomoću tri tačke koje predstavljaju temena trougla.
- **Mnogougao** je figura koja se zadaje pomoću niza tačaka koje predstavljaju temena mnogougla. Pretpostaviti da je mnogougao konveksan.
- **Crtež** je pravougaonik koji može da sadrži proizvoljan broj figura u obliku generičkog niza. Boja figure predstavlja boju podloge crteža. Koordinate sadržanih figura računaju se u odnosu na donje levo teme crteža. U slučaju preklapanja figura kasnije dodata figura prekriva ranije dodatu figuru (boju tačke u crtežu određuje figura koja je poslednja dodata crtežu i sadrži tu tačku – ako takve figure nema, uzima se boja podloge).

Napisati na jeziku C++ **program** koji napravi jedan crtež od nekoliko figura konstantnih parametara (ne treba ništa učitavati) i posle čitajući nekoliko parova koordinata tačaka ispisuje boju tih tačaka u crtežu.

Rešenje:



```
// niz4.h - Generička klasa nizova.

#ifndef _niz4_h_
#define _niz4_h_

#include <iostream>
using namespace std;

class GIndeks {}; // KLASA GREŠAKA: Nedoovoljen indeks.
inline ostream& operator<<(ostream& d, const GIndeks&)
{ return d << "*** Nedoovoljen indeks! ***"; }

class GPrazan {}; // KLASA GREŠAKA: Niz je prazan.
inline ostream& operator<<(ostream& d, const GPrazan&)
{ return d << "*** Niz je prazan! ***"; }

template <typename T> // GENERIČKA KLASA NIZOVA:
class Niz {
    T* niz; // Elementi niza.
    int kap, duz; // Kapacitet i dužina niza.
    void kopiraj(const Niz& n); // Kopiranje u objekat.
public:
    explicit Niz(int k=10) // Stvaranje praznog niza.
    { niz = new T [kap = k]; duz = 0; }
    Niz(const Niz& n) { kopiraj(n); } // Konstruktor kopije.
    ~Niz() { delete [] niz; } // Destruktor.
    Niz& operator=(const Niz& n) { // Dodela vrednosti.
        if (this != &n) { delete [] niz; kopiraj(n); }
        return *this;
    }
    int vel() const { return duz; } // Dužina niza.
    Niz& dodaj(const T& t); // Dodavanje na kraj.
    T uzmi(); // Uzimanje s kraja.
    T& operator[](int i) { // Dohvatanje elementa
        if (i<0 || i>=duz) throw GIndeks(); // - promenljivog niza,
        return niz[i];
    }
    const T& operator[](int i) const { // - nepromenljivog niza.
        if (i<0 || i>=duz) throw GIndeks();
        return niz[i];
    }
    Niz& isprazni() { duz = 0; return *this; } // Pražnjenje niza.
}; // class Niz<T>

template <typename T> // METODE KLASA NIZOVA:
void Niz<T>::kopiraj(const Niz& n) { // Kopiranje u objekat.
    niz = new T [kap = n.kap];
    duz = n.duz;
    for (int i=0; i<duz; i++)
        niz[i] = n.niz[i];
}
```

```
template <typename T>
Niz<T>& Niz<T>::dodaj(const T& t) { // Dodavanje na kraj.
    if (duz == kap) {
        T* pom = new T [kap+=5];
        for (int i=0; i<duz; i++) pom[i] = niz[i];
        delete [] niz; niz = pom;
    }
    niz[duz++] = t;
    return *this;
}

template <typename T>
T Niz<T>::uzmi() { // Uzimanje s kraja.
    if (!duz) throw GPrazan();
    T t = niz[--duz];
    if (kap-duz>10) {
        T* pom = new T [kap-=5];
        for (int i=0; i<duz; i++) pom[i] = niz[i];
        delete [] niz; niz = pom;
    }
    return t;
}

#endif
```

```
// boja.h - Klasa boja.
```

```
#ifndef _boja_h_
#define _boja_h_

#include <iostream>
using namespace std;

namespace Figure {
    class GBoja {}; // KLASA GREŠAKA: Neispravna boja.
    inline ostream& operator<<(ostream& it, const GBoja&)
    { return it << "*** Neispravna boja! ***"; }

    class Boja { // KLASA BOJA:
        double cc, zz, pp;
    public:
        Boja() { cc = zz = pp = 1; } // Konstruktor.
        Boja(double c, double z, double p) {
            if (c<0 || c>1 || z<0 || z>1 || p<0 || p>1)
                throw GBoja();
            cc = c; zz = z; pp = p;
        }
        friend ostream& operator<<(ostream& it, const Boja& b) // Pisanje.
        { return it << '(' << b.cc << ', ' << b.zz << ', ' << b.pp << ')'; }
    }; // class Boja
} // namespace Figure

#endif
```

```
// tacka9.h - Klasa tačka u ravni.
#ifndef _tacka9_h_
#define _tacka9_h_

#include <cmath>
using namespace std;

namespace Figure {
    class Tacka {
        double xx, yy; // Koordinate tačke.
    public:
        Tacka(double x=0, double y=0) // Konstruktor.
        { xx = x; yy = y; }
        double x() const { return xx; } // Apscisa tačke.
        double y() const { return yy; } // Ordinata tačke.
        double rastojanje(Tacka t=Tacka()) const { // Rastojanje do
            return sqrt(pow(xx-t.xx,2) + pow(yy-t.yy,2)); // zadate tačke.
        }
    }; // class Tacka
} // namespace Figure

#endif
```

```
// figura3.h - Apstraktna klasa obojenih figura u ravni.
#ifndef _figura3_h_
#define _figura3_h_

#include "boja.h"
#include "tacka9.h"
#include <iostream>
using namespace std;

namespace Figure {
    class GNePripada {}; // KLASA GREŠAKA: Tačka ne pripada figuri.
    inline ostream& operator<<(ostream& it, const GNePripada&)
    { return it << "*** Tacka ne pripada figuri! ***"; }

    class Figura { // KLASA FIGURA:
        static int posId; // Poslednji identifikator
        int id; // Identifikator figure.
        Boja b; // Osnovna boja figure.
    public:
        explicit Figura(Boja bb=Boja()) // Nova figura dobija
        : b(bb), id(++posId) {} // nov identifikator.
        Figura(const Figura& fig) // Kopija dobija nov
        : b(fig.b), id(++posId) {} // identifikator.
        Figura& operator=(const Figura& fig) // Levom operandu se ne
        { b = fig.b; return *this; } // menja identifikator.
        virtual ~Figura() {} // Virtuelan destruktor.
        virtual Figura* kopija() const =0; // Kopiranje.
        int dohvId() const { return id; } // Identifikator.
        Boja boja() const { return b; } // Osnovna boja figure.
        virtual bool pripada(const Tacka& T) const=0; // Da li tačka pripada?
        virtual Boja boja(const Tacka& T) const { // Boja tačke.
            if (!pripada(T)) throw GNePripada();
            return b;
        }
    }; // class Figura
} // namespace Figure

#endif
```

```
// figura3.C - Staticko polje klase figura u ravni.
```

```
#include "figura3.h"

int Figure::Figura::posId = 0;
```

```
// krug3.h - Klasa obojenih krugova u ravni.
```

```
#ifndef _krug3_h_
#define _krug3_h_

#include "figura3.h"

namespace Figure {
    class Krug: public Figura {
        Tacka C; // Centar.
        double r; // Poluprečnik.
    public:
        explicit Krug(Tacka cc=Tacka(), double rr=1, Boja bb=Boja())
        : Figura(bb), C(cc), r(rr) {}
        Krug* kopija() const { return new Krug(*this); } // Kopiranje.
        bool pripada(const Tacka& T) const // Da li tačka pripada?
        { return C.rastojanje(T) <= r; }
    }; // class Krug
} // namespace Figure

#endif
```

```
// pravoug3.h - Klasa obojenih pravougaonika u ravni.
```

```
#ifndef _pravoug3_h_
#define _pravoug3_h_

#include "figura3.h"

namespace Figure {
    class Pravoug: public Figura {
    protected:
        Tacka A; // Donje levo teme.
        double sir, vis; // Širina i visina.
    public:
        // Konstruktor.
        explicit Pravoug(Tacka P=Tacka(), double s=1, double v=1, Boja b=Boja())
        : Figura(b), A(P), sir(s), vis(v) {}
        Pravoug* kopija() const { return new Pravoug(*this); } // Kopiranje.
        bool pripada(const Tacka& T) const { // Da li tačka pripada?
            return A.x() <= T.x() && T.x() <= A.x()+sir &&
                A.y() <= T.y() && T.y() <= A.y()+vis;
        }
    }; // class Pravoug
} // namespace Figure

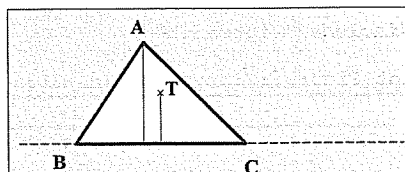
#endif
```

```
// trougao5.h - Klasa obojenih trouglova u ravni.
```

```
#ifndef _trougao5_h_
#define _trougao5_h_
```

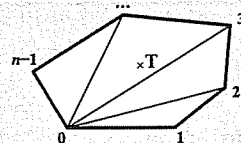
```
#include "figura3.h"
```

```
namespace Figure {
    class Trougao: public Figura {
        Tacka A, B, C; // Temena.
    public: // Konstruktor.
        Trougao(Tacka P, Tacka Q, Tacka R,
            Boja b=Boja())
            : Figura(b), A(P), B(Q), C(R) {}
        Trougao* kopija() const { return new Trougao(*this); } // Kopiranje.
        bool pripada(const Tacka& T) const { // Da li tačka pripada?
            return provera(T,A,B,C) && provera(T,B,C,A) && provera(T,C,A,B);
        }
    private:
        // Da li se tačke P i Q nalaze s iste strane duži AB?
        static bool provera(Tacka P, Tacka Q, Tacka A, Tacka B) {
            double t = B.x()-A.x(), u = B.y()-A.y();
            return ((P.x()-A.x())/t-(P.y()-A.y())/u)*
                ((Q.x()-A.x())/t-(Q.y()-A.y())/u) >=0;
        }
    }; // class Trougao
} // namespace Figure
#endif
```



Tačka T pripada trouglu ako za svako teme A, B i C važi da se to teme i tačka T nalaze s iste strane naspramne ivice.

$$p = \frac{x_P - x_A}{x_B - x_A} \frac{y_P - y_A}{y_B - y_A} \begin{cases} < 0 - \text{iznad} \\ = 0 - \text{na} \\ > 0 - \text{ispod} \end{cases}$$



Tačka T pripada konveksnom mnogouglu ako pripada nekom od trouglova dobijenih povlačenjem dijagonala iz jednog temena.

```
// mnogougao2.h - Klasa obojenih mnogouglova.
```

```
#ifndef _mnogougao2_h_
#define _mnogougao2_h_
```

```
#include "figura3.h"
#include "niz4.h"
```

```
namespace Figure {
    class Mnogougao: public Figura {
        Niz<Tacka> temena; // Temena mnogougla.
    public: // Konstruktor.
        explicit Mnogougao(const Niz<Tacka>& tem, Boja b=Boja())
            : Figura(b), temena(tem) {}
        Mnogougao* kopija() const { return new Mnogougao(*this); } // Poli. kop.
        bool pripada(const Tacka& T) const; // Da li tačka pripada?
    }; // class Mnogougao
} // namespace Figure
#endif
```

```
// mnogougao2.C - Metode klase mnogouglova u ravni.
```

```
#include "mnogougao2.h"
#include "trougao5.h"
```

```
bool Figure::Mnogougao::pripada(const Tacka& T) const { // Da li pripada?
    for (int i=1; i<temena.vel()-1; i++)
        if (Trougao(temena[0],temena[i],temena[i+1]).pripada(T)) return true;
    return false;
}
```

```
// crtez1.h - Klasa obojenih crteža.
```

```
#ifndef _crtez1_h_
#define _crtez1_h_
```

```
#include "pravoug3.h"
#include "niz4.h"
```

```
namespace Figure {
    class Crtez: public Pravoug {
        Niz<Figura*> figure; // Figure u crtežu.
    public: // Konstruktor.
        void kopiraj(const Crtez& crt); // Kopiranje crteža.
        void brisi(); // Uništavanje crteža.
        Crtez(const Tacka& T, // Konstruktori.
            double sir, double vis, Boja bb=Boja())
            : Pravoug(T, sir, vis, bb) {}
        Crtez(const Crtez& crt): Pravoug(crt) { kopiraj(crt); }
        ~Crtez() { brisi(); } // Destrutor.
        Crtez& operator=(const Crtez& crt) { // Dodela vrednosti.
            if (this != &crt) {
                brisi(); Pravoug::operator=(crt); kopiraj(crt);
            }
            return *this;
        }
        Crtez* kopija() const { return new Crtez(*this); } // Kopiranje.
        Crtez& dodaj(const Figura& fig) // Dodavanje figure kopiranjem.
        { figure.dodaj(fig.kopija()); return *this; }
        Crtez& dodaj(Figura* fig) // Dodavanje figure preuzimanjem.
        { figure.dodaj(fig); return *this; }
        Boja boja(const Tacka& T) const; // Boja tačke.
    }; // class Crtez
} // namespace Figure
#endif
```

```
// crtez1.C - Metode klase crteza.
```

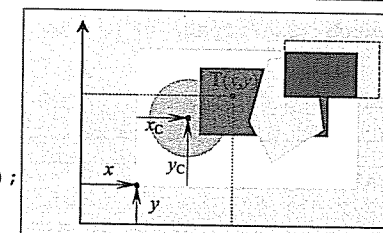
```
#include "crtez1.h"
```

```
// Kopiranje crteža.
void Figure::Crtez::kopiraj(const Crtez& crt) {
    for (int i=0; i<crt.figure.vel(); i++)
        figure.dodaj(crt.figure[i]->kopija());
}

// Uništavanje crteža.
void Figure::Crtez::brisi() {
    for (int i=0; i<figure.vel(); i++)
        delete figure[i++];
    figure.isprazni();
}
```

```
// Boja tačke.
```

```
Figure::Boja Figure::Crtez::boja(const Tacka& T) const {
    if (!pripada(T)) throw GNePripada();
    Tacka T1(T.x()-A.x(), T.y()-A.y());
    for (int i=figure.vel()-1; i>=0; i--)
        try { return figure[i]->boja(T1); } catch (GNePripada) {}
    return Figura::boja();
}
```



- Koordinate tačke T(x,y) treba preračunati u odnosu na donji levi ugao crteža.
- Boju tačke određuje prva figura po obrnutom redosledu u odnosu na redosled dodavanja.

// figura3t.C - Ispitivanje klasa obojenih figura u ravni.

```
#include "boja.h"
#include "tacka9.h"
#include "krug3.h"
#include "pravoug3.h"
#include "trougao5.h"
#include "mnougao2.h"
#include "crtez1.h"
using namespace Figure;
#include <iostream>
using namespace std;

int main() {
    Crtez crt(Tacka(3,1), 15, 12, Boja(.9,.9,.9));
    crt.dodaj(new Pravoug(Tacka(-2,5), 11, 3, Boja(1,0,0)))
        .dodaj(new Mnougao(Niz<Tacka>().dodaj(Tacka(8,2))
            .dodaj(Tacka(11,4))
            .dodaj(Tacka(12,8))
            .dodaj(Tacka(7,7))
            .dodaj(Tacka(6,4)),
            Boja(0,1,0)
        ))
        .dodaj(&(new Crtez(Tacka(9,6), 9, 6, Boja(.5,.5,.5)))
            ->dodaj(new Trougao(Tacka(1,1),Tacka(5,1),Tacka(1,3),
                Boja(0,1,1)))
            .dodaj(new Pravoug(Tacka(4,4), 4, 2, Boja(1,0,1)))
        )
        .dodaj(new Krug(Tacka(5,6), 3, Boja(0,0,1)));

    while (true) {
        cout << "Tacka (x,y)? "; double x, y; cin >> x >> y;
        if (x == 1E38) break;
        try {
            cout << "Boja tacke: " << crt.boja(Tacka(x,y)) << endl;
        } catch (GNePripada g) { cout << g << '\n'; }
    }

    return 0;
}
```

```
% figura3t
Tacka (x,y)? 6 2
Boja tacke: (0.9,0.9,0.9)
Tacka (x,y)? 14 8.5
Boja tacke: (0,1,0)
Tacka (x,y)? 17 9
Boja tacke: (0.5,0.5,0.5)
Tacka (x,y)? 2 4
*** Tacka ne pripada figuri! ***
Tacka (x,y)? 1e38 0
```

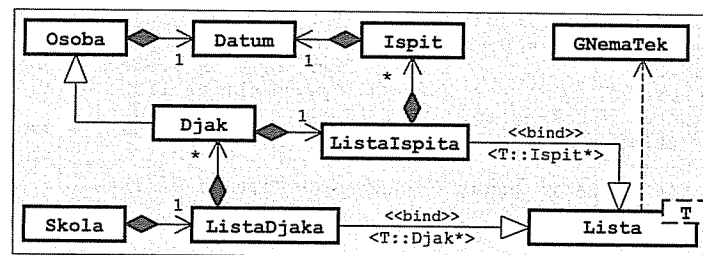
Zadatak 6.5 Generičke liste; datumi, osobe, ispiti, đaci i škole

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorom i operatorom za dodelu vrednosti koji su potrebni za bezbedno korišćenje klasa; greške prijavljivati izuzecima tipa jednostavnih klasa koje su opremljene pisanjem teksta poruke):

- Generička **lista** podataka nekog tipa stvara se prazna posle čega se podaci dodaju jedan po jedan na kraju liste. Može da se postavi na prvi element liste, da se prelazi na sledeći element u odnosu na tekući, da se ispita da li postoji tekući element, da se pristupi podatku u tekućem elementu sa mogućnošću promene vrednosti podatka i da se tekući element izbaci iz liste (pri tome sledeći element postaje tekući). Greška je ako tekući element ne postoji u momentu pokušaja pristupanja elementu ili izbacivanja elementa.
- **Datum** se zadaje pomoću dana, meseca i godine. Pri stvaranju datuma nije potrebno proveravati ispravnost parametara. Komponente datuma mogu da se dohvate. Datum može da se upiše u izlazni tok (it<<datum).
- Za **osobu** se zna ime i datum rođenja. Podaci o osobi mogu da se dohvataju, da se promene i mogu da se upišu u izlazni tok (it<<osoba). Osoba ne sme da se kopira ni na koji način.
- **Ispit** se zadaje pomoću jednoslovne šifre predmeta, dobijene ocene i datuma polaganja ispita. Moguće ocene su od 0 do 10 (ne treba proveravati). Ispit je položen ako je ocena 6 ili viša. Podaci o ispitu mogu da se dohvataju. Ispit ne sme da se kopira ni na koji način.
- **Đak** je osoba koja je polagala izvestan broj ispita. Stvara se s praznom listom ispita posle čega se ispiti dodaju jedan po jedan (djak+=&ispit; đak postane vlasnik ispita). Može da se odredi srednja ocena položenih ispita. U izlazni tok se pored opštih podataka piše i srednja ocena položenih ispita.
- **Škola** ima ime i sadrži listu đaka koji pohađaju školu. Stvara se prazna posle čega se đaci dodaju jedan po jedan (škola+=djak; škola ne postane vlasnik đaka). Može da se dohvati đak s najvišom srednjom ocenom položenih ispita bez mogućnosti promene podataka o tom đaku. Škola ne sme da se kopira ni na koji način.

Napisati na jeziku C++ **program** koji napravi školu s dva đaka sa po tri polagana ispita i ispiše na glavnom izlazu podatke o đaku s boljim prosekom.

Rešenje:




```
// lista7.h - Generičke klasa listi.

#ifndef _lista7_h_
#define _lista7_h_

#include <iostream>
using namespace std;

class GNemaTek {}; // KLASA GREŠAKA: Nema tekućeg elementa.
inline ostream& operator<<(ostream& it, const GNemaTek&)
{ return it << "*** Nema tekućeg elementa! ***"; }

template <typename T> // GENERIČKA KLASA LISTI:
class Lista {
    struct Elem { // Element liste:
        T pod; // - sadržani podatak,
        Elem* sled; // - sledeći element,
        Elem(const T& p, Elem* s=0) // - konstruktor.
        { pod = p; sled = s; }
    };
    Elem *prvi, *posl; // Prvi i poslednji element.
    mutable Elem *tek, *pret; // Tekući i prethodni element.
    void kopiraj(const Lista& lst); // Kopiranje u objekat.
public:
    Lista() // Stvaranje prazne liste.
    { prvi = posl = tek = pret = 0; }
    Lista(const Lista& lst) { kopiraj(lst); } // Konstruktor kopije.
    ~Lista() { isprazni(); } // Destruktor.
    Lista& operator=(const Lista& lst) { // Dodela vrednosti.
        if (this != &lst) { isprazni(); kopiraj(lst); }
        return *this;
    }
    Lista& dodaj(const T& t) { // Dodavanje na kraj.
        posl = (!prvi ? prvi : posl->sled) = new Elem(t);
        return *this;
    }
    Lista& naPrvi() // Postavljanje na prvi element:
    { tek = prvi; pret = 0; return *this; } // - promenljive liste,
    const Lista& naPrvi() const
    { tek = prvi; pret = 0; return *this; } // - nepromenljive liste.
    Lista& naSled() { // Pomeranje na sledeći element:
        pret = tek; // - promenljive liste,
        if (tek) tek = tek->sled;
        return *this;
    }
    const Lista& naSled() const { // - nepromenljive liste.
        pret = tek;
        if (tek) tek = tek->sled;
        return *this;
    }
    bool imaTek() const { return tek != 0; } // Ima li tekućeg?
    T& dohvTek() { // Pristup tekućem podatku:
        if (!tek) throw GNemaTek(); // - promenljive liste,
        return tek->pod;
    }
    const T& dohvTek() const { // - nepromenljive liste.
        if (!tek) throw GNemaTek();
        return tek->pod;
    }
};
```

```
Lista& izbaciTek() { // Izbacivanje tekućeg elementa.
    if (!tek) throw GNemaTek();
    Elem* stari = tek;
    tek = tek->sled;
    (!pret ? prvi : pret->sled) = tek;
    if (!tek) posl = pret;
    delete stari;
    return *this;
}
Lista& isprazni(); // Pražnjenje liste.
}; // class Lista<T>

template <typename T> // METODE KLASA LISTI:
void Lista<T>::kopiraj(const Lista& lst) { // Kopiranje u objekat.
    prvi = posl = tek = pret = 0;
    for (Elem* pok=lst.prvi; pok; pok=pok->sled) {
        Elem* novi = new Elem(pok->pod);
        posl = (!prvi ? prvi : posl->sled) = novi;
        if (pok == lst.tek) tek = novi;
        if (pok == lst.pret) pret = novi;
    }
}

template <typename T>
Lista<T>& Lista<T>::isprazni() { // Pražnjenje liste.
    while (prvi) { Elem* stari = prvi; prvi = prvi->sled; delete stari; }
    posl = tek = pret = 0;
    return *this;
}

#endif

// datum3.h - Klasa kalendarskih datuma.

#ifndef _datum3_h_
#define _datum3_h_

#include <iostream>
using namespace std;

class Datum {
    short d, m, g; // Dan, mesec i godina.
public:
    Datum(short dd, short mm, short gg) // Stvaranje datuma.
    { d = dd; m = mm; g = gg; }
    Datum() { d = m = g = 0; } // Prazan datum.
    short dan() const { return d; } // Dohvatanje delova datuma.
    short mes() const { return m; }
    short god() const { return g; }
    friend ostream& operator<<(ostream& it, const Datum& dat) // Pisanje.
    { return it << dat.d << '.' << dat.m << '.' << dat.g << '.'; }
};

#endif
```

```
// osoba4.h - Klasa osoba.

#ifndef _osoba4_h_
#define _osoba4_h_

#include "datum3.h"
#include <cstring>
#include <iostream>
using namespace std;

class Osoba {
    char* ime; // Ime.
    Datum rodj; // Datum rodenja.
    Osoba(const Osoba&) {} // Ne sme da se kopira.
    void operator=(const Osoba&) {} // Ne sme da se dodeljuje.
public:
    Osoba(const char* ii, const Datum& rr) { // Konstruktor.
        ime = strcpy(new char [strlen(ii)+1],ii);
        rodj = rr;
    }
    virtual ~Osoba() { delete [] ime; } // Destruktor.
    Osoba& staviIme(const char* ii) { // Menjanje imena.
        delete [] ime;
        ime = strcpy(new char [strlen(ii)+1],ii);
        return *this;
    }
    Osoba& staviRodj(const Datum& rr) // Menjanje datuma rod.
    { rodj = rr; return *this; }
    const char* dohvIme() const { return ime; } // Dohvatanje imena.
    const Datum& dohvRodj() const { return rodj; } // Dohvatanje dat. rod.
protected:
    virtual void pisi(ostream& it) const // Pisanje.
    { it << ime << " (" << rodj << ')'; }
    friend ostream& operator<<(ostream& it, const Osoba& oso)
    { oso.pisi(it); return it; }
};

#endif
```

```
// ispit.h - Klasa ispita.
```

```
#ifndef _ispit_h_
#define _ispit_h_

class Ispit {
    char sif; // Šifra ispita.
    short oce; // Ocena.
    Datum pol; // Datum polaganja.
    Ispit(const Ispit&) {} // Ne sme da se kopira.
    void operator=(const Ispit&) {} // Ne sme da se dodeljuje.
public:
    Ispit(char si, short oc, Datum po) // Konstruktor.
    { sif = si; oce = oc; pol = po; }
    char dohvSif() const { return sif; } // Dohvatanje šifre.
    short dohvOce() const { return oce; } // Dohvatanje ocene.
    Datum dohvPol() const { return pol; } // Dohvatanje datuma polaganja.
};

#endif
```

```
// djak.h - Klasa daka.
```

```
#ifndef _djak_h_
#define _djak_h_

#include "osoba4.h"
#include "lista7.h"
#include "ispit.h"
#include <iostream>
using namespace std;

class Djak: public Osoba {
    Lista<Ispit*> ispiti; // Lista ispita.
    void pisi(ostream& it) const // Pisanje.
    { Osoba::pisi(it); it << ": " << srOce(); }
public:
    Djak(const char* ime, const Datum& rodj) // Konstruktor.
    : Osoba(ime, rodj) {}
    ~Djak(); // Destruktor.
    Djak& operator+=(Ispit* isp) // Dodavanje ispita.
    { ispiti.dodaj(isp); return *this; }
    double srOce() const; // Srednja ocena.
};

#endif
```

```
// djak.C - Metode klase daka.
```

```
#include "djak.h"

Djak::~Djak() { // Destruktor.
    for (ispiti.naPrvi(); ispiti.imaTek(); ispiti.naSled())
        delete ispiti.dohvTek();
}

double Djak::srOce() const { // Srednja ocena.
    double s = 0; int n = 0;
    for (ispiti.naPrvi(); ispiti.imaTek(); ispiti.naSled()) {
        int k = ispiti.dohvTek()->dohvOce();
        if (k > 5) { s += k; n++; }
    }
    return n ? s/n : 0;
}
```

```
// skola.h - Klasa škola.
```

```
#ifndef _skola_h_
#define _skola_h_

#include "lista7.h"
#include "djak.h"
#include <cstring>
using namespace std;

class Skola {
    char* ime; // Ime škole.
    Lista<Djak*> djaci; // Lista daka.
    Skola(const Skola&); // Ne sme da se kopira.
    void operator=(const Skola&) {} // Ne sme da se dodeljuje.
};
```

```

public:
    Skola(const char* ii)           // Konstruktor.
    { ime = strcpy(new char [strlen(ii)+1], ii); }
    ~Skola() { delete [] ime; }    // Destruktor.
    Skola& operator+=(Djak& djak)  // Dodavanje daka.
    { djaci.dodaj(&djak); return *this; }
    const Djak* najbolji() const;  // Dak s najvišim prosekom.
};

#endif

```

// skola.C - Metode klase škola.

```

#include "skola.h"

const Djak* Skola::najbolji() const { // Dak s najvišim prosekom.
    Djak* naj = 0; double max = 0;
    for (djaci.naPrvi(); djaci.imaTek(); djaci.naSled()) {
        Djak* tek = djaci.dohvTek();
        double m = tek->srOce();
        if (m > max) { max = m; naj = tek; }
    }
    return naj;
}

```

// skolat.C - Ispitivanje klasa za obradu škola.

```

#include "djak.h"
#include "skola.h"
#include <iostream>
using namespace std;

int main() {
    Skola sk("ETF, Beograd");
    Djak marko("Marko", Datum(12,5,1986));
    Djak zoran("Zoran", Datum(31,7,1984));
    sk += marko; sk += zoran;
    marko += new Ispit('a', 10, Datum(12, 6,2006));
    marko += new Ispit('b', 7, Datum(24, 6,2006));
    marko += new Ispit('c', 5, Datum( 1, 7,2006));
    zoran += new Ispit('c', 9, Datum( 1, 7,2006));
    zoran += new Ispit('a', 10, Datum(12, 6,2006));
    zoran += new Ispit('d', 7, Datum(20, 6,2006));
    cout << *sk.najbolji() << '\n';
    return 0;
}

```

```

% skolat
Zoran (31.7.1984.): 8.66667

```

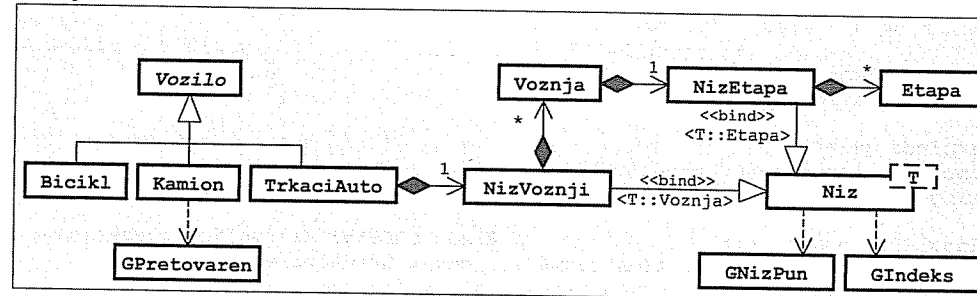
Zadatak 6.6 Vozila, bicikli, kamioni, generički nizovi, etape, vožnje i trkački automobili

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorem i operatorom za dodelu vrednosti koji su potrebni za bezbedno korišćenje klasa; greške prijavljivati izuzecima tipa jednostavnih klasa koje su opremljene pisanjem teksta poruke):

- Apstraktno **vozilo** ima zadatu sopstvenu težinu. Može da se dohvati naziv vrste vozila, da se odredi težina vozila i da se vozilo upiše u izlazni tok (`it<<vozilo`). Piše se naziv vrste vozila i sopstvena težina vozila.
- **Bicikl** je vozilo.
- **Kamion** je vozilo zadate sopstvene težine i nosivosti. Stvara se bez tovara posle čega može da se doda tovar zadate težine (`kamion+=tovar`) i da se skine tovar zadate težine (`kamion-=tovar`). Greška je ako se kamion pretovari. Skidanjem previše tovara težina tovara postane jednaka nuli. U izlazni tok upisuje se i trenutna težina tovara na kamionu.
- Generički **niz** može da sadrži elemente nekog tipa. Stvara se prazan zadatog kapaciteta (podrazumevano 20) posle čega elementi mogu da se dodaju pojedinačno (`niz+=element`). Greška je ako se niz prepuni. Može da se dohvati broj elemenata u nizu, da se pristupi elementu zadatog rednog broja (`niz[i]`) i da se niz isprazni. Greška je ako se pokuša pristupiti nepostojećem elementu.
- **Etapa** vožnje zadaje se pomoću realne dužine etape i brzine vožnje koja se ne menja u etapi. Mogu da se dohvate parametri etape i da se izračuna vreme kretanja u etapi.
- **Vožnja** sadrži generisan niz etapa zadatog kapaciteta (podrazumevano 10). Stvara se prazna posle čega se etape dodaju pojedinačno (`voznja+=etapa`). Može da se odredi ukupna dužina vožnje, ukupno trajanje i srednja brzina kretanja u toku vožnje, da se vožnja upiše u izlazni tok (`it<<voznja`). Piše se dužina, trajanje i srednja brzina vožnje.
- **Trkački auto** je vozilo koje sadrži generisan niz vožnji. Stvara se s praznim nizom zadatog kapaciteta. Može da se započne nova vožnja zadatog kapaciteta za etape, da se poslednje započetoj vožnji doda nova etapa, da se odredi vožnja s najvećom srednjom brzinom. U izlazni tok upisuje se i vožnja s najvećom srednjom brzinom.

Napisati na jeziku C++ **program** koji napravi trkački auto s dve vožnje koje sadrže po tri etape, ispiše auto na glavnom izlazu kao i vožnju s najvećom srednjom brzinom. Koristiti konstantne parametre (ne treba ništa učitivati s glavnog ulaza).

Rešenje:



```
// vozilo3.h - Apstraktna klasa vozila.

#ifndef _vozilo3_h_
#define _vozilo3_h_

#include <iostream>
using namespace std;

class Vozilo {
    float sopTez; // Sopstvena težina.
public:
    Vozilo(float tez) { sopTez = tez; } // Inicijalizacija.
    virtual ~Vozilo() {} // Virtuelan destruktor.
    virtual const char* vrsta() const = 0; // Naziv vrste vozila.
    virtual float tezina() const { return sopTez; } // Težina vozila.
protected:
    virtual void pisi(ostream& it) const // Pisanje.
    { it << vrsta() << ' ' << sopTez; }
    friend ostream& operator<<(ostream& it, const Vozilo& v)
    { v.pisi(it); return it; }
};

#endif
```

```
// bicikl.h - Definicija klase bicikala.
```

```
#ifndef _bicikl_h_
#define _bicikl_h_

#include "vozilo3.h"

class Bicikl: public Vozilo {
public:
    explicit Bicikl(float tez): Vozilo(tez) {} // Inicijalizacija.
    const char* vrsta() const { return "Bicikl"; } // Naziv vrste vozila.
};

#endif
```

```
// kamion.h - Klasa kamiona.
```

```
#ifndef _kamion_h_
#define _kamion_h_

#include "vozilo3.h"
#include <iostream>
using namespace std;

class GPretovaren {}; // KLASA GREŠKE: Kamion je pretovaren.
inline ostream& operator<<(ostream& it, const GPretovaren&) {
    return it << "*** Kamion je pretovaren! ***"; }

class Kamion: public Vozilo { // KLASA KAMIONA:
    float nos, tov; // Nosivost i trenutna težina tovara.
public:
    Kamion(float sTez, float ns): Vozilo(sTez) // Stvaranje praznog kamiona.
    { nos = ns; tov = 0; }
    const char* vrsta() const { return "Kamion"; } // Naziv vrste vozila.
    float tezina() const { return Vozilo::tezina() + tov; } // Težina vozila.
};
```

```
Kamion& operator+=(float tovar) { // Dodavanje tovara.
    if (tov+tovar > nos) throw GPretovaren();
    tov += tovar;
    return *this;
}

Kamion& operator-=(float tovar) { // Skidanje tovara.
    tov -= tovar;
    if (tov<0) tov = 0;
    return *this;
}

private:
    void pisi(ostream& it) // Pisanje.
    { Vozilo::pisi(it); it << ' ' << tov; }
};

#endif
```

```
// niz5.h - Generička klasa nizova.
```

```
#ifndef _niz5_h_
#define _niz5_h_

#include <iostream>
using namespace std;

class GNizPun {}; // KLASA GREŠAKA: Niz je prepunjen.
inline ostream& operator<<(ostream& it, const GNizPun&)
{ return it << "*** Niz je pun! ***"; }

class GIndeks {}; // KLASA GREŠAKA: Indeks izvan opsega.
inline ostream& operator<<(ostream& it, const GIndeks&)
{ return it << "*** Nedoovoljen indeks! ***"; }

template <typename T> // GENERIČKA KLASA NIZOVA:
class Niz {
    T* niz; // Elementi niza.
    int kap, duz; // Kapacitet i dužina niza.
    void kopiraj(const Niz& n); // Kopiranje u objekat.
    void brisi() { delete [] niz; } // Brisanje objekta.
public:
    explicit Niz(int k=20) { // Stvaranje praznog niza.
        niz = new T [kap = k];
        duz = 0;
    }
    Niz(const Niz& n) { kopiraj(n); } // Konstruktor kopije.
    ~Niz() { brisi(); } // Dstruktor.
    Niz& operator=(const Niz& n) { // Dodela vrednosti.
        if (this!=&n) { brisi(); kopiraj(n); }
        return *this;
    }
    int duzina() const { return duz; } // Dužina niza.
    Niz& operator+=(const T& e) { // Dodavanje elementa.
        if (duz == kap) throw GNizPun();
        niz[duz++] = e;
        return *this;
    }
};
```

```

T& operator[](int i) {                // Pristup elementu
    if (i<0 || i>=duz) throw GIndeks(); // - promenljivog niza,
    return niz[i];
}
const T& operator[](int i) const {      // - nepromenljivog niza.
    if (i<0 || i>=duz) throw GIndeks();
    return niz[i];
}
};

template <typename T>                  // METODE KLASJE NIZOVA:
void Niz<T>::kopiraj(const Niz& n) {    // Kopiranje u objekat.
    niz = new T [kap = n.kap];
    duz = n.duz;
    for (int i=0; i<duz; i++) niz[i] = n.niz[i];
}

#endif

```

// etapa.h - Klasa etapa voznje.

```

#ifndef _etapa_h_
#define _etapa_h_

class Etapa {
    float duz, brz;                // Dužina etape i brzina voznje.
public:
    Etapa () { duz = brz = 0; }      // "Prazna" etapa.
    Etapa(float d, float b) { duz = d, brz = b; } // Inicijalizacija.
    float duzina() const { return duz; } // Dužina etape.
    float brzina() const { return brz; } // Brzina voznje.
    float vreme() const { return duz / brz; } // Vreme voznje.
};

#endif

```

// voznja.h - Klasa voznji.

```

#ifndef _voznja_h_
#define _voznja_h_

#include "niz5.h"
#include "etapa.h"
#include <iostream>
using namespace std;

class Voznja {
    Niz<Etapa> etape;                // Niz etapa voznje.
public:
    explicit Voznja(int k=10): etape(10) {} // Stvaranje prazne voznje.
    Voznja& operator+=(const Etapa& e) {    // Dodavanje etape.
        etape += e;
        return *this;
    }
    float duzina() const;              // Dužina voznje.
    float trajanje() const;            // Vreme voznje.
    float srBrzina() const             // Srednja brzina voznje.
    { return duzina() / trajanje(); }
}

```

```

friend ostream& operator<<(ostream& it, const Voznja& v) { // Pisanje.
    return it << "(s=" << v.duzina() << ",t=" << v.trajanje()
        << ",v=" << v.srBrzina() << ')';
}
};

#endif

```

// voznja.C - Metode klase voznji.

```

#include "voznja.h"

float Voznja::duzina() const {        // Dužina voznje.
    float d = 0;
    for (int i=0; i<etape.duzina(); d+=etape[i++].duzina());
    return d;
}

float Voznja::trajanje() const {       // Vreme voznje.
    float t = 0;
    for (int i=0; i<etape.duzina(); t+=etape[i++].vreme());
    return t;
}

```

// tkackiauto.h - Klasa trkačkog automobila.

```

#ifndef _tkackiauto_h_
#define _tkackiauto_h_

#include "vozilo3.h"
#include "niz5.h"
#include "voznja.h"
#include "etapa.h"

class TrkackiAuto: public Vozilo {
    Niz<Voznja> voznje;                // Niz voznji.
public:
    TrkackiAuto(float sTez, int kap) // Stvaranje automobila.
        : Vozilo(sTez, voznje(kap)) {}
    const char* vrsta() const { return "Trkacki auto"; } // Naziv vrste.
    TrkackiAuto& novaVoznja(int kap) { // Početak nove voznje.
        voznje += Voznja(kap);
        return *this;
    }
    TrkackiAuto& dodajEtapu(const Etapa& e) { // Dodavanje nove etape.
        voznje[voznje.duzina()-1] += e;
        return *this;
    }
    const Voznja& najbrza() const;      // Nalaženje najbrže voznje.
};

#endif

```

```
// trkackiauto.C - Metoda klase trkačkih automobila.
```

```
#include "trkackiauto.h"
#include "voznja.h"

const Voznja& TrkackiAuto::najbrza() const { // Nalaženje najbrže vožnje.
    double max = voznje[0].srBrzina();
    int imax = 0;
    for (int i=1; i<voznje.duzina(); i++){
        double m = voznje[i].srBrzina();
        if (m > max) { max = m; imax = i; }
    }
    return voznje[imax];
}
```

```
// trkackiautot.C - Ispitivanje klase trkačkih automobila.
```

```
#include "trkackiauto.h"
#include "voznja.h"
#include "etapa.h"
#include <iostream>
using namespace std;

void main() {
    try {
        TrkackiAuto ta(800, 10);
        ta.novaVoznja(3)
            .dodajEtapu(Etapa(100, 50))
            .dodajEtapu(Etapa(200, 80))
            .dodajEtapu(Etapa(150, 70))
            .novaVoznja(3)
            .dodajEtapu(Etapa(300, 70))
            .dodajEtapu(Etapa(150, 60))
            .dodajEtapu(Etapa(200, 70)) ;
        cout << ta << endl;
        const Voznja& v = ta.najbrza();
        cout << "Najbrza voznja: " << v << endl;
    } catch (GNizPun g) { cout << g << endl; }
    catch (GIndeks g) { cout << g << endl; }
}
```

```
% automobilt
Trkacki auto 800
Najbrza voznja: (s=450,t=6.64286,v=67.7419)
```

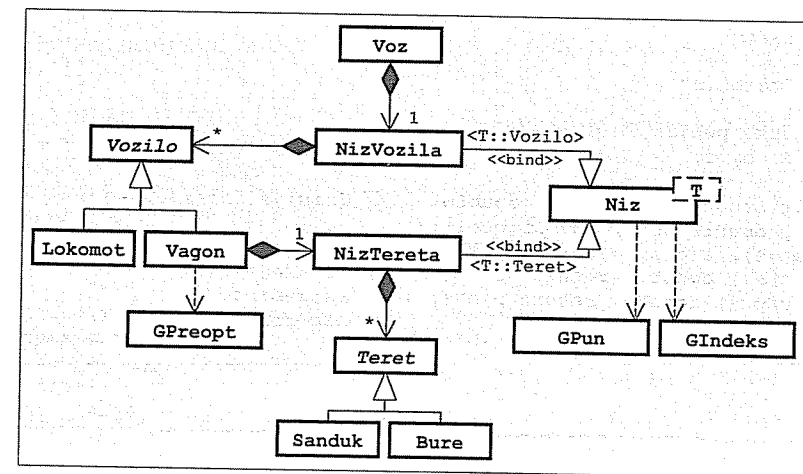
Zadatak 6.7 Tereti, sanduci, burad, generički nizovi, vozila, lokomotive, vagoni i vozovi

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorom i operatorom za dodelu vrednosti koji su potrebni za bezbedno korišćenje klase; greške prijavljivati izuzecima tipa jednostavnih klasa koje su opremljene pisanjem teksta poruke):

- Apstraktan **teret** ima jedinstven, automatski generisan celobrojan identifikator i zadatu specifičnu težinu. Može da se dohvati jednoslovna oznaka vrste, da se napravi kopija, da se odredi zapremina i težina i da se upiše u izlazni tok (`it<<teret`; piše se oznaka vrste i identifikator).
- **Sanduk** je teret oblika kvadra zadatih dužina ivica. Oznaka vrste je **S**.
- **Bure** je teret oblika valjka zadatog poluprečnika osnove i visine. Oznaka vrste je **B**.
- **Niz** sadrži pokazivače na stvari nekog polimorfnog tipa. Stvara se prazan zadatog kapaciteta posle čega se stvari dodaju pojedinačno po adresi, bez kopiranja (`niz+=&stvar`; greška je ako se niz prepuni). Može da se dohvati broj stvari u nizu i da se pristupi stvari sa zadatim rednim brojem (`niz[ind]`; greška je ako je indeks izvan opsega).
- Apstraktno **vozilo** ima zadatu sopstvenu težinu. Može da se odredi ukupna težina i vučna snaga vozila i da se vozilo upiše u izlazni tok (`it<<vozilo`). Vozilo ne može da se kopira ni na koji način. Vučna snaga predstavlja težinu tereta koji vozilo može da povuče.
- **Lokomotiva** je vozilo zadate vučne snage. U izlazni tok se piše u obliku **L** (`ukTežina|vučnaSnaga`).
- **Vagon** je vozilo koje sadrži generisan niz tereta. Stvara se prazan zadatog kapaciteta posle čega se tereti dodaju pojedinačno (`vagon+=&teret`). Vučna snaga vagona je 0. U izlazni tok upisuje se u obliku **V** (`ukTežina|teret, ..., teret`).
- **Voz** se sastoji od generisanog niza vozila. Stvara se prazan zadatog kapaciteta posle čega se vozila dodaju pojedinačno (`voz+=&vozilo`; greška je ako se voz preopteretiti, tj. ako ukupna težina svih vozila i tereta premaši ukupnu vučnu snagu vozila u vozu). Može da se dohvati broj vozila u vozu, da se proveriti da li bi se voz preopteretio priključivanjem zadatog vozila i da se voz upiše u izlazni tok (`it<<voz`; pišu se sadržana vozila, po jedno vozilo u redu). Voz ne može da se kopira ni na koji način.

Napisati na jeziku C++ **program** koji napravi voz s jednom lokomotivom i dva vagona sa po dva tereta. Koristiti konstantne parametre (ne treba ništa učitalavati s glavnog ulaza).

Rešenje:



```
// teret.h - Apstraktna klasa tereta.
```

```
#ifndef _teret_h_
#define _teret_h_
```

```
#include <iostream>
using namespace std;
```

```
class Teret {
    static int posId;           // Poslednji identifikator.
    int id;                     // Identifikator tereta.
    double sigma;               // Specifična težina.
public:
    explicit Teret(double s=1) // Nov objekat dobija nov ident.
    { id = ++posId; sigma = s; }
    Teret(const Teret& t)       // Kopija dobija nov identifikator.
    { id = ++posId; sigma = t.sigma; }
    virtual ~Teret() {}        // Virtuelan destruktor.
    Teret& operator=(const Teret& t) // Levom operandu se ne menja id.
    { sigma = t.sigma; }
    virtual char vrsta() const // Oznaka vrste.
    { return 'T'; }
    virtual Teret* kopija() const // Kopiranje.
    { return new Teret(*this); }
    virtual double zapr() const // Zapremina.
    { return sigma; }
    double tezina() const { return zapr() * sigma; } // Težina.
    friend ostream& operator<<(ostream& it, const Teret& t) // Pisanje.
    { return it << t.vrsta() << t.id; }
};
```

```
#endif
```

```
// teret.C - Statičko polje klase tereta.
```

```
#include "teret.h"
```

```
int Teret::posId = 0;           // Poslednji identifikator.
```

```
// sanduk.h - Klasa sanduka.
```

```
#ifndef _sanduk_h_
#define _sanduk_h_
```

```
#include "teret.h"
```

```
class Sanduk: public Teret {
    double a, b, c;             // Dužine ivica kvadra.
public:
    explicit Sanduk(double s=1, double // Inicijalizacija.
    aa=1, double bb=1, double cc=1):
    Teret(s), a(aa), b(bb), c(cc) {}
    char vrsta() const { return 'S'; } // Oznaka vrste.
    double zapr() const { return a*b*c; } // Zapremina.
    Sanduk* kopija() const // Kopiranje.
    { return new Sanduk(*this); }
};
```

```
#endif
```

```
// bure.h - Klasa buradi.
```

```
#ifndef _bure_h_
#define _bure_h_
```

```
#include "teret.h"
```

```
class Bure: public Teret {
    double r, h;                // Poluprečnik i visina valjka.
public:
    explicit Bure(double s=1, double rr=1, double hh=1): // Inicijalizacija.
    Teret(s), r(rr), h(hh) {}
    char vrsta() const { return 'B'; } // Oznaka vrste.
    double zapr() const // Zapremina.
    { return r * r * 3.14159 * h; }
    Bure* kopija() const { return new Bure(*this); } // Kopiranje.
};

#endif
```

```
// niz6.h - Generička klasa nizova pokazivača na elemente.
```

```
#ifndef _niz6_h_
#define _niz6_h_
```

```
#include <iostream>
using namespace std;
```

```
class GIndeks {}; // KLASA GREŠAKA: Nedoovoljen indeks.
inline ostream& operator<<(ostream& it, const GIndeks&)
{ return it << "*** Nedoovoljen indeks! ***"; }
```

```
class GPun {}; // KLASA GREŠAKA: Niz je pun.
inline ostream& operator<<(ostream& it, const GPun&)
{ return it << "*** Niz je pun! ***"; }
```

```
template <typename T> // GENERIČKA KLASA NIZOVA:
class Niz {
    T** niz;                // Niz pokazivača na elemente.
    int kap, duz;            // Kapacitet i dužina niza.
    void kopiraj(const Niz& n); // Kopiranje u objekat.
    void brisi();            // Brisanje objekta.
public:
    explicit Niz(int k=10) // Stvaranje praznog niza.
    { niz = new T* [kap = k]; duz = 0; }
    Niz(const Niz& n) { kopiraj(n); } // Konstruktor kopije.
    ~Niz() { brisi(); } // Destruktor.
    Niz& operator=(const Niz& n) { // Dodela vrednosti.
        if (this != &n) { brisi(); kopiraj(n); }
        return *this;
    }
    int duzina() const { return duz; } // Dužina niza.
    Niz& operator+=(T* t) { // Preuzimanje elementa po adresi.
        if (duz == kap) throw GPun();
        niz[duz++] = t;
        return *this;
    }
};
```

```

T& operator[](int i) {                // Pristup elementu
    if (i<0 || i>=duz) throw GIndeks(); // - promenljivog niza,
    return *niz[i];
}
const T& operator[](int i) const {     // - nepromenljivog niza.
    if (i<0 || i>=duz) throw GIndeks();
    return *niz[i];
}
};

template <typename T>                // METODE KLASA NIZOVA:
void Niz<T>::kopiraj(const Niz& n) {   // Kopiranje u objekat.
    niz = new T* [kap = n.kap];
    duz = n.duz;
    for (int i=0; i<duz; i++) niz[i] = n.niz[i]->kopija()
}

template <typename T>
void Niz<T>::brisi() {                // Brisanje objekta.
    for (int i=0; i<duz; delete niz[i++]);
    delete [] niz;
}
#endif

```

// vozilo4.h - Apstraktna klasa vozila.

```

#ifndef _vozilo4_h_
#define _vozilo4_h_

#include <iostream>
using namespace std;

class Vozilo {
    double sopTez;                // Sopstvena težina.
    Vozilo(const Vozilo&) {}       // Ne sme da se kopira.
    void operator=(const Vozilo&) {} // Ne sme da se dodeljuje.
public:
    explicit Vozilo(double st) { sopTez = st; } // Inicijalizacija.
    virtual ~Vozilo() {}           // Virtuelan destruktor.
    virtual double ukTezina() const // Ukupna težina.
    { return sopTez; }
    virtual double vucnaSnaga() const =0; // Vučna snaga.
private:
    virtual void pisi(ostream& it) const =0; // Pisanje.
    friend ostream& operator<<(ostream& it, const Vozilo& v)
    { v.pisi(it); return it; }
};
#endif

```

```

// lokomot2.h - Klasa lokomotiva.

#ifndef _lokomot2_h_
#define _lokomot2_h_

#include "vozilo4.h"

class Lokomot: public Vozilo {
    double vSnaga;                // Vučna snaga.
    void pisi(ostream& it) const // Pisanje.
    { it << "L(" << ukTezina() << '|' << vSnaga << ')'; }
public:
    Lokomot(double st, double vs): Vozilo(st), vSnaga(vs) {} // Stvaranje.
    double vucnaSnaga() const { return vSnaga; } // Vučna snaga.
};

#endif

```

// vagon.h - Klasa vagona.

```

#ifndef _vagon_h_
#define _vagon_h_

#include "vozilo4.h"
#include "niz6.h"
#include "teret.h"

class Vagon: public Vozilo {
    Niz<Teret> tereti;            // Niz tereta.
    void pisi(ostream& it) const; // Pisanje.
public:
    Vagon(double st, int k): Vozilo(st), tereti(k) {} // Stvaranje.
    Vagon& operator+=(Teret* t) // Preuzimanje tereta po adresi.
    { tereti += t; return *this; }
    double ukTezina() const;      // Ukupna težina.
    double vucnaSnaga() const {return 0;} // Vučna snaga.
};

#endif

```

// vagon.C - Metode klase vagona.

```

#include "vagon.h"

void Vagon::pisi(ostream& it) const { // Pisanje.
    it << "V(" << ukTezina() << "|";
    for (int i=0; i<tereti.duzina(); i++) {
        if (i > 0) it << ',';
        it << tereti[i];
    }
    it << ')';
}

double Vagon::ukTezina() const { // Ukupna težina.
    double t = Vozilo::ukTezina();
    for (int i=0; i<tereti.duzina(); i++)
        t += tereti[i].tezina();
    return t;
}

```



```
// voz2.h - Klasa vozova.

#ifndef _voz2_h_
#define _voz2_h_

#include "niz6.h"
#include "vozilo4.h"
#include <iostream>
using namespace std;

class GPreopt {};           // KLASA GREŠAKA: Voz je preopterećen.
inline ostream& operator<<(ostream& it, const GPreopt&)
{ return it << "*** Voz je preopterećen! ***"; }

class Voz {                 // KLASA VOZOVA:
    Niz<Vozilo> vozila;      // Niz vozila.
    Voz(const Voz&) {}       // Ne sme da se kopira.
    void operator=(const Voz&) {} // Ne sme da se dodeljuje.
public:
    explicit Voz(int k): vozila(k) {} // Stvaranje praznog voza.
    int duzina() const           // Dužina voza.
    { return vozila.duzina(); }
    bool preopt(Vozilo* v) const; // Da li bi se preopteretio?
    Voz& operator+=(Vozilo* v) { // Preuzimanje vozila po adresi.
        if (preopt(v)) throw GPreopt();
        vozila += v;
        return *this;
    }
    friend ostream& operator<<(ostream& it, const Voz& v); // Pisanje.
};

#endif
```

```
// voz2.C - Metode i funkcije uz klasu vozova.

#include "voz2.h"

bool Voz::preopt(Vozilo* v) const { // Da li bi se preopteretio?
    double teret = 0, snaga = 0;
    for (int i=0; i<vozila.duzina(); i++) {
        teret += vozila[i].ukTezina();
        snaga += vozila[i].vucnaSnaga();
    }
    return teret + v->ukTezina() > snaga + v->vucnaSnaga();
}

ostream& operator<<(ostream& it, const Voz& v) { // Pisanje.
    for (int i=0; i<v.vozila.duzina(); i++) it << v.vozila[i] << endl;
    return it;
}
```

```
// voz2t.C - Ispitivanje klase vozova.

#include "voz2.h"
#include "lokomot2.h"
#include "vagon.h"
#include "sanduk.h"
#include "bure.h"
#include <iostream>

int main() {
    try {
        Voz voz(3);
        voz += new Lokomot(200, 3000);
        Vagon* vag = new Vagon(100, 5);
        *vag += new Sanduk(3, 2, 1, 3);
        *vag += new Bure(2, 2, 4);
        voz += vag;
        vag = new Vagon(150, 10);
        *vag += new Sanduk(4, 2, 10, 3);
        *vag += new Bure(6, 3, 8);
        voz += vag;
        cout << voz;
    } catch (GPreopt g) { cout << g << endl;
    } catch (GPun g) { cout << g << endl;
    }
    return 0;
}
```

```
% voz2t
L(200|3000)
V(218.531|S1,B2)
V(1747.17|S3,B4)
```

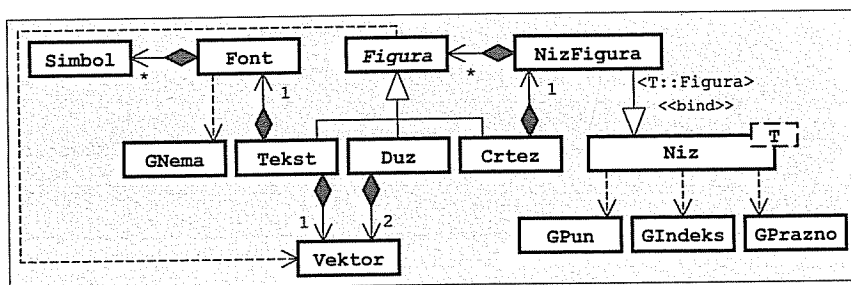
Zadatak 6.8 Simboli, fontovi, vektori, duži, tekstovi, generički nizovi i crteži

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorom i operatorom za dodelu vrednosti koji su potrebni za bezbedno korišćenje klase; greške prijavljivati izuzecima tipa jednostavnih klase koje su opremljene pisanjem teksta poruke):

- **Simbol** sadrži jedan znak (char) i odnos širine i visine pri iscrtavanju znaka. Može da se dohvati sadržani znak i da se odredi širina iscrtavanja za datu visinu.
- **Font** sadrži proizvoljan broj simbola. Stvara se prazan posle čega se simboli dodaju pojedinačno (font+=simb). Ako već postoji simbol za dati znak, stari simbol se zamenjuje novim. Može da se dohvati broj simbola u fontu i da se pristupi simbolu koji sadrži zadati znak (font[zn] – greška je ako znak ne postoji).
- **Vektor** u ravni zadaje se realnim komponentama u pravcu osa. Može da se vektoru doda drugi vektor (vekt1+=vekt2) i da se upiše u izlazni tok (it<<vekt) u obliku (x,y).
- Apstraktna **figura** u ravni može da se pomeri za zadati vektor pomaka (fig+=vekt), da se napravi kopija figure i da se figura upiše u izlazni tok (it<<fig).
- **Duž** je figura koja sadrži dva vektora položaja krajnjih tačaka. U izlazni tok upisuju se vektori položaja krajnjih tačaka u obliku *vektor1-vektor2*".
- **Tekst** je figura koja sadrži zadatu nisku znakova koja se iscrtava simbolima zadate visine primenom zadatog fonta sa zadatim vektorom položaja donjeg levog temena prvog simbola. Može da se dohvati visina i odredi širina teksta. U izlazni tok upisuje se niska znakova, položaj, visina i širina u obliku *nizZnakova [vektorPoložaja, visina, širina]*.
- **Niz** sadrži zadati broj pokazivača na podatke tipa neke klase sa mogućnošću kopiranja. Može da se dohvati kapacitet niza, da se postavi kopija podatka na prvo slobodno mesto (niz +=pod; greška je ako je niz pun), da se izbaci podatak sa zadatog mesta, da se pristupi podatku na zadatom mestu (niz[k]; greška je ako je indeks izvan opsega ili ako je zadato mesto prazno) i da se niz upiše u izlazni tok (it<<niz) u obliku *<podatak # ... # podatak>*".
- **Crtež** je figura koja sadrži niz figura zadatog kapaciteta. Stvara se prazan posle čega se figure dodaju pojedinačno.

Napisati na jeziku C++ **program** koji napravi font s tri simbola i crtež koji sadrži jednu duž i jedan tekst, ispiše crtež na glavnom izlazu, pomeri crtež za dati pomak i ponovo ispiše crtež na glavnom izlazu. Korištiti konstantne parametre (ne treba ništa učitavati s glavnog ulaza).

Rešenje:



```

// simbol.h - Klasa simbola.
#ifndef _simbol_h_
#define _simbol_h_

class Simbol {
    char zn; // Kôd znaka.
    double odnos; // Odnos širine i visine.
public:
    Simbol(char z, double odn) {zn = z; odnos = odn;} // Stvaranje.
    char znak() const { return zn; } // Dohvatanje znaka.
    double sirina(double vis) const { return vis * odnos; } // Širina.
};

#endif

// font.h - Klasa fontova.
#ifndef _font_h_
#define _font_h_

#include "simbol.h"
#include <iostream>
using namespace std;

class GNema {}; // KLASA GREŠAKA: Nema simbola.
inline ostream& operator<<(ostream& it, const GNema&)
{ return it << "*** Nema simbola! ***"; }

class Font { // KLASA FONTOVA:
    struct Elem { // Element liste simbola.
        Simbol sim; Elem* sled;
        Elem(const Simbol& s): sim(s) { sled = 0; }
    };
    Elem* prvi, *posl; // Prvi i poslednji element.
    int brSimb; // Dužina liste.
    void kopiraj(const Font& f); // Kopiranje u objekat.
    void brisi(); // Brisanje objekta.
public:
    Font() { prvi = posl = 0; brSimb = 0; } // Inicijalizacija.
    Font(const Font& f) { kopiraj(f); } // Konstruktor kopije.
    ~Font() { brisi(); } // Destruktor.
    Font& operator=(const Font& f) { // Dodela vrednosti.
        if (this != &f) { brisi(); kopiraj(f); }
        return *this;
    }
    int brSimbola() const { return brSimb; } // Broj simbola.
    Simbol& operator[](char zn); // Pristup simbolu promenljivog
    const Simbol& operator[](char zn) const // ... i nepromenljivog fonta.
    { return const_cast<Font&>(*this)[zn]; }
    Font& operator+=(const Simbol& s) { // Dodavanje simbola:
        try { // - zamena postojećeg,
            (*this)[s.znak()] = s;
        } catch (GNema) { // - dodavanje novog.
            brSimb++;
            posl = (!prvi ? prvi : posl->sled) = new Elem(s);
        }
        return *this;
    }
};

#endif

```

```
// font.C - Metode klase fontova.

#include "font.h"

void Font::kopiraj(const Font& f) {           // Kopiranje u objekat.
    prvi = posl = 0; brSimb = f.brSimb;
    Elem *tek = prvi, *pret = 0;
    for (Elem *tek=f.prvi; tek; tek=tek->sled)
        posl = (!prvi ? prvi : posl->sled) = new Elem(tek->sim);
}

void Font::brisi() {                         // Brisanje objekta.
    while (prvi) { Elem* stari = prvi; prvi = prvi->sled; delete stari; }
}

Simbol& Font::operator[](char zn) {          // Pristup znaku nepromenljivog
    for (Elem* tek=prvi; tek; tek=tek->sled) // fonta.
        if (tek->sim.znak() == zn) return tek->sim;
    throw GNema();
}
```

```
// vektor4.h - Klasa vektora u ravni.

#ifndef _vektor4_h_
#define _vektor4_h_

#include <iostream>
using namespace std;

class Vektor {
    double x, y;                               // Komponente vektora.
public:
    Vektor(double a=0, double b=0) {x=a; y=b;} // Inicijalizacija.
    Vektor& operator+=(const Vektor& v)         // Dodavanje (zbir) vektora.
        { x += v.x; y += v.y; return *this; }
    friend ostream& operator<<(ostream& it, const Vektor& v) // Pisanje.
        { return it<<'('<<v.x<<','<<v.y<<')'; }
};

#endif
```

```
// figura4.h - Apstraktna klasa figura u ravni.

#ifndef _figura4_h_
#define _figura4_h_

#include "vektor4.h"
#include <iostream>
using namespace std;

class Figura {
public:
    virtual ~Figura() {} // Virtuelan destruktor.
    virtual Figura& operator+=(const Vektor& v) =0; // Pomeranje figure.
    virtual Figura* kopija() const =0; // Kopiranje.
private:
    virtual void pisi(ostream& it) const =0; // Pisanje.
    friend ostream& operator<<(ostream& it, const Figura& f)
        { f.pisi(it); return it; }
};

#endif
```

```
// duz2.h - Klasa duži.

#ifndef _duz2_h_
#define _duz2_h_

#include "figura4.h"
#include <iostream>
using namespace std;

class Duz: public Figura {
    Vektor A, B;                               // Vektori položaja krajeva.
    void pisi(ostream& it) const { it << A << '-' << B; } // Pisanje.
public:
    Duz(const Vektor& P, const Vektor& Q): A(P), B(Q) {} // Inicijalizacija.
    Duz& operator+=(const Vektor& v)                // Pomeranje.
        { A += v; B += v; return *this; }
    Duz* kopija() const {return new Duz(*this);}    // Kopiranje.
};

#endif
```

```
// tekst4.h - Klasa tekstova.

#ifndef _tekst4_h_
#define _tekst4_h_

#include "figura4.h"
#include "vektor4.h"
#include "Font.h"
#include <iostream>
using namespace std;

class Tekst: public Figura {
    char* tks;                                // Sadržani tekst.
    double vis;                               // Visina znakova.
    Font fnt;                                 // Korišćeni font.
    Vektor poz;                               // Položaj u ravni.
    void pisi(ostream& it) const              // Pisanje.
        { it << tks << '[' << poz << ',' << vis << ',' << sirina() << ']; }
public:
    Tekst(const char* tks, double h, const Font& f, // Inicijalizacija.
           const Vektor& p): fnt(f), poz(p) {
        this->tks = strcpy(new char[strlen(tks)+1], tks);
        vis = h;
    }
    Tekst(const Tekst& t): fnt(t.fnt), poz(t.poz) { // Konstruktor kopije.
        tks = strcpy(new char [strlen(t.tks)+1], t.tks);
        vis = t.vis;
    }
    ~Tekst() { delete [] tks; }                // Dstruktor.
    Tekst& operator=(const Tekst& t) {
        if (this != &t) {
            tks = strcpy(new char [strlen(t.tks)+1], t.tks);
            vis = t.vis; fnt = t.fnt; poz = t.poz;
        }
    }
    double visina() const { return vis; }      // Visina znakova.
    double sirina() const;                     // Širina teksta.
};
```

```

Tekst& operator+=(const Vektor& v)           // Pomeranje.
{ poz += v; return *this; }
Tekst* kopija() const { return new Tekst(*this); } // Kopiranje.
};

#endif

```

```
// tekst4.C - Metoda klase tekstova.
```

```

#include "tekst4.h"

double Tekst::sirina() const {           // Širina teksta.
    double s = 0;
    for (unsigned i=0; i<strlen(tks); s += fnt[tks[i++]].sirina(vis));
    return s;
}

```

```
// niz7.h - Generička klasa nizova pokazivača na podatke.
```

```

#ifndef _niz7_h_
#define _niz7_h_

#include <iostream>
using namespace std;

class GPun {};           // KLASA GREŠAKA: Niz je pun.
inline ostream& operator<<(ostream& it, const GPun&)
{ return it << "**** Niz je pun! ****"; }

class GIndeks {};        // KLASA GREŠAKA: Nedozvoljen indeks.
inline ostream& operator<<(ostream& it, const GIndeks&)
{ return it << "**** Nedozvoljen indeks! ****"; }

class GPrazno {};        // KLASA GREŠAKA: Mesto je prazno.
inline ostream& operator<<(ostream& it, const GPrazno&)
{ return it << "**** Mesto je prazno! ****"; }

template <class T>
class Niz {               // GENERIČKA KLASA NIZOVA:
    T** niz;              // Niz pokazivača na elemente.
    int kap;              // Kapacitet niza.
    void kopiraj(const Niz& n); // Kopiranje u objekat.
    void brisi();          // Brisanje objekta.
public:
    explicit Niz(int k=10); // Stvaranje praznog niza.
    Niz(const Niz& n) { kopiraj(n); } // Konstruktor kopije.
    ~Niz() { brisi(); } // Destruktor.
    Niz& operator=(const Niz& n) { // Dodela vrednosti.
        if (this != &n) { brisi(); kopiraj(n); }
        return *this;
    }
    int kapac() const { return kap; } // Kapacitet niza.
    Niz& operator+=(const T& t);      // Dodavanje elementa.
    Niz& izbaci(int i) {              // Izbacivanje elementa.
        if (i>=0 && i<kap) { delete niz[i]; niz[i] = 0; }
        return *this;
    }
}

```

```

T& operator[](int i) {           // Pristup elementu
    if (i<0 || i>=kap) throw GIndeks(); // - promenljivog niza,
    if (!niz[i]) throw GPrazno ();
    return *niz[i];
}

const T& operator[](int i) const { // - nepromenljivog niza.
    if (i<0 || i>=kap) throw GIndeks();
    if (!niz[i]) throw GPrazno();
    return *niz[i];
}

template <class T>              // Pisanje.
friend ostream& operator<<(ostream& it, const Niz<T>& n);

template <class T>              // METODE I FUNKCIJE UZ KLASU NIZOVA
void Niz<T>::kopiraj(const Niz& n) { // Kopiranje u objekat.
    niz = new T* [kap = n.kap];
    for (int i=0; i<kap; i++) niz[i] = n.niz[i] ? n.niz[i]->kopija() : 0;
}

template <class T>
void Niz<T>::brisi() {          // Brisanje objekta.
    for (int i=0; i<kap; delete niz[i++]);
    delete [] niz;
}

template <class T>              // Stvaranje praznog niza.
Niz<T>::Niz(int k) {
    niz = new T* [kap = k];
    for (int i=0; i<k; niz[i++]=0);
}

template <class T>
Niz<T>& Niz<T>::operator+=(const T& t) { // Dodavanje elementa.
    int i = 0; while (i<kap && niz[i]) i++;
    if (i == kap) throw GPun();
    niz[i] = t.kopija();
    return *this;
}

template <class T>              // Pisanje.
ostream& operator<<(ostream& it, const Niz<T>& n) {
    it << '<';
    for (int i=0; i<n.kap; i++) {
        if (i) it << " # ";
        if (n.niz[i]) it << *n.niz[i];
    }
    return it << '>';
}

#endif

```

```
// crtez2.h - Klasa crteža.
```

```

#ifndef _crtez2_h_
#define _crtez2_h_

#include "figura4.h"
#include "niz7.h"
#include <iostream>
using namespace std;

```

```

class Crtez: public Figura {
    Niz<Figura> niz; // Niz figura.
    void pisi(ostream& it) const { it << niz; } // Pisanje.
public:
    explicit Crtez(int kap): niz(kap) {} // Stvaranje praznog crteža.
    Crtez& operator+=(const Figura& f) // Dodavanje figure.
    { niz += f; return *this; }
    Crtez& operator+=(const Vektor& v); // Pomeranje crteža.
    Crtez* kopija() const { return new Crtez(*this); } // Kopiranje.
};
#endif

```

// crtez2.C - Metode klase crteža.

```
#include "crtez2.h"
```

```

Crtez& Crtez::operator+=(const Vektor& v) { // Pomeranje crteža.
    for (int i=0; i<niz.kapac(); i++)
        try { niz[i] += v; } catch(GPrazno) {}
    return *this;
}

```

// crtez2t.C - Ispitivanje klase crteža.

```

#include "font.h"
#include "simbol.h"
#include "crtez2.h"
#include "vektor4.h"
#include "duz2.h"
#include "tekst4.h"
#include <iostream>
using namespace std;

int main() {
    try {
        Font fnt;
        fnt += Simbol('m', 0.8);
        fnt += Simbol('a', 0.5);
        fnt += Simbol('!', 0.1);
        Crtez crt(2);
        crt += Duz(Vektor(1, 1), Vektor(2, 3));
        crt += Tekst("mama!", 1, fnt, Vektor(2, 1));
        cout << crt << endl;
        crt += Vektor(2, 1);
        cout << crt << endl;
    } catch (GNema g) { cout << g << endl; }
    } catch (GPun g) { cout << g << endl; }
    } catch (GIndeks g) { cout << g << endl; }
    } catch (GPrazno g) { cout << g << endl; }
    }
    return 0;
}

```

```

% crtez2t
<(1,1)-(2,3) # mama![(2,1),1,2.7]>
<(3,2)-(4,4) # mama![(4,2),1,2.7]>

```

Zadatak 6.9 Akteri, časovnici, proizvodi i generička skladišta, proizvođači i potrošači

Napisati na jeziku C++ sledeće klase (klase opremiti onim konstruktorima, destruktorom i operatorom za dodelu vrednosti koji su potrebni za bezbedno korišćenje klasa; greške prijavljivati izuzecima tipa jednostavnih klasa koje su opremljene pisanjem teksta poruke):

- Apstraktan **akter** ima jedinstven, automatski generisan celobrojan identifikator i može da mu se saopšti da je proteklo određeno vreme. U slučajnim vremenskim intervalima između najkraćeg i najdužeg trajanja, koji se zadaju prilikom stvaranja, izvrši neku apstraktnu radnju. Akter može da se upiše u izlazni tok (`it<<akt`) i tada se piše njegov identifikator.
- **Časovnik** služi za obaveštavanje prijavljenih aktera o protoku vremena. Stvara se bez prijavljenih aktera, posle čega akteri mogu pojedinačno da se prijavljuju (`cas+=&akt`) i odjavljuju (`cas-=&akt`). Kad se časovnik pokrene (uz zadavanje koraka promene i ukupnog vremena), on odbrojava vreme (uvećavanjem vremena za zadati korak) i obaveštava sve prijavljene aktere o protoku koraka vremena do zadatog ukupnog vremena uz ispisivanje trenutnog vremena na glavnom izlazu na početku svakog ciklusa obaveštavanja. Časovnik ne može da se kopira ni na koji način.
- **Skladište** može da sadrži zadat broj stvari nekog tipa. Može da se stavi jedna stvar u skladište i da se uzme jedna stvar iz skladišta po redosledu dodavanja. Greška je ako se pokuša staviti stvar u puno skladište ili uzeti stvar iz praznog skladišta.
- **Proizvođač** je akter čija se radnja sastoji od stvaranja i stavljanja u zadato skladište jedne stvari nekog tipa uz ispisivanje na glavnom izlazu svog identifikatora i iza toga stavljenе stvari ili poruke o grešci.
- **Potrošač** je akter čija se radnja sastoji od uzimanja iz zadatog skladišta jedne stvari nekog tipa uz ispisivanje na glavnom izlazu svog identifikatora i iza toga uzete stvari ili poruke o grešci.
- **Proizvod** sadrži slučajan ceo broj u opsegu od 0 do 999. Može da se upiše u izlazni tok (`it<<pro`) i tada se piše vrednost sadržanog broja.

Napisati na jeziku C++ **program** koji napravi jedno skladište proizvoda, napravi jedan časovnik, prijavi časovniku nekoliko proizvođača proizvoda i nekoliko potrošača proizvoda i pokrene časovnik da sa nekim korakom radi izvesno vreme. Koristiti konstantne parametre (ne treba ništa učitavati s glavnog ulaza).

Rešenje:

```

// akter.h - Apstraktna klasa
// aktera.

```

```

#ifndef _akter_h_
#define _akter_h_

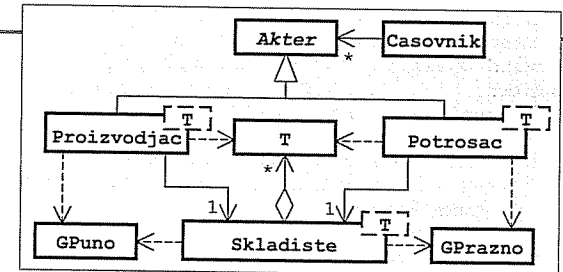
#include <cstdlib>
#include <iostream>
using namespace std;

```

```

class Akter {
    static int posId; // Poslednji identifikator.
    int id; // Identifikator aktera.
    double minT, maxT; // Min i max vreme između dve radnje.
    double t; // Preostalo vreme do sledeće radnje.
    void kopiraj(const Akter& a) // Kopiranje u objekat (bez ident.!).
    { minT = a.minT; maxT = a.maxT; t = a.t; }
    double slucT() // Generisanje slučajnog vremena.
    { return minT + (maxT-minT)*rand()/(RAND_MAX+1); }
    virtual void radnja() =0; // Radnja aktera.
}

```



```

public:
    Akter(double tMin=0, double tMax=1)           // Inicijalizacija.
    { id = ++posId; minT = tMin; maxT = tMax; t = 0; }
    Akter(const Akter& a) { id = ++posId; kopiraj(a); } // Konstr. kopije.
    virtual ~Akter() {}
    Akter& operator=(const Akter& a) {             // Dodela vrednosti.
        kopiraj(a);
        return *this;
    }
    void proteklo(double dt) {                     // Saopštavanje proteklog vremena.
        if ((t -= dt) <= 0) { radnja(); t = slucT(); }
    }
    friend ostream& operator<<(ostream& it, const Akter& a) // Pisanje.
    { return it << " " << a.id; }
};

#endif

```

// akter.C - Statičko polje klase aktera.

```
#include "akter.h"
```

```
int Akter::posId = 0;           // Poslednji identifikator.
```

// casovnik.h - Klasa časovnika.

```

#ifndef _casovnik_h_
#define _casovnik_h_

#include "akter.h"

class Casovnik {
    struct Elem {                // Element liste aktera.
        Akter* akter; Elem* sled;
        Elem(Akter* akt) { akter = akt; sled = 0; }
    };
    Elem *prvi, *posl;           // Prvi i poslednji element.
    Casovnik(const Casovnik&) {} // Ne sme da se kopira.
    void operator=(const Casovnik&) {} // Ne sme da se dodeljuje.
public:
    Casovnik() { prvi = posl = 0; } // Stvaranje praznog časovnika.
    ~Casovnik() {} // Destruktor.
    Casovnik& operator+=(Akter* akt) { // Prijavljivanje aktera.
        posl = (!prvi ? prvi : posl->sled) = new Elem(akt);
        return *this;
    }
    Casovnik& operator==(Akter* akt); // Odjavljivanje aktera.
    void radi(double dt, double maxT); // Rad časovnika.
};

#endif

```

// casovnik.C - Metode klase časovnika.

```
#include "casovnik.h"
```

```

Casovnik::~Casovnik() {                // Destruktor.
    while (prvi) { Elem* stari = prvi; prvi = prvi->sled; delete stari; }
    posl = 0;
}

```

```

Casovnik& Casovnik::operator==(Akter* akt) { // Odjavljivanje aktera.
    Elem *tek = prvi, *pret = 0;
    while (prvi && prvi->akter!=akt) { pret = tek; tek = tek->sled; }
    if (tek) {
        (!pret ? prvi : pret->sled) = tek->sled;
        delete tek;
        if (tek == posl) posl = pret;
    }
    return *this;
}

void Casovnik::radi(double dt, double maxT) { // Rad časovnika.
    for (double t=0; t<=maxT; t+=dt) {
        cout << "Vreme: " << t << endl;
        for (Elem* tek=prvi; tek; tek=tek->sled) tek->akter->proteklo(dt);
    }
}

```

// skladiste.h - Generička klasa skladišta stvari.

```

#ifndef _skladiste_h_
#define _skladiste_h_

#include <iostream>
using namespace std;

class GPuno {}; // KLASA GREŠAKA: Skladište je puno.
inline ostream& operator<<(ostream& it, const GPuno&)
{ return it << "*** Skladiste je puno! ***"; }

class GPrazno {}; // KLASA GREŠAKA: Skladište je prazno.
inline ostream& operator<<(ostream& it, const GPrazno&)
{ return it << "*** Skladiste je prazno! ***"; }

template <typename T> // GENERIČKA KLASA SKLADIŠTA:
class Skladiste {
    T* niz; // Niz stvari.
    int kap, duz; // Kapacitet i popunjenost niza.
    int prvi, posl; // Indeks prve i poslednje stvari.
    void kopiraj(const Skladiste& s); // Kopiranje u objekat.
    void brisi() { delete [] niz; } // Brisanje objekta.
public:
    explicit Skladiste(int k=10) { // Stvaranje praznog skladišta.
        niz = new T [kap = k];
        prvi = posl = duz = 0;
    }
    void stavi(const T& t) { // Stavljanje u skladište.
        if (duz == kap) throw GPuno();
        niz[posl++] = t;
        if (posl == kap) posl = 0;
        duz++;
    }
    T uzmi() { // Uzimanje iz skladišta.
        if (duz == 0) throw GPrazno();
        T t = niz[prvi++];
        if (prvi == kap) prvi = 0;
        duz--;
        return t;
    }
};

```

```

template <typename T>                // METODE KLASKE SKLADIŠTA:
void Skladiste<T>::kopiraj(const Skladiste& s) { // Kopiranje u objekat.
    niz = new T [kap = s.kap];
    for (int i=0; i<kap; i++) niz[i] = s.niz[i];
    prvi = s.prvi; posl = s.posl; duz = s.duz;
}

#endif

```

// proizvodjac.h - Generička klasa proizvođača.

```

#ifndef _proizvodjac_h_
#define _proizvodjac_h_

#include "akter.h"
#include "skladiste.h"

template <typename T>
class Proizvodjac: public Akter {
    Skladiste<T>* sklad;                // Odredišno skladište.
public:
    Proizvodjac(Skladiste<T>* s, double tMin=0, // Inicijalizacija.
                double tMax=1): Akter(tMin, tMax)
    { sklad = s; }
    void radnja() {                    // Radnje proizvođača.
        cout << *this << ": ";
        try {
            T t = T();
            sklad->stavi(t);
            cout << "stavio " << t << endl;
        } catch (GPuno g) { cout << g << endl; }
    }
};

#endif

```

// potrosac2.h - Generička klasa potrošača.

```

#ifndef _potrosac2_h_
#define _potrosac2_h_

#include "akter.h"
#include "skladiste.h"

template <typename T>
class Potrosac: public Akter {
    Skladiste<T>* sklad;                // Izvorišno skladište.
public:
    Potrosac(Skladiste<T>* s, double tMin=0, // Inicijalizacija.
             double tMax=1): Akter(tMin, tMax)
    { sklad = s; }
    void radnja() {                    // Radnja potrošača.
        cout << *this << ": ";
        try { cout << "uzeo " << sklad->uzmi() << endl; }
        catch (GPrazno g) { cout << g << endl; }
    }
};

#endif

```

// proizvod2.h - Klasa proizvoda.

```

#ifndef _proizvod2_h_
#define _proizvod2_h_

#include <cstdlib>
#include <iostream>
using namespace std;

class Proizvod {
    int broj;                                // Sadržani broj.
public:
    Proizvod() { broj = 1000 * rand() / (RAND_MAX+1); } // Inicijalizacija.
    friend ostream& operator<<(ostream& it, const Proizvod& p) // Pisanje.
    { return it << p.broj; }
};

#endif

```

// skladistet.C - Ispitivanje klase skladišta.

```

#include "skladiste.h"
#include "proizvod2.h"
#include "casovnik.h"
#include "proizvodjac.h"
#include "potrosac2.h"

int main() {
    Skladiste<Proizvod> s(3);
    Casovnik c;
    Akter* akteri[5];
    c += akteri[0] = new Proizvodjac<Proizvod>(&s, 2, 4);
    c += akteri[1] = new Proizvodjac<Proizvod>(&s, 1, 5);
    c += akteri[2] = new Proizvodjac<Proizvod>(&s, 1.5, 4.5);
    c += akteri[3] = new Potrosac<Proizvod>(&s, 1.5, 2.5);
    c += akteri[4] = new Potrosac<Proizvod>(&s, 1, 3);
    c.radi(0.25, 10);
    for (int i=0; i<5; delete akteri[i++]);
    return 0;
}

```

```

% skladistet
Vreme: 0
1: stavio 808
2: stavio 479
3: stavio 895
4: uzeo 808
5: uzeo 479
Vreme: 0.25
Vreme: 0.5
Vreme: 0.75
Vreme: 1
Vreme: 1.25
Vreme: 1.5
5: uzeo 895
Vreme: 1.75
Vreme: 2
Vreme: 2.25
4: *** Skladiste je prazno! ***
Vreme: 2.5

```

```

    2: stavio 513
Vreme: 2.75
Vreme: 3
Vreme: 3.25
    1: stavio 14
Vreme: 3.5
Vreme: 3.75
Vreme: 4
    3: stavio 364
Vreme: 4.25
    5: uzeo 513
Vreme: 4.5
    4: uzeo 14
Vreme: 4.75
    2: stavio 445
Vreme: 5
Vreme: 5.25
Vreme: 5.5
    1: stavio 4
Vreme: 5.75
    5: uzeo 364
Vreme: 6
    3: stavio 531
Vreme: 6.25
    2: *** Skladiste je puno! ***
Vreme: 6.5
Vreme: 6.75
Vreme: 7
    4: uzeo 445
Vreme: 7.25
Vreme: 7.5
Vreme: 7.75
    1: stavio 663
    5: uzeo 4
Vreme: 8
Vreme: 8.25
Vreme: 8.5
Vreme: 8.75
    4: uzeo 531
Vreme: 9
Vreme: 9.25
    3: stavio 607
Vreme: 9.5
    5: uzeo 663
Vreme: 9.75
    2: stavio 519
Vreme: 10

```

7 Standardna biblioteka

Zadatak 7.1 Stekovi i redovi tačaka neograničenih kapaciteta

Korišćenjem standardnih generičkih klasa za zbirke podataka napisati na jeziku C++ klase za stekove i za redove tačaka u ravni neograničenih kapaciteta.

Napisati na jeziku C++ programe za ispitivanje prethodnih klasa.

Rešenje:

```
// stek2.h - Klasa stekova tačaka pomoću klase vector<>.

#ifndef _stek2_h_
#define _stek2_h_

#include "tacka4.h" ➡ Zadatak 4.5
#include "greska.h" ➡ Zadatak 5.6
#include <vector>
#include <iostream>
using namespace std;

class Stek2 {
    vector<Tacka> niz; // Sadržaj steka.
public:
    void stavi(const Tacka& t) // Stavljanje na vrh.
    { niz.push_back(t); }
    Tacka uzmi() { // Uzimanje sa vrha.
        if (prazan()) throw Usluge::Greska("Stek je prazan!");
        Tacka t = niz.back();
        niz.pop_back();
        return t;
    }
    unsigned vel() const { return niz.size(); } // Broj elemenata na steku.
    bool prazan() const { return vel() == 0; } // Da li je prazan?
    void prazni() { niz.clear(); } // Pražnjenje.
    friend ostream& operator<<(ostream& it, const Stek2& s); // Pisanje.
};

#endif
```

// stek2.C - Metode i funkcije uz klasu stekova tačaka pomoću vector<>.

```
#include "stek2.h"

ostream& operator<<(ostream& it, const Stek2& s) { // Pisanje.
    for (unsigned i=s.vel(); i>0; it <<s.niz[--i]<<' ');
    return it;
}
```

// stek3.h - Klasa stekova tačaka pomoću klase list<>.

```
#ifndef _stek3_h_
#define _stek3_h_

#include "tacka4.h" ➡ Zadatak 4.5
#include "greska.h" ➡ Zadatak 5.6
#include <list>
#include <iostream>
```

```
using namespace std;

class Stek3 {
    list<Tacka> niz; // Sadržaj steka.
public:
    void stavi(const Tacka& t) // Stavljanje na vrh.
    { niz.push_back(t); }
    Tacka uzmi() { // Uzimanje sa vrha.
        if (prazan()) throw Usluge::Greska("Stek je prazan!");
        Tacka t = niz.back();
        niz.pop_back();
        return t;
    }
    unsigned vel() const { return niz.size(); } // Broj elemenata na steku.
    bool prazan() const { return vel() == 0; } // Da li je prazan?
    void prazni() { niz.clear(); } // Pražnjenje.
    friend ostream& operator<<(ostream& it, const Stek3& s); // Pisanje.
};

#endif
```

// stek3.C - Metode i funkcije uz klasu stekova tačaka pomoću list<>.

```
#include "stek3.h"

ostream& operator<<(ostream& it, const Stek3& s) { // Pisanje.
    for (list<Tacka>::const_iterator i=s.niz.end();
        i!=s.niz.begin(); it <<!--i<<' ');
    return it;
}
```

// stek3t.C - Ispitivanje klasa stekova tačaka.

```
#include "stek2.h"
#include "stek3.h"
#include "tacka4.h" ➡ Zadatak 4.5
#include <iostream>
using namespace std;

int main() {
    Stek2 s2; Stek3 s3;
    for (int i=0; i<5; i++) {
        s2.stavi(Tacka(i, 2*i));
        s3.stavi(Tacka(2*i, i));
    }
    cout << "Napunjeni stek2: " << s2 << endl;
    cout << "Napunjeni stek3: " << s3 << endl;
    cout << "Praznjenje stek2: ";
    while (!s2.prazan()) cout << s2.uzmi() << ' '; cout << endl;
    cout << "Praznjenje stek3: ";
    while (!s3.prazan()) cout << s3.uzmi() << ' '; cout << endl;
    return 0;
}
```

```
% stek3t
Napunjeni stek2:  (4,8) (3,6) (2,4) (1,2) (0,0)
Napunjeni stek3:  (8,4) (6,3) (4,2) (2,1) (0,0)
Praznjenje stek2: (4,8) (3,6) (2,4) (1,2) (0,0)
Praznjenje stek3: (8,4) (6,3) (4,2) (2,1) (0,0)
```

// red4.h - Klasa redova tačaka pomoću klase deque<>.

```
#ifndef _red4_h_
#define _red4_h_

#include "tacka4.h" ➔ Zadatak 4.5
#include "greska.h" ➔ Zadatak 5.6
#include <deque>
#include <iostream>
using namespace std;

class Red4 {
    deque<Tacka> niz; // Sadržaj reda.
public:
    void stavi(const Tacka& t) // Stavljanje na kraj.
    { niz.push_back(t); }
    Tacka uzmi() { // Uzimanje sa početka.
        if (prazan()) throw Usluge::Greska("Red je prazan!");
        Tacka t = niz.front();
        niz.pop_front();
        return t;
    }
    unsigned vel() const { return niz.size(); } // Broj elemenata u redu.
    bool prazan() const { return vel() == 0; } // Da li je prazan?
    void prazni() { niz.clear(); } // Pražnjenje.
    friend ostream& operator<<(ostream& it, const Red4& r); // Pisanje.
};

#endif
```

// red4.C - Metode i funkcije uz klasu redova tačaka pomoću deque<>.

```
#include "red4.h"

ostream& operator<<(ostream& it, const Red4& r) { // Pisanje.
    for (unsigned i=0; i<r.vel(); i++) it <<r.niz[i++]<<' ';
    return it;
}
```

// red5.h - Klasa redova tačaka pomoću klase list<>.

```
#ifndef _red5_h_
#define _red5_h_

#include "tacka4.h" ➔ Zadatak 4.5
#include "greska.h" ➔ Zadatak 5.6
#include <list>
#include <iostream>
using namespace std;
```

```
class Red5 {
    list<Tacka> niz; // Sadržaj reda.
public:
    void stavi(const Tacka& t) // Stavljanje na kraj.
    { niz.push_back(t); }
    Tacka uzmi() { // Uzimanje sa početka.
        if (prazan()) throw Usluge::Greska("Red je prazan!");
        Tacka t = niz.front();
        niz.pop_front();
        return t;
    }
    unsigned vel() const { return niz.size(); } // Broj elemenata u redu.
    bool prazan() const { return vel() == 0; } // Da li je prazan?
    void prazni() { niz.clear(); } // Pražnjenje.
    friend ostream& operator<<(ostream& it, const Red5& r); // Pisanje.
};

#endif
```

// red5.C - Metode i funkcije uz klasu redova tačaka pomoću list<>.

```
#include "red5.h"

ostream& operator<<(ostream& it, const Red5& r) { // Pisanje.
    for (list<Tacka>::const_iterator i=r.niz.begin();
        i!=r.niz.end(); i++) it <<*i++<<' ';
    return it;
}
```

// red5t.C - Ispitivanje klasa redova tačaka.

```
#include "red4.h"
#include "red5.h"
#include "tacka4.h" ➔ Zadatak 4.5
#include <iostream>
using namespace std;

int main() {
    Red4 r4; Red5 r5;
    for (int i=0; i<5; i++) {
        r4.stavi(Tacka(i, 2*i));
        r5.stavi(Tacka(2*i, i));
    }
    cout << "Napunjeni red4: " << r4 << endl;
    cout << "Napunjeni red5: " << r5 << endl;
    cout << "Praznjenje red4: ";
    while (!r4.prazan()) cout << r4.uzmi() << ' '; cout << endl;
    cout << "Praznjenje red5: ";
    while (!r5.prazan()) cout << r5.uzmi() << ' '; cout << endl;
    return 0;
}
```

```
% red5t
Napunjeni red4:  (0,0) (1,2) (2,4) (3,6) (4,8)
Napunjeni red5:  (0,0) (2,1) (4,2) (6,3) (8,4)
Praznjenje red4: (0,0) (1,2) (2,4) (3,6) (4,8)
Praznjenje red5: (0,0) (2,1) (4,2) (6,3) (8,4)
```

Zadatak 7.2 Predmeti, tela, sfere, kvadri i sklopovi

Napisati na jeziku C++ sledeće klase (sve klase opremiti potrebnim konstruktorima, destruktorom, operatorom = za bezbedno korišćenje i metodom za stvaranje kopije):

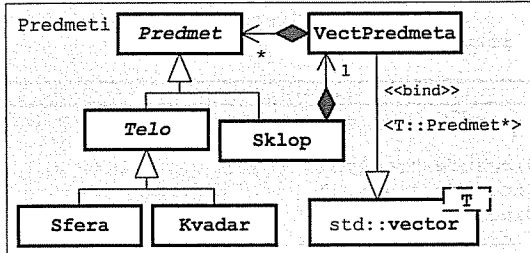
- **Predmet** ima jedinstven, automatski generisan celobrojan identifikator. Može da mu se izračuna zapremina i težina, da se dohvati ime njegove klase i da se upiše u izlazni tok (`it<<pr`) u obliku *klasa/id*, gde su: *klasa* – ime klase predmeta, a *id* – identifikator.
- **Telo** je homogen predmet koji ima određenu specifičnu težinu.
- **Sfera** i **kvadar** su tela zadatih dimenzija. Sfera se u izlazni tok upisuje u obliku *klasa/id(σ ;r)*, a kvadar u obliku *klasa/id(σ ;a,b,c)*, gde su: σ – specifična težina, *r* poluprečnik sfere, a *a*, *b* i *c* – dužine ivica kvadra.
- **Sklop** je predmet koji se sastoji od više raznovrsnih predmeta. Stvara se prazan posle čega može da se dodaje proizvoljan broj predmeta (`sklop+=pr`). U izlazni tok upisuju se u obliku *klasa/id[*predmet predmet ... predmet*]*, gde je: *predmet* – rezultat pisanja jednog sadržanog predmeta, svaki predmet u zasebnom redu.

Napisati na jeziku C++ **funkciju** koja pročita iz ulaznog toka jedan predmet proizvoljnog tipa.

Napisati na jeziku C++ **program** koji s glavnog ulaza pročita niz predmeta, ispiše ih na glavnom izlazu i na kraju pronade i ispiše na glavnom izlazu predmet s najvećom zapreminom i s najvećom težinom.

Rešenje:

```
// predmet3.h - Apstraktna klasa predmeta.
//
#ifndef _predmet3_h_
#define _predmet3_h_
#include <iostream>
#include <iomanip>
#include <typeinfo>
#include <cstring>
using namespace std;
namespace Predmeti {
class Predmet {
    static int posId;           // Poslednji identifikator.
    int id;                    // Identifikator predmeta.
public:
    Predmet(): id(++posId) {} // Nov predmet dobija nov broj.
    Predmet(const Predmet&): id(++posId) {} // Kopija dobija nov broj.
    virtual ~Predmet() {} // Virtuelan destruktor.
    Predmet& operator=(const Predmet&) // Levom operandu se ne menja
    { return *this; } // broj.
    virtual Predmet* kopija() const = 0; // Kopiranje.
    virtual double V() const = 0; // Zapremina.
    virtual double Q() const = 0; // Težina.
    const char* vrsta() const // Ime klase.
    { return strchr(typeid(*this).name(), ':') + 1; }
protected:
    static int nivo; // Nivo pisanja predmeta.
    virtual void pisi(ostream& it) const // Pisanje.
    { it << setw(nivo*2) << "" << vrsta() << '/' << id << ' '; }
    friend ostream& operator<<(ostream& it, const Predmet& p)
    { p.pisi(it); return it; }
}; // class Predmet
} // namespace Predmeti
#endif
```



```
// predmet3.C - Statička polja apstraktne klase predmeta.
```

```
#include "predmet3.h"
```

```
int Predmeti::Predmet::posId = 0, Predmeti::Predmet::nivo = 0;
```

```
// telo2.h - Apstraktna klasa tela.
```

```
#ifndef _telo2_h_
```

```
#define _telo2_h_
```

```
#include "predmet3.h"
```

```
namespace Predmeti {
class Telo: public Predmet {
    double sigma; // Specifična težina.
public:
    Telo(double s=1) { sigma = s; } // Stvaranje.
    double spTez() const { return sigma; } // Specifična težina.
    double Q() const { return sigma * V(); } // Težina.
protected:
    void pisi(ostream& it) const // Pisanje.
    { Predmet::pisi(it); it << '(' << sigma << ';'; }
}; // class Telo
} // namespace Predmeti
```

```
#endif
```

```
// sfera4.h - Klasa sfera.
```

```
#ifndef _sfera4_h_
```

```
#define _sfera4_h_
```

```
#include "telo2.h"
```

```
namespace Predmeti {
class Sfera: public Telo {
    double r; // Poluprečnik.
public:
    Sfera(double rr=1, double ss=1) // Stvaranje.
    : Telo(ss) { r = rr; }
    Sfera* kopija() const // Kopiranje.
    { return new Sfera(*this); }
    double V() const // Zapremina.
    { return 4 * r * r * r * 3.14159 / 3; }
private:
    void pisi(ostream& it) const // Pisanje.
    { Telo::pisi(it); it << r << ' '; }
}; // class Sfera
} // namespace Predmeti
```

```
#endif
```

```
// kvadar5.h - Klasa kvadara.
#ifndef _kvadar5_h_
#define _kvadar5_h_
#include "telo2.h"

namespace Predmeti {
    class Kvadar: public Telo { double a, b, c; // Ivice.
    public:
        Kvadar(double aa=1, double bb=1, double cc=1, // Stvaranje.
            double ss=1): Telo(ss) { a = aa; b = bb; c = cc; }
        Kvadar* kopija() const { return new Kvadar(*this); } // Kopiranje.
        double V() const { return a * b * c; } // Zapremina.
    private:
        void pisi(ostream& it) const // Pisanje.
        { Telo::pisi(it); it << ' ' << b << ' ' << c << ' '; }
    }; // class Kvadar
} // namespace Predmeti

#endif
```

```
// sklop.h - Klasa sklopova predmeta.
#ifndef _sklop_h_
#define _sklop_h_
#include "predmet3.h"
#include <vector>
using namespace std;

namespace Predmeti {
    class Sklop: public Predmet {
        vector<Predmet*> niz; // Sadržani predmeti.
        void kopiraj(const Sklop&); // Kopiranje u objekat.
        void brisi(); // Pražnjenje.
        void pisi(ostream&) const; // Pisanje.
    public:
        // Inicijalizacija:
        Sklop() {} // - kao prazan sklop,
        Sklop(const Sklop& s) // - kopijom drugog sklopa,
        { Predmet(s) { kopiraj(s); }
        Sklop(const Predmet& p) // - kopiranjem predmeta,
        { Predmet() { niz.push_back(p.kopija()); }
        Sklop(Predmet* p) // - preuzimanjem predmeta.
        { Predmet() { niz.push_back(p); }
        ~Sklop() { brisi(); } // Uništavanje.
        Sklop& operator=(const Sklop& s) { // Dodela vrednosti.
            if (this != &s) { brisi(); Predmet::operator=(s); kopiraj(s); }
            return *this;
        }
        Sklop& operator+=(const Predmet& p) // Dodavanje predmeta kopiranjem,
        { niz.push_back(p.kopija()); return *this; }
        Sklop& operator+=(Predmet* p) // i preuzimanjem.
        { niz.push_back(p); return *this; }
        Sklop* kopija() const { return new Sklop(*this); } // Kopiranje.
        double V() const; // Zapremina.
        double Q() const; // Težina.
    }; // class Sklop
} // namespace Predmeti

#endif
```

```
// sklop.C - Metode klase sklopova predmeta.
#include "sklop.h"

namespace Predmeti {
    void Sklop::kopiraj(const Sklop& s) { // Kopiranje u objekat.
        for (unsigned i=0; i<s.niz.size(); i++)
            niz.push_back(s.niz[i]->kopija());
    }

    void Sklop::brisi() { // Pražnjenje.
        for (unsigned i=0; i<niz.size(); delete niz[i++]);
        niz.clear();
    }

    void Sklop::pisi(ostream& it) const { // Pisanje.
        Predmet::pisi(it); it << '[' << endl; nivo++;
        for (unsigned i=0; i<niz.size(); it << *niz[i++]<<endl;
            --nivo; it << setw(2*nivo+1) << '];'
    }

    double Sklop::V() const { // Zapremina.
        double v = 0;
        for (unsigned i=0; i<niz.size(); v+=niz[i++]->V());
        return v;
    }

    double Sklop::Q() const { // Težina.
        double q = 0;
        for (unsigned i=0; i<niz.size(); q+=niz[i++]->Q());
        return q;
    }
} // namespace Predmeti
```

```
// predmet3t.C - Ispitivanje klase predmeta.
#include "sfera4.h"
#include "kvadar5.h"
#include "sklop.h"
using namespace Predmeti;
#include <iostream>
#include <string>
using namespace std;

Predmet* citaj(istream& ut) { // Čitanje jednog predmeta.
    string vrs; ut >> vrs;
    if (vrs == "Sfera") {
        double s, r; ut >> s >> r;
        return new Sfera(r, s);
    } else if (vrs == "Kvadar") {
        double s, a, b, c; ut >> s >> a >> b >> c;
        return new Kvadar(a, b, c, s);
    } else if (vrs == "Sklop") {
        Sklop* s = new Sklop;
        while (Predmet* p = citaj(ut)) *s += p;
        return s;
    } else return 0;
}
```

```

int main() { // Glavna funkcija.
    vector<Predmet*> niz;
    cout << "\nProcitano:\n\n";
    while (Predmet* p = citaj(cin)) {
        cout << *p << " V=" << p->V() << " Q=" << p->Q() << "\n\n";
        niz.push_back(p);
    }
    vector<Predmet*>::iterator i = niz.begin(), iVmax = i, iQmax = i;
    double Vmax = 0, Qmax = 0;
    for (; i!=niz.end(); i++) {
        if ((*i)->V() > Vmax) { Vmax = (*i)->V(); iVmax = i; }
        if ((*i)->Q() > Qmax) { Qmax = (*i)->Q(); iQmax = i; }
    }
    cout << "\nNajvecu zapreminu ima:\n\n" << **iVmax << "\n\n";
    cout << "Najvecu tezinu ima:\n\n" << **iQmax << "\n\n";
    return 0;
}

```

% predmet3t <predmet3t.pod

Procitano:

Sfera/1 (2;2.5) V=65.4498 Q=130.9

Kvadar/2 (6;2,3,4) V=24 Q=144

Sklop/3 [
 Sfera/4 (1;0.5)
 Kvadar/5 (0.5;0.2,0.3,0.4)
] V=0.547598 Q=0.535598

Sklop/6 [
 Kvadar/7 (3;1,0.3,0.4)
 Sklop/8 [
 Sklop/9 [
 Sfera/10 (1;0.2)
 Kvadar/11 (5;0.2,3,4)
]
 Kvadar/12 (0.5;0.2,1,0.2)
]
] V=2.59351 Q=12.4135

Najvecu zapreminu ima:

Sfera/1 (2;2.5)

Najvecu tezinu ima:

Kvadar/2 (6;2,3,4)

predmet3t.pod

```

Sfera 2 2.5
Kvadar 5 2 3 4
Sklop
  Sfera 1 .5
  Kvadar .5 .2 .3 .4
  *
Sklop
  Kvadar 3 1 .3 .4
  Sklop
    Sklop
      Sfera 1 .2
      Kvadar 5 .2 3 4
    *
    Kvadar .5 .2 1 .2
  *
  *
  *

```

Zadatak 7.3 Obrada sekvencijalne tekstualne datoteke

U sekvencijalnoj tekstualnoj datoteci u svakom redu se nalazi jedan ceo broj n i n realnih brojeva. Napisati na jeziku C++ program koji u novu sekvencijalnu datoteku upiše one redove iz početne datoteke u kojima je srednja vrednost realnih brojeva veća od nule i na kraju ispiše na glavnom izlazu zbir srednjih vrednosti realnih brojeva u svim redovima početne datoteke.

Rešenje:

```

// srvred.C - Srednja vrednost brojeva u zapisima tekstualne datoteke.

#include <fstream>
#include <iostream>
using namespace std;

int main() {
    ifstream ul ("srvred.pod");
    ofstream izl("srvred.rez");
    double z = 0;
    while (true) {
        int n; ul >> n;
        if (!ul) break;
        double* a = new double [n];
        double s = 0;
        for (int i=0; i<n; i++) { ul >> a[i]; s += a[i]; }
        if (n) s /= n;
        z += s;
        if (s > 0) {
            izl << n;
            for (int i=0; i<n; i++) izl << ' ' << a[i++];
            izl.put('\n');
        }
        delete [] a;
    }
    cout << "Zbir srednjih vrednosti= " << z << endl;
    return 0;
}

```

srvred.pod

```

6 1 2 3 4 5 6
8 -1 -2 -3 -4 -5 -6 -7 -8
5 4 -2 5 -7 1
7 1 -2 3 -4 5 -6 7
4 4 -5 6 -7

```

% srvred

Zbir srednjih vrednosti= -0.728571

srvred.rez

```

6 1 2 3 4 5 6
5 4 -2 5 -7 1
7 1 -2 3 -4 5 -6 7

```

Zadatak 7.4 Obrada rečenica u tekstualnoj sekvencijalnoj datoteci

Napisati na jeziku C++ funkciju za prepisivanje sadržaja sekvencijalne tekstualne datoteke u drugu sekvencijalnu tekstualnu datoteku uz pretvaranje početnih slova rečenica u velika, a svih ostalih slova u mala. Kraj rečenice obeležava se tačkom (.), znakom uzvika (!) ili znakom pitanja (?).

Napisati na jeziku C++ program koji čita parove imena datoteka i poziva prethodnu funkciju sve dok umesto imena jedne od datoteka ne pročitati tri zvezdice (***)

Rešenje:

```
// recenice.C - Sredivanje rečenica.

#include <fstream>
#include <iostream>
#include <cctype>
#include <string>
using namespace std;

void recenice(istream& ul, ostream& izl) {
    int zn; bool prvi = true;
    while ((zn = ul.get()) != EOF) {
        if (isupper(zn))
            { if (!prvi) zn = tolower(zn); else prvi = false; }
        else if (islower(zn))
            { if (prvi) { zn = toupper(zn); prvi = false; } }
        else if (zn=='.' || zn=='!' || zn=='?')
            prvi = true;
        izl.put(zn);
    }
}

int main() {
    ifstream ul; ofstream izl; string ulaz, izlaz;
    while (true) {
        printf("Ime ulazne datoteke? "); cin >> ulaz;
        if (ulaz == "****") break;
        printf("Ime izlazne datoteke? "); cin >> izlaz;
        if (izlaz == "****") break;
        ul.open(ulaz.c_str()); izl.open(izlaz.c_str());
        recenice(ul, izl);
        izl.close(); ul.close();
    }
    return 0;
}

% recenice
Ime ulazne datoteke? recenice.pod
Ime izlazne datoteke? recenice.rez
Ime ulazne datoteke? ****

recenice.pod
Prva, lepa recenica. dRUGA
ruZNa ReCeNiCa. OVA RECENICA SE
ZAVRSAVA NA KRAJU REDA.

u ovoj recenici ima i suvisnih
razmaka. DANAS JE 2. JANUAR 2006.

recenice.rez
Prva, lepa recenica. Druga
ruzna recenica. Ova recenica se
zavrsava na kraju reda.

U ovoj recenici ima i suvisnih
razmaka. Danas je 2. Januar 2006.
```

Zadatak 7.5 Obrada sekvencijalne binarne datoteke

Sekvencijalna binarna datoteka sadži zapise promenljive dužine čiji je sadržaj jedan ceo broj n i n realnih brojeva. Napisati na jeziku C++ programe za:

- formiranje jedne takve datoteke čitajući podatke iz sekvencijalne tekstualne datoteke;
- ispisivanje sadržaja jedne takve datoteke na glavnom izlazu; i
- razvrstavanje zapisa jedne takve datoteke u dve, tako da u jednoj budu zapisi u kojima je srednja vrednost podataka veća od nule, a u drugoj zapisi s negativnom srednjom vrednošću.

Imena datoteka za obradu programima dostavljaju se kao parametri programa.

Rešenje:

```
// sekpravi.C - Formiranje sekvencijalne binarne datoteke.

#include <fstream>
#include <iostream>
using namespace std;

int main(int, const char* varg[]) {
    try {
        ifstream ulaz (varg[1]); if (!ulaz) throw 1;
        ofstream izlaz(varg[2], ios::binary); if (!izlaz) throw 2;
        int n;
        while (ulaz >> n) {
            double* x = new double [n];
            for (int i=0; i<n; ulaz >> x[i++]);
            izlaz.write((char*)&n, sizeof(int));
            izlaz.write((char*)x, n*sizeof(double));
            delete [] x;
        }
    } catch (int g) {
        cout << "*** Greska pri otvaranju datoteke " << varg[g] << " ***\a\n";
        return g;
    }
    return 0;
}

// sekprik.C - Prikazivanje sadržaja sekvencijalne binarne datoteke.

#include <fstream>
#include <iostream>
#include <iomanip>
using namespace std;

int main(int, const char* varg[]) {
    try {
        ifstream ulaz(varg[1], ios::binary); if (!ulaz) throw 1;
        int n;
        cout << fixed << setprecision(2);
        while (ulaz.read((char*)&n, sizeof(int))) {
            double* x = new double [n];
            ulaz.read((char*)x, n*sizeof(double));
            cout << setw(3) << n << ' ';
            for (int i=0; i<n; i++)
                cout << setw(7) << x[i] <<
                    (i==n-1 ? "\n" : i%8==7 ? "\n" : " ");
            delete [] x;
        }
    }
}
```

```

catch (int g) {
    cout << "*** Greska pri otvaranju datoteke " << varg[g] << " ***\a\n";
    return g;
}
return 0;
}

```

// sekradi.c - Obrada sekvencijalne binarne datoteke.

```

#include <fstream>
#include <iostream>
using namespace std;

int main(int, const char* varg[]) {
    try {
        ifstream ulaz(varg[1], ios::binary); if (!ulaz) throw 1;
        ofstream poz (varg[2], ios::binary); if (!poz ) throw 2;
        ofstream neg (varg[3], ios::binary); if (!neg ) throw 3;
        int n;
        while (ulaz.read((char*)&n, sizeof(int))) {
            double* x = new double [n];
            ulaz.read((char*)x, n*sizeof(double));
            double s = 0;
            for (int i=0; i<n; s+=x[i++]);
            if (s) {
                (s>0 ? poz : neg).write((char*)&n, sizeof(int));
                (s>0 ? poz : neg).write((char*)x, n*sizeof(double));
            }
            delete [] x;
        }
    } catch (int g) {
        cout << "*** Greska pri otvaranju datoteke " << varg[g] << " ***\a\n";
        return g;
    }
    return 0;
}

```

sek.pod

```

3 1 2 3
4 -1 -2 -3 -4
5 0 0 0 0
10 1 -1 2 -2 3 -3 4 -4 5 -5
2 123 -456
2 -123 456

```

```

% sekpravi sek.pod sek.dat
% sekprkri sek.dat
3 1.00 2.00 3.00
4 -1.00 -2.00 -3.00 -4.00
5 0.00 0.00 0.00 0.00 0.00
10 1.00 -1.00 2.00 -2.00 3.00 -3.00 4.00 -4.00
5.00 -5.00
2 123.00 -456.00
2 -123.00 456.00
% sekradi sek.dat sek.poz sek.neg
% sekprkri sek.poz
3 1.00 2.00 3.00
2 -123.00 456.00
% sekprkri sek.neg
4 -1.00 -2.00 -3.00 -4.00
2 123.00 -456.00

```

Zadatak 7.6 Obrada relativne binarne datoteke

Zapisi fiksne dužine u relativnoj binarnoj datoteci sadrže naziv artikla (`char[30]`), količinu artikla (`float`) i jediničnu cenu artikla (`float`). Redni broj zapisa u datoteci (1, 2, 3, ...) predstavlja istovremeno i šifru artikla na koji se odnosi dati zapis. Napisati na jeziku C++ program za ažuriranje sadržaja te datoteke koji iz neke tekstualne datoteke čita niz parova šifara artikala i iznosa promene količine tih artikala. U toku obrade datoteke treba da se izračuna i ukupna promena vrednosti robe kao posledica pročitanih promena količina. Rezultat treba ispisati na glavnom izlazu.

Napisati na jeziku C++ neophodne pomoćne programe koji omogućuju da se prethodni program ispita.

Rešenje:

// magacin.C - Obrada relativne binarne datoteke.

```

#include <fstream>
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    struct Zapis { char naziv[30]; float kolicina, cena; };
    Zapis zapis;
    float ukupno=0, promena; int sifra, duzina = sizeof zapis;
    fstream datot("magacin.dat", ios::in | ios::out | ios::binary);
    ifstream prom("promene.txt");
    while (prom >> sifra >> promena) {
        datot.seekg((long)(sifra-1)*duzina);
        datot.read((char*)&zapis, duzina);
        zapis.kolicina += promena;
        ukupno += zapis.cena * promena;
        datot.seekp(-(long)duzina, ios::cur);
        datot.write((char*)&zapis, duzina);
    }
    cout << "Ukupna promena vrednosti robe je "
        << setprecision(2) << fixed << ukupno << endl;
    return 0;
}

```

Pomoćni programi:

// magpravi.C - Stvaranje početne datoteke.

```

#include <fstream>
using namespace std;

int main() {
    struct Zapis { char naziv[30]; float kolicina, cena; };
    Zapis zapis;
    ifstream pod("magacin.pod");
    ofstream dat("magacin.dat", ios::binary);
    while (pod >> zapis.naziv >> zapis.kolicina >> zapis.cena)
        dat.write((char*)&zapis, sizeof(Zapis));
    return 0;
}

```

```
// magpisi.C - Ispisivanje sadržaja datoteke.
```

```
#include <fstream>
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    struct Zapis { char naziv[30]; float kolicina, cena; };
    Zapis zapis; int sifra = 0;
    ifstream dat("magacin.dat", ios::binary);
    cout << fixed << setprecision(2);
    while (dat.read((char*)&zapis, sizeof(Zapis)))
        cout << setw(2) << ++sifra << ' ' << left
            << setw(20) << zapis.naziv << ' ' << right
            << setw(10) << zapis.kolicina << ' '
            << setw(10) << zapis.cena << endl;
    return 0;
}
```

magacin.pod

jabuke	10	50
kruske	15	60
mandarine	5	55
pomorandze	8	50
banane	20	45

promene.txt

5	-5
3	20
2	-10
4	5
1	-1

```
% magpravi
% magpisi
1 jabuke          10.00    50.00
2 kruske          15.00    60.00
3 mandarine       5.00     55.00
4 pomorandze      8.00     50.00
5 banane          20.00    45.00
% magacin
Ukupna promena vrednosti robe je 475.00
% magpisi
1 jabuke          9.00     50.00
2 kruske          5.00     60.00
3 mandarine      25.00    55.00
4 pomorandze     13.00    50.00
5 banane         15.00    45.00
```

Zadatak 7.7 Klasa rečnika

Napisati na jeziku C++ klasu za rečnik koji za svaku reč može da sadrži više prevoda. Predvideti:

- dodavanje prevoda za datu reč (reč već može da postoji u rečniku),
- brisanje svih prevoda i samo odabranog prevoda date reči,
- dohvatanje svih prevoda date reči,
- upisivanje svih prevoda date reči u izlazni tok, *i*
- upisivanje sadržaja celog rečnika u izlazni tok, po jedan red za svaku reč.

Napisati na jeziku C++ program koji prvo, čitajući parove *reč – prevod*, sastavi početni sadržaj rečnika a posle kroz dijalog s korisnikom rukuje rečnikom.

Rešenje:

```
// recnik.h - Klasa rečnika.
```

```
#ifndef _recnik_h_
#define _recnik_h_
```

```
#include <string>
#include <set>
#include <map>
#include <iostream>
using namespace std;
```

```
class Recnik {
    map<string, set<string> > niz; // Sadržaj rečnika.
public:
    // Dodavanje prevoda reči.
    Recnik& dodaj(const string& rec, const string& prevod) {
        niz[rec].insert(prevod);
        return *this;
    }
```

```
    // Brisanje svih prevoda reči.
    Recnik& brisi(const string& rec) {
        niz.erase(rec);
        return *this;
    }
```

```
    // Brisanje jednog prevoda reči.
    Recnik& brisi(const string& rec, const string& prevod) {
        map<string, set<string> >::iterator i = niz.find(rec);
        if (i != niz.end()) i->second.erase(prevod);
        return *this;
    }
```

```
    // Dohvatanje svih prevoda reči.
    const set<string>* prevodi(const string& rec) const {
        map<string, set<string> >::const_iterator i = niz.find(rec);
        if (i != niz.end()) return &i->second;
        else return 0;
    }
```

```
    // Pisanje svih prevoda reči.
    void pisiPrevode(ostream& it, const string& rec) const;
```



```
// Pisanje sadržaja rečnika.
friend ostream& operator<<(ostream& it, const Recnik& recnik);
};

#endif
```

```
// recnik.C - Metode i funkcije uz klasu rečnika.
```

```
#include "recnik.h"
```

```
// Pisanje svih prevoda reči.
```

```
void Recnik::pisiPrevode(ostream& it, const string& rec) const {
    map<string, set<string> >::const_iterator i = niz.find(rec);
    if (i != niz.end()) {
        for (set<string>::const_iterator j=i->second.begin();
             j!=i->second.end(); j++) {
            if (j != i->second.begin()) it << ", ";
            it << *j;
        }
    }

    // Pisanje sadržaja rečnika.
    ostream& operator<<(ostream& it, const Recnik& recnik) {
        for (map<string, set<string> >::const_iterator i=recnik.niz.begin();
             i!=recnik.niz.end(); i++) {
            it << i->first << ": ";
            for (set<string>::const_iterator j=i->second.begin();
                 j!=i->second.end(); j++) {
                if (j != i->second.begin()) it << ", ";
                it << *j;
            }
            it << "." << endl;
        }
        return it;
    }
}
```

```
// recnikt.C - Ispitivanje klase rečnika.
```

```
#include "recnik.h"
#include <fstream>
#include <iostream>
#include <string>
#include <cctype>
using namespace std;
```

```
int main() {
    Recnik r;
    string rec, prevod;
    ifstream ulaz("recnikt.pod");
    while (ulaz >> rec >> prevod) r.dodaj(rec, prevod);
    cout << "Pocetni sadrzaj recnika:\n\n" << r;
```

```
while (true) {
    cout << "\n1. Trazenje prevoda\n"
           "2. Dodavanje prevoda\n"
           "3. Brisanje prevoda\n"
           "4. Brisanje svih prevoda\n"
           "5. Prikazivanje recnika\n"
           "0. Zavrsetak rada\n\n"
           "Vas izbor? ";

    char izb; cin >> izb;
    if (izb == '0') break;
    switch (izb) {
        case '1': // Trazenje prevoda:
            cout << "Rec?      "; cin >> rec;
            cout << "Prevodi:   "; r.pisiPrevode(cout, rec); cout << endl;
            break;
        case '2': // Dodavanje prevoda:
            cout << "Rec?      "; cin >> rec;
            cout << "Prevod?    "; cin >> prevod;
            if (prevod != "") r.dodaj(rec, prevod);
            break;
        case '3': // Brisanje prevoda:
            cout << "Rec?      "; cin >> rec;
            cout << "Prevod?    "; cin >> prevod;
            r.brisi(rec, prevod);
            break;
        case '4': // Brisanje svih prevoda:
            cout << "Rec?      "; cin >> rec;
            cout << "Prevodi:   "; r.pisiPrevode(cout, rec); cout << endl;
            cout << "Brisati?    ";
            { string odg; cin >> odg;
              if (toupper(odg[0] == 'D')) r.brisi(rec);
            }
            break;
        case '5': // Prikazivanje rečnika:
            cout << endl << r;
            break;
    }
}

return 0;
```

```
% recnikt
```

```
Pocetni sadrzaj recnika:
```

```
beo: glimmering, light, white.
crn: black, dark, dirty, sunburned.
crven: red.
dobar: good, kind.
glas: rumor, sound, voice, vote.
gotov: completed, finished, ready.
jezik: language, speech, tongue.
obim: circumference, scope, volume.
plav: blond, blue.
potvrda: acknowledgement, certificate, confirmation.
prihod: income, vevenue.
zelen: green, immature, inexperienced, vegetables.
zut: yellow.
```

1. Traženje prevoda
2. Dodavanje prevoda
3. Brisanje prevoda
4. Brisanje svih prevoda
5. Prikazivanje rečnika
0. Završetak rada

```
Vas izbor? 1
Rec?      plav
Prevodi:   blond, blue
...
Vas izbor? 1
Rec?      roze
Prevodi:
...
Vas izbor? 2
Rec?      roze
Prevod?   pink
...
Vas izbor? 3
Rec?      zelen
Prevod?   green
...
Vas izbor? 3
Rec?      zelen
Prevod?   green
...
Vas izbor? 4
Rec?      gotov
Prevodi:   completed, finished, ready
Brisati?   d
...
Vas izbor? 4
Rec?      ljubicast
Prevodi:
Brisati?   d
...
Vas izbor? 4
Rec?      zelen
Prevodi:   immature, inexperienced, vegetables
Brisati?   n
...
Vas izbor? 5

beo: glimmering, light, white
crn: black, dark, dirty, sunburned
crven: red
dobar: good, kind
glas: rumor, sound, voice, vote
gotov: completed, finished, ready
jezik: language, speech, tongue
obim: circumference, scope, volume
plav: blond, blue
potvrda: acknowledgement, certificate, confirmation
prihod: income, vevenue
roze: pink
zelen: immature, inexperienced, vegetables
zut: yellow
...
Vas izbor? 0
```

recnikt.pod

```
prihod income
prihod vevenue
potvrda acknowledgement
potvrda certificate
potvrda confirmation
crn black
gotov completed
obim scope
gotov finished
beo light
dobar good
zelen green
zelen immature
beo glimmering
zelen inexperienced
obim circumference
beo white
dobar kind
jezik language
zut yellow
plav blond
plav blue
crn sunburned
glas voice
obim volume
glas vote
glas sound
gotov ready
crven red
crn dark
crn dirty
jezik speech
jezik tongue
glas rumor
zelen vegetables
```

Zadatak 7.8 Klasa relativnih binarnih datoteka i obrada liste u relativnoj binarnoj datoteci

Napisati na jeziku C++ klasu relativnih binarnih datoteka sa zapisima fiksne dužine. Predvideti:

- stvaranje nove prazne datoteke sa zapisima zadate dužine,
- otvaranje postojeće datoteke,
- upisivanje jednog zapisa na određeno mesto u datoteci,
- čitanje zapisa sa zadatim rednim brojem iz datoteke,
- dohvaćanje trenutnog broja zapisa u datoteci, i
- zatvaranje datoteke.

Napisati na jeziku C++ programe za stvaranje uređene liste u relativnoj binarnoj datoteci i za prikazivanje sadržaja takve liste. Elementi liste sastoje se od celobrojne šifre kao kriterijuma za uređivanje i niza znakova od 50 elemenata.

Rešenje:

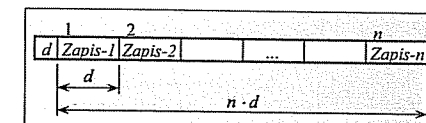
// rfile.h - Klasa relativnih binarnih datoteka.

```
#include <fstream>
using namespace std;

class RFile : fstream {
public:
    class Greska {           // KLASA GREŠAKA.
    public:                   // Šifre gresaka:
        enum Gre { DUZ,      // - neispravna dužina zapisa,
                    BRO,     // - neispravan redni broj zapisa,
                    VEL,     // - datoteka nije ceo broj zapisa,
                    ZAT,     // - datoteka nije otvorena,
                    OTV,     // - nije uspešno otvaranje,
                    POZ,     // - nije uspešno pozicioniranje,
                    PIS,     // - nije uspešno upisivanje,
                    CIT,     // - nije uspešno čitanje,
                    ZAP      // - zapis ne postoji.
        };

    private:
        Gre gre;              // Šifra greške.
        static const char* por[]; // Tekstualni opisi grešaka.
    public:
        Greska(Gre g) { gre = g; }
        friend ostream& operator<<(ostream& it, const Greska& g);
    };

    // ČLANOVI GLAVNE KLASKE:
private:
    long duz_zap, bro_zap, tek_zap, tek_poz; // bro_zap== -1 - nije otvorena.
    void otvori(const char* ime, long duz=0); // Otvaranje datoteke (duz>0
                                              // - stvaranje nove).
public:
    RFile(): fstream() { bro_zap = -1; } // Inic. bez otvaranja datoteke.
    RFile(const char* ime, long duz=0)   // Inic. s otvaranjem datoteke.
        : fstream() { otvori(ime, duz); }
    ~RFile() { close(); } // Uništavanje objekta.
    RFile& open(const char* ime, long duz=0) // Otvaranje datoteke.
        { otvori(ime, duz); return *this; }
    bool open() { return bro_zap >= 0; } // Da li je datoteka otvorena?
    RFile& write(const void* niz, long zap=0); // Pisanje zap (zap==0 - sekv).
    RFile& read(void* niz, long zap=0); // Čitanje zap (zap==0 - sekv).
```



```
RFile& close(); // Zatvaranje datoteke.
long recn() { return bro_zap<0 ? -1 : bro_zap; } // Broj zapisa u datoteci
long recl() { return bro_zap<0 ? -1 : duz_zap; } // Dužina zapisa.
};
```

// rfile.C - Metode i statička polja klase relativnih datoteka.

```
#include "rfile.h"
```

```
const char* RFile::Greska::por[] = { // PORUKE O GREŠKAMA:
    "Neispravna dužina zapisa", // DUZ
    "Neispravan redni broj zapisa", // BRO
    "Datoteka nije ceo broj zapisa", // VEL
    "Datoteka nije otvorena", // ZAT
    "Nije uspelo otvaranje", // OTV
    "Nije uspelo pozicioniranje", // POZ
    "Nije uspelo upisivanje", // PIS
    "Nije uspelo citanje", // CIT
    "Zapis ne postoji" // ZAP
};
```

```
ostream& operator<<(ostream& it, const RFile::Greska& g)
{ return it << "*** " << RFile::Greska::por[g.gre] << "! ***\n\a"; }
```

```
void RFile::otvori(const char* ime, long duz) { // OTVARANJE DATOTEKE.
    if (duz < 0) throw Greska(Greska::DUZ);
    if (bro_zap >= 0) close(); // Datot. otvorena zatvori.
    if (duz) { // Stvaranje nove datoteke.
        fstream::open(ime, ios::in);
        if (!is_open()) throw Greska(Greska::OTV);
        fstream::open(ime, ios::in | ios::out | ios::binary);
        if (!is_open()) throw Greska(Greska::OTV);
        if (!fstream::write((char*)&duz, sizeof(long)) ||
            !flush()) throw Greska(Greska::PIS);
        duz_zap = duz;
        bro_zap = 0;
    } else { // Otvaranje stare datoteke.
        fstream::open(ime, ios::in);
        if (!is_open()) throw Greska(Greska::OTV);
        fstream::close();
        fstream::open(ime, ios::in | ios::out | ios::binary);
        if (!is_open()) throw Greska(Greska::OTV);
        if (!fstream::read((char*)&duz_zap, sizeof(long)))
            throw Greska(Greska::CIT);
        if (!seekg(0l, ios::end)) throw Greska(Greska::POZ);
        long k = tellg() - (long)sizeof(long);
        if (k % duz_zap) throw Greska(Greska::VEL);
        bro_zap = k / duz_zap;
    }
    tek_zap = 0;
    tek_poz = sizeof(long);
}
```

```
RFile& RFile::close() { // ZATVARANJE DATOTEKE.
    fstream::close();
    bro_zap = -1;
    return *this;
}
```

```
RFile& RFile::write(const void* niz, long zap) { // PISANJE ZAPISA.
    if (bro_zap == -1) throw Greska(Greska::ZAT);
    if (zap < 0) throw Greska(Greska::BRO);
    if (zap) { // Direktan pristup.
        tek_poz = (zap - 1) * duz_zap + sizeof(long);
        tek_zap = zap;
    } else { // Sekvencijalan pristup.
        tek_poz += duz_zap;
        tek_zap++;
    }
    if (tek_zap > bro_zap) bro_zap = tek_zap;
    if (!seekp(tek_poz)) throw Greska(Greska::POZ);
    if (!fstream::write((char*)niz, duz_zap) ||
        !flush()) throw Greska(Greska::PIS);
    return *this;
}
```

```
RFile& RFile::read(void* niz, long zap) { // ČITANJE ZAPISA.
    if (bro_zap == -1) throw Greska(Greska::ZAT);
    if (zap < 0) throw Greska(Greska::BRO);
    if (zap) { // Direktan pristup.
        tek_poz = (zap - 1) * duz_zap + sizeof(long);
        tek_zap = zap;
    } else { // Sekvencijalan pristup.
        tek_poz += duz_zap;
        tek_zap++;
    }
    if (zap > bro_zap) throw Greska(Greska::ZAP);
    if (!seekg(tek_poz)) throw Greska(Greska::POZ);
    if (!fstream::read((char*)niz, duz_zap)) throw Greska(Greska::CIT);
    return *this;
}
```

	sif	txt	nar
1	0	0	6
2	5	petak	5
3	2	utorak	7
4	7	nedelja	0
5	6	subota	4
6	1	ponedeljak	3
7	3	sreda	8
8	4	cetvrtak	2

// lista8pravi.C - Obrazovanje uredene liste u relativnoj datoteci.

```
#include "rfile.h"
#include <iostream>
#include <iomanip>
using namespace std;
```

```
struct Zapis { int sif; char txt[50]; unsigned nar; };
```

```
int main() {
    try {
        RFile dat("lista8.dat", sizeof(Zapis));
        Zapis zap; zap.sif = zap.txt[0] = zap.nar = 0; // Glava liste.
        dat.write(&zap, 1);
        ifstream pod("lista8.pod"); // Tekstualna datoteka s podacima.
    }
}
```

```

while (pod >> zap.sif >> zap.txt) {
    cout << setw(5) << zap.sif << "    " << zap.txt << endl;
    unsigned nov = dat.recn() + 1; dat.write(&zap, nov);
    int sif = zap.sif;
    dat.read(&zap, 1); unsigned pre = 1, tek = zap.nar;
    while(tek && (dat.read(&zap, tek), zap.sif < sif))
        { pre = tek; tek = zap.nar; }
    dat.read(&zap, pre); zap.nar = nov; dat.write(&zap, pre);
    dat.read(&zap, nov); zap.nar = tek; dat.write(&zap, nov);
}
} catch (RFile::Greska g) { cout << g; }
return 0;
}

```

// lista8pisi.C - Prikazivanje sadržaja liste iz relativne datoteke.

```

#include "rfile.h"
#include <iostream>
#include <iomanip>
using namespace std;

struct Zapis { int sif; char txt[50]; unsigned nar; };

int main() {
    try {
        RFile dat("lista8.dat");
        Zapis zap;
        dat.read(&zap, 1);
        while (zap.nar) {
            dat.read(&zap, zap.nar);
            cout << setw(5) << zap.sif << "    " << zap.txt << endl;
        }
    } catch (RFile::Greska g) { cout << g; }
    return 0;
}

```

```

% lista8pravi
5 petak
2 utorak
7 nedelja
6 subota
1 ponedeljak
3 sreda
4 cetvrtak

% lista8pisi
1 ponedeljak
2 utorak
3 sreda
4 cetvrtak
5 petak
6 subota
7 nedelja

```

lista8.pod

```

5 petak
2 utorak
7 nedelja
6 subota
1 ponedeljak
3 sreda
4 cetvrtak

```

Издавач

АКАДЕМСКА МИСАО

Приморска 21, Београд
тел./факс: (+381 11) 3218 354

www.akademska-misao.rs
office@akademska-misao.rs
knjizara@akademska-misao.rs

CIP - Каталогизација у публикацији
Народна библиотека Србије, Београд

CIP

004.432.2C++ (075.8)(076)
004.42.045(075.8)(076)

КРАУС, Ласло Л., 1949-

Rešeni zadaci iz programskog jezika C++ /
Laslo Kraus. - 3. dopunjeno izd. - Beograd :
Akademska misao, 2011 (Beograd : Planeta
print). - 302 str. : graf. prikazi ; 24 cm

Tiraž 300. - Preporučena literatura: str. 6.

ISBN 978-86-7466-411-7

а) Програмски језик "C++" - Задаци б)
Објектно оријентисано програмирање - Задаци
COBISS.SR-ID 186276620
